OCR A level NEA

Aadam Anwar

Table of Contents

Analysis ***1***

What will I be making? **1**

Why am I making it? **1**

Why This Problem Is Amenable to a Computational Approach **1**

Existing Systems **3**

MyFitnessPal Summary: 3

MyFitnessPal Evaluation: 3

Table 1: Features i will take from MyFitnessPal 10

Table 2: Features i will not take from MyFitnessPal 10

Other existing systems (Cronometer,LoseIt,Lifesum,CalAI): 11

Cronometer 11

Lose It! 13

Lifesum 15

Cal Ai 17

Overall existing systems analysis : 18

Stakeholder Analysis **20**

Stakeholder 1 : General Health Conscious Users 20

Stakeholder 2 : Diabetic Patients 20

Stakeholder 3 : Fitness Coaches / Personal Trainers 21

Stakeholder 4 : Elderly Users (65+) 21

Interview Questions : 22

Justification of Questions: 23

Interviews **24**

Interview 1 24

Interview 2 26

Interview 3 28

Interview 4 30
Conclusion: 32
Summary Table (Features i will add to meet the stakeholders needs): 35
Features i will implement: 35
Summary: Feature Inspiration from Existing Systems 38
Limitations of the Proposed Solution 39
Hardware Requirements 41
Software Requirements: 42
Success Criteria: 43
Design 46
Decomposition Diagram : 46
Justification of Decomposition Diagram 46
Data Structures, Algorithms, Calculations and Libraries: 47
Authentication Menu: 48
Main Menu: 52
Log Food: 53
Log Workout: 62
View Stats: 62
Settings: 64
Whole program 65
Tkinter in the Program: 66
Overall program flow : 71
User Interface and Usability Features 72
Usability Features and GUI design: 72
Database Design 84
SQL Tables: 84
ER diagram for database structure: 86

Truth tables **87**
Authentication menu truth tables 87
Main menu truth tables 90
Log food truth tables 91
Log workout truth tables 92
View stats truth tables 93
Settings truth tables 93
Class diagram **93**
Key Classes and Variables **95**
SignUp Class 95
Login Class 96
LogFood Class 96
Log Food Listbox Variables 97
Stack Class 98
LogWorkout Class 98
Log Workout Listbox Variables 98
ViewStats Class 99
Settings Class 99
Calculated Variables (Algorithms) **100**
Daily calorie calculation 100
Binary search variables 101
Test data **101**
Development test data: 101
Post development Test Data 104
Usability Testing 111
Iterative development process ***112***
Iteration 1 - Authentication System **112**

Development 113
Iteration 1-Testing 121
Iteration 2 - Food Logging 129
Development 129
Iteration 2-Testing 136
Iteration 3 - Workout Logging 144
Development 144
Iteration 3 - Testing 151
Iteration 4 - View Stats 160
Development 160
Iteration 4 - Testing 165
Iteration 5 − Settings 169
Development 169
Iteration 5 - Testing 174
GUI Testing 178
Justification for Test 5.9 Failure 184
Remaining testing 185
Database testing: 188
Post Development Testing: 191
Analysis of post development testing 198
Usability Testing: 198
Tester 1- Student : 198
Tester 2 - Type 2 diabetic: 199
Analysis of Usability Testing 200
Evaluation: 201
Evaluation against the Success criteria defined during the analysis phase: 201
Summary of success criteria evaluation: 213

Further development from success criteria : 214

Usability Features Evaluation 215

Summary of usability features 216

Further development from usability tests and interviews: 217

Further Development 218

Limitations 219

Maintenance 220

Code: 221

Analysis

What will I be making?

I plan to make a health and fitness program that helps users track key metrics, including their diet, caloric intake, exercise, and weight. The program should include estimated caloric intake recommendations based on the user's goals, whether their goal is to gain weight, lose weight, or maintain their weight, whilst also tracking macronutrient intake.

Why am I making it?

The "Health Survey for England,2024" showed that 30% of adults aged 16 and over were living with obesity, and 66% were either overweight or living with obesity. Also the burden on the NHS due to obesity and related illnesses is well recognised. The monetary cost each year, uplifted for inflation, was estimated at £6.5 billion in 2022 (Source: Frontier Economics, 2022).In order for people to reduce their weight and become more healthy, adopting healthier habits such as eating clean food and exercising works but the most renowned tactic for weight loss is calorie tracking as consuming less calories than one has burned on a daily basis leads to weight loss and this is shown in numerous studies and is even promoted by the NHS through a programme called "Better Health".

I am making this because improving fitness is a problem that is common among individuals. Lots of Individuals wish to better themselves but lack an easy way to monitor this. With fitness tracking applications, one is able to easily monitor their weight gain, weight loss, and the intake of carbohydrates and sugar for individuals who need to monitor this for a particular illness like diabetes. It also allows individuals to analyse their food habits and avoid certain foods that may pose any risks to their health and wellbeing. This program will make it easier for users to stay on top of their goals and see how they are progressing over time.

Why This Problem Is Amenable to a Computational Approach

This problem is well suited to a computational solution since it is based on structured input data and defined algorithms. Manual maintaining information such as body mass index, calorie consumption and physical activities is very labour intensive, prone to human error and also very time consuming. Computers can carry these operations out quickly, reliably and at a large scale.

Several features make this problem ideal for computational methods:

Structured and repetitive calculations: Calculations such as BMR, TDEE, BMI, and calories/macronutrients per day are calculated by mathematical formulae. While it would take minutes to do these manually, a software application can do it within milliseconds. This will also ensure that users receive accurate, personalised recommendations without needing to understand any of the underlying mathematics.

Data storage and retrieval at scale: The management of a food database that holds thousands of records, alongside users information about their diets and fitness sessions for weeks or months, calls for a storage method with fast retrieval of data. Relational SQL databases are specifically designed for this purpose, allowing for the storage of large amounts of data, such as nutritional information for more than 5,000 foods and retrieving particular foods within a short amount of time by means of optimised querying. Manual logging on paper would mean users would have to flip through large logs searching for past records, while a computer program can retrieve past meals or fitness sessions for any day through a database query immediately.

Aggregation of data and feedback in real time: This application automatically aggregates caloric consumption as well as the macronutrients (protein, carbohydrates, and fats) over several meals in one day's time and shows the results in real time. Typing out such data manually after each meal will be quite time consuming and error prone, especially when keeping track of 10-15 different types of food items in a day. A computational solution can make these calculations instantly and also provide immediate feedback on the caloric and macronutrient requirements still left for the day.

Input validation and error prevention: A computational solution can validate the user inputs to maintain the integrity of the data by refusing negative weight inputs, non numerical age inputs, and activity level outside the valid range. These procedures can be easily implemented into a computer programs.

Consistency and accuracy across users: The same algorithms and formulas are applied to every user, ensuring all of the calculations are consistent. A nutritionist might make slight variations in BMR calculations or even forget to account for activity levels, but a computational approach applies the same formula exactly as specified every time. This consistency is particularly important for users like diabetics who will require precise carbohydrate tracking for insulin dosing. A calculation error could have serious health consequences. Computational methods guarantee that the same input will always produce the same output, providing reliability that manual methods just don't have.

Scalability for multiple users: While a single individual could in theory track their own data manually, a computational solution can handle hundreds or even thousands of users simultaneously without any additional effort. Each user's data can be stored separately in a database with unique identifiers, and the program will retrieve and process only the relevant records for the user. This scalability means the same program can serve a single person or an entire gym's clients without being modified, whereas manual tracking scales poorly ,each additional user would require separate physical logs and individual calculations.

Visual data representation: Humans are poor at identifying patterns in raw numerical data, but are much better at interpreting visual information. Making a visualisation manually would require graph papers and rulers, as well as a considerable amount of time, making this very impractical. Tools such as matplotlib will allow me to make professional graphs in seconds from numeric data.

Overall, a computational approach will ensure repeatability and scalability. It enables real time feedback, visualisation, and can carry out complex calculations that would be significantly harder to manage manually. By using algorithms, databases, and user interfaces, the system will be able to deliver personalised health insights efficiently and effectively.

Existing Systems

I decided to look into existing systems and after some research, I have gathered the most widely used applications for health and fitness within the United Kingdom. By doing this, I was able to gain a greater understanding of how people who are interested in fitness and calorie tracking interact with these systems and how it is used to aid them in reaching their fitness goals.

MyFitnessPal Summary:

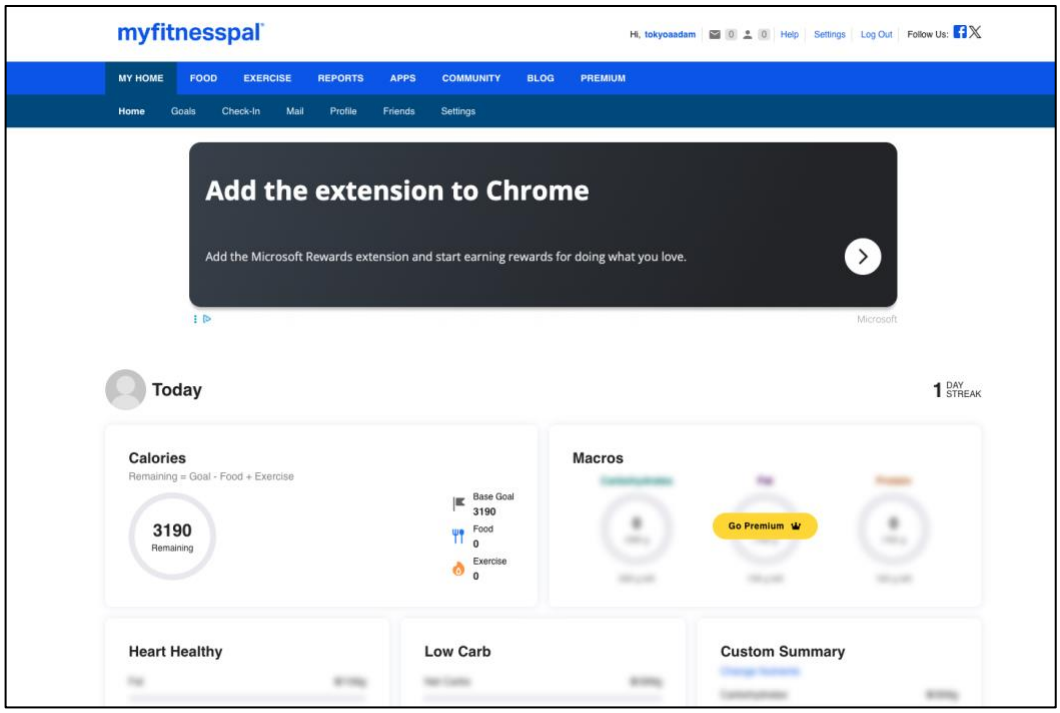
MyFitnessPal is a digital health and fitness tracking application used globally. It is designed to assist users in managing various aspects of their wellbeing including calorie tracking, exercise logging, and goal setting.

- The platform offers personalisation options to suit individual health objectives and dietary preferences.
- MyFitnessPal also supports integration with multiple fitness devices such as garmin watches and apple watches and also with other apps such as garmin connect, enabling data sharing and progress monitoring.
- MyFitnessPal is the chosen application used by individuals aiming to improve their nutrition, fitness, and overall lifestyle.
- As someone who uses MyFitnessPal myself, I find one feature specifically interesting: the ability to connect to wearables and applications. Myfitnesspal allows data from wearables to be sent directly to Myfitnesspal and vice versa, allowing users to be able to see all of their information in one central application. I will be taking this feature into account and will consider implementing something like this in my own project.

Cost: Free version and a paid version at £80 for an annual subscription

URL: https://www.myfitnesspal.com/

MyFitnessPal Evaluation:



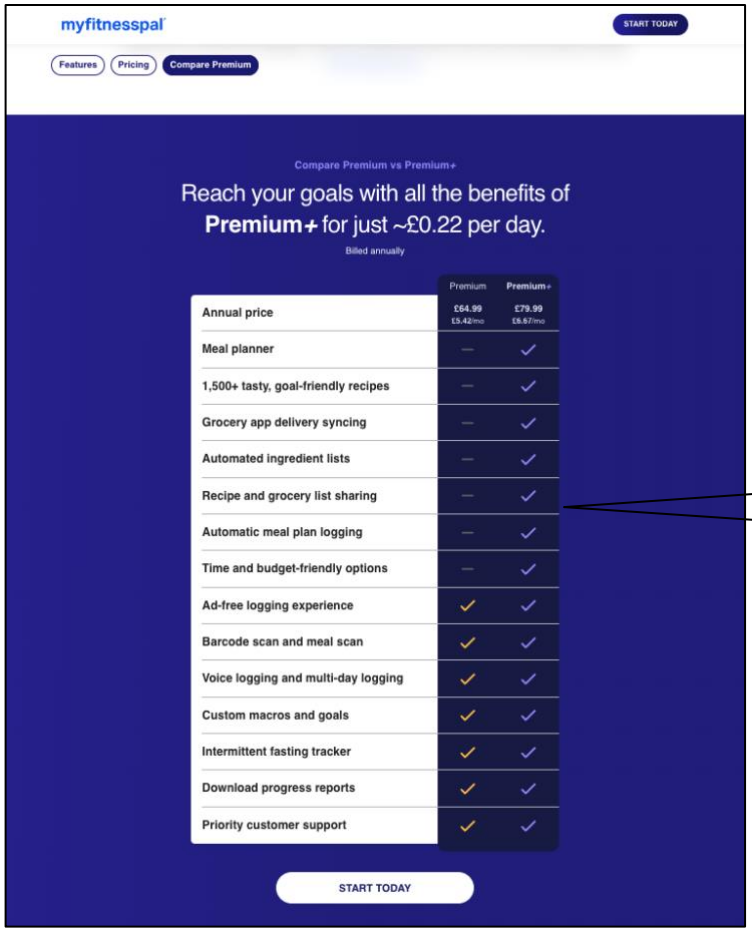
Pros and Cons :

Offers a large food database, making calorie and nutrient tracking efficient as it is unlikely the food the user is looking for won't be found .	Many useful features are locked behind a paywall, limiting free users.
Integrates with various fitness devices and apps for easy progress monitoring.	Adverts can disrupt the user experience unless a premium subscription is purchased.
Has an efficient interface for certain aspects that allows quick and straightforward logging of meals and exercise.	Community driven food data entries can be inaccurate and can lead to undesired outcomes for the user.

A key drawback of MyFitnessPal:

Too many features are locked behind a paywall

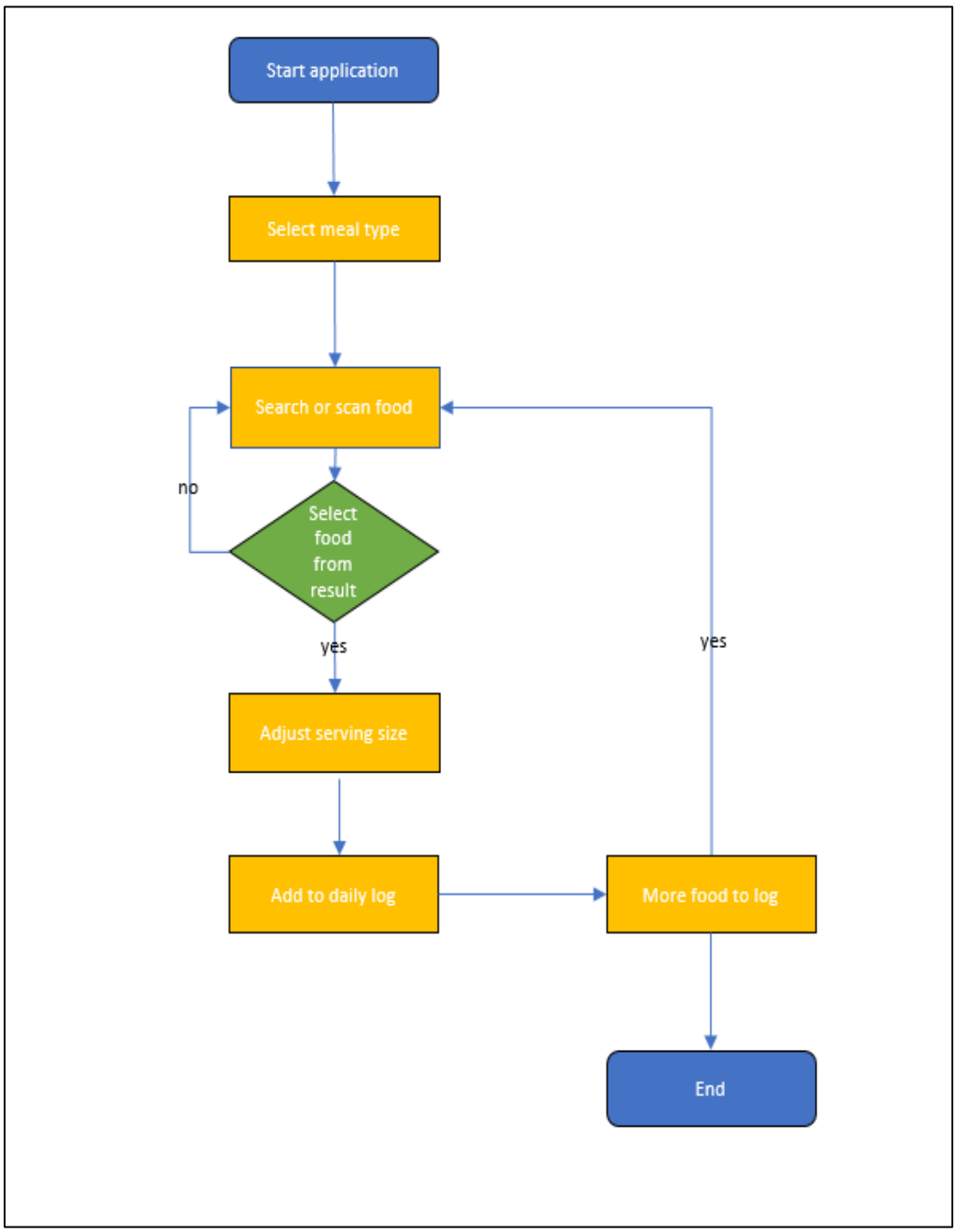
One of the biggest issues people run into with MyFitnessPal is how the app pushes its premium subscription. Over time, more and more useful features are being moved behind the paywall, which means the free version feels increasingly limited.



List of features only available to premium users

The result is that the free tier doesn't offer the same value it once did, and users who want a full tracking experience often feel the need to pay for the premium membership.

The flowchart below is a simple illustration of how the food logging process works.



The current process is as follows:

1.Meal type is selected

● The user will be asked to choose which meal they would like to track food for

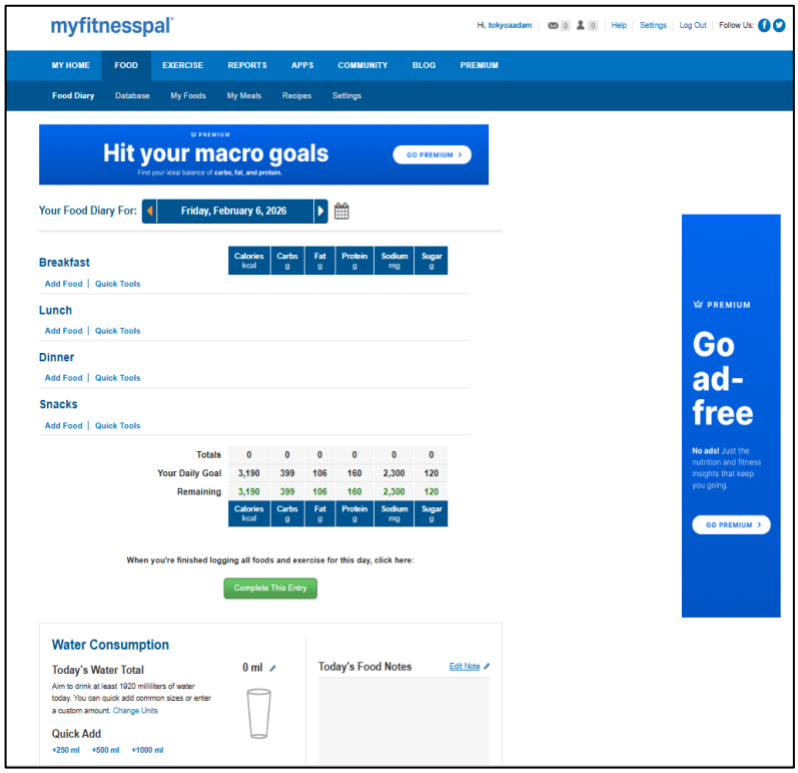
2. User will select food

● The user will select the food for their meal from the extensive food database. The food item can either be searched up or the barcode of the food item can be scanned .

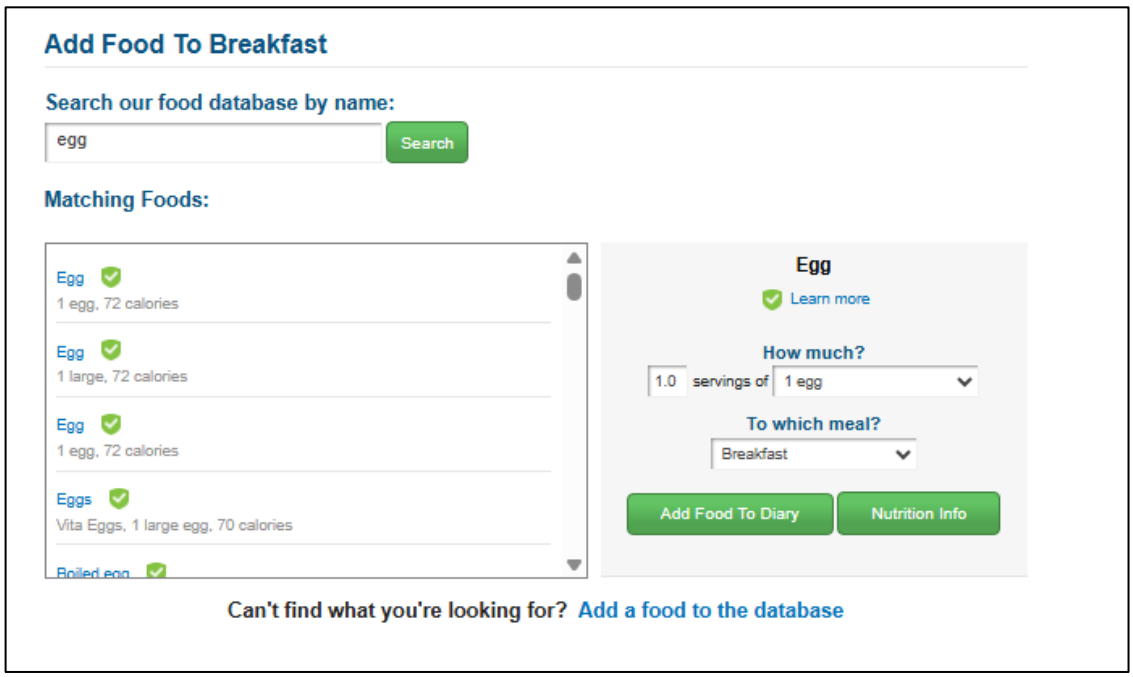
Though this is a simplified summary of how the existing system operates , it gives me an opportunity to clearly evaluate the overall process of the system, allowing me to take inspiration from these processes and use them for my own fitness tracking program.

The screenshots below showcase the main features:

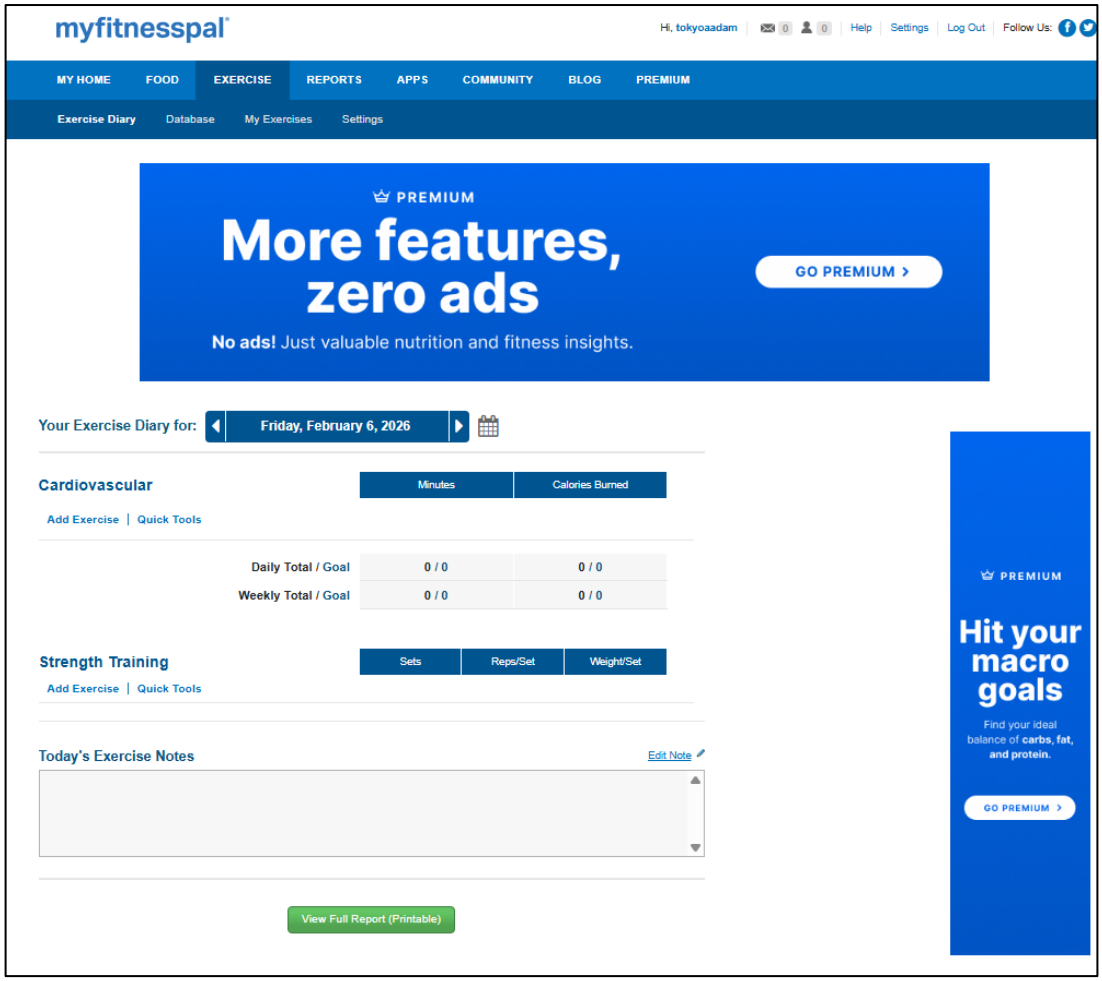
The main food logging page



The food logging process



The workout logging page

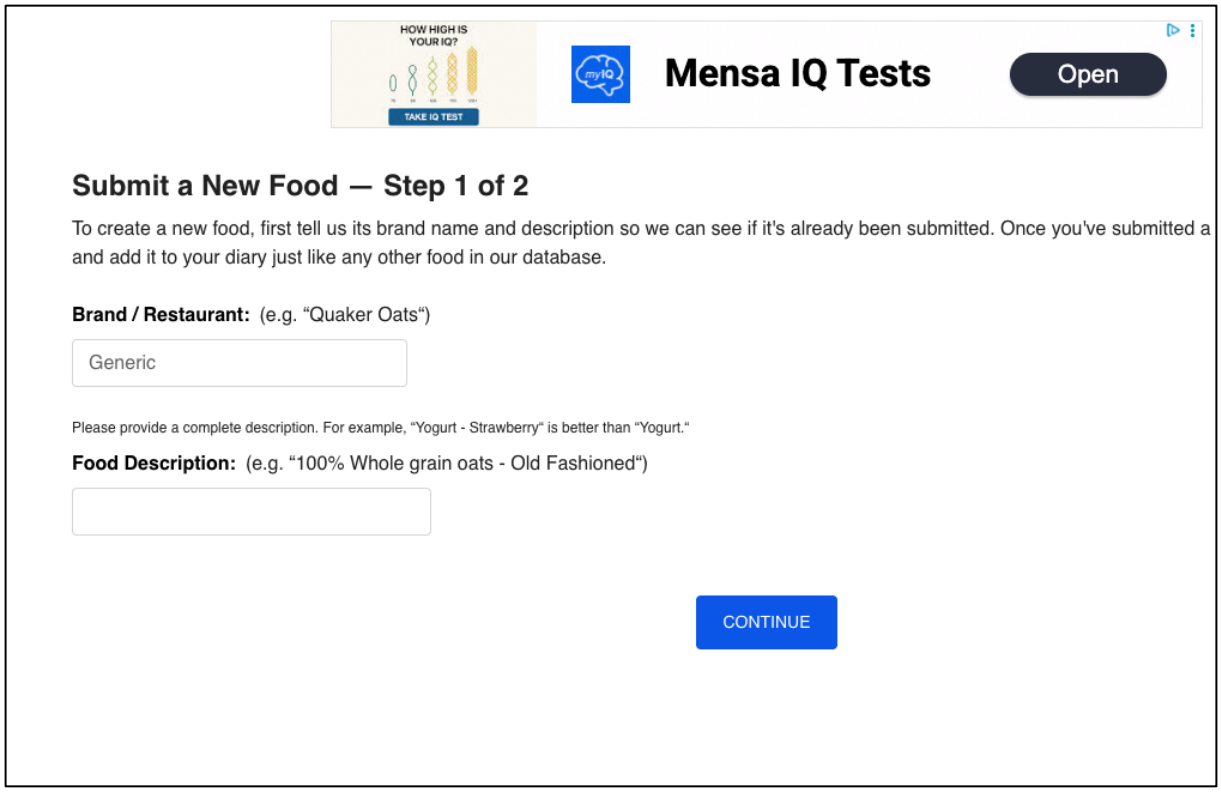


- The layout of the food logging and workout logging pages feel quite cluttered, with multiple buttons, tabs, adverts, and icons, making it difficult for users, especially for beginners, to understand where to log their meals, how to edit entries, or how to review their daily totals.
- Food logging screens frequently display too much information at once, and the inconsistent placement of features like portion sizes, macros, and meal categories slows down this process.
- The workout logging section is not very intuitive. This lack of UI clarity increases the likelihood of users making errors and ultimately discourages consistent tracking. For a health app that relies on daily engagement, a confusing interface becomes a barrier to long term user satisfaction and retention.

The User Generated Database :

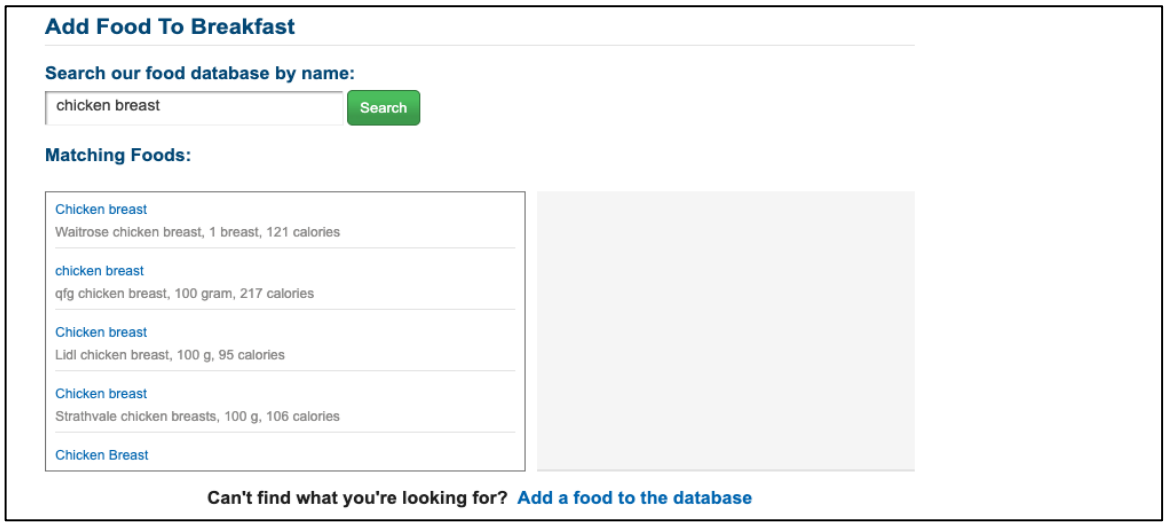
The most significant issue in existing fitness tracking applications is their reliance on their user generated food database. While being marketed as a "feature" offering millions of

entries, this approach creates an unreliable system that undermines the entire purpose of calorie tracking.



This means the same food can have multiple conflicting entries with very different calorie and macronutrient values.

Inconsistent and conflicting entries:



For example, searching for "Chicken Breast" on MyFitnessPal returns over 50 different entries ranging from 60 calories to upwards of 250 calories. With very few results being USDA approved. This means that users must guess which entry is correct every time they log food. This problem extends beyond just confusion. Users attempting weight loss on a 500

calorie daily deficit can unknowingly select incorrect entries that add an extra 400-500 or more calories per week, effectively eliminating their deficit and wasting all of their progress.

Health risks for medical users

For users who have medical issues, the consequences are significantly more severe.

Incorrect food data directly impacts dosing calculations, and errors of 30-40% between database entries can lead to serious health consequences.

Database pollution and duplicate entries

Beyond health risks, the database is plagued with duplicate entries.

Once a user submits an incorrect entry, it remains in the database and does not get removed. These errors add up over time as more users submit inaccurate food entries, making the database progressively worse rather than better.

Key Features I will and will not implement :

Table 1: Features i will take from MyFitnessPal

Feature	Justification
Convenient logging	Shows that quick, easy logging is essential for consistent user engagement.
Clean interface	The Cluttered ad filled UI highlights the need for a simple, user friendly design.

Table 2: Features i will not take from MyFitnessPal

Feature	Justification
Advertisements	Ads interrupt logging, clutter the screen, and frustrate users . My program will be ad free.
Scanning meal behind a paywall	Locking essential features behind subscriptions reduces user satisfaction so I will not include a paid feature.
Intermittent fasting tools behind paywall	Restricting useful features damages trust and pushes users to competitors.

Paid subscription model	Over monetisation risks user retention and long term growth of the platform.

Wearable integration feature:

Initially I mentioned that I wanted to implement wearable integration. However upon investigation I discovered that it requires external API access and network connectivity, all of which are beyond the scope of my project. As a result, I decided to instead consider adding a workout feature in my app where users can directly input data from their own wearables and track their workouts on the app. This helps achieve the same benefit of being able to track all of their food and workout data on the same app without having wearable data directly integrated.

Other existing systems (Cronometer,LoseIt,Lifesum,CalAI):

Cronometer

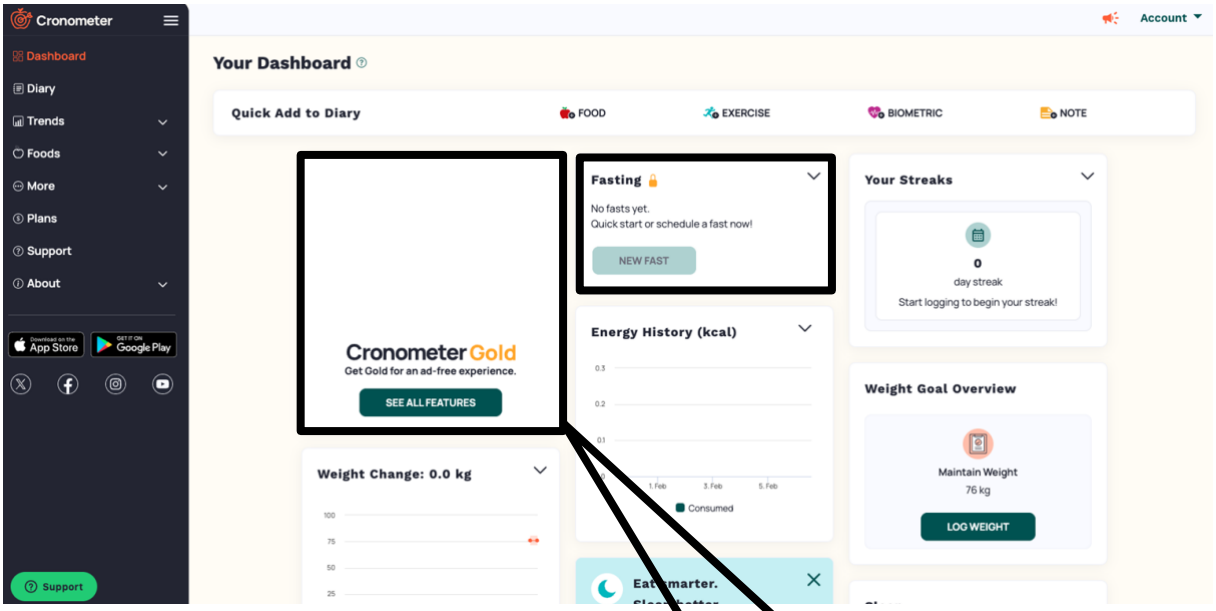
Cronometer is widely known for its scientific accuracy and detailed nutritional tracking.

It offers one of the most precise databases available, with verified USDA approved entries rather than user generated ones, making it one of the better choices for people who need accurate micronutrient data, such as athletes, bodybuilders, or individuals with medical dietary requirements. It also integrates well with wearables like MyFitnessPal and provides robust statistical insights.

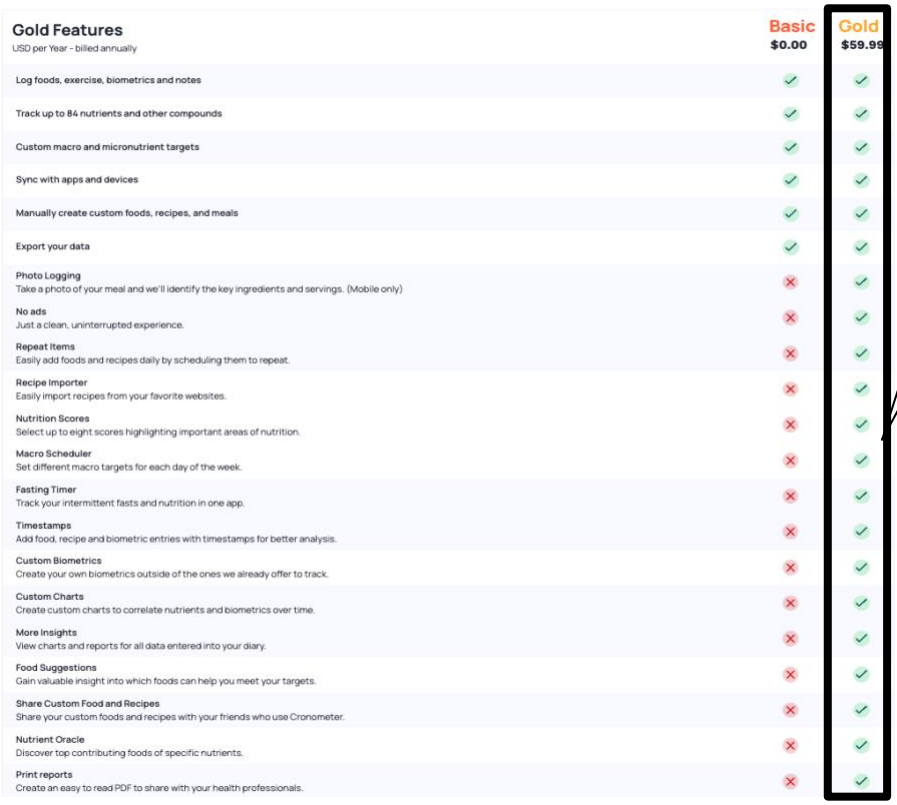
However, Cronometer can feel too technical for casual / new users as there is a lot of nutritional information. The free version is functional, but many advanced features like recipe import, fasting tools, and deeper analytics require a premium subscription.

Cost: Free version and a paid version at £59.99 for an annual subscription

URL: https://cronometer.com/



Upon account creation we are shown that many features important to a fitness tracker are behind a paywall .



List of features that are only accessible to premium users

Overall Pros and Cons:

Verified food database Accurate, reliable nutritional data unlike user generated databases	Overly technical interface. May appear too complex for beginners and casual users
Detailed nutritional tracking Ideal for users with specific dietary or medical needs	Advanced features are paywalled such Recipe import and deeper analytics locked behind subscription
Functional free tier with Core tracking accessible without payment	

Evaluation:

Cronometer is well suited for experienced users and those with specific dietary needs due to its USDA verified database and detailed nutritional tracking. However, its overly technical interface does make it less accessible for beginners or casual users seeking a simple logging experience. Additionally, locking advanced features behind a paywall limits its appeal as a fully free solution.

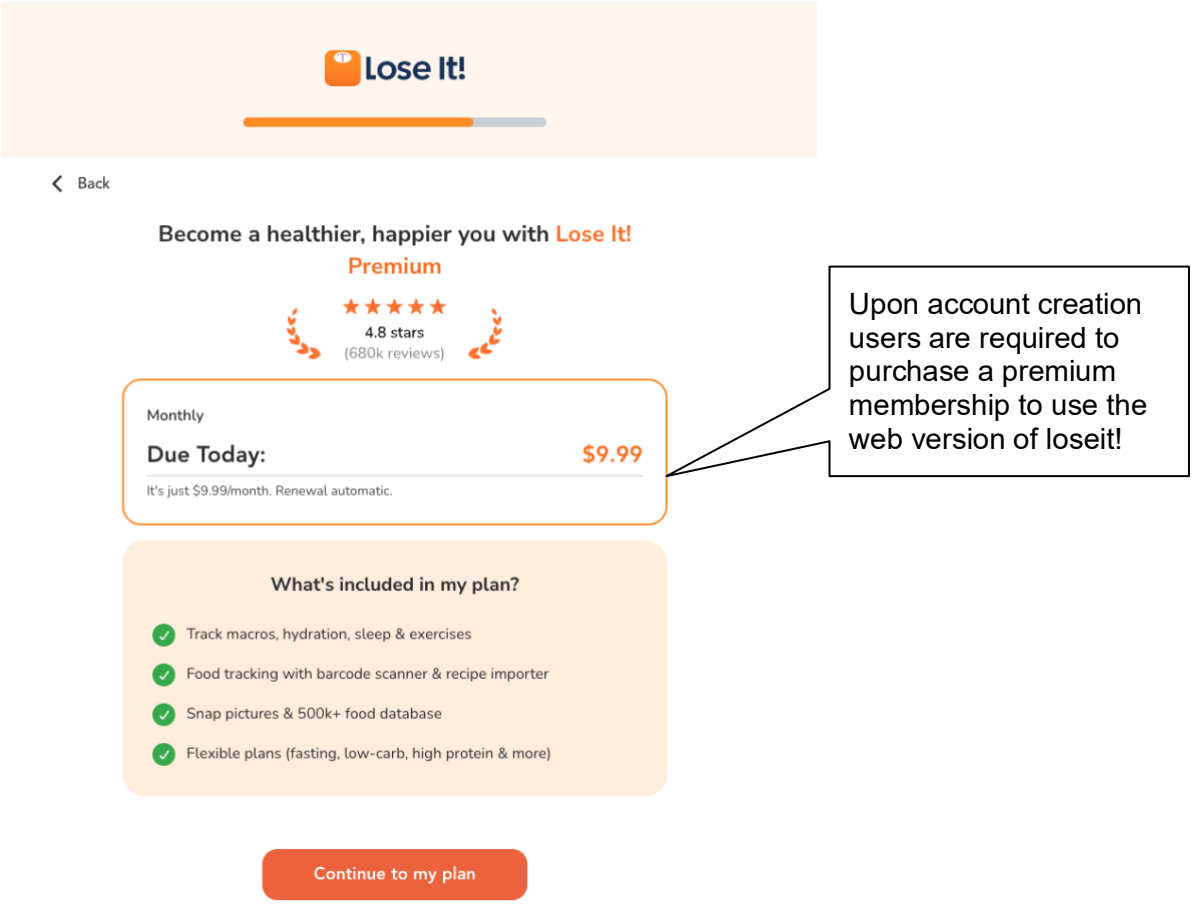
Lose It!

Lose It! Is a simple, approachable, and beginner friendly calorie tracking app. Its design, barcode scanning feature, and simple logging system make it easy for new users to get started. The app also includes features such as challenges, badges, and social features that help keep users motivated.

However, the food database of Lose It! is highly dependent on user inputs and is equally as prone to errors as MyFitnessPal. Despite being a user friendly app, it is not great when it comes to nutritional details. Moreover, all the functionalities of Lose It! are accessible only to premium users via the website.

Cost: Free version for mobile users and a paid version for roughly £9.99 a month (price fluctuates)

URL: https://www.loseit.com/



No free access for desktop users

"Lose It! markets itself as having a free plan online, but the website forces users to select a paid subscription (monthly or annual) with no option to create or access a free account. This deceptive design attempts to tricks users into believing they can use the website for free, only to discover that essential functionality requires payment, undermining trust and creating frustration for budget conscious users who were misled by the marketing.

Overall Pros and Cons:

Simple, beginner friendly design makes the website easy to navigate and encourages new users to engage	User generated food database. Inaccuracies in data reduce reliability and could lead to users not reaching their desired outcome
Fast logging system which reduces friction and encourages consistent use	Lacks nutritional depth. No advanced macro tracking limits usefulness for serious users
Barcode scanner as it speeds up food logging significantly	Premium paywall for web users. All features are locked behind a subscription. This makes the

	program less accessible to people who are looking for a free calorie tracking app

Evaluation

Lose It! is well suited for beginners due to its simple interface and features, however its user generated food database introduces the same inaccuracy issues we had seen in MyFitnessPal. All of the web features are locked behind a premium subscription, reinforcing my decision to keep my program completely free. Its lack of nutritional depth highlights the importance of incorporating advanced macro tracking into my own program.

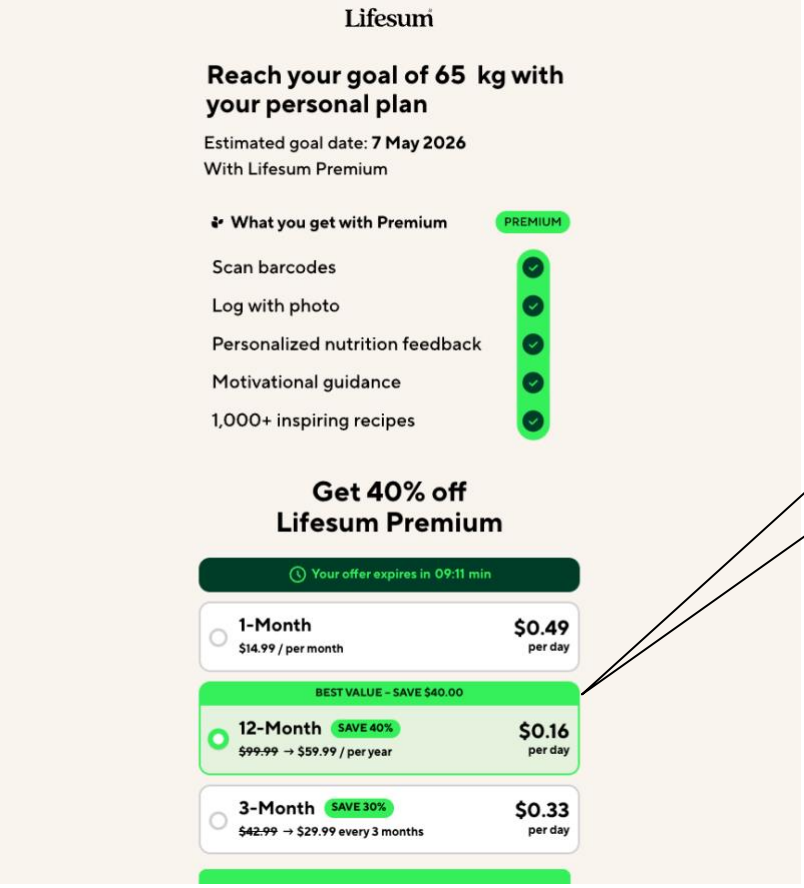
Lifesum

Lifesum stands out with its modern and visually appealing user interface, as well as having a focus not just on calorie counting but also on changing your habits. Lifesum offers personalised diet plans, recipe suggestions, and numerous recipes for healthy eating, thus catering to individuals who prefer more structure and encouragement.

Furthermore, the app also enables you to monitor your fluid consumption, emotions, and general routine. One drawback of Lifesum is that all functions are available only to users who have paid for them (web version). Lifesum also has a significantly smaller food database than the other competitors mentioned.

Cost: Free version and a paid version for roughly £59.99-£99.99 a year

URL:https://www.loseit.com/



Upon account creation, users are greeted with a paywall that can not be passed

Overall Pros and Cons:

Visually engaging interface motivates inexperienced and elderly users	No free tier for web users makes it inaccessible for budget conscious users on a desktop
Structured plans provide clear direction for users new to fitness tracking	Smaller food database, reducing convenience and accuracy
Habit Tracking. Water intake and mood tracking encourages broader healthy lifestyle changes	Focuses on aesthetics rather than precise nutritional data

Evaluation

Lifesum's modern design and guided plans make it particularly motivating for inexperienced and elderly users who need structure and visual encouragement rather than straight data. However, its entirely paywalled model makes it inaccessible to users seeking a free solution on a desktop.

Cal Ai

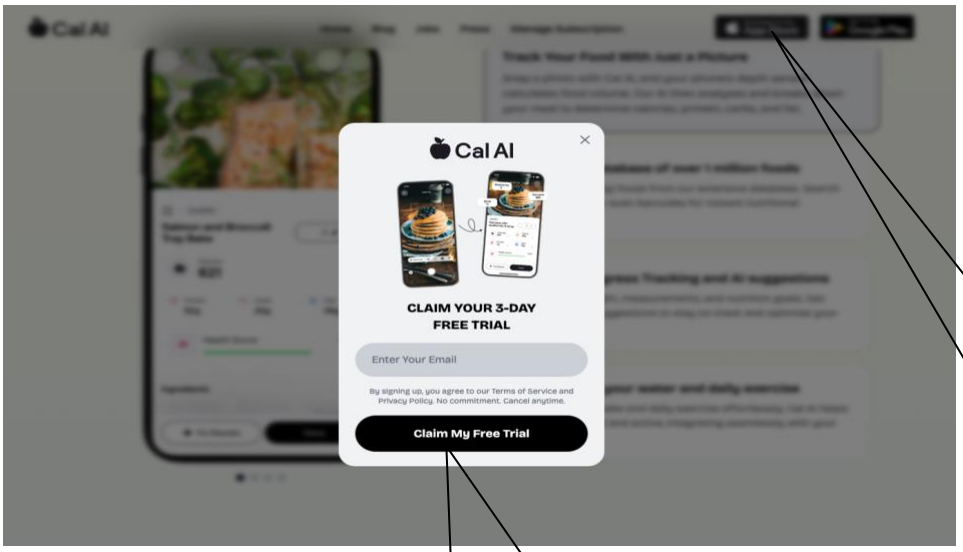
Calorie AI introduces an interesting and new concept by using artificial intelligence to automatically identify foods from photos and estimate their nutritional content.

In theory, this approach removes the need for manual logging and could significantly reduce the time taken to track food, keep users more consistent and overall improve the user experience.

However, in practice, the AI based tracking is highly inaccurate, often misidentifying foods, guessing portion sizes incorrectly, or failing to recognise mixed meals all together. This leads to unreliable calorie and macro estimates, which defeats the purpose of a tracking app that relies on precision. The inconsistency becomes even more problematic when users depend on accurate data for their weight management or medical dietary needs.

Cost: No free version, paid version for roughly £10 a month

URL:https://www.loseit.com/



Mobile device is required (due to the need for a camera so the AI can examine food)

All features require a paid subscription

Overall Pros and Cons:

AI food recognition removes need for manual logging, speeding up the tracking process	Inaccurate food identification. Frequently misidentifies foods leading to unreliable nutritional data
Clean and usable UI for users who want to track their food quickly	Unreliable for medical users as inconsistent data is dangerous for users tracking for health conditions such as diabetes.

Evaluation

Calorie AI's photo based logging is a new concept that could simplify the user experience, however its poor accuracy makes it unsuitable for users who rely on precise data for weight management or medical needs. This reinforces my decision to use manual entry with a verified database, prioritising accuracy over convenience.

Overall existing systems analysis :

After analysing MyFitnessPal, Cronometer, Lose It!, Lifesum, and Calorie AI, I was able to clearly identify which features I will adopt in my own Health and Fitness tracking system and which ones I will avoid.

Qualities i will be take inspiration from :

The review highlighted the value of :

- clean interfaces
- accurate databases,
- simple logging tools, and helpful progress tracking features.

All of which contribute to a smooth user experience.

Specifically, I will adopt :

- Meal specific logging from MyFitnessPal (separating breakfast, lunch, and dinner for clear organisation).
- verified database approach from Cronometer (ensuring all nutritional data is USDA sourced for accuracy).
- workout integration from MyFitnessPal (combining calorie intake and exercise tracking in one system).

- visual design principles from Lifesum (using clear layouts and progress graphs to maintain user engagement).

Additionally, I will implement offline functionality which all competitors lack, ensuring users can track their health data without internet dependency or cloud storage vulnerabilities.

Qualities i will not be take inspiration from :

The primary drawbacks across these platforms are qualities such as :

- aggressive paywalls,
- intrusive adverts,
- inaccurate AI food recognition,
- cluttered layouts, and
- inconsistent data quality.

This showed me exactly what not to implement. This comparison made it clear that my system should prioritise accuracy, transparency, and ease of use, without relying on unreliable AI tracking or locking essential features behind a subscription. Another feature I will not take inspiration from is:

- having a user generated database

This leads to inconsistent and conflicting entries with calorie variations for identical foods, creates health risks for diabetic users who require precise carbohydrate data for insulin dosing, and makes the overall program less user friendly and intimidating to people who are new to the system by forcing them to guess which of 50+ duplicate entries is correct.

Furthermore, I will avoid deceptive free tier marketing like Lose It! and Lifesum, where apps advertise free plans but force users into paid subscriptions through website registration blocks and feature paywalls.

These websites all leverage pricing psychology to try and get users to upgrade to a premium subscription by carefully shaping the experience of the free tier. This is done by providing enough functionality to build a daily habit, but holding advanced features like detailed macro tracking, custom goals, and ad free navigation to create a sense of missing out.I will not be charging for my program as this concept of charging users for basic features undermines user trust and creates accessibility barriers for individuals managing health conditions on limited budgets.

Stakeholder Analysis

Before any system can be designed or built, it is essential that I identify and understand the people who will use or be affected by it.

By clearly defining who our stakeholders are, I can ensure the Health and Fitness Tracker is built to solve real problems users currently face rather than making assumptions about what users might need.

I have identified four key stakeholder groups for the proposed Health and Fitness Tracker application.

For each group i will describe

- Who they are
- Why do they require a fitness tracking solution
- Why is it appropriate to their needs

Following the stakeholder profiles, a set of interview questions has been prepared so that I can gather an insight into stakeholders needs before any development begins.

Stakeholder 1 : General Health Conscious Users

Who they are: Individuals from the age of 16 to 45 who are trying to improve their health and fitness, whether that's by trying to lose weight, building muscle or trying to eat more mindfully.

Why do they need a fitness tracker?

- **Calorie awareness:** Without tracking, most people significantly underestimate or overestimate their daily calorie intake. A fitness tracker will help provide a place to track calories ,provide accurate caloric data on foods and help users be more mindful of their food consumption.
- **Progress motivation:** Seeing consistent data logged over days and weeks will motivate users to stick to healthy habits as they can visualise their progress.

Why the solution is appropriate to their needs: Having a fitness tracker will allow them to log every meal they eat and see how many calories they have consumed and how many calories they have remaining for the day. Without this there is no reliable and efficient way to keep track of their food consumption and exercise.

Stakeholder 2 : Diabetic Patients

Who they are: Individuals with type 1 or type 2 diabetes who need to carefully monitor their carbohydrate and sugar intake as part of their daily life.

Why do they need a fitness tracker?

- **Precise carbohydrate tracking:** A tracker with a verified, accurate food database will provide precise macronutrient tracking. This will remove guessing, which could otherwise cause dangerous blood sugar fluctuations.
- **Meal by meal breakdown:** Knowing the carbohydrates in a single meal (not just a daily total) is critical for calculating how much insulin to take before eating. Meal specific logging is therefore an essential feature.
- **Database reliability:** User submitted databases like MyFitnessPal's contain inconsistent entries for the same food. For a diabetic person, this inconsistency is a genuine health risk, not just an inconvenience as it may be to other stakeholders.

Why the solution is appropriate to their needs: Individuals with diabetes or other medical conditions will be able to quickly track their food/meals and be able to see the exact carbohydrate content of their meals to be able to use their medications accordingly.

Stakeholder 3 : Fitness Coaches / Personal Trainers

Who they are: fitness coaches/personal trainers who work with multiple clients and need to monitor both their clients food intake and training to give informed guidance.

Why do they need a Fitness Tracker?

- **Nutrition training alignment:** A coach needs to be able to see what their client is eating and how much they are working out in one place. Without this, it would be significantly more difficult to determine whether poor results are due to under/over eating or insufficient training.
- **Reliable client data:** Coaches cannot give accurate advice based on rough estimates. A tracker with a verified database ensures the calorie and macro data they review is from a trustworthy source.
- **Personalised calorie targets:** Each client has different body metrics, activity levels, and goals. Automatic TDEE and BMR calculation means each client gets a personalised target rather than a generic figure.

Why the solution is appropriate to their needs: Having a fitness tracker means a coach can see exactly what a client has consumed and what exercise they have completed in a single application, rather than relying on their client's memory. For example, if a client is not losing weight despite claiming to eat well, the coach can see their meal logs and immediately see whether they are hitting their calorie target or if they are consuming more .

Stakeholder 4 : Elderly Users (65+)

Who they are: People aged 65+ who are looking to maintain a healthy weight, manage age related health conditions and monitor their diet as part of an effort to preserve their quality of life.

Why do they need a Fitness Tracker?

- **Managing age related health conditions:** Older people are more affected by conditions such as obesity, type 2 diabetes, high blood pressure, and osteoporosis, all of which are directly influenced by diet and activity levels. A tracker gives them a tool to support the dietary changes that their doctors may recommend to them.
- **Preventing muscle loss:** After the age of 60, the body loses muscle mass at a much faster rate than usual unless protein intake and physical activity are maintained. Lots of elderly users are not aware of how much protein they are consuming, making macro tracking particularly valuable.
- **Simplicity and accessibility:** Elderly users can often feel intimidated by cluttered interfaces. A clean, minimal design with large buttons and clear labels is not just a preference for this group but more of an accessibility requirement.

Why the solution is appropriate to their needs: Having a fitness tracker is useful for elderly people, as many doctors recommend that elderly users increase their protein intake to prevent muscle loss and prevent the weakening of bones. They can log their meals and see how many grams of protein they have consumed that day and how much more they still need.

Interview Questions :

These questions will be asked to all four stakeholder groups. The aim is to gather broad insight into user needs, current habits, and expectations before any development begins.

1. Do you currently keep track of what you eat or how active you are? If so, how do you do it?

2. What is your main health or fitness goal right now?

3. Have you ever used a health or fitness tracking app before? If so, which one, and what did you think of it?

4. What is the most frustrating or difficult part of trying to stay healthy or hit your fitness goals?

5. How important is it to you that an app like this is completely free to use, with no subscription or hidden charges?

6. How easy or difficult do you find it to use apps and software on a computer?

7. How important is it that the calorie and nutritional information in the app is accurate and comes from a verified, reliable source rather than being submitted by other users?

8. How would you want the app to show your progress? Would numbers and totals be sufficient, or would you prefer visual charts and graphs?

9. When it comes to the layout of an app, what makes an interface feel easy to use for you or would you prefer a more sophisticated technical user interface?

10. How important is it to you that the app also tracks your workouts and exercise, not just your food intake? What details would you want to log, such as the type of exercise, how long you did it, or calories burned?

Justification of Questions:

1. This will give me an idea of whether my stakeholders already have any system in place for monitoring their health, helping me understand the baseline I am designing for.

2. This will show me what each stakeholder is actually trying to achieve, ensuring the features I prioritise directly reflect the real needs of each group rather than making assumptions.

3. This will tell me whether stakeholders have prior experience with existing apps and what frustrated them, helping me decide what to include or avoid in my own system.

4. Identifying stakeholders' biggest frustrations helps me justify why a computational solution is needed and what it must address.

5. This will show how strongly stakeholders feel about cost as a barrier. If the majority find it important, this validates my decision to make the application completely free with no paywalled features.

6. This will give me an understanding of the technical confidence of each group, directly influencing how simple the interface needs to be, particularly for elderly users with limited software experience.

7. This will show how much stakeholders value data accuracy. Particularly for diabetic patients who rely on precise carbohydrate values, this justifies my decision to use a verified only food database rather than user submitted entries.

8. This will inform whether stakeholders prefer visual graphs or plain numerical summaries.

9. This will give me direct insight into what makes an interface feel accessible and usable across all groups, informing key design decisions such as how many options to display on each page and how many steps each action should require.

10. This will show how important workout tracking is to each stakeholder group and what level of detail they expect to log, justifying the inclusion of the workout logging feature and which fields are worth capturing.

Interviews

Now that I have identified the stakeholders of my program, I will be conducting interviews on all 4 of the key stakeholder groups. After the interview, I will create a list of features I will implement.

Interview 1

Stakeholder group: General Health Conscious User

Age: 19

Name: Liam Carter

Interview transcript:

Q1: Do you currently keep track of what you eat or how active you are? If so, how do you do it?

Liam Carter - “Not really, I do try and eat reasonably and go to the gym a few times a week but I don’t have a real system. I sometimes take a rough mental note of what I have eaten but I never write any of it down.”

Q2: What is your main health or fitness goal right now?

Liam Carter - “I want to lose about 8 kg before the summer. I have been trying for a few months but I am not really seeing much progress and I think it is because I have no idea how many calories I am actually eating.”

Q3: Have you ever used a health or fitness tracking app before? If so, which one and what did you think of it?

Liam Carter - “I tried MyFitnessPal for about two weeks. It had loads of features but honestly it felt overwhelming. There were ads everywhere and half the things I wanted to do were locked unless I paid. I deleted it.”

Q4: What is the most frustrating or difficult part of trying to stay healthy or hit your fitness goals?

Liam Carter - “Not knowing whether what I am doing is actually working. I could be going to the gym three times a week and eating what I think is healthy but still not losing weight. Without tracking I have no idea where I am going wrong.”

Q5: How important is it to you that an app like this is completely free to use, with no subscription or hidden charges?

Liam Carter - “Very important. I am a student so I am not paying monthly for a fitness app. If it asks me to pay I will just stop using it.”

Q6: How easy or difficult do you find it to use apps and software on a computer?

Liam Carter - “Pretty confident. I use a laptop every day for university so I am comfortable with most software.”

Q7: How important is it that the calorie and nutritional information is accurate and comes from a verified source rather than being submitted by other users?

Liam Carter - “Not so important. When I go to log food I don’t care too much I just want to know roughly how much I am consuming ,as long as the food is accurate enough then its fine for me.”

Q8: How would you want the app to show your progress ,would numbers and totals be sufficient, or would you prefer visual charts and graphs?

Liam Carter - “Definitely graphs. Numbers alone do not mean much to me. If I can see a bar chart showing my calorie intake going down over the week that is far more motivating than just reading a number.”

Q9: When it comes to the layout of an app, what makes an interface feel easy to use for you?

Liam Carter - “Keep it simple. I do not want ten buttons on the same screen. Just show me what I need ,log food, log workout, see my stats. That is it. Clean and minimal.”

Q10: How important is it that the app also tracks your workouts, and what details would you want to log?

Liam Carter - “Very important. Tracking food without tracking exercise is only half the picture. I would want to log the type of workout, how long I did it, and roughly how many calories I burned so I can see my full daily balance.

Features I will consider:

- Clean, minimal interface with only essential buttons on each page
- Bar charts and graphs to visualise calorie intake over time
- Integrated workout tracking with type, duration, and calories burned
- Verified food database with no user submitted entries
- Fully free platform with no subscription or advertisements

Interview 2

Stakeholder group: Diabetic Patient

Age: 47

Name: Susan Malik

Interview transcript:

Q1: Do you currently keep track of what you eat or how active you are? If so, how do you do it?

Susan Malik – “Yes, I have to. I write everything down in a notebook , the food, rough carbohydrate amounts, and when I ate. It takes a long time and I am not always confident the values I am writing down are correct.”

Q2: What is your main health or fitness goal right now?

Susan Malik – “Managing my blood sugar levels. My doctor has told me I need to reduce my carbohydrate intake and lose some weight. For me tracking food is not optional, it is part of managing my condition.”

Q3: Have you ever used a health or fitness tracking app before? If so, which one and what did you think of it?

Susan Malik – “I tried MyFitnessPal briefly but the food database was unreliable. I searched for the same food on different days and got completely different carbohydrate values. When you are diabetic that is not just annoying, it is dangerous. I went back to my notebook.”

Q4: What is the most frustrating or difficult part of trying to stay healthy or hit your fitness goals?

Susan Malik – “The inaccuracy of food data available to me. I need to know exactly how many grams of carbohydrates are in what I am eating before each meal. If that number is wrong my insulin dose could be wrong, and that has serious consequences.”

Q5: How important is it to you that an app like this is completely free to use, with no subscription or hidden charges?

Susan Malik – “Very important. I already spend a significant amount on medication and medical appointments. I should not have to pay to track my own food.”

Q6: How easy or difficult do you find it to use apps and software on a computer?

Susan Malik – “Reasonably comfortable. I use a computer at work so I am familiar with the basics, though I prefer things to be straightforward with no unnecessary complexity.”

Q7: How important is it that the calorie and nutritional information is accurate and comes from a verified source rather than being submitted by other users?

Susan Malik – “Extremely important.This is my biggest requirement. The data needs to be from a trustworthy source. A wrong carbohydrate figure is not just an inconvenience for me, it directly affects my health.”

Q8: How would you want the app to show your progress ,would numbers and totals be sufficient, or would you prefer visual charts and graphs?

Susan Malik – “Both would be ideal. I need to see the exact numbers for carbohydrates per meal, but a weekly graph showing my overall intake trends would help me and my doctor spot patterns.”

Q9: When it comes to the layout of an app, what makes an interface feel easy to use for you?

Susan Malik – “I want to be able to open it and immediately see what I have eaten today and the carbohydrate totals per meal without having to search through menus. Simple, direct, and uncluttered.”

Q10: How important is it that the app also tracks your workouts, and what details would you want to log?

Susan Malik – “Quite important. Exercise affects my blood sugar too so knowing I have been active helps me understand my readings. I would want to log the type of activity and duration at minimum.”

Features I will consider:

- Verified food database sourced from a reliable authority such as the USDA
- Meal specific carbohydrate and macro display (per meal, not just daily total)
- Weekly graphs showing carbohydrate and calorie trends
- Clean layout with immediate visibility of today's totals on opening
- Workout logging including activity type and duration

Interview 3

Stakeholder group: Fitness Coach / Personal Trainer

Age: 31

Name: Jordan Phillips

Interview transcript:

Q1: Do you currently keep track of what you eat or how active you are? If so, how do you do it?

Jordan Phillips – “Personally, yes, I use a spreadsheet. For my clients it is much harder ,most of them send me WhatsApp messages with photos of what they ate or just describe their day verbally. It is completely inconsistent and hard to analyse.”

Q2: What is your main health or fitness goal right now?

Jordan Phillips – “Professionally, my goal is to get my clients results. That means making sure their nutrition supports their training. Right now I cannot do that properly because the data I get from them is unreliable.”

Q3: Have you ever used a health or fitness tracking app before? If so, which one and what did you think of it?

Jordan Phillips – “Yes, I have used MyFitnessPal and recommended it to clients. The database is huge which is useful but the user submitted entries are a real problem. I have had clients logging wildly inaccurate values without realising it, which makes my job harder.”

Q4: What is the most frustrating or difficult part of trying to stay healthy or hit your fitness goals?

Jordan Phillips – “Getting clients to track consistently. If the app is complicated or requires too many steps to log a meal, they stop doing it after a few days. The simpler the process, the more likely they are to stick with it.”

Q5: How important is it to you that an app like this is completely free to use, with no subscription or hidden charges?

Jordan Phillips – “Essential. I cannot ask 15 clients to all pay for a subscription. If it is free they will all use it. If it costs money half of them will not bother.”

Q6: How easy or difficult do you find it to use apps and software on a computer?

Jordan Phillips – “Very confident. I use software daily for programming workouts and tracking client progress.”

Q7: How important is it that the calorie and nutritional information is accurate and comes from a verified source rather than being submitted by other users?

Jordan Phillips – “Very important. I base my dietary recommendations on the data clients give me. If that data is inaccurate because of bad database entries, my advice is worthless. Verified data is non-negotiable for me professionally.”

Q8: How would you want the app to show your progress ,would numbers and totals be sufficient, or would you prefer visual charts and graphs?

Jordan Phillips – “Graphs without question. When I sit down with a client I want to pull up a bar chart of their calorie intake over the last week and immediately see where they went wrong. That is a much more powerful conversation than showing them a spreadsheet of numbers.”

Q9: When it comes to the layout of an app, what makes an interface feel easy to use for you?

Jordan Phillips – “Minimal and task focused. The app should do one thing at a time. Log food, log workout, view stats. Each on its own clean page with no distractions. The more cluttered it is the less my clients will use it.”

Q10: How important is it that the app also tracks your workouts, and what details would you want to log?

Jordan Phillips – “Extremely important. Nutrition and training are two sides of the same coin. I need to see both together. The key details are workout type, duration, and calories burned so I can check whether a client is in a calorie deficit or surplus on any given day.”

Features I will consider:

- Integrated workout tracking showing type, duration, and calories burned
- Weekly bar charts for both calorie intake and workout duration
- Minimal, task focused interface with no clutter or unnecessary buttons
- Verified food database to ensure data my clients log is trustworthy
- Separate pages for each main function: food, workout, stats and settings

Interview 4

Stakeholder group: Elderly Users

Age: 68

Name: Margaret Collins

Interview transcript:

Q1: Do you currently keep track of what you eat or how active you are? If so, how do you do it?

Margaret Collins – “Not formally, no. I try to eat sensibly and I go for a walk most days but I have never written any of it down. My doctor has been telling me I should keep more of an eye on what I eat since I was diagnosed with high blood pressure.”

Q2: What is your main health or fitness goal right now?

Margaret Collins – “To lose a bit of weight and eat more healthily. Nothing extreme, I just want to feel better and take some pressure off my heart. My doctor mentioned that even losing a few kilograms would make a real difference.”

Q3: Have you ever used a health or fitness tracking app before? If so, which one and what did you think of it?

Margaret Collins – “No, I have never used one. I would not really know where to start with something like that. I have a laptop that I use for emails and shopping but I have not really used apps beyond that.

Q4: What is the most frustrating or difficult part of trying to stay healthy or hit your fitness goals?

Margaret Collins – “Not knowing whether what I am doing is making any difference. I could be eating what I think is healthy and walking every day but with no way of measuring it I genuinely do not know if I am making progress.”

Q5: How important is it to you that an app like this is completely free to use, with no subscription or hidden charges?

Margaret Collins – “Very important. I am on a pension so I would not want to be paying monthly for something like this. If it were free I would be much more likely to give it a proper try.”

Q6: How easy or difficult do you find it to use apps and software on a computer?

Margaret Collins – “I would say I am a beginner. I can manage emails and simple websites but anything with too many menus or small buttons gets confusing quite quickly. I need things to be clearly labelled and straightforward.”

Q7: How important is it that the calorie and nutritional information is accurate and comes from a verified source rather than being submitted by other users?

Margaret Collins – “Very important. If I am using it to help with a health condition I need to be able to trust what it is telling me. I would not want to be guessing whether the information is correct.”

Q8: How would you want the app to show your progress , would numbers and totals be sufficient, or would you prefer visual charts and graphs?

Margaret Collins – “I think I would prefer graphs. A simple chart showing whether I am eating more or less than I should each day would be much easier for me to understand than a page of numbers.”

Q9: When it comes to the layout of an app, what makes an interface feel easy to use for you?

Margaret Collins – “Big clear buttons, not too many options at once, and obvious labels on everything. I get put off immediately if I open something and there are twenty different things on the screen. I would want it to be as simple as possible , just the basics I actually need.”

Q10: How important is it that the app also tracks your workouts, and what details would you want to log?

Margaret Collins – “Reasonably important. I would not call what I do workouts exactly, but being able to log a walk or some light exercise and see it recorded would be motivating. Just the type of activity and how long I did it would be more than enough for me.”

Features I will consider:

- Very clean and minimal interface with large, clearly labelled buttons
- Simple graphs to visualise daily and weekly progress without overwhelming detail

- Ability to log light activity such as walking with just type and duration
- Fully free with no subscription so it is accessible to all users

Conclusion:

Now that all interviews have been completed i will be analysing the features suggested by the stakeholders and make a conclusive list of features that i will need to implement.

What I Have Taken Away from the Interviews :

Each feature below is directly justified by evidence gathered from the interviews.

1. Clean, minimal Interface with only essential buttons

Every single stakeholder, regardless of age or background, indicated that a simple design was important to them.

- Liam described wanting "keep it simple. I do not want ten buttons on the same screen. Just show me what I need, log food, log workout, see my stats. That is it. Clean and minimal."
- Jordan stated "the app should do one thing at a time. Log food, log workout, view stats. Each on its own clean page with no distractions. The more cluttered it is the less my clients will use it."
- Susan stated "I want to be able to open it and immediately see what I have eaten today and the carbohydrate totals per meal without having to search through menus. Simple, direct, and uncluttered."
- Margaret stated "big clear buttons, not too many options at once, and obvious labels on everything. I get put off immediately if I open something and there are twenty different things on the screen."

This was the most consistently raised point across all four interviews and therefore the most important design requirement. I will implement a minimal interface where each page focuses on a single task, with no advertisements, unnecessary buttons, or decorative elements that could distract or confuse users.

2. Integrated workout tracking (Type, Duration, Calories Burned)

All four stakeholders indicated that workout tracking was either very important or important to them.

- Liam stated "tracking food without tracking exercise is only half the picture. I would want to log the type of workout, how long I did it, and roughly how many calories I burned so I can see my full daily balance."
- Jordan described it as "extremely important" and stated "nutrition and training are two sides of the same coin. I need to see both together. The key details are workout type, duration, and calories burned so I can check whether a client is in a calorie deficit or surplus on any given day."

- Susan said "exercise affects my blood sugar too so knowing I have been active helps me understand my readings. I would want to log the type of activity and duration at minimum."
- Margaret said "being able to log a walk or some light exercise and see it recorded would be motivating. Just the type of activity and how long I did it would be more than enough for me."

I will implement a dedicated workout logging page where users can record the type of exercise, duration in minutes, and estimated calories burned, with a daily summary showing total workouts, total duration, and total calories burned.

3. Visual graphs and charts to display progress

Every stakeholder expressed a preference for visual graphs over plain numbers when viewing their progress.

- Liam stated "if I can see a bar chart showing my calorie intake going down over the week that is far more motivating than just reading a number."
- Jordan described wanting "when I sit down with a client I want to pull up a bar chart of their calorie intake over the last week and immediately see where they went wrong. That is a much more powerful conversation than showing them a spreadsheet of numbers."
- Susan stated "a weekly graph showing my overall intake trends would help me and my doctor spot patterns."
- Margaret said she felt "a simple chart showing whether I am eating more or less than I should each day would be much easier for me to understand than a page of numbers."

I will implement three graphs: a bar chart showing calorie intake over the last 7 days, a bar chart showing workout duration over the last 7 days, and a pie chart showing today's macronutrient breakdown.

4. Verified food database with no user submitted entries

Data accuracy was raised as a concern by 3 stakeholders.

- Jordan stated that verified data is "non-negotiable" professionally and that "if that data is inaccurate because of bad database entries, my advice is worthless."
- Susan said that "a wrong carbohydrate figure is not just an inconvenience for me, it directly affects my health", making this a genuine medical requirement rather than a preference.
- Margaret said "if I am using it to help with a health condition I need to be able to trust what it is telling me. I would not want to be guessing whether the information is correct."

I will implement a verified only food database where no user can submit or modify entries, ensuring every nutritional value is accurate and consistent.

5. Fully Free Platform with No Subscription or Advertisements

Cost and advertisements were flagged as significant barriers by every stakeholder.

- Liam stated "I am a student so I am not paying monthly for a fitness app. If it asks me to pay I will just stop using it."
- Jordan said "I cannot ask 15 clients to all pay for a subscription. If it is free they will all use it. If it costs money half of them will not bother."
- Susan said "I already spend a significant amount on medication and medical appointments. I should not have to pay to track my own food."
- Margaret said "I am on a pension so I would not want to be paying monthly for something like this. If it were free I would be much more likely to give it a proper try."

I will implement the application as a completely free platform with no subscription tiers, no paywalled features, and no advertisements anywhere in the interface.

6. Personalised daily calorie target calculated automatically

All four stakeholders indicated that personalised calorie targets were important to them.

- Liam stated "I have been trying for a few months but I am not really seeing results and I think it is because I have no idea how many calories I am actually eating."
- Jordan stated "each client has different body metrics, activity levels, and goals. Automatic TDEE and BMR calculation means each client gets a scientifically grounded target rather than a generic figure."
- Susan stated "my doctor has told me I need to reduce my carbohydrate intake and lose some weight", meaning an automatic calculation based on her current weight and activity level would provide a reliable and medically relevant baseline.
- Margaret stated "I would not want to have to work out any numbers myself. If the app could just tell me what I should be eating based on my details that would be much easier for me."

I will implement the Mifflin-St Jeor equation combined with activity level multipliers to automatically calculate each user's BMR and TDEE upon registration, producing a personalised daily calorie target that updates whenever their weight changes.

7. Meal Specific Logging (Breakfast, Lunch, Dinner)

- Susan specifically highlighted "I need to see the exact numbers for carbohydrates per meal" as this directly informs her insulin dosing before each meal.
- Jordan also stated "I need to see both together. The key details are workout type, duration, and calories burned so I can check whether a client is in a calorie deficit or surplus on any given day", indicating that meal-specific breakdown is essential for professional client analysis.

I will implement three separate meal categories, breakfast, lunch, and dinner each with its own display listbox, so users can see exactly what was consumed at each point in the day alongside the associated nutritional values.

Summary Table (Features i will add to meet the stakeholders needs):

After analysing the interviews with the stakeholders i have created a summary list of all the features i will be implementing based on the interviews:

Feature	Stakeholders Who Raised It	Priority
Clean, minimal interface	All four	Essential
Integrated workout tracking	All four	Essential
Visual graphs and charts	All four	Essential
Verified food database	Susan, Jordan and Margaret	High
Fully free, no subscription	All four	Essential
Personalised calorie target	All four	Essential
Meal specific logging	Susan and Jordan	High

Features i will implement:

1. SHA-256 Password Hashing

- **What it does:** Passwords are converted into a hash value before being stored in the database.
- **Why I will include it:** This is for basic security. If someone accesses the database file, they cannot see users' actual passwords. MyFitnessPal has had security breaches in the past, so implementing proper password security from the start will protect users accounts.
- **Stakeholders who will benefit**: All users (account security)

2. Personalised Calorie Calculation

- **What it does:** Calculates daily calorie target based on weight, height, age, gender, activity level, goal weight, and timeframe using the Mifflin-St Jeor equation.
- **Why I will include it:** Standard feature across all platforms. Calculations provide personalised targets that adapt to each user's specific body and goals.
- **Stakeholders who will benefit**: All users (accurate tracking)

3. Verified food database (No user submissions, all foods being USDA verified)

- **What it does**: Users cannot add foods to the database. Only verified entries exist.
- **Why I will include it:** To limit unreliable data. By stopping user submissions like Cronometer does, I can ensure every entry is for the most part accurate. The trade off is a smaller database, but all the entries will be reliable.
- **Stakeholders who will benefit**:
  Diabetic patients (accurate carb data for insulin dosing)
  Nutritionists (trustworthy data for client analysis)
  General users (no guessing which entry is correct)

4. Free platform - All features accessible

- **What it does**: No premium subscription, no paid only features, no advertisements.
- **Why I will include it:** MyFitnessPal, Lose It!, and Lifesum all lock essential features behind £8-10/month subscriptions. A free app makes health tracking accessible to all users.
- **Stakeholders who will benefit**:
  Diabetic patients (need macro tracking for medical reasons)
  General users on limited budgets
  Fitness coaches (clients can access full features without paying)

5. Minimalistic user interface

- **What it does**: Clean, simple layout with clear labels. Each page focuses on one task (Log Food, Log Workout, View Stats, Settings).
- **Why I will include it:** Most platforms GUI feels overwhelming to users with too many things on screen at once. A simple design will reduce stress for new users and create a more user friendly experience.
- **Stakeholders who will benefit**: General users new to calorie tracking and older users

6. Meal specific organisation (Breakfast, Lunch, Dinner)

- **What it does**: Three separate displays, one for each meal type. Users click meal buttons to log food to that meal.

- **Why I will include it:** Inspired by MyFitnessPal's meal categorisation but simplified. Makes it easy to see what was eaten at each meal without scrolling through a mixed list.
- **Stakeholders who will benefit**:
  Diabetic patients (check carbs consumed at specific meals for insulin dosing)
  General users (quickly review daily eating patterns)

7. Undo last item button

- **What it does**: "Undo Last Entry" button removes the most recently logged food item instantly.
- **Why I will include it:** To make the process of removing recently logged foods easier for all users.
- **Stakeholders who will benefit**:
  All users (quick error correction)

8. Real time calorie and macro display

- **What it does**: Shows daily totals for calories, protein, carbs, and fat at the top of the Log Food page. Displays "calories remaining" based on daily goal. Updates automatically when meals are added or removed.
- **Why I will include it:** Most platforms display 20+ micronutrients which overwhelms casual users. I will create a platform that focuses only on the four metrics that matter most (calories, protein, carbs, fat), similar to MyFitnessPal's macro display.
- **Stakeholders who will benefit**:
  Diabetic patients (prominent carb display for insulin dosing)
  General users (see remaining calories to make informed decisions)
  Fitness coaches (check if clients are hitting protein targets)

9. Integrated workout tracking

- **What it does**: Users log workouts with type, duration, and calories burned. Today's workouts are displayed with a summary showing total duration and total calories burned.
- **Why I will include it**: Allows users to see both food intake and exercise output in one app instead of using separate apps .
- **Stakeholders who will benefit:**
  Fitness coaches (view client nutrition and exercise together)
  General users (see if they're in a caloric deficit or surplus)

10. Delete workout functionality

- **What it does:** Users can select a logged workout and delete it if entered incorrectly.
- **Why I will include it:** Standard feature in all competitor apps. Allows users to correct mistakes in workout logging.
- **Stakeholders who will benefit:**
  All users (error correction)

11. Data visualisation with graphs

- **What it does:** Three graphs:
  Calorie intake over the last 7 days (bar chart)
  Workout duration over the last 7 days (bar chart)
  Today's macronutrient breakdown (pie chart)
- **Why I will include it:** MyFitnessPal locks graphs behind a £9.99/month premium membership. Cronometer offers free graphs, but these are overly technical. I will make graphs that provide essential visual progress tracking without payment barriers, using simple chart types that are easy for users to understand.
- **Stakeholders who will benefit:**
  General users (identify eating patterns and consistency)
  Fitness coaches (review client progress visually)
  Nutritionists (see client macro distribution)

12. Password change functionality

- **What it does:** Users can update their password through the Settings page.
- **Why I will include it:** Standard security feature in all apps. If users suspect their account is being targeted, they can change their password without losing logged data.
- **Stakeholders who will benefit:**
  All users (account security)

13. Automatic meal/workout loading

- **What it does:** When users open the Log Food or Log Workout pages, today's meals/workouts automatically load and display current totals.
- **Why I will include it:** Similar to MyFitnessPal's auto refresh but without requiring a manual pull to refresh.
- **Stakeholders who will benefit:** All users (immediate view of daily progress)

14. User statistics display

- **What it does:** The View Stats page shows name, current weight, height, and BMI.
- **Why I will include it:** Standard feature in all of the competitor apps. Provides users with a quick overview of their current body metrics.
- **Stakeholders who will benefit:** General users (track weight and BMI changes) and fitness coaches

Summary: Feature Inspiration from Existing Systems

Inspired by MyFitnessPal:

- Meal specific organisation (Breakfast, Lunch, Dinner)
- Real time macro display
- Automatic meal loading
- Integrated workout tracking

Inspired by Cronometer:

- Verified only database (no user submissions)

Inspired by Lose It!:

- Graph data visualisations

Original Features (Not in Existing Systems):

- Completely free with no ads (all competitors have paywalls or ads)
- Clean, minimalist interface
- Undo last meal entry

Limitations of the Proposed Solution

While the Health & Fitness Tracker will successfully address key weaknesses in existing fitness applications, several limitations do exist due to the scope of the coursework project and design choices that prioritise core functionality over advanced features.

1. Database portability and distribution

- **Limitation:** The file path for the database is hard coded, meaning that it will only work on the computer where it was written. It is not possible to distribute the software elsewhere without making changes to the source code.
- **Impact:** Without any technical skills to make necessary changes to the program code, the stakeholders cannot run the program. This limits the testing of the program with the stakeholders.
- **Why this limitation exists:** Using an absolute file path will simplify the development during the coursework project. For a project demonstrating core functionality, this approach is acceptable.
- **Justification:** The limitation is technical. In a production version, using relative file paths would resolve this issue without changing any of the core functionality.

2. Desktop only application

- **Limitation:** The application will be built using Tkinter, which creates desktop applications only. No mobile version for iOS or Android will be made.
- **Impact:** Users cannot track meals or workouts while away from their computer. This will reduce convenience compared to competitors like MyFitnessPal that have mobile apps, particularly for users who want to log food immediately after eating at restaurants or track workouts at the gym.
- **Why this limitation exists:** Developing mobile applications requires different programming frameworks and is beyond the scope of a coursework project. Tkinter was chosen because it is built into Python, and allows simple development of a desktop interface.
- **Justification:** Desktop applications are appropriate for users who plan their meals in advance or log food at the end of the day at home. The core tracking functionality,

database design, and algorithms will all work identically on desktop, demonstrating all of the required concepts for project.

3. No Barcode Scanning

- **Limitation:** Users must manually type food names to search the database. There is no barcode scanning functionality to get nutritional information from product packaging.
- **Impact:** Logging foods requires typing the food name into the search box, whereas barcode scanning apps allow users to point their phone camera at the barcode.
- **Why this limitation exists:** Implementing barcode scanning requires a camera, barcode reading libraries, and access to barcode databases. This adds significant technical complexity beyond the coursework scope.
- **Justification:** I will use a binary search algorithm to provide fast food lookup for typed searches. The desktop application format also makes camera integration impractical. Barcode scanning is a convenience feature and is not really essential to the core tracking functionality that this project demonstrates.

4. Stack based Item undo is limited to the current session

- **Limitation:** The undo functionality will use a stack, which will be cleared when the application closes. Users can only undo meals logged in the current session, not meals from previous days or earlier sessions.
- **Impact:** If a user logs a meal, closes the app, and then later realises they logged the wrong food, they cannot use the undo button.
- **Why this limitation exists:** The stack was designed for immediate error correction during active logging sessions, not for historical data deletion.
- **Justification:** The primary use case for undo is correcting mistakes immediately after they occur.

5. No recipe Import or meal planning features

- **Limitation:** Users cannot import recipes from websites, create custom recipes by combining multiple ingredients, or generate meal plans. Each food must be logged individually.
- **Impact:** Users who cook homemade meals with many ingredients must search for and log each ingredient separately, which will be time consuming.
- **Why this limitation exists:** This is a complex feature. This feature is complex enough to be a separate software project and is far beyond the scope of this coursework.
- **Justification:** The core tracking functionality allows users to log individual foods, which is the fundamental requirement for calorie counting. Users can still track homemade meals by logging each ingredient, although this may be tedious. Meal planning features are for convenience and are not essential to the problem of tracking actual calorie consumption.

6. No data export function

- **Limitation:** Users cannot export their meal logs or workout history to CSV files or other formats for external analysis or to share with healthcare professionals.
- **Impact:** Users cannot easily share their tracking data with nutritionists, personal trainers, or doctors. This also means they cannot perform analysis in spreadsheet programs or create their own visualisations outside of the application.
- **Why this limitation exists:** Implementing CSV export requires additional file handling and formatting data appropriately for external programs. This feature is deprioritised in favour of core tracking functionality within the project timeframe.
- **Justification:** The application will provide built in graphs for visualising progress and all data will remain accessible through the View Stats page. Export functionality would enhance professional use cases, but it is not required for individual users to benefit from the calorie tracking system.

Overall Justification for Limitations

The application will successfully demonstrate all of the required programming concepts including algorithms, data structures, database normalisation, and user interface design, all while still solving the fundamental problem of providing accurate, accessible calorie and macronutrient tracking. The limitations are mostly related to convenience rather than the core computational solution itself.

Hardware Requirements

Hardware	Justification
Mouse and keyboard	To allow users to navigate the system
50Mb of secondary storage space	The code and the SQL database are unlikely to take up much memory
Monitor	To display the application to the user

Memory (RAM): Minimum: 2GB RAM Recommended: 4GB RAM	Python, Tkinter, and SQLite together will likely use around 50-150Mb at runtime. 2GB provides sufficient headroom for the program and operating system to run concurrently without slowdowns.
Display and Resolution: Minimum: 1024x768 resolution Recommended: 1366x768 or higher	Tkinter GUI windows have fixed minimum dimensions and below this resolution, interface elements may overlap or become truncated, making the application unusable.
1 GHz single core processor	The app performs lightweight operations such as BMR calculations and GUI rendering. Not intensive tasks and do not require a high spec processor.
Optional Hardware: Barcode Scanner or Webcam (for future versions)	If barcode scanning is implemented, a USB barcode scanner or a webcam can be used to read product codes. This enhances user experience by reducing manual data entry.

Software Requirements:

Operating system: Windows 10/11, macOS 15.1+, or Linux	The application will be developed and tested on macOS 15.1 so I can guarantee full compatibility on that platform. Python and all libraries are cross platform so Windows and Linux are supported, however these have not been formally tested as part of this project.

IDE: Visual Studio Code or PyScripter	An IDE is required to open, edit and run the source code. Only needed by me or any future developers .
Python 3.13.2	Version 3.13.2 is the version the application will be developed and tested on.
matplotlib 3.5+	Used to generate visual graphs. The only library that will require separate installation all other libraries are built into Python.

Success Criteria:

I developed a success criteria based on the stakeholders needs identified during the analysis phase and also the features to address the limitations of the existing systems. This will give me measurable criteria to guide me through the development process and I will also use it to evaluate the system that I create.

Objective 1: User Authentication

1.1 - User can create an account with username and password

1.2 - User can login to their account

1.3 - Multiple accounts can be created so that multiple users can access the system

1.4 - Passwords should be run under a hashing algorithm (SHA-256) to securely store user passwords in the database

1.5 - User can change their password through the settings page

Objective 2: Food Logging and Management

2.1 - Users can search for foods in the database

2.2 - Binary search algorithm is implemented for efficient food lookup in sorted databases

2.3 - Users can log foods to specific meals (breakfast, lunch, dinner)

2.4 - Food entries display nutritional information including calories, protein, carbs, and fat

2.5 - Users can undo their last meal entry using stack data structure

2.6 - Today's meals are automatically loaded when the log food page is opened

Objective 3: Calorie and Macro Tracking

3.1 - Daily calorie intake is calculated and displayed

3.2 - Daily macronutrients (protein, carbs, fat) are calculated and displayed

3.3 - Calories remaining based on user's daily goal are calculated and shown

3.4 - User's daily calorie goal is calculated using Mifflin St Jeor equation and activity level

3.5 - Daily calorie goal automatically recalculates when user updates weight or goal weight in settings

3.6 - BMI is calculated and updated when weight changes

Objective 4: Workout Tracking

4.1 - Users can log workouts with type, duration, and calories burned

4.2 - Today's workout summary displays total workouts, duration, and calories burned

4.3 - Individual workouts are listed with all details

4.4 - Users can delete selected workouts from today's logs

4.5 - Workout data is validated to ensure numeric values and positive numbers

Objective 5: Data Visualization and Statistics

5.1 - Bar graph showing calorie intake over the last 7 days using matplotlib

5.2 - Bar graph showing workout duration over the last 7 days using matplotlib

5.3 - Pie chart showing macronutrient breakdown for today using matplotlib

5.4 - User statistics including name, weight, height, and BMI are displayed

5.5 - Graphs use timedelta to calculate dates for the past week

Objective 6: Database Management

6.1 - A database table for user accounts is present including username, passwordHash, name, age, gender, height, weight, activity level, BMI, daily calories, and goal weight

6.2 - A database table for foods is present including foodId, name, calories, protein, carbs, and fat

6.3 - A database for meal logs is present including logId, userId, date, mealType, foodId, quantity, and total nutritional values

6.4 - A database table for workout logs is present including logId, userId, date, workoutType, duration, and calories burned

6.5 - Information can be stored, retrieved, and deleted from these databases

6.6 - Database maintains referential integrity with foreign key relationships

6.7 - Database design follows Second Normal Form (2NF) with no partial dependencies

Objective 7: Settings and User Profile Management

7.1 - Users can update their current weight

7.2 - Users can update their goal weight

7.3 - Daily calorie target automatically recalculates when weights are updated

7.4 - BMI automatically recalculates when weight is updated

7.5 - Users can change their password

7.6 - Input validation ensures only valid numeric values are accepted for weight

Objective 8: User Interface and Navigation

8.1 - Main menu provides clear navigation to all main features

8.2 - All pages have back buttons to return to main menu

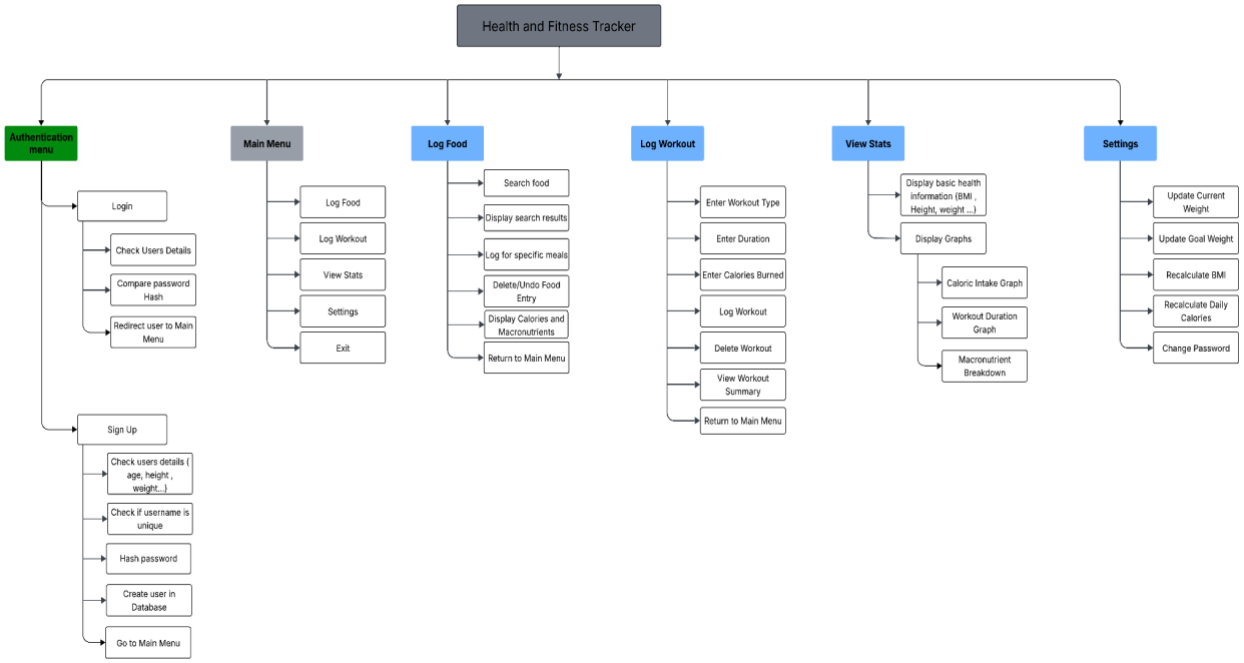
8.3 - Application uses consistent styling with colours and fonts

8.4 - Listboxes display information in readable formatted text

8.5 - Application window has appropriate title and dimensions

Design

Decomposition Diagram :



Justification of Decomposition Diagram

The decomposition diagram above shows the modules which I have broken the problem down into. During development I can work on all of these modules independently and then combine them to create the final application. I have broken the problem down into six main sections: Authentication menu, Main menu, Log food, Log workout, View stats and settings.

Authentication Menu:

This section involves how a user will be able to either log into an existing account or sign up to create a new account. The user will enter their details, which are then checked to make sure they are valid.

For login, the system checks the user's details and compares the password hash to the one stored in the database before redirecting the user to the main menu.

For sign up, the system checks the entered details, such as age, height, weight and it will make sure they are all valid. It will check that the username is unique, hash the password and create the user in the database before redirecting them to the main menu.

Main Menu:

This module involves creating the central menu where a user will be able to access different parts of the application. From here they can go to any of the other modules such as Log Food, Log Workout, View Stats, Settings or they can choose to exit the program. This acts as the central point of the system and will allow the user to move easily between the different functions of the application.

Log Food:

This section involves allowing a user to search for foods and displaying the search results from the database.

The user can then log foods for specific meals such as breakfast, lunch or dinner. This section will also include how the user will be able to delete or undo a food entry, display the calories and macronutrients for the foods that they have logged, and users will be able to return to the main menu.

Log Workout:

This section takes care of the processes involved in logging data about their workouts. Users will be able to log workouts and add information about the workout type, duration, and calories burned. A user will be able to log a workout and also delete an existing workout. Overall, this section helps the user keep track of their workouts and view all of their completed workouts.

View Stats:

This view stats section involves displaying the user's basic health information such as BMI, height and weight. It also includes displaying graphs of the user's data to provide information about their caloric intake, workout duration and macronutrient breakdown.

This allows the user to review their progress over time and better understand patterns in their health and fitness data.

Settings:

This module involves allowing a user to update their current weight and goal weight, as well as recalculate their BMI and daily calorie target when their weight/goals change.

They are also able to change their password from this section.

This helps keep the user's password secure if they feel their security is being breached, It serves to keep their information accurate and ensure the program continues to generate correct calorie and BMI values.

I decided that these were the main sections of my solution and defined separate sub sections within these that will build up each part of the application.

Data Structures, Algorithms, Calculations and Libraries:

For the Health and Fitness Tracker to function efficiently, I will need to use a range of different data structures, algorithms, libraries and calculations throughout the program.

Different parts of the program require different structures depending on the task being carried out. Below, I will explain the main data structures, algorithms, calculations and libraries that will be used in each section of the system and why they are appropriate to my solution.

Authentication Menu:

Libraries:

Library	Use
tkinter	Creates login and sign up interface
sqlite3	Stores and checks account details
hashlib	Hashes and verifies passwords

Algorithms:

Hashing algorithm:

I will implement SHA-256 as the hashing algorithm to securely store users passwords.

SHA-256 is part of the SHA-2 family, which was developed by the National Security Agency (NSA) and is widely used across many industries for its reliability.

Strong Collision Resistance: SHA-256 produces a unique 256 bit (64-character hexadecimal) hash for each input, making it very unlikely for two different passwords to generate the same hash (collision).

Built in Support: SHA-256 is supported in Python via the hashlib library, making this easy to integrate into my program.

Performance: Unlike slower algorithms like bcrypt or Argon2, SHA-256 offers fast computation, which is ideal for my project where performance and simplicity are key.

How the hash function will be implemented: Passwords are hashed using SHA-256 immediately upon account registration and stored in the database as hexadecimal strings.

During login, the entered password is hashed again and compared to the stored hash to verify the identity of the user.

Hashing class :

```
class Passwordhasher():
    def hash(self, password):
        return hashlib.sha256(password.encode("utf8")).hexdigest()

```

Login and Signup:

Login Algorithm pseudocode :

Below is the pseudocode for my login function. The function is responsible for verifying the identify of the user before granting them access to the system. When a user enters their username and password and clicks login, the following algorithm will run.

```
FUNCTION login():

    username = get text from usernameEntry field
    password = get text from passwordEntry field

    CONNECT to database

    result = SELECT id, passwordHash FROM Userstab WHERE username = username

    CLOSE database connection

    IF result exists THEN
        hashedInput = SHA256(password)
        IF hashedInput == result.passwordHash THEN
            SET currentUserId = result.id
            proceed to MainMenu
            RETURN
        ENDIF
    ENDIF

    show error "Invalid credentials"

END FUNCTION
```

Account registration algorithm pseudocode :

The registration function will handle the creation of new accounts. When a user enters their details and clicks sign up the following algorithm will execute. The algorithm validates all inputs to ensure correct data is added to the database. When all of the inputs are valid, the password will then be hashed and all of the necessary calculations will be carried out, and then the information will be inserted into the database.

```
FUNCTION register():

    username = get text from usernameEntry field
    password = get text from passwordEntry field
    name = get text from nameEntry field
    ageStr = get text from ageEntry field
    heightStr = get text from heightEntry field
    weightStr = get text from weightEntry field
    goalweightStr = get text from goalweightEntry field
    weeksStr = get text from weeksEntry field

    IF ageStr is not a int THEN
        show error "Please enter valid age"
        RETURN
    ENDIF

    IF heightStr is not a float THEN
        show error "Please enter valid height"
        RETURN
    ENDIF

    IF weightStr is not a float THEN
        show error "Please enter valid weight"
        RETURN
    ENDIF

    IF goalweightStr is not a float THEN
        show error "Please enter valid goal weight"
        RETURN
    ENDIF

    IF weeksStr is not a int THEN
        show error "Please enter valid weeks"
        RETURN
    ENDIF


    hashed = SHA256(password)

    carry out all neccesary calculations (BMI,BMR,TDEE)

    CONNECT to database

    existing = SELECT username FROM Users WHERE username = username


    IF existing record found THEN
        CLOSE database connection
        show error "Username already exists"
        RETURN
    ENDIF

    INSERT INTO Users (username, passwordHash, name, age, gender,
                       heightCm, initialWeight, activityLevel, bmi,
                       dailycalories, goalweight)
    VALUES (username, hashed, name, age, gender, height, weight,
            activity, bmi, dailycalories, goalweight)

    COMMIT and CLOSE database connection

    proceed to MainMenu

END FUNCTION
```

Calculations:

BMR (Basal metabolic rate):

To calculate BMR i will be using Mifflin St Jeor's Equation as it is known to be the most accurate and reliable BMR equation:

Variables:

W-Weight

H-Height

A-Age

For men the equation is:

BMR = 10W + 6.25H - 5A + 5

For women the equation is:

BMR = 10W + 6.25H - 5A - 161

TDEE calculations (Total daily energy expenditure):

TDEE is calculated by multiplying BMR by an activity level multiplier:

- Sedentary: BMR × 1.2
- Light: BMR × 1.375
- Moderate: BMR × 1.55
- Active: BMR × 1.725
- Very Active: BMR × 1.9

Calculate Daily Calorie Adjustment

The daily adjustment is calculated based on the users weight goal and time frame:

- Weight change needed = Goal Weight - Current Weight (kg)
- Total calorie change = Weight change × 7700 (calories per kg of body weight)
- Daily adjustment = Total calorie change ÷ (Weeks × 7)

Final Daily Calorie Target

- Daily Calories = TDEE + Daily Adjustment

A minimum threshold of 1200 calories per day is going to be enforced for health and safety reasons.

This calorie calculation method doesn't have a formal name but it's based on the Harris Benedict Principle.

This approach uses:

1. **Mifflin St jeor equation** for BMR calculation
2. **Harris Benedict activity multipliers** for calculating TDEE from BMR

3. **Energy balance/caloric deficit-surplus model** for the weight change adjustment (based on the principle that 1kg of body weight equals approximately 7700 calories)

The overall method is the standard method used by nutritionists and fitness professionals to calculate personalised daily calorie targets based on individual goals and timeframes.

BMI Calculation:

```
userbmi = weight / ((height / 100) ** 2)

```

Daily calories and BMR calculation:

```
def calculatedailycalories(self, weight, goalweight, height, age, gender, activity, weeks):
        # Calculate BMR using Mifflin-St Jeor equation
        if gender.upper() == 'MALE':
            bmr = 10 * weight + 6.25 * height - 5 * age + 5
        else:
            bmr = 10 * weight + 6.25 * height - 5 * age - 161

        # Multiply BMR by activity level to get TDEE
        activity_levels = {
            'SEDENTARY': 1.2,
            'LIGHT': 1.375,
            'MODERATE': 1.55,
            'ACTIVE': 1.725,
            'VERY ACTIVE': 1.9
        }
        tdee = bmr * activity_levels.get(activity.upper(), 1.2)

        # Calculate daily calorie adjustment to reach goal
        weight_change = goalweight - weight
        kcal_change = weight_change * 7700  # 7700 kcal per kg
        daily_adjustment = kcal_change / (weeks * 7)

        dailycalories = tdee + daily_adjustment

        # Minimum 1200 calories for health
        if dailycalories < 1200:
            return 1200
        return dailycalories
```

A dictionary structure will be used to store the activity level multipliers, as this allows the program to quickly match the user's activity level to the correct numerical value when calculating calorie targets.

Main Menu:

The main menu will act as a central navigation page and I will use a dictionary to store and access the different tkinter pages so that the user can move efficiently between sections of the program.

It will work like this

- User clicks a navigation button on the main menu, which calls a function called showPage(), passing the page name as a string parameter
- showPage() uses the page name as a key to look up the frame in the dictionary

- The correct frame is retrieved and tkraise() is called which will bring it to the front of the window
- The user can interact with the features on the current page
- The user can then choose to click the Back to Main Menu button which calls showPage("MainMenu")
- The main menu frame will then be retrieved from the dictionary and raised back to the front

Libraries:

Library	Use
tkinter	Displays navigation buttons and frame/interface

Log Food:

Algorithms:

Binary search algorithm:

```
def binarySearch(foodList, searchTerm):

    left = 0
    right = length of foodList - 1
    results = []

    WHILE left <= right
        mid = (left + right) // 2
        middleFood = foodList[mid][0]

        IF searchTerm == middleFood
            results.append(foodList[mid])
            RETURN results

        ELIF searchTerm < middleFood
            right = mid - 1

        ELSE
            left = mid + 1
    END WHILE

    RETURN results
```

Choosing an algorithm:

When designing a searching algorithm for the food logging aspect of my code, I considered 2 options: binary search and linear search.

Linear search works by checking every item in the list one by one until either the item is found or the end of the list is reached.

Binary search works by repeatedly halving the list. It begins by looking at the middle item of the list, if the search term matches the middle item, then the search is complete and the item is found. If the search term is greater than the middle item (alphabetically) this process is done again but only on the right sublist. If the search term is lower than the middle item (alphabetically) this happens again but on the left sublist.

Why I chose Binary Search over Linear Search:

Although linear search is easier to program, given that we're dealing with a food database which has the potential to store 10,000+ items, binary search poses significantly more efficient:

Big O notation is used to describe how an algorithm's performance scales as the input size grows. It measures the worst case scenario for time complexity. Using Big O notation I compared the performance of both searching algorithms to see how well they would scale for a large database.

Linear Search - O(n):

- **n** represents the number of items in the list
- Linear search must check each item one by one until it finds the target
- Worst case: the food is at the very end or doesn't exist, so it checks ALL items
- For 10,000 foods: 10,000 comparisons
- For 100,000 foods: 100,000 comparisons
- **Growth:** If you double the data, you double the time (linear relationship)

Binary Search - O(log n):

- **log n** means logarithmic growth (specifically $\log_2 n$ - base 2)
- Binary search cuts the search space in half with each comparison
- For 10,000 foods: $\log_2(10{,}000) = 13.3$, so maximum 14 comparisons
- For 100,000 foods: $\log_2(100{,}000) = 16.6$, so maximum 17 comparisons
- **Growth:** If you multiply data by 10, you only add 3 comparisons (logarithmic relationship)

Verdict:

- Binary search is significantly faster for large datasets.
- For 100,000 foods: Linear needs 100,000 checks, Binary search only needs 17.

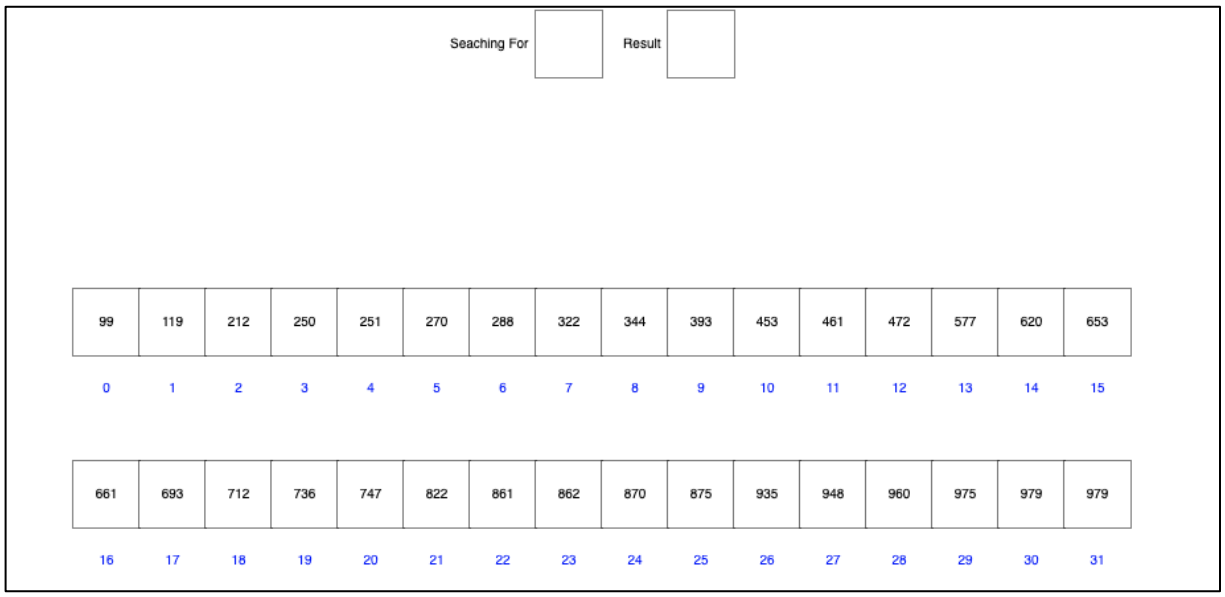
Trade off: Binary search requires sorted data. I will handle this by using (ORDER by name) in my SQL query before the search is performed.

Visualisation of how the binary search will work :

Source: https://www.cs.usfca.edu/~galles/visualization/Search.html

The food names are pulled from the database and are ordered by name

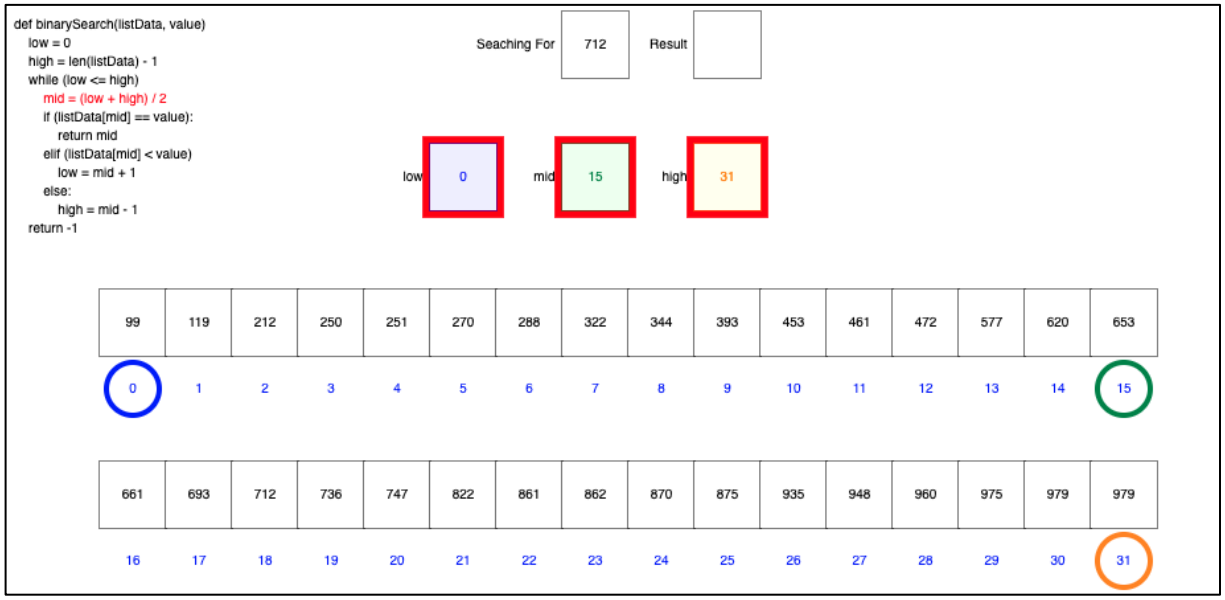
Each number in this list represents a different food name



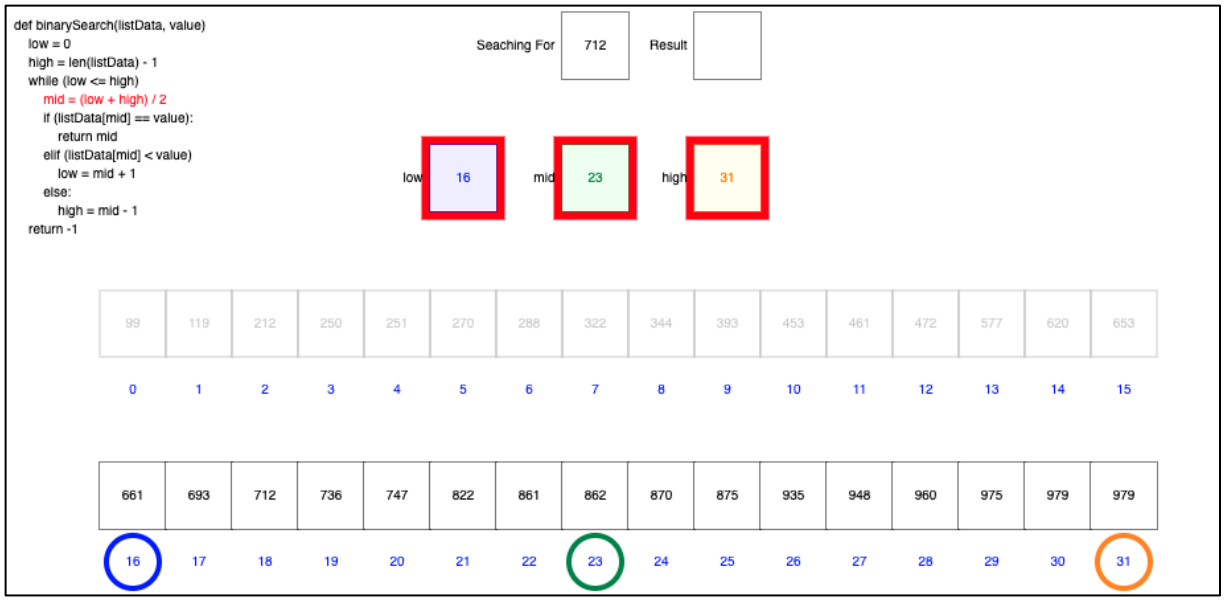
The user has now input the food they want to search for and it is set as the search term.

In this case I will be using the number “712”.

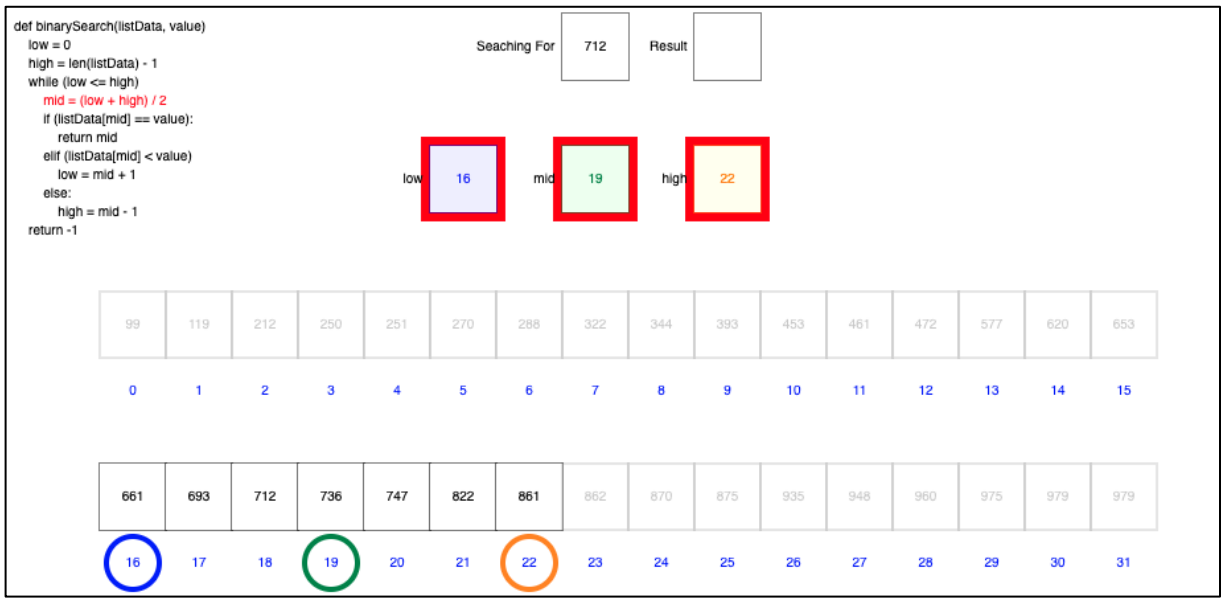
The first item in the list, middle item and the last item are set as pointers and the search term is compared to the middle item .



In this case, the middle element is != to the search term ,the search term is larger than the middle item so we repeat this process on the right sublist.



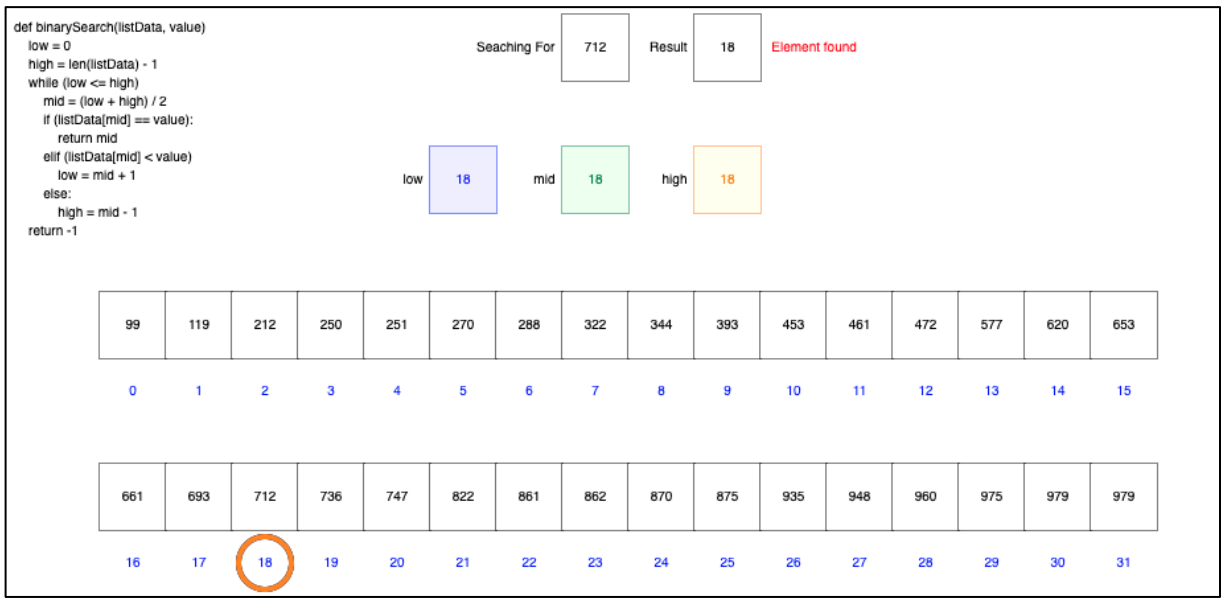
The middle element is not equal to the search term, the search term is smaller, so this repeats again but on the left sublist.



The middle element is still not equal to the search term, it is smaller so this repeats again on the left sublist.



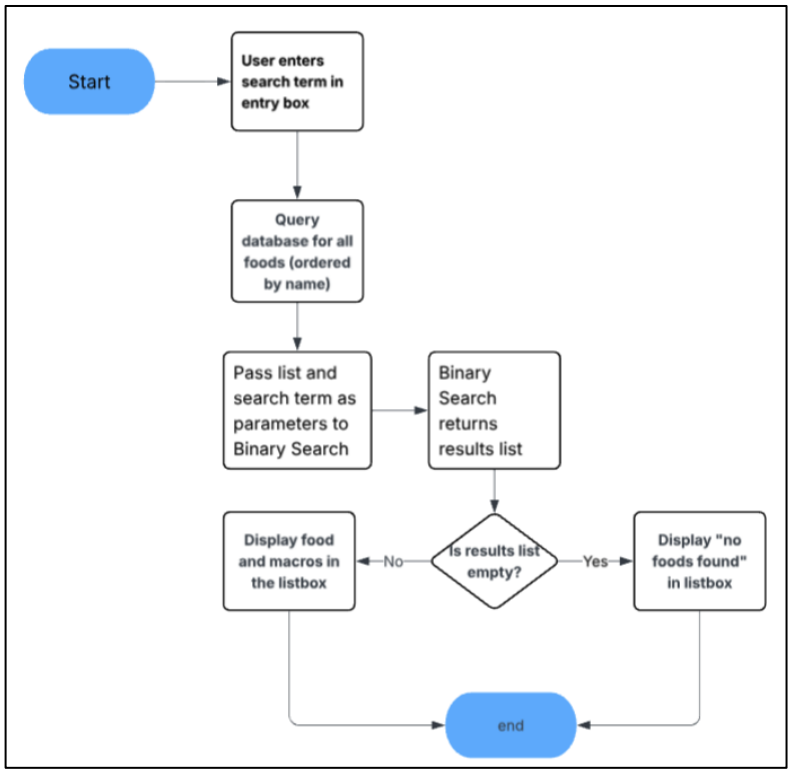
This repeats one more time on the right sublist. This sublist only contains one item, which is “712”. All 3 pointers are set to 712 and so the item is found.



For binary search, this took 5 comparisons. If linear search were to be used, it would’ve taken 19 comparisons. Binary search becomes even more efficient for larger databases, which will be needed for my health and fitness tracker in the future. This visualisation solidifies my choice of binary search over linear search.

The flowchart below shows how the food search process works.

Food search process flowchart :



Data Structures:

STACK

Stack for Undo Functionality :

A stack is a Last In First Out (LIFO) data structure; you can only add or remove from the top.

Stack Operations:

- **Push:** Add a meal to the top of the stack (when logging food)
- **Pop:** Remove and return the most recent meal from the top (for undo)
- **Peek:** Look at the top item without removing it (to show what would be undone)
- **isEmpty:** Check if there are any meals to undo

Stack class code:

```python
class Stack:
    def __init__(self):
        self.items = []

    # Add item to top of stack
    def push(self, item):
        self.items.append(item)

    # Remove and return top item
    def pop(self):
        if len(self.items) == 0:
            return None
        return self.items.pop()

    # View top item without removing
    def peek(self):
        if len(self.items) == 0:
            return None
        return self.items[-1]

    # Check if stack is empty
    def isEmpty(self):
        if len(self.items) == 0:
            return True
        return False

    # Get number of items in stack
    def size(self):
        return len(self.items)
```

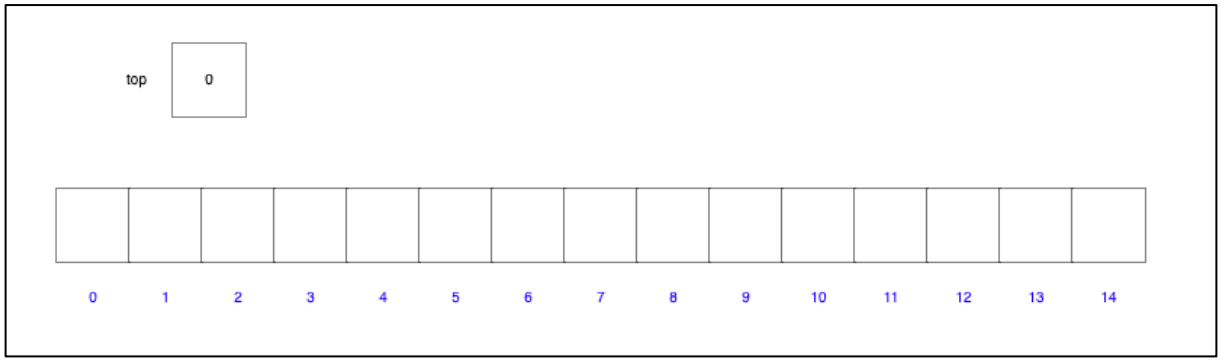
How it works for meal logging:

1. Every time a user logs a meal, we push that meal's information onto a stack
2. When the user clicks "Undo Last Meal", we pop the most recent meal from the stack
3. We then delete that meal from the database
4. The stack always keeps track of the order in which the meals were logged

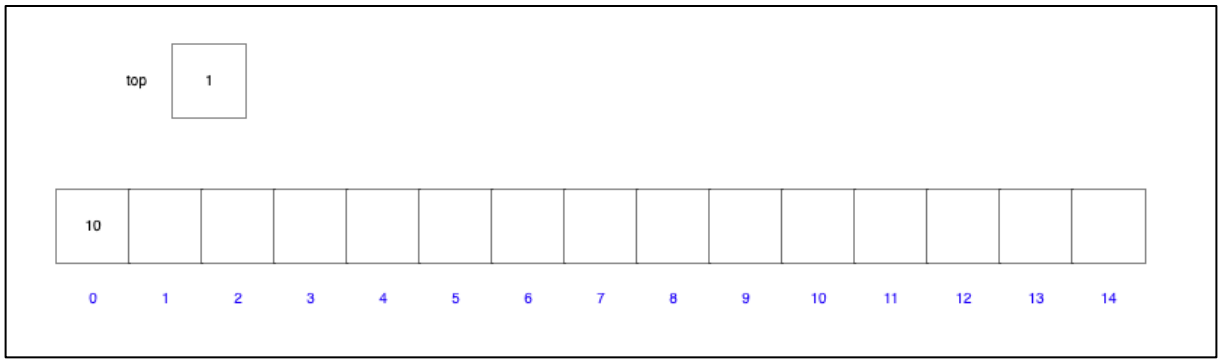
Visualisation of how the stack will work :

Source: https://www.cs.usfca.edu/~galles/visualization/StackArray.html

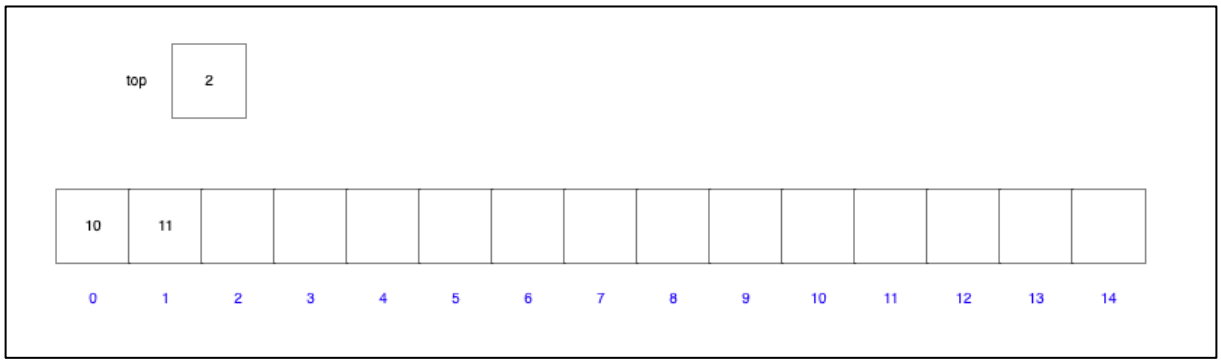
The stack is created



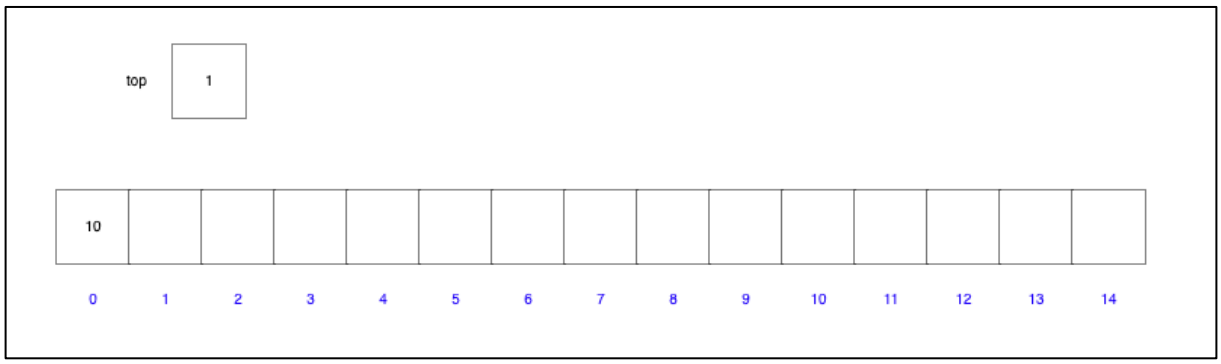
User logs "CHICKEN" → Database assigns logId = 10 → Push 10 onto stack



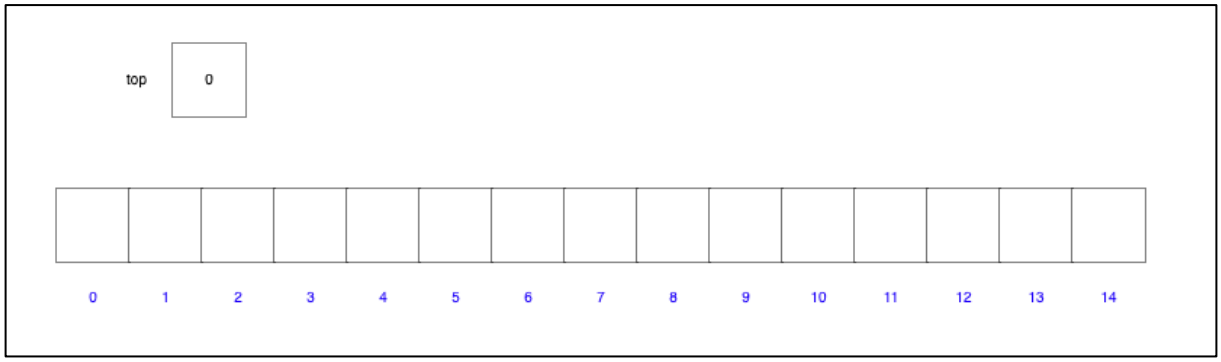
User logs "RICE" → Database assigns logId = 11 → Push 11 onto stack



User clicks "Undo" → Pop 11 from stack → Delete meal with logId = 11 from database

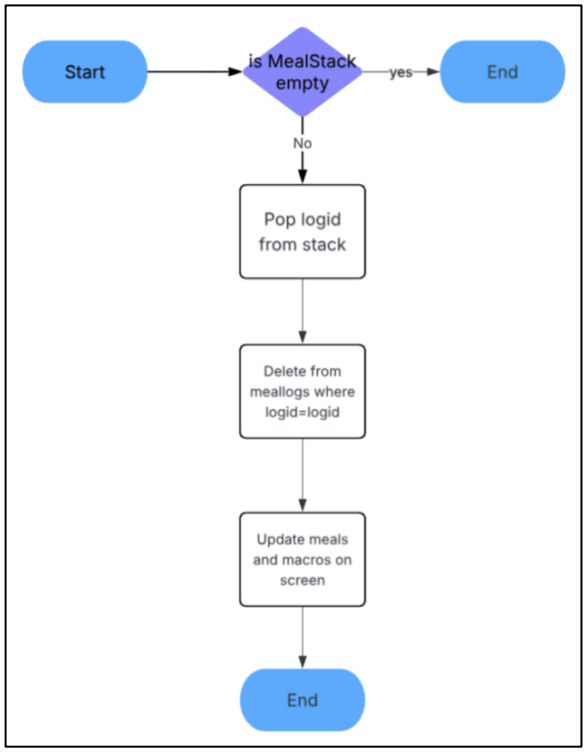


User clicks "Undo" again → Pop 10 from stack → Delete meal with logId = 10 from database



The flowchart below illustrates how the stack will be used for the undo functionality. Every time a food is logged the logid is pushed onto the stack. When the user clicks undo the following procedure will happen

Undo last meal entry flowchart:



Libraries:

Library	Use
tkinter	Creates food logging interface,buttons,listboxes,input fields
sqlite3	Stores and retrieves meals
datetime	Records current date meals

Log Workout:

This section will use a workout logs table in the database to store workout information.

Data structures:

Lists may also be used temporarily to display workout summaries and today's workout entries.

Libraries:

Library	Use
tkinter	**Creates workout interface,buttons,input fields and listboxes**
sqlite3	**Manages workout records**
datetime	**Records workout dates**

View Stats:

View stats functionality

In the View stats section there will be three types of graphs to summarise the user's data, and each is for a specific reason:

1. **Calorie intake over the last 7 days – Bar graph**: A bar graph will be used to show calorie intake because it clearly displays the total calories for each day side by side making it easy to compare intake across the week and spot trends.

2. **Workout minutes over the last 7 days – Bar graph**: Another bar graph will be used to show daily workout duration. Like calories, workout minutes are daily values, and bars make it simple to compare each day's activity visually.

3. **Macronutrient breakdown (daily) – Pie chart**: A pie chart will be used to show the proportion of protein, carbs, and fat consumed daily. Pie charts are ideal for displaying percentages, so the user can immediately see which macronutrient is

most abundant in their intake.

Each graph type is justified based on the nature of the data. Bar graphs are used for comparing daily totals over time, and Pie charts are used for showing the percentage composition of a user's daily intake. The implementation of this requires proficiency in matplotlib, a Python library that is commonly used to analyse data for programmers.

However, graphing is required to help people see how their caloric, physical, or macronutrient intakes change over time, enabling them to modify their goals and reach their desired results in terms of health and fitness.

The most complex part of designing how I will implement graphs is how I will get it to present data for the last 7 days. By using datetime to retrieve the current date I was able to think of this solution which I will implement in my program.

Graph date algorithm:

```
FOR i = 0 TO 6:
    date = today minus (6 - i) days
    query database for that date
    IF no data THEN value = 0
    append to list

plot list as bar chart
```

Data structures:

For the graphing features, I will use lists to temporarily store values before they are plotted. For example, one list will store the dates for the previous seven days, while another list will store the calorie totals for those dates. The same approach will be used for workout duration data.

Libraries:

Library	Use
tkinter	**Displays stats via a listbox, interface,buttons**
sqlite3	**Retrieves stored data**

datetime	**Handles recent dates (last 7 days)**
matplotlib.pyplot	**Creates charts and graphs**

Settings:

Calculations:

If the user updates their weight or goal weight, calculations will be carried out in order to recalculate BMI and daily calorie targets.

- The same calculations used during account creation will be used here.

UpdateSettings Algorithm psuedocode:

The UpdateSettings algorithm will allow users to change their weight and goal weight and will automatically recalculate all the necessary fields like TDEE, BMI and BMR. The algorithm works by checking the fields are valid, it then retrieves the user's current data from the database and then applies any changes.

```
FUNCTION updateSettings():

    GET weightStr, goalWeightStr FROM input fields

    IF both fields are empty THEN
        RETURN
    ENDIF

    RETRIEVE current user data from database

    IF weightStr is not empty THEN
        IF weightStr is not valid THEN
            show error "Please enter valid weight"
            RETURN
        ENDIF
        UPDATE weight in database
    ENDIF

    IF goalWeightStr is not empty THEN
        IF goalWeightStr is not valid THEN
            show error "Please enter valid goal weight"
            RETURN
        ENDIF
        UPDATE goal weight in database
    ENDIF

    recalculate BMI and UPDATE in database

    recalculate daily calories
    IF dailyCalories < 1200 THEN
        SET dailyCalories = 1200
    ENDIF
    UPDATE daily calories in database

    CLEAR input fields

END FUNCTION
```

Data structures:

A dictionary structure will also be used to store the activity level multipliers, as this will allow the program to quickly match the user's activity level to the correct numerical multiplier when their calorie targets are being recalculated.

Algorithms:

For password updates, the hashing algorithm will be used to store the new password in the database as its SHA-256 hash and not as plaintext.

Libraries:

Library	Use
tkinter	Creates settings screen,buttons, input fields
sqlite3	Updates user data
hashlib	Hashes new passwords

Whole program

Dictionary Data Structure in the HealthTrackerApp Class:

I implemented a dictionary to store all of the application pages in the HealthTrackerApp class. A dictionary is a key value pair data structure that will provide efficient page navigation throughout the application.

Fast Access: Dictionaries provide a constant time complexity for searching, this is significantly faster than using a list, which would require a linear search to find the correct page.

Readable Code: Using string keys like "Login", "MainMenu", and "LogFood" makes navigation calls self explanatory. Calling self.pages["Login"] is clear compared to accessing pages through indexes like pages[2].

Scalability: Adding new pages just requires inserting a new key value pair without changing the existing code.

How it's implemented:

Dictionary key value:

key = page name

value = page/screen object

This allows the program to quickly access and display whichever screen is necessary.

dictionary data structure:

```
pages = {}

pages["AuthMenu"] = AuthMenu
pages["SignUp"] = SignUp
pages["Login"] = Login
pages["MainMenu"] = MainMenu
pages["LogFood"] = LogFood
pages["LogWorkout"] = LogWorkout
pages["ViewStats"] = ViewStats
pages["Settings"] = Settings

page = pages[pageName]
show page

```

Tkinter in the Program:

I will use Tkinter as the GUI library for my program because it allows me to create an interactive desktop application in Python.

This is appropriate for my project because the system requires:

- user input fields
- Buttons
- labels
- areas where stored information can be displayed clearly

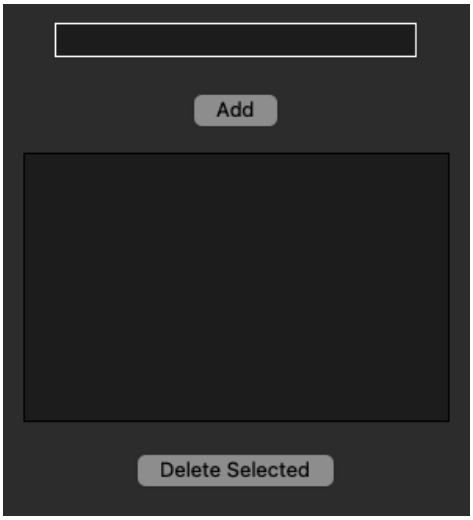
Tkinter provides all of these components and allows them to be organised into separate frames, this makes it suitable for my modular program structure.

Each main section of the application, such as Authentication, Log Food, Log Workout, View Stats, and Settings, can be created as its own frame and then it can be shown when it is required. This supports the structure diagram and makes navigation between sections straightforward.

Tkinter Listboxes:

Listboxes will be used all throughout the program as a way of displaying multiple pieces of related information in a clear format. A tkinter listbox is suitable for this because it can show several lines of data at once, making it useful for presenting data such as food logs ,workout logs, statistical summaries, and also to display search results without needing to create new windows or overcrowd the interface. This will help keep the UI organised and easy for the user to understand.

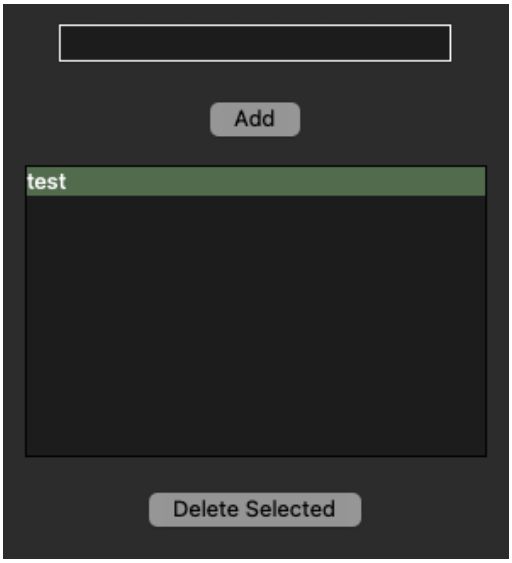
Empty listbox:



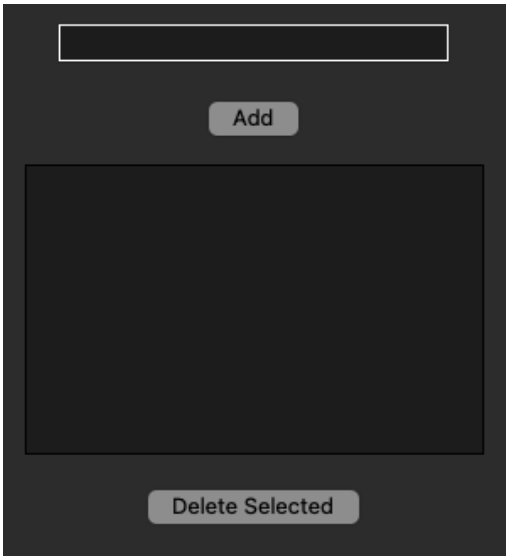
Item added to listbox:



Selecting item using curselection() function in tkinter:



Deleting selected item:



This demonstration shows how listboxes can be used in my program.

Below is an in depth explanation of how I plan to use listboxes in my program.

Log Food:

In the Log Food section, listboxes will be used to display food search results and also to show foods logged under breakfast, lunch, and dinner , allowing the user to assign food to different meals. A separate listbox will be used to display daily calorie and macronutrient totals so that users can see their overall calorie consumption and all the nutritional values at the same time in an orderly fashion.

Log Workout:

In the Log Workout section, listboxes will be used to display the workouts logged for the current day, as well as a workout summary showing the total number of workouts, total

duration, and total calories burned. This allows the user to review both detailed and summarised information all in the same module of the program.

View Stats:

In the View Stats section, a listbox will be used to display user information such as name, weight, height, and BMI. This provides a simple and readable way to present core statistics.

Overall, listboxes are an appropriate GUI component for this program because they allow data retrieved from the database to be displayed clearly and viewed in groups. They will also make it easier for the user to interact with large amounts of information across different sections of the system.

The tables below show the tkinter components that are going to be used in the program and their purposes:

Main Window

Tkinter Item	What it does
tk.Tk()	Creates the main application window (root window).
tk.Frame()	A container used to group widgets into sections or separate pages.

Widgets

Tkinter Item	What it does
tk.Label()	Displays text or images (e.g. titles or instructions).
tk.Button()	Creates a clickable button that runs a function when pressed.
tk.Entry()	Creates a single line text box for user input.
tk.Listbox()	Displays a list of selectable items.

Layout Methods

Tkinter Item	What it does
pack()	Places widgets in order (top to bottom or side by side).
tkraise()	Brings a frame to the front to make it visible.

Window Configuration Methods

Tkinter Item	What it does
title()	Sets the title of the application window.
geometry()	Sets the size of the window (e.g. "400x300").
configure()	Changes window or widget properties (e.g. background colour).
config()	Short form of configure(), used to update widget settings.

Input / List Handling Methods

Tkinter Item	What it does
get()	Retrieves the current input or selected value.
insert()	Adds text or items into a widget.
delete()	Removes text or items from a widget.
curselection()	Returns the index of the selected item in a Listbox.

Control Methods

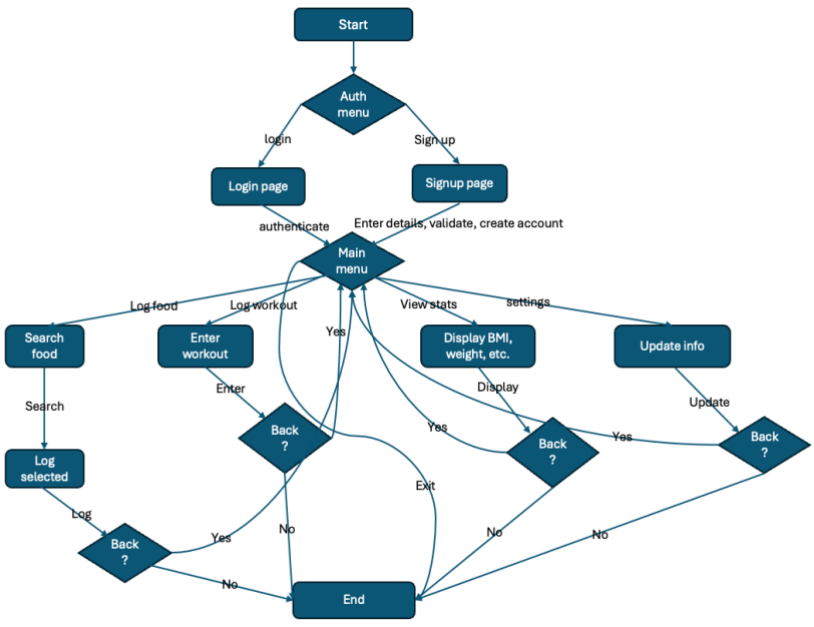
Tkinter Item	What it does
messagebox.showerror()	Displays an error popup message.
destroy()	Closes the application window.
mainloop()	Starts the event loop so the GUI runs and responds to input.

Tkinter Constant

Tkinter Item	What it does
tk.END	Refers to the end position in a widget (used when inserting/deleting).

Overall program flow :

The program flow diagram for the Fitness App is shown below. It outlines the key components and how the program should be structured to serve as a blueprint for when I develop the Health and Fitness tracker.



User Interface and Usability Features

Main menu navigation with buttons for all key tasks.

Minimal steps per action (log food in one screen: choose meal, search/scan, confirm).

Numeric validation for fields like age, weight, height, and food quantities.

Instant feedback after actions (daily calorie totals update immediately).

Consistent layout using Tkinter frames to avoid a cluttered UI.

Error prompts shown for invalid entries.

Usability Features and GUI design:

1.Authentication Menu

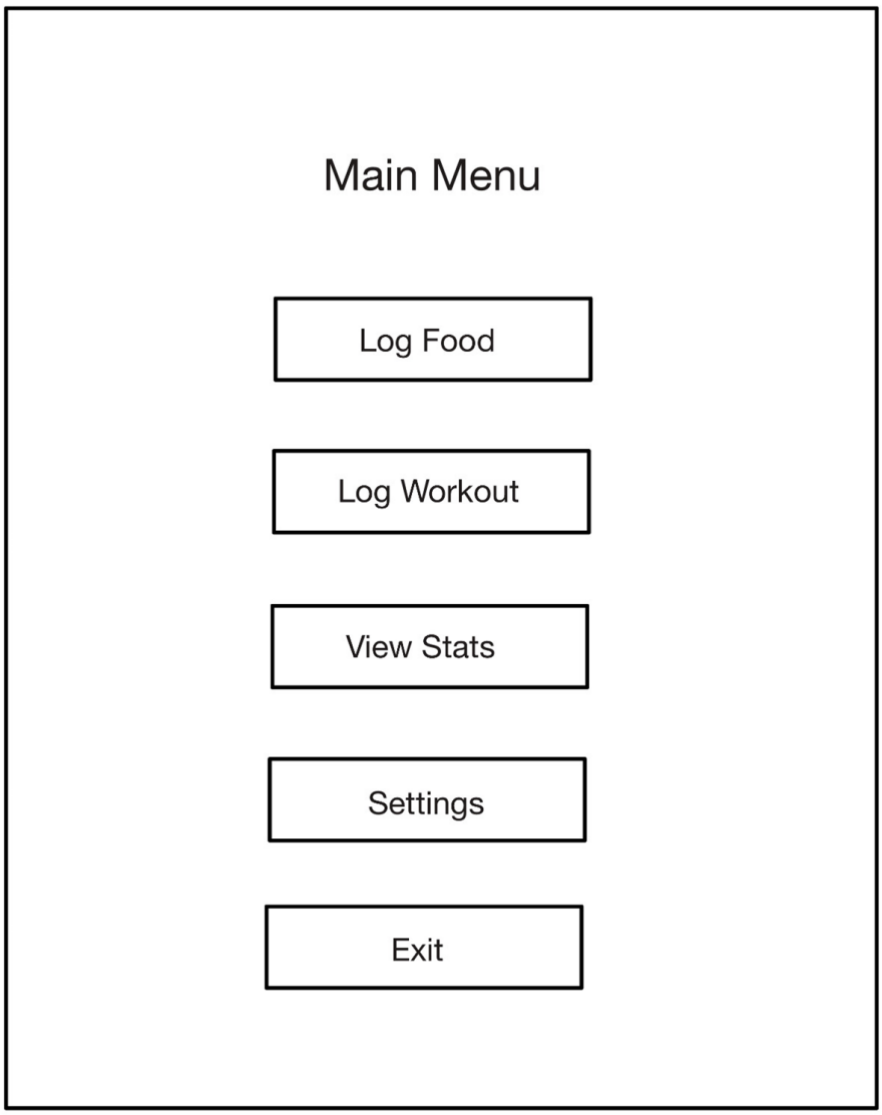
- Design: Two vertically stacked buttons (Login , Sign Up) centred on screen



- Justification: Vertical layout with Large buttons accommodate users with visual impairments. Single click access to all features to prevent confusion about navigation.

2. Main Menu Navigation

- Design: Five vertically stacked buttons (Log Food, Log Workout, View Stats, Settings, Exit) centred on screen



- Justification: Vertical layout with Large buttons accommodate users with visual impairments. Single click access to all features prevents confusion about navigation.

3. Real Time Feedback

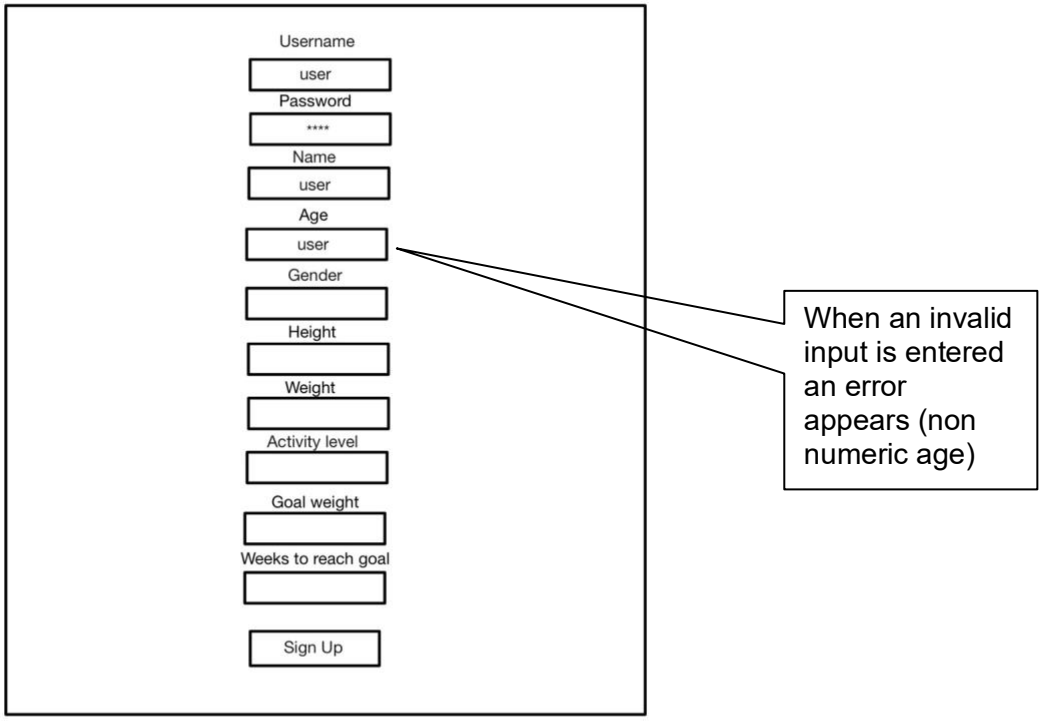
- Design: Daily totals listbox auto updates after every meal logged and workout logged
- Justification: Immediate feedback prevents user uncertainty. Calories remaining displayed instantly helps users make informed decisions throughout the day.

4. Colour Scheme

- Design: White background ,default black text, consistent across all pages
- Justification: High contrast ensures readability. White keeps the design minimalistic for users. Consistent colours reduce the cognitive load when navigating between pages**.**

5. Error Messages

- Design: Pop up message boxes with specific error text ("Please enter valid age")



Error message:



- Justification: Specific errors are guide users to correct mistakes. To ensure errors are noticed as user will not be able to proceed if the error isn't dealt with.

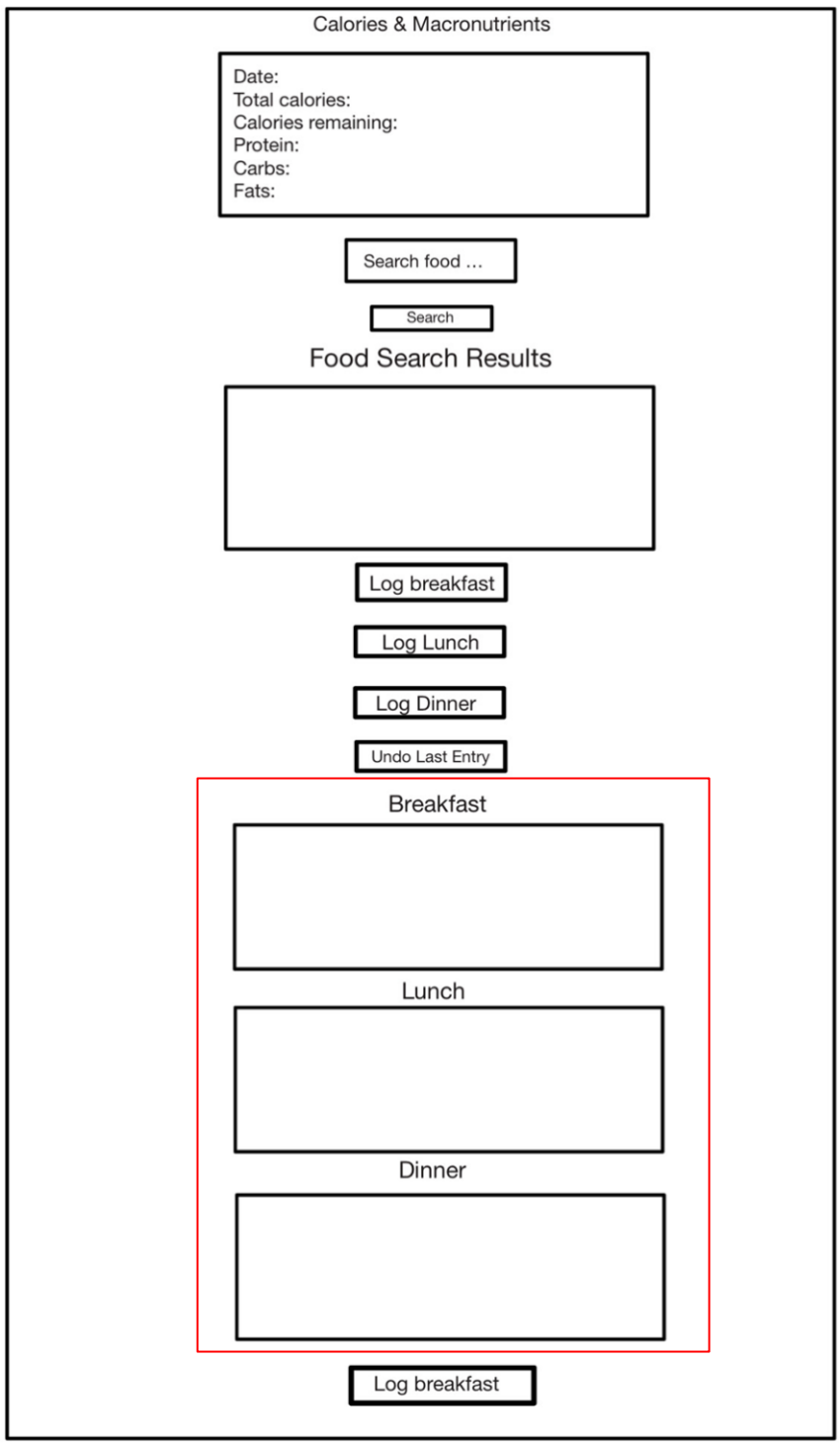
6. Automatic Page Refresh

- Design: LogFood and LogWorkout pages will call refresh methods when displayed

- Justification: Users see current data immediately without manual refresh. Prevents confusion from previous data.

7. Log food Meal Specific Listboxes

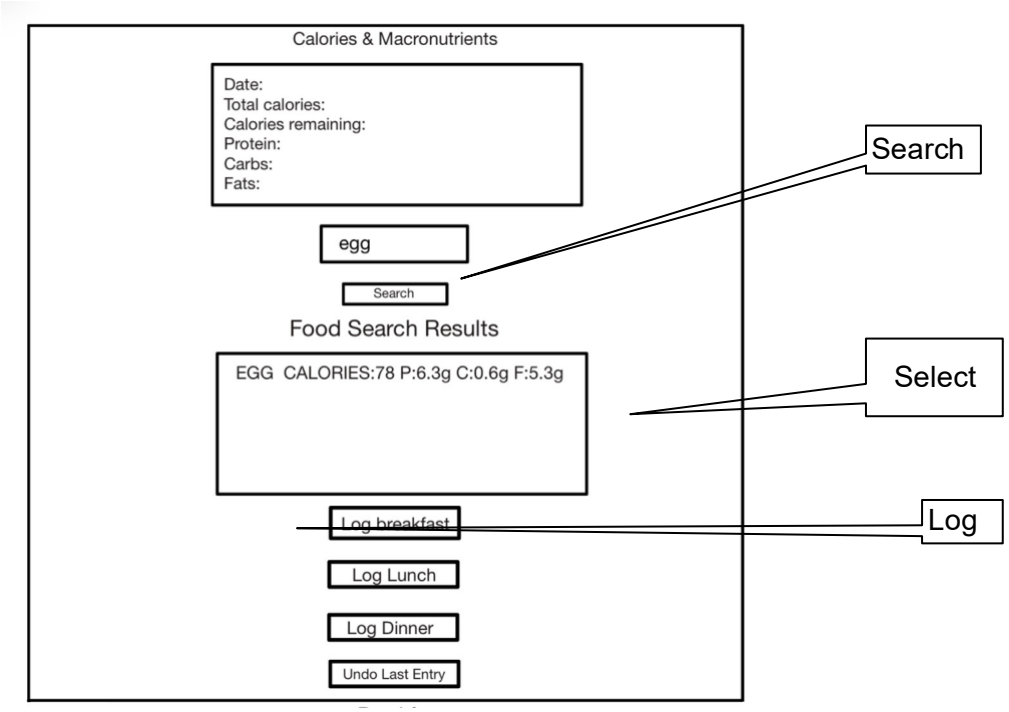
- Design: Three separate listboxes labeled "Breakfast", "Lunch", "Dinner"



- Justification: Removed all unnecessary features, Separation which prevents scrolling through mixed meal data. Fixed height maintains consistent layout for overall usability and user experience.

8. Log Food-Minimal Input Steps

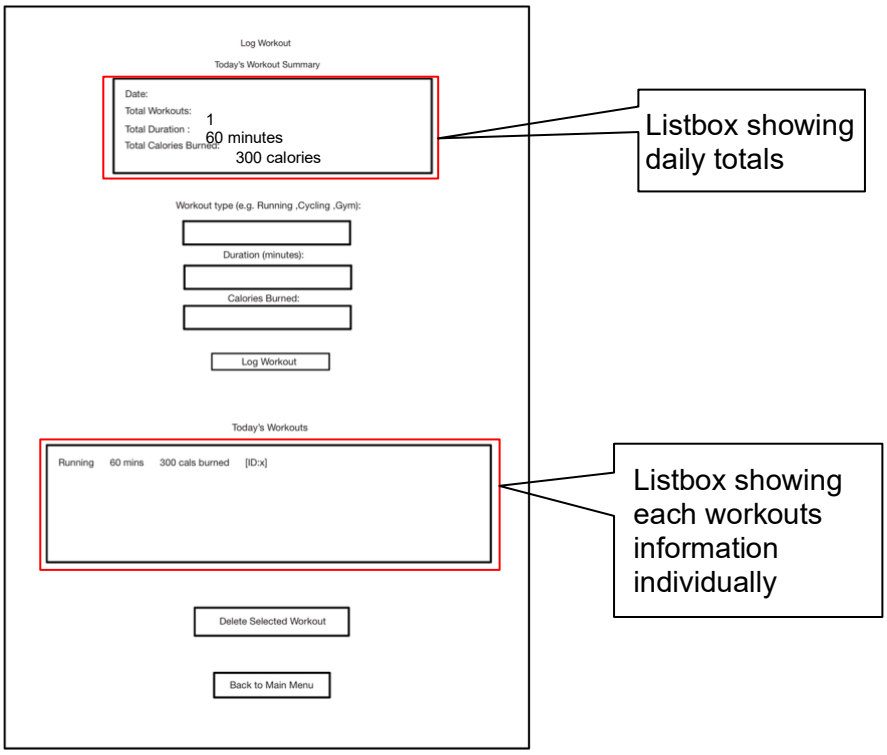
- Design: Food logging requires 3 steps: Search → Select → Click meal button



- Justification: Fewer steps reduces abandonment rate and helps to keep users logs consistent.

9. Log workout Specific listboxes

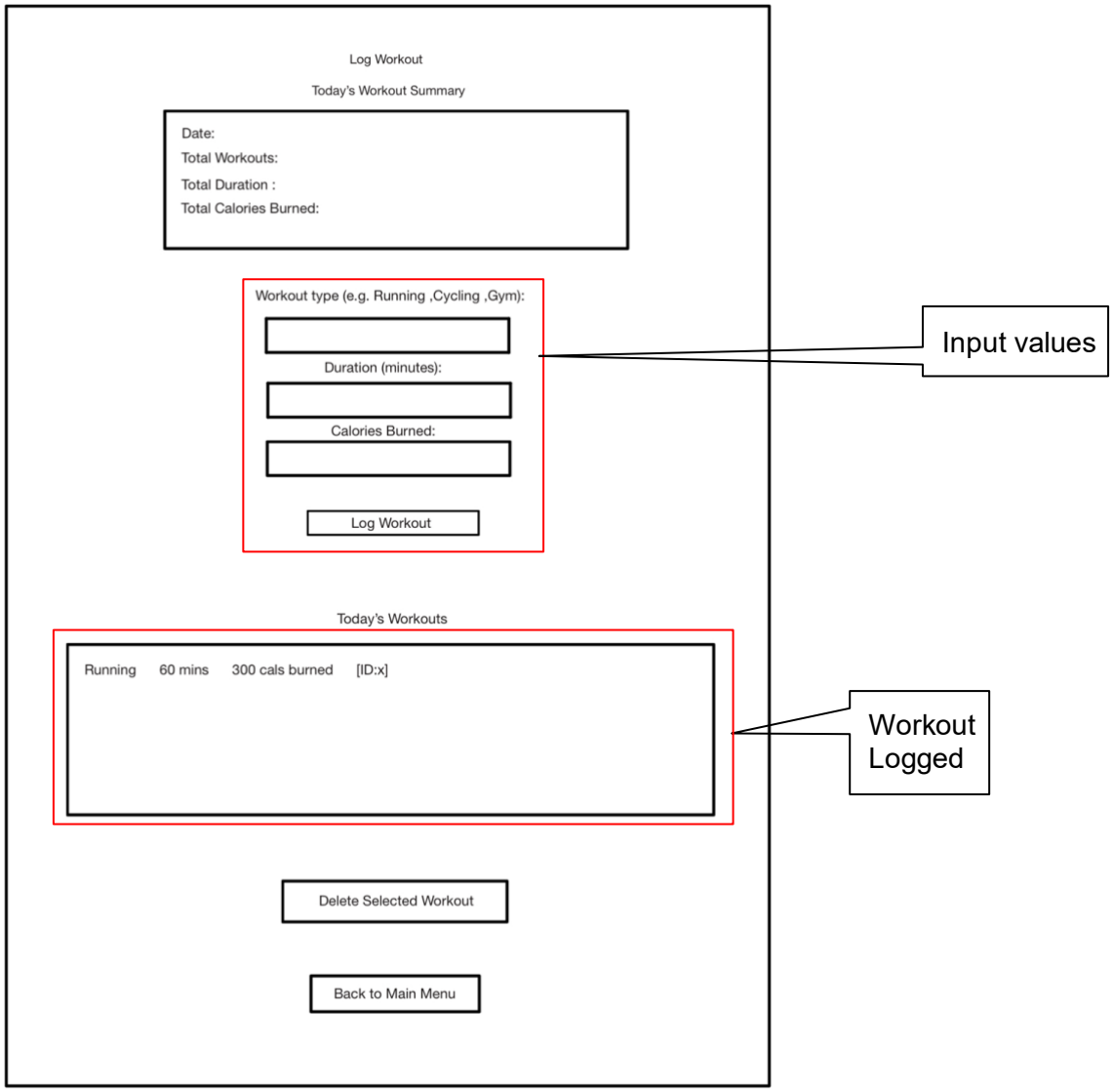
- Design: 2 listboxes labeled “todays workout summary” and “ todays workouts”



- Justification: Removed all unnecessary features. Fixed height maintains consistent layout for overall usability and user experience.

10. Log Workout Minimal Input Steps

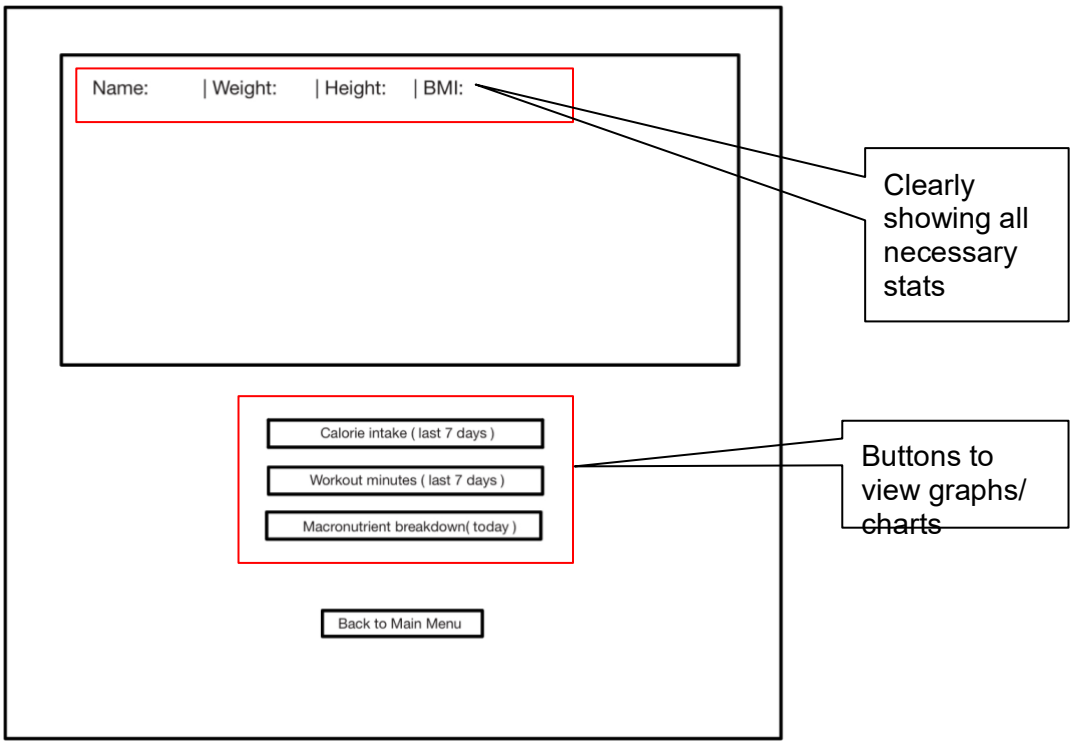
- Design: Workout logging requires 4 steps: Input workout type → Input Duration→ Input calories burned→Log Workout



- Justification: Fewer steps reduces abandonment rate and helps to keep users logs consistent.

11. View Stats

- Design: Listbox showing stats (Name ,Weight ,Height ,BMI). 3 Vertically stacked buttons, one for each graph/ chart.



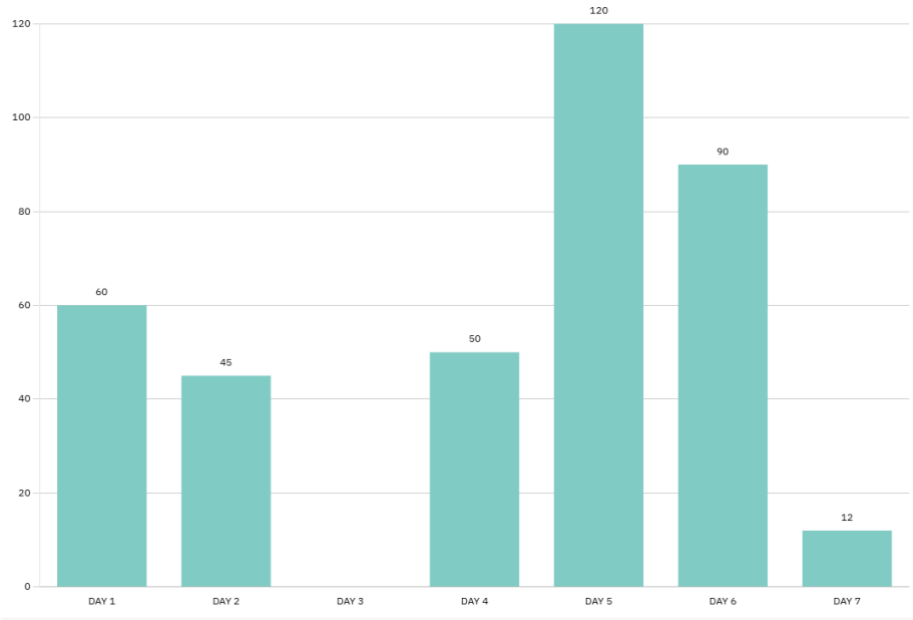
- Justification: Users are immediately presented with all necessary statistics. 3 Vertically stacked buttons to prevent confusion .

12. Graph Visualization in View stats

- Design: Bar graphs for 7 day trends, pie chart for daily macros

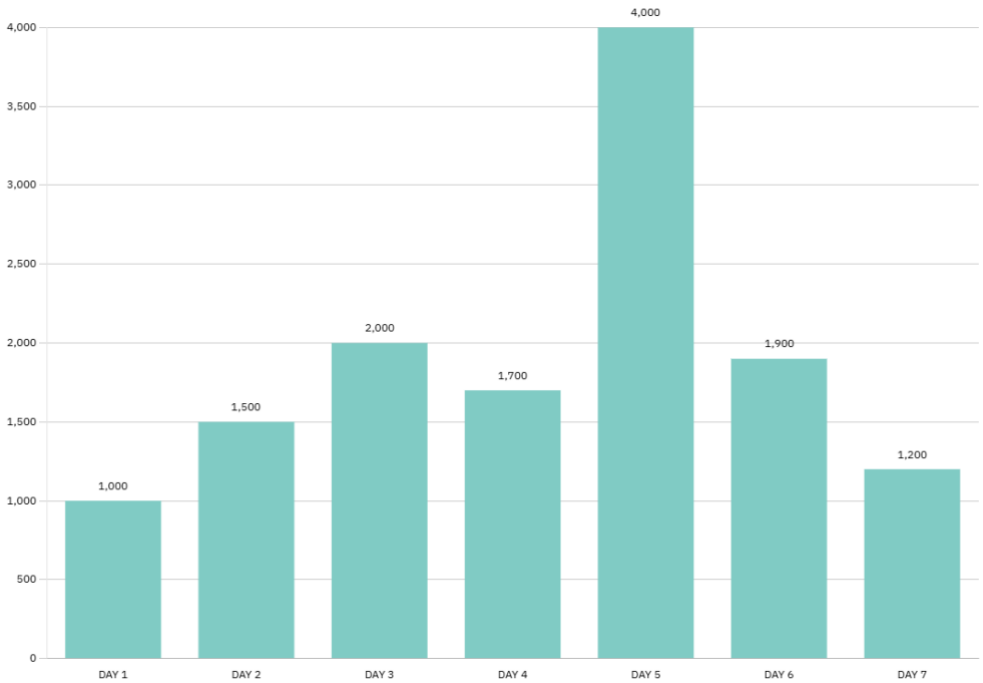
Workout duration over last 7 days (bar chart)

Y axis =minutes **X axis =Date**



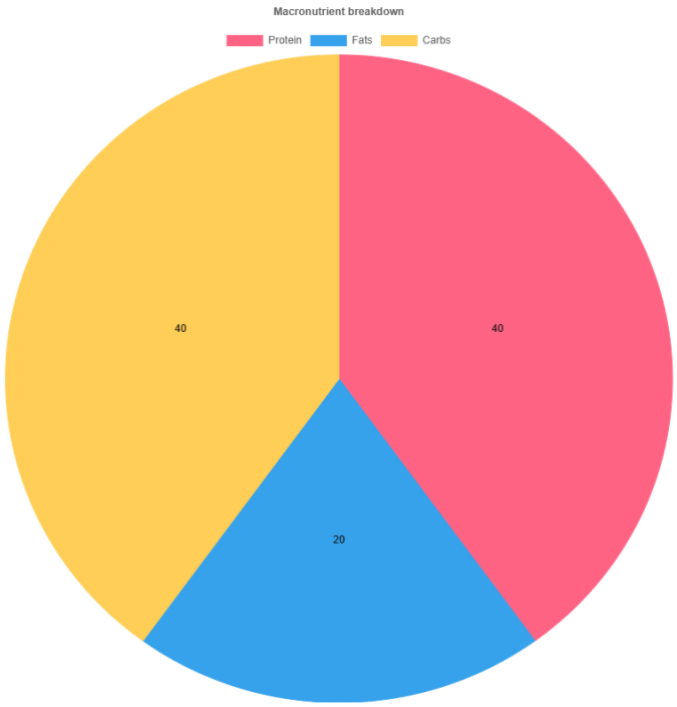
Calorie intake over last 7 days (bar chart)

Y axis =Total calories **X axis =Date**



Today's macronutrient breakdown (pie chart)

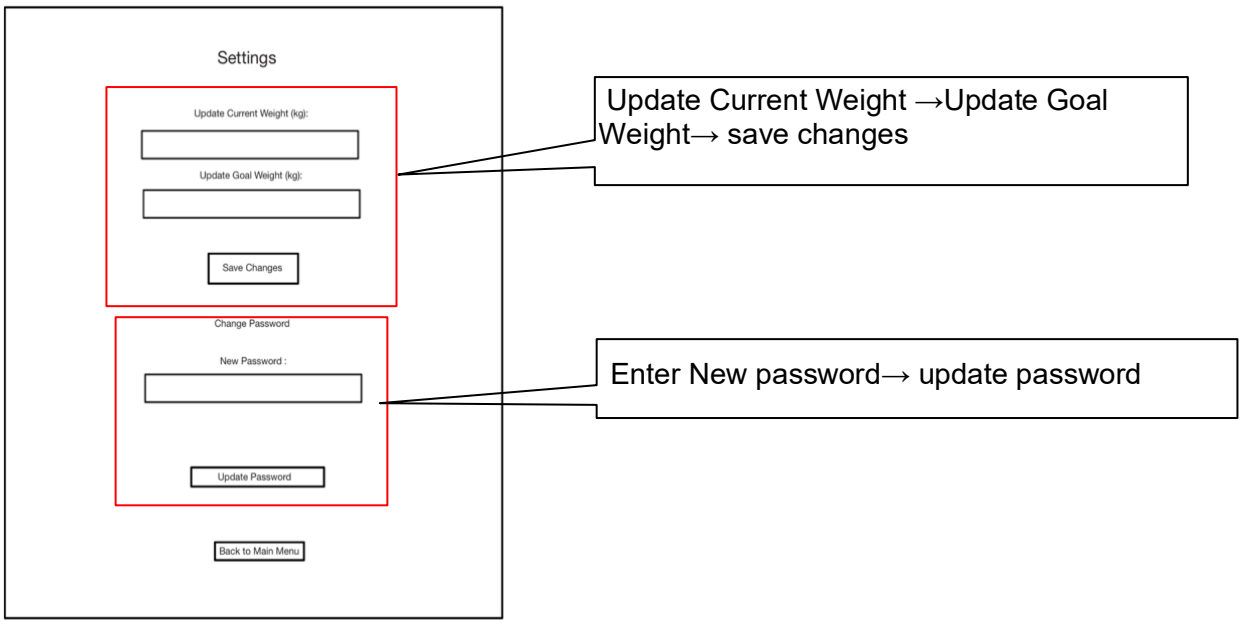
Each section will be shown as a percentage of the days total calories



- Justification: Bar graphs show temporal patterns. Pie charts show proportional relationships. Colour coded for visual distinction.

13. Setting page layout

- Design: Page has 2 sections (update weight , change password) . Both Require minimal steps as shown below



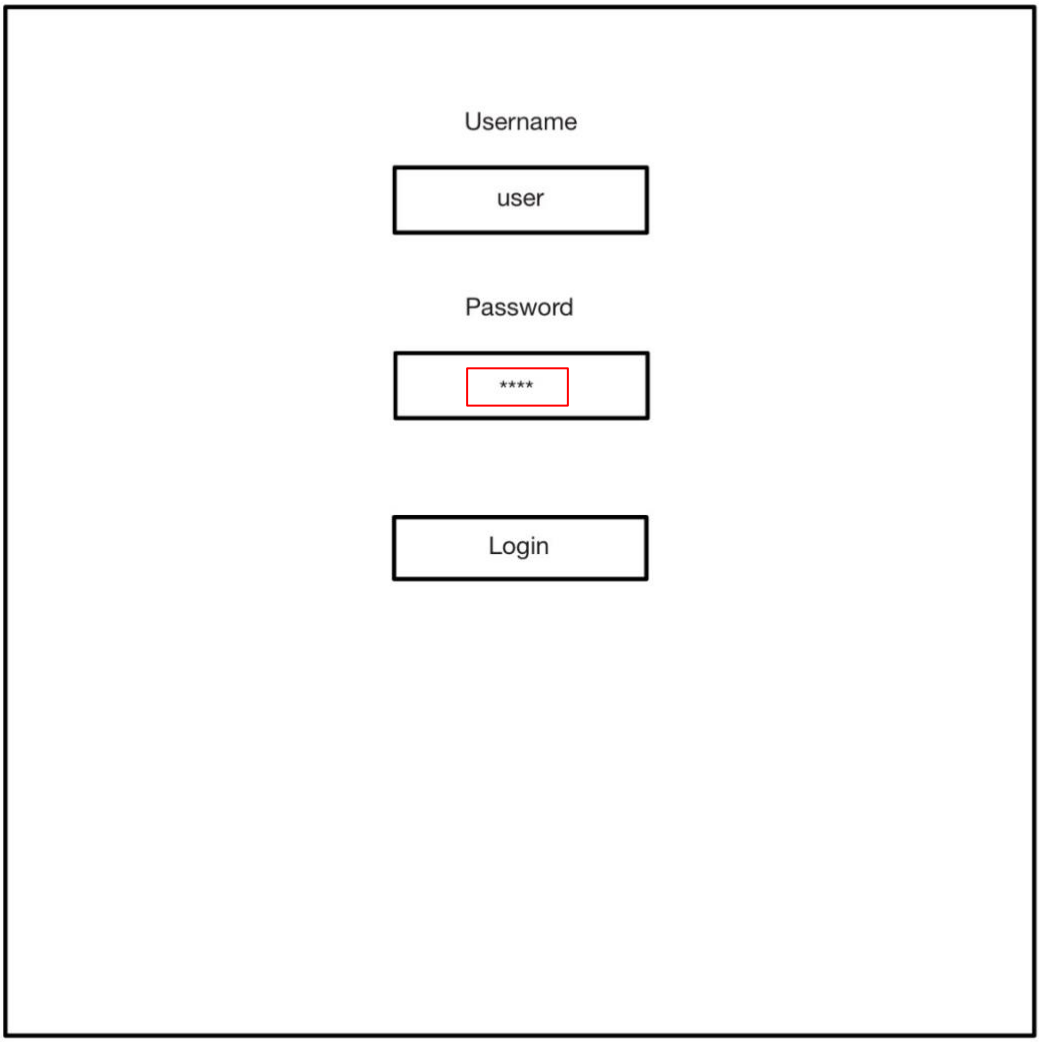
- Justification: Fewer steps reduces abandonment rate and helps to keep users logs consistent. Minimal features to prevent confusion for new / elderly users.

14. Consistent Button Placement

- Design: "Back to Main Menu" button always at bottom of each page
- Justification: Predictable navigation will reduce confusion for users

15. Input Field Masking

- Design: Password fields masked with *



- Justification: Masked passwords prevent shouldering.

Database Design

SQL Tables:

Database Structure (Normalisation Level)

The database has intentionally been normalised into second normal form (2NF). Every table meets the requirements of the first normal form (1NF) all the attributes have atomic values, and repeating groups do not exist. Additionally, every table uses a single attribute primary key, which prevents any partial dependencies hence, the second normal form conditions are met.

Justification for Using Second Normal Form

The database was deliberately normalised to second normal form rather than third normal form in order to reduce query complexity and improve performance.

The users table violates 3NF as it stores certain derived values meaning it has non key dependencies, such as BMI and daily calorie targets which have dependencies on non key attributes like height, weight, and age .This is to simplify data retrieval. This design choice is appropriate for the fitness tracking system, where these values are frequently accessed and recalculated in real time.

Below is the SQL code I used to create the database tables in SQLite:

```sql
CREATE TABLE Users (
    userId INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    passwordHash TEXT NOT NULL,
    name TEXT,
    age INTEGER CHECK(age > 0),
    gender TEXT CHECK(gender IN ('male', 'female', 'other')),
    heightCm REAL CHECK(heightCm > 0),
    initialWeight REAL CHECK(initialWeight > 0),
    activityLevel TEXT CHECK(activityLevel IN ('low', 'moderate', 'high')),
    goal TEXT CHECK(goal IN ('lose', 'maintain', 'gain'))
);

CREATE TABLE Food (
    foodId INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    calories REAL NOT NULL,
    protein REAL NOT NULL,
    carbs REAL NOT NULL,
    fat REAL NOT NULL
);

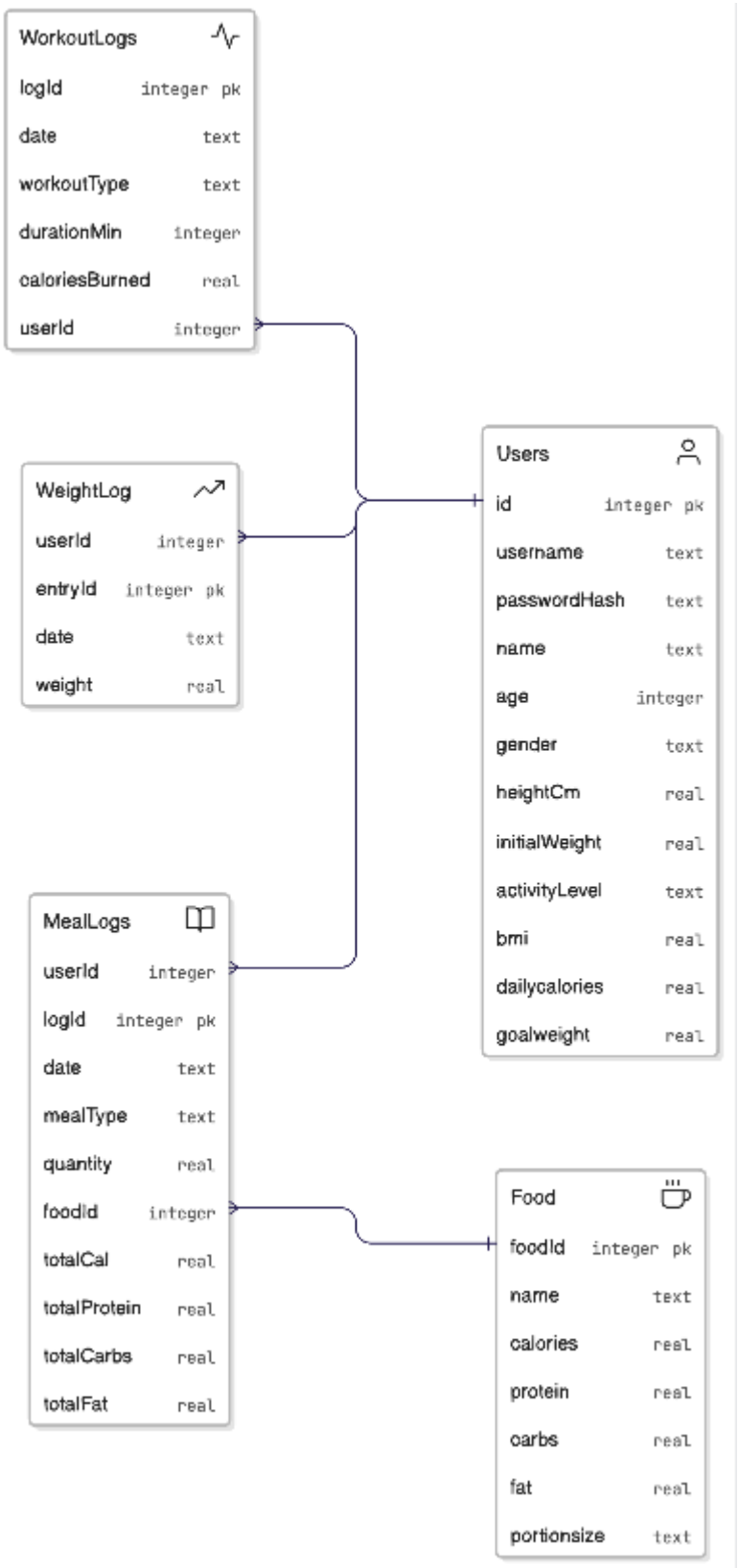
CREATE TABLE MealLogs (
    logId INTEGER PRIMARY KEY AUTOINCREMENT,
    userId INTEGER NOT NULL,
    date TEXT NOT NULL,
    mealType TEXT CHECK(mealType IN ('breakfast', 'lunch', 'dinner', 'snack')),
    foodId INTEGER NOT NULL,
    quantity REAL CHECK(quantity > 0),
    totalCal REAL,
    totalProtein REAL,
    totalCarbs REAL,
    totalFat REAL,
    FOREIGN KEY(userId) REFERENCES Users(userId),
    FOREIGN KEY(foodId) REFERENCES Food(foodId)
);
```

```sql
CREATE TABLE WorkoutLogs (
    logId INTEGER PRIMARY KEY AUTOINCREMENT,
    userId INTEGER NOT NULL,
    date TEXT NOT NULL,
    workoutType TEXT,
    durationMin INTEGER CHECK(durationMin > 0),
    caloriesBurned REAL,
    FOREIGN KEY(userId) REFERENCES Users(userId)
);

CREATE TABLE WeightLog (
    entryId INTEGER PRIMARY KEY AUTOINCREMENT,
    userId INTEGER NOT NULL,
    date TEXT NOT NULL,
    weight REAL CHECK(weight > 0),
    FOREIGN KEY(userId) REFERENCES Users(userId)
);
```

Below is the ER diagram showing the relationships between the tables in the database:

ER diagram for database structure:



Truth tables

Authentication menu truth tables

Considering the Sign Up/registration section is the most logic heavy section, I have made tables for the logical/mathematical operations that are going to be used during sign up to showcase how the system will validate inputs:

Registration:

1. Username validation:

Username Input	Username in Database	Result
"johndoe"	False	Accept, proceed to insert
"admin"	True	Show error "Username already exists", return
"newuser2024"	False	Accept, proceed to insert

2. Age validation:

Age Input	ageStr.isdigit()	Result
"25"	True	Accept, convert to int(25)
"30.5"	False	Show error "Please enter valid age", return
"abc"	False	Show error "Please enter valid age", return

3. Height validation:

Height Input	isValidFloat()	Result
"175"	True	Accept, convert to float(175.0)

"180.5"	True	Accept, convert to float(180.5)
"abc"	False	Show error "Please enter valid height", return

4. Weight / goal weight validation:

Weight Input	isValidFloat()	Result
"70"	True	Accept, convert to float(70.0)
"75.5"	True	Accept, convert to float(75.5)
"abc"	False	Show error "Please enter valid weight", return

6. Weeks validation:

Weeks Input	weeksStr.isdigit()	Result
"12"	True	Accept, convert to int(12)
"8"	True	Accept, convert to int(8)
"12.5"	False	Show error "Please enter valid weeks", return

7. Gender input (No Validation):

Gender Input	gender.upper()	Stored Value
"male"	"MALE"	"MALE"
"FEMALE"	"FEMALE"	"FEMALE"
"other"	"OTHER"	"OTHER"

8. BMI Calculation:

Weight	Height	BMI Formula	Result
70	175	70 / ((175/100)²)	22.86
80	180	80 / ((180/100)²)	24.69
90	185	90 / ((185/100)²)	26.30

9.TDEE Calculations :

Gender	BMR	Sedentary (x1.2)	Light (x1.375)	Moderate (x1.55)	Active (x1.725)	Very Active (x1.9)
MALE (BMR 1805)	1805	2166	2482	2798	3114	3430
FEMALE (BMR 1370)	1370	1644	1884	2124	2364	2603
MALE (BMR 1699)	1699	2039	2336	2633	2931	3228

If user enters an invalid activity level the multiplier is set to an automatic multiplier of x1.2

10. Daily calories calculation - Weight change adjustment:

Current Weight	Goal Weight	Weeks	Weight Change	Daily Adjustment
80	75	12	-5kg	(-5 * 7700) / (12 * 7) = -458 cal/day

70	75	10	+5kg	(5 * 7700) / (10 * 7) = +550 cal/day
90	85	16	-5kg	(-5 * 7700) / (16 * 7) = -344 cal/day

If a user's calculated daily calories is <1200 their daily calories will be set to 1200 to ensure a safe minimum.

Login:

Login validation:

Username Input	Password Input	Username in Database	Password Hash Match	Result
"john_d"	"Pass123!"	True	True	Login success, navigate to Main Menu
"john_d"	"wrongpass"	True	False	Show error "Invalid credentials", return
"unknown"	"Pass123!"	False	N/A	Show error "Invalid credentials", return
""	"Pass123!"	False	N/A	Show error "Invalid credentials", return

Main menu truth tables

The main menu has no validation as it contains no user input. It will consist of solely navigation buttons to direct users to other sections of the program so it does not require any validation or truth tables.

Log food truth tables

Food Search Validation:

Search Input	Search Term Empty	Results List Empty	Result
"APPLE"	False	False	Display food with name, calories, protein, carbs, fat in the foodsearchlistbox
"MANGO"	False	True	Display "No foods found" in foodsearchlistbox
""	True	N/A	Display "No foods found" in foodsearchlistbox
"app"	False	True	Display "No foods found"

Food Selection Validation:

Food Selected in Listbox	"NAME: " in Selection	Result
Yes, valid food selected	True	Food logged to meal, logId pushed onto stack, display refreshed
No selection made	N/A	Silently returns, nothing logged

Undo Validation:

Stack Empty	Result
False	Pop last logId, delete from MealLogs table, refresh display
True	Nothing happens, no error shown

Log workout truth tables

Duration Validation:

Duration Input	Is it an integer?	Is it > 0?	Result
"30"	Yes	Yes	Accept, convert to int(30)
"thirty"	No	N/A	Show error "Please enter valid numeric values for duration"
"-30"	No	No	Show error "Duration must be a positive number"
"0"	Yes	No	Show error "Duration must be a positive number"
""	N/A	N/A	N/A (assume user is not logging a workout and they accidently clicked)

Calories Burned Validation:

Calories Input	Is it an integer?	Is it > 0?	Result
"300"	Yes	Yes	Accept, convert to int(300)
"three hundred"	No	N/A	Show error "Please enter valid numeric values for calories"
"-100"	Yes	No	Show error "Calories burned must be a positive number"

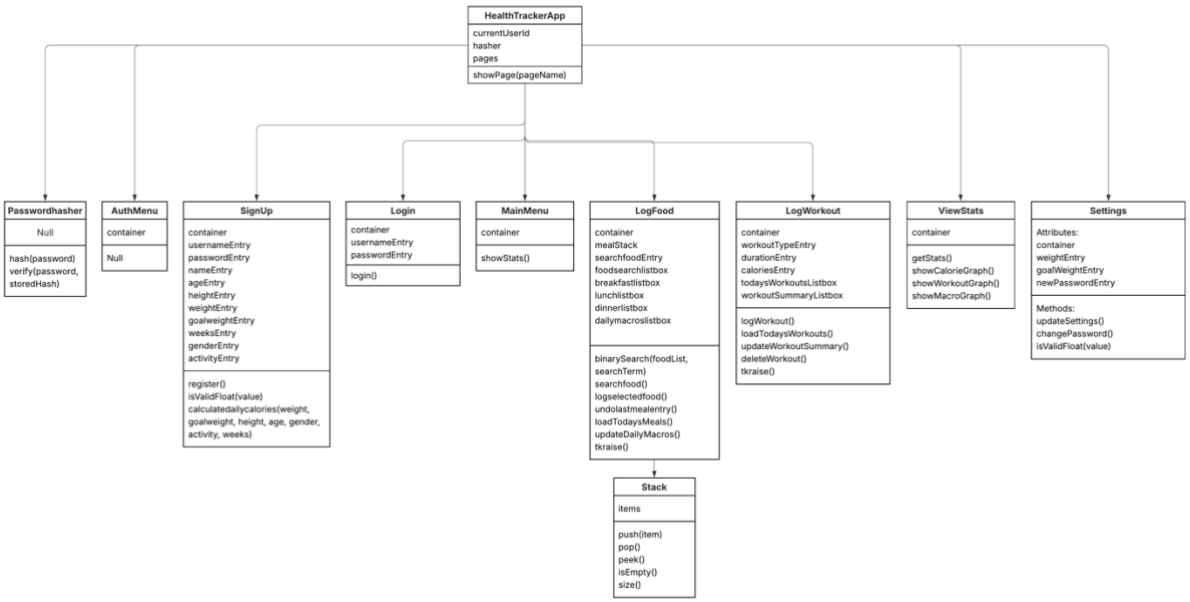
View stats truth tables

View stats does not have any validation as it only displays data from the database.There is no user inputs and therefore there is no validation.

Settings truth tables

Settings has elements from registration (weight ,goalweight) so the logic is the same for those elements. Settings also has a change password feature but this section has no logic as there is no invalid password a user could enter. I deliberately chose to not enforce rules such as “minimum length” and “must contain numbers/ special characters” for passwords as it allows for flexibility. These validations exist on other systems for security reasons but since this project is not public this is not necessary. This could be changed easily in future development if it is deemed necessary.

Class diagram



Classes for each module:

Below shows which module each class falls into:

Authentication Menu module

- AuthMenu
- SignUp
- Login

Main Menu module

- MainMenu

Log Food module

- LogFood
- Stack

Log Workout module

- LogWorkout

View stats module

- ViewStats

Settings module

- Settings

Justification of Class Design:

The application has been divided into separate class each with their own specific purpose. I have decided that each page of the application should be implemented as its own class inheriting from tk.Frame. This will allow each page to be treated as its own and can be brought to the front by tkraise().

This design allows me to continue my modular structure as I have shown in the decomposition diagram. The HealthTrackerApp class will act as the central point and will manage the navigation between each of the different pages using the dictionary I decided on earlier.

Buttons

I decided to not include buttons as attributes in my class diagram because they will only be used to navigate and call methods, including them would just make the diagram more complex to understand and they don't really provide any additional information about the structure of my classes.

Listboxes

I did decide to include listboxes as attributes in my class diagrams to visualise how and in which pages listboxes will be used.

Relationships

The HealthTrackerApp will inherit from tk.Tk (this is a built in Tkinter class that creates the main application window). All other classes inherit from tk.Frame (this is a tkinter class that creates containers for widgets).

The HealthTrackerApp is connected to all of the other pages. It will create an instance of every class and store them in the pages dictionary. This means that HealthTrackerApp manages all the classes and is responsible for displaying the correct page.

The LogFood class will contain an instance of Stack for the undo functionality.

Now that all key Classes, their attributes and their methods have been defined I will move onto the key variables within these classes.

Key Classes and Variables

SignUp Class

Variable Name	Data Type	Purpose
username	String	Username retrieved from entry field
password	String	Password retrieved from entry field
hashed	String	SHA-256 hash of the password
name	String	Name retrieved from entry field
age	Integer	Validated age converted from string
height	Float	Validated height in cm
weight	Float	Validated weight in kg
goalweight	Float	Validated goal weight in kg
weeks	Integer	Validated timeframe in weeks
gender	String	Gender converted to uppercase
activity	String	Activity level converted to uppercase
userbmi	Float	Calculated BMI
bmr	Float	Basal metabolic rate (Mifflin-St Jeor)
tdee	Float	Total daily energy expenditure

dailycalories	Float	The users final daily calorie target

Login Class

Variable Name	Data Type	Purpose
username	String	Username retrieved from entry field
password	String	Password retrieved from entry field
result	Tuple	Database query result containing then users id and passwordHash

LogFood Class

Variable Name	Data Type	Purpose
searchedfood	String	Search term converted to uppercase
allFoods	List	All foods retrieved from database sorted alphabetically
left	Integer	Left boundary of binary search space
right	Integer	Right boundary of binary search space
mid	Integer	Middle index of current search space
midFood	String	Food name at current mid position
results	List	Matching food items returned by binary search
foodId	Integer	ID of selected food from database
calories	Float	Calorie value of selected food
protein	Float	Protein value of selected food

carbs	Float	Carbohydrate value of selected food
fat	Float	Fat value of selected food
todayDate	String	Today's date (YYYY-MM-DD)
quantity	Float	Serving quantity (defaulted to 1.0)
totalCal	Float	Total calories for the serving
totalProtein	Float	Total protein for the serving
totalCarbs	Float	Total carbs for the serving
totalFat	Float	Total fat for the serving
dailycalories	Float	User’s daily calorie target
caloriesRemaining	Integer	Remaining calories for the day

Log Food Listbox Variables

Variable Name	Data Type	Purpose
foodsearchlistbox	tk.Listbox	Displays food search results
breakfastlistbox	tk.Listbox	Displays breakfast meals
lunchlistbox	tk.Listbox	Displays lunch meals
dinnerlistbox	tk.Listbox	Displays dinner meals
dailymacroslistbox	tk.Listbox	Displays daily calorie and macro totals

Stack Class

Variable Name	Data Type	Purpose	Validation and Justification
items	List of integers	Stores meal log IDs	Python list provides efficient append and pop operations

LogWorkout Class

Variable Name	Data Type	Purpose
workoutType	String	Workout type retrieved from entry field
duration	Integer	Validated duration in minutes
caloriesBurned	Integer	Validated calories burned
todayDate	String	Today's date (YYYY-MM-DD)
totalDuration	Integer	Total workout duration today
totalCalories	Integer	Total calories burned today
workoutCount	Integer	Number of workouts logged today

Log Workout Listbox Variables

Variable Name	Data Type	Purpose
todaysWorkoutsListbox	tk.Listbox	Displays today's workouts
workoutSummaryListbox	tk.Listbox	Displays workout totals

ViewStats Class

Variable Name	Data Type	Purpose
dates	List (String)	Last 7 dates (MM-DD)
calories	List (Integer)	Daily calorie intake (7 days)
minutes	List (Integer)	Daily workout duration (7 days)
protein	Float	Today's protein intake
carbs	Float	Today's carbohydrate intake
fat	Float	Today's fat intake
todayDate	String	Today's date (YYYY-MM-DD)

Settings Class

Variable Name	Data Type	Purpose
currentWeight	Float	Current weight
currentGoalWeight	Float	Goal weight
height	Float	User height
age	Integer	User age

gender	String	User gender
activity	String	Activity level
newBMI	Float	Recalculated BMI
bmr	Float	Recalculated BMR
tdee	Float	Recalculated TDEE
weight_change	Float	Difference between goal and current weight
daily_adjustment	Float	Daily calorie adjustment
dailycalories	Float	Updated daily calorie target
hashed	String	SHA-256 hash of new password

Calculated Variables (Algorithms)

Daily calorie calculation

Variable Name	Data Type	Purpose
bmr	Float	Basal Metabolic Rate
tdee	Float	Total daily energy expenditure
dailycalories	Float	Final daily calorie target

Binary search variables

Variable Name	Data Type	Purpose
left, right, mid	Integer	Control search boundaries

Test data

Development test data:

This test data is going to be used during the iterative development of the solution.

Account Testing (iteration-1)

Objective	Test	Input	Expected Output
1.1 Create account	Create new user	username john_d, password Pass123!, name John Doe, age 17, height 175, weight 70, gender MALE, activity MODERATE, goal 65, weeks 10	Account created, user stored in Users
1.2 Create account	Create second user	username jane_s, password Jane456!, name Jane Smith, age 18, height 160, weight 60, gender FEMALE, activity LIGHT, goal 55, weeks 12	Account created, user stored in Users
1.3 Create account	Duplicate username	username john_d again	Error “Username already exists.”
1.4 Login	Correct credentials	username john_d, password Pass123!	Login success, main menu shown
1.5 Login	Wrong password	username john_d, password wrong	Error “Invalid credentials.”
1.6 Hashing	Check stored password	DB check for john_d password . Compare hashes	Password stored as SHA-256 hash

1.7 Change password	Update password	new password NewPass123!	Password hash updated, login works with new password

Food Logging Testing (iteration-2)

Objective	Test	Input	Expected Output
2.1 Search food	Exact search	APPLE	Food shown with macros
2.2 Search food	Partial search	APP	“No foods found.”
2.3 Binary search	Sorted DB search	Any exact food name	Result found via binary search
2.4 Log food	Log lunch	Select CHICKEN BREAST, lunch	Entry appears under lunch list
2.5 Food display	Nutritional display	Log any food	List shows calories, protein, carbs, fat
2.6 Undo last meal	Undo	After logging food, click Undo	Last meal entry removed
2.7 Undo after restart	Restart app, click Undo	None	No change (stack empty)
2.8 Auto Load meals	Open Log Food page	Existing meals today	Meals shown automatically

Workout Testing (iteration-3)

Objective	Test	Input	Expected Output

3.1 Log workout	Add workout	Type Running, duration 30, calories 300	Workout logged
3.2 Validation	Non-numeric duration	duration 30.5	Rejected
3.3 Validation	Negative calories	calories -100	Rejected
3.4 Summary	Auto totals	Multiple workouts today	Summary totals shown
3.5 Delete workout	Delete by list	Select workout, delete	Entry removed

Statistics and Graph Testing (iteration-4)

Objective	Test	Input	Expected Output
4.1 Graph calories	7 day data	Meals across 7 days	Bar chart shows 7 days
4.2 Graph workouts	7 day data	Workouts across 7 days	Bar chart shows 7 days
4.3 Macro pie	Today macros	Meals today	Pie chart displays macros
4.4 Stats	View stats	Open View Stats	Name, weight, height, BMI shown

Settings and UI Testing (iteration-5)

Objective	Test	Input	Expected Output
5.1 Update weight	Settings	New weight 72	Weight updated
5.2 Update goal	Settings	Goal 68	Goal updated

5.3 Validation	Invalid weight	70kg	Rejected
5.4 Navigation	Main menu	Click each button	Each page opens
5.5 Back buttons	Back navigation	Back from each page	Returns to main menu
5.6 Styling	UI check	Open app	Consistent fonts/colours
5.7 Readable lists	UI check	Meals/workouts list	Readable formatted text
5.8 Window	UI check	Launch app	Title correct
5.9 Window	Ui check	Launch app	Size correct

Database Testing

Objective	Test	Input	Expected Output
6.1 Users table	Check table has the required fields	DB schema	Users table has required fields
6.2 Food table	Check table has the required fields	DB schema	Food table has required fields
6.3 MealLogs table	Check table has the required fields	DB schema	MealLogs table has required fields
6.4 WorkoutLogs table	Check table has the required fields	DB schema	WorkoutLogs table has required fields

Post development Test Data

After I have developed my program I will need to test it to ensure the experience is of standard for users. I have planned the test before starting the development so that it can be used to check if the criteria has been fully met as I initially defined.

Objective	**Expected Output**	**Actual Outcome**
1.1 **The User can create an account**	Account created, user stored in Users table	
1.2 The User can Login	Login success, main menu shown	
1.3 Multiple accounts can be created	Successful, multiple accounts have been created	
1.4 The Users Password is hashed	Password stored as SHA-256 hash (64-character hex, not plain text)	
1.5 The User can change password	Password hash updated, login works with new password	
2.1 The User can search food	Food shown with name, calories, protein, carbs, fat	

2.2 Binary search is used for lookup of foods	Binary search is used to search for foods	
2.3 The User can log food for breakfast	Entry appears under Breakfast listbox	
2.3 The User can log food for lunch	Entry appears under Lunch listbox	
2.3 The User can log food for dinner	Entry appears under Dinner listbox	
2.4 Nutritional display	Listbox shows: NAME, CALORIES, P: Xg, C: Xg, F: Xg	
2.5 The User can undo meal entries	Most recently logged meal removed from listbox and database	
2.6 Auto load meals	All today's meals automatically displayed	
3.1 The User can view daily calories	Total calories for today displayed	

3.2 Macros display	Protein, carbs and fat totals shown in grams	
3.3 The User can view calories remaining	Calories remaining = daily goal minus consumed	
3.4 Daily goal calculation	Daily calorie goal calculated from BMR, TDEE and weekly adjustment	
3.5 Recalculate on update of weight	Daily calorie target automatically recalculated	
3.6 BMI update	BMI recalculated and updated	
4.1 The User can log workouts	Workout logged and appears in Today's Workouts listbox	
4.2 The User can view their workout summary	Total workouts: 2, Duration: 75 min, Calories burned: 700	
4.3 The User can view their workout details	Listbox shows type, duration, calories and ID	

4.4 The User can delete a workout	Workout removed from listbox and database	
4.5 Validation	Error "Please enter valid numeric values for duration"	
4.5 Validation	Error "Calories must be a positive number"	
5.1 The User can view Calorie graph	Bar chart with 7 bars and MM-DD date labels	
5.2 The User can view Workout graph	Bar chart with 7 bars showing duration in minutes	
5.3 The User can view Macro pie chart	Pie chart with Protein, Carbs, Fat sections and percentages	
5.4 User statistics are shown	Listbox shows Name, weight, height and BMI	
5.5 Graphs use timedelta to calculate dates for past week	Graphs display values for last week (using timedelta)	

6.1 Users table	Users table has all required fields including bmi and dailycalories	
6.2 Food table	Food table has foodId, name, calories, protein, carbs, fat	
6.3 MealLogs table	MealLogs has logId, userId, date, mealType, foodId, quantity, totals	
6.4 WorkoutLogs table	WorkoutLogs has logId, userId, date, workoutType, durationMin, caloriesBurned	
6.5 Information can be stored, retrieved, and deleted from the database	All operations complete successfully	
6.6 Referential integrity	Data linked correctly via foreign keys	
6.7 Second Normal Form	All tables use single primary keys, no partial dependencies	

7.1 Update weight	Weight updated to 72 in database	
7.2 Update goal weight	Goal weight updated to 68 in database	
7.3 Calorie recalculation	Daily calorie target automatically recalculated	
7.4 BMI recalculation	BMI recalculated and stored	
7.5 Change password	Password hash updated in database	
7.6 Input validation	Error "Please enter valid weight", input rejected	
8.1 The User can navigate the main menu	Each page opens correctly	
8.2 Back buttons	Each back button returns to Main Menu	
8.3 Consistent styling	Consistent fonts, colours and layout throughout	

8.4 Readable lists	Formatted text e.g. "NAME: X CALORIES: Y P: Zg"	
8.5 Window title and size	Title "Health & Fitness Tracker", consistent 600×500 dimensions	

Usability Testing

After developing the program I will get potential users of the software to test it and get their feedback .This will help me identify any missing functionality that the app may have.

This will also allow me to see if I have met usability criteria as well as see if the application solves the issue it was built to solve.

I will interview candidates and use this set of questions that were specifically designed to understand how users find the usability and function of the program.

Questions:

1. Were you able to use the app to log your food, workouts, and personal details successfully?

Justification: This question tests whether users can complete the main tasks the app was designed for.

2. Did the app help you keep track of your calories, macros, and fitness progress more easily?

Justification: This is useful because it tests whether the app is actually helpful and makes tracking simpler for the user.

3. Was the application easy to use?

Justification: This gives an overall judgement on usability and shows whether the interface is simple and user friendly.

4. Was it clear what each part of the application was for?

Justification: This tests whether the layout and navigation were understandable and whether users could tell what each feature did.

5. Which feature did you find most helpful when using the app?

Justification: This helps identify which parts of the app were most useful and successful from the user's point of view.

6. Were there any problems or issues you encountered while using the app?

Justification: This is important because it helps reveal usability problems or any faults in the program.

7. Were there any features you think would improve the app further?

Justification: This is useful because it gathers feedback for future improvements and shows whether the app fully met the users needs.

8. Would you use this app again, or could you see yourself using it in the long term?

Justification: This tests overall satisfaction and whether the app is practical enough for repeated everyday use.

9. Do you have any other comments or suggestions?

Justification: This gives users the opportunity to mention anything else that was not covered by the other questions.

Iterative development process

Iteration 1 - Authentication System

This iteration implements the Authentication Menu module that was identified in the decomposition diagram. This module is responsible for user registration and login.

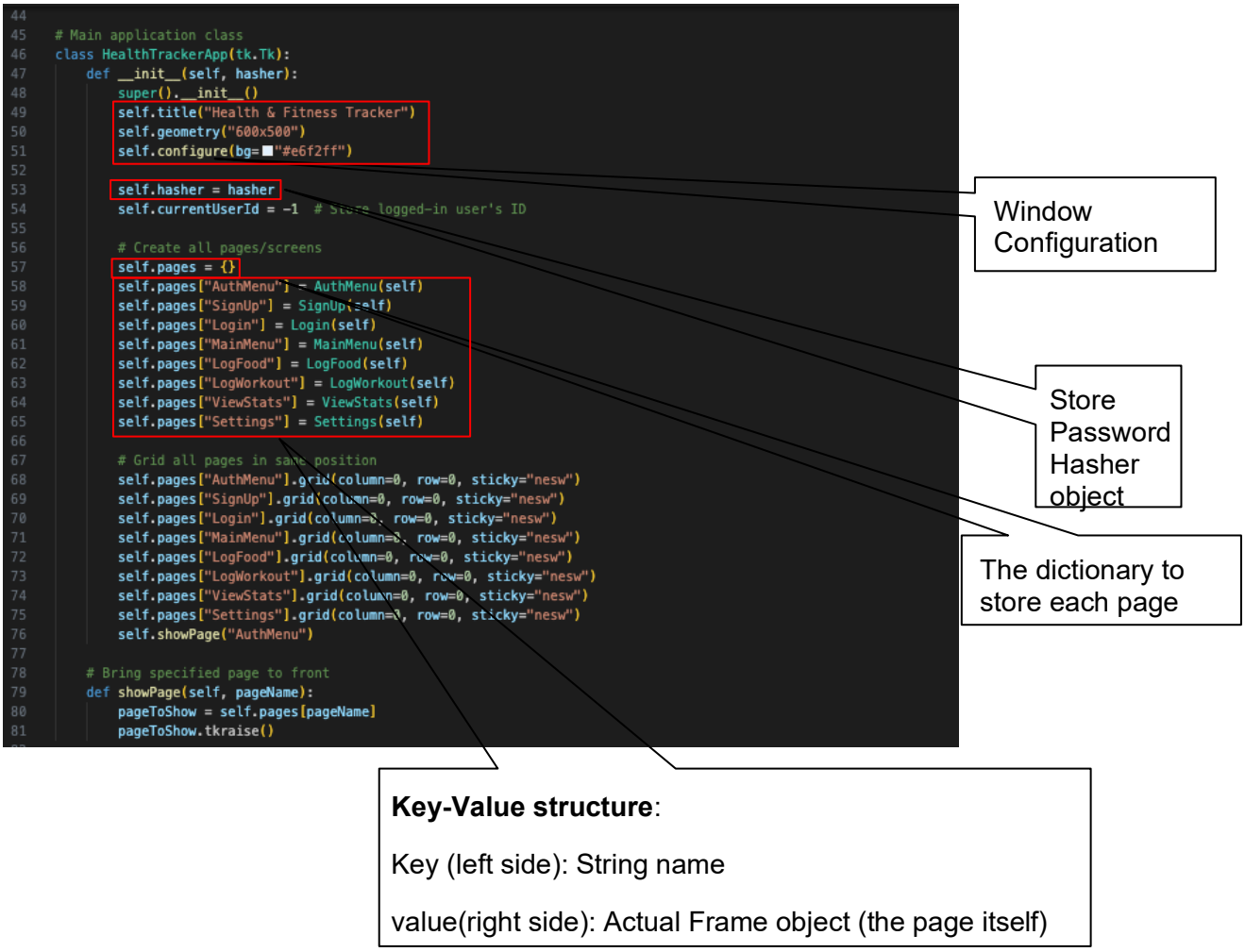
Development

I began by Integrating all of the required libraries to support database operations, hashing, graphing and the GUI.

I configured and tested the database connection, to ensure there was a reliable communication between the application and the backend database.



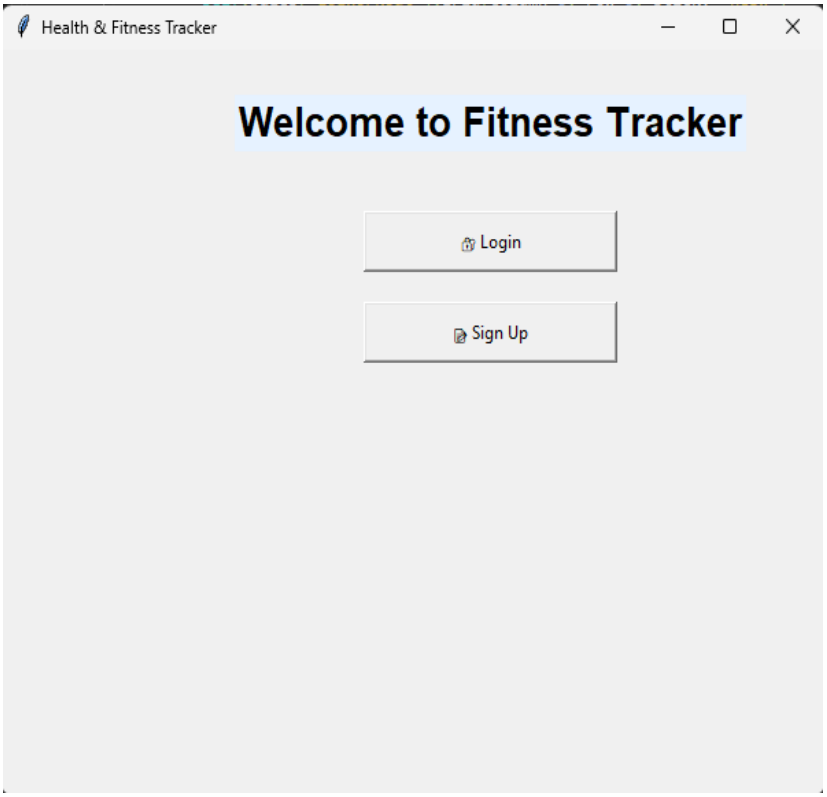
I then implemented the primary application class to serve as the central entry point for the system.



I then created the authentication menu to provide an entry point for users, enabling login and registration.

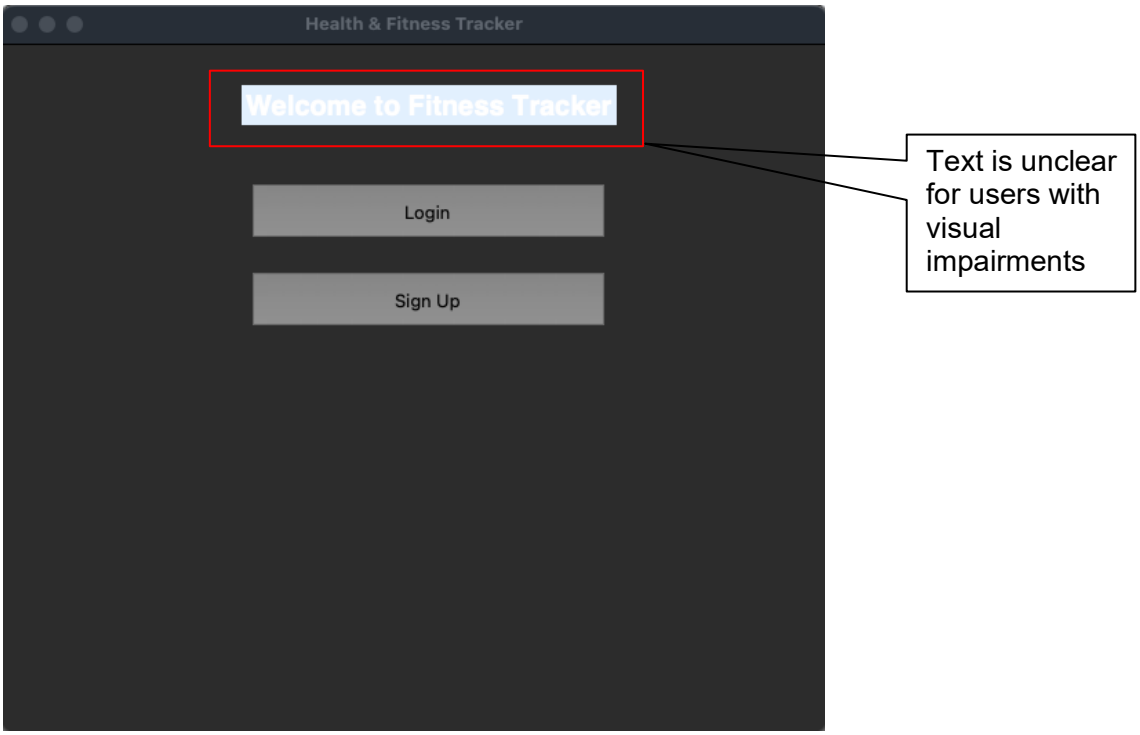
Original code:

```
# Initial login/signup screen
class AuthMenu(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        tk.Label(self, text="Welcome to Fitness Tracker", font=("Helvetica", 20, "bold"), bg="#e6f2ff").pack(pady=30)
        tk.Button(self, text="Login", width=25, height=2, command=lambda: container.showPage("Login")).pack(pady=10)
        tk.Button(self, text="Sign Up", width=25, height=2, command=lambda: container.showPage("SignUp")).pack(pady=10)
```



GUI Colour Issue:

When coding the Authentication menu screen I was testing with different colour backrounds to see if any lower contrast colours would make it easier to read but whilst testing the GUI I realised the blue background colour of the text “Welcome to Fitness Tracker” could make the experience less favourable on devices with dark mode settings.

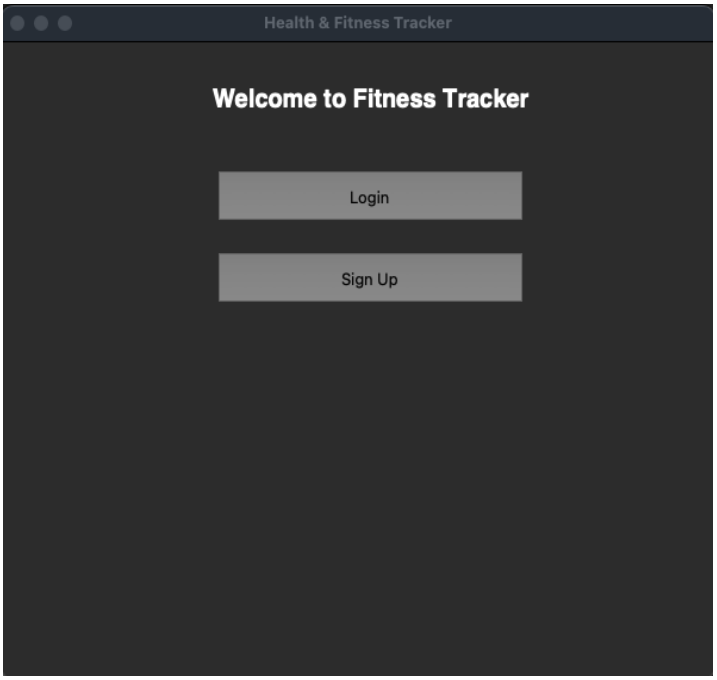


I decided to remove this colour from the authentication menu as it would allow better usability for users who may have visual impairments.

Edited code:

```
# Initial login/signup screen
class AuthMenu(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        tk.Label(self, text="Welcome to Fitness Tracker", font=("Helvetica", 20, "bold")).pack(pady=30)
        tk.Button(self, text="Login", width=25, height=2, command=lambda: container.showPage("Login")).pack(pady=10)
        tk.Button(self, text="Sign Up", width=25, height=2, command=lambda: container.showPage("SignUp")).pack(pady=10)
```

How the authentication menu screen looks after :



Now that the Main authentication menu screen is finished I decided to move on to the sign up frame.

The sign up frame :

```python
class SignUp(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.usernameLabel = tk.Label(self, text="Username")
        self.usernameLabel.pack()
        self.usernameEntry = tk.Entry(self)
        self.usernameEntry.pack()
        self.passwordLabel = tk.Label(self, text="Password")
        self.passwordLabel.pack()
        self.passwordEntry = tk.Entry(self)
        self.passwordEntry.pack()
        self.passwordEntry.config(show="*")
        self.nameLabel = tk.Label(self, text="Name")
        self.nameLabel.pack()
        self.nameEntry = tk.Entry(self)
        self.nameEntry.pack()
        self.ageLabel = tk.Label(self, text="Age")
        self.ageLabel.pack()
        self.ageEntry = tk.Entry(self)
        self.ageEntry.pack()
        self.genderLabel = tk.Label(self, text="Gender")
        self.genderLabel.pack()
        self.genderEntry = tk.Entry(self)
        self.genderEntry.pack()
        self.heightLabel = tk.Label(self, text="Height (cm)")
        self.heightLabel.pack()
        self.heightEntry = tk.Entry(self)
        self.heightEntry.pack()
        self.weightLabel = tk.Label(self, text="Weight (kg)")
        self.weightLabel.pack()
        self.weightEntry = tk.Entry(self)
        self.weightEntry.pack()
        self.activityLabel = tk.Label(self, text="Activity Level (Sedentary, Light, Moderate, Active, Very Active)")
        self.activityLabel.pack()
        self.activityEntry = tk.Entry(self)
        self.activityEntry.pack()
        self.goalLabel = tk.Label(self, text="Goal (e.g. Lose Weight, Maintain, Gain Muscle)")
        self.goalLabel.pack()
        self.goalEntry = tk.Entry(self)
        self.goalEntry.pack()
        self.goalweightLabel = tk.Label(self, text="Goal weight (kg)")
        self.goalweightLabel.pack()
        self.goalweightEntry = tk.Entry(self)
        self.goalweightEntry.pack()
        self.weeksLabel = tk.Label(self, text="the number of weeks you want to take to reach your goal weight within")
        self.weeksLabel.pack()
        self.weeksEntry = tk.Entry(self)
        self.weeksEntry.pack()
```

Database Table Structure Issue:

I was facing issues with Database so I modified the users table by shifting the logic from the database layer into the code. This approach improves the flexibility of my code as it would be easier to change the logic in python and it still works to ensure data integrity.

```sql
CREATE TABLE Users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    passwordHash TEXT NOT NULL,
    name TEXT,
    age INTEGER,
    gender TEXT,
    heightCm REAL,
    initialWeight REAL,
    activityLevel TEXT,
    goal TEXT,
    bmi REAL,
    dailycalories REAL,
    goalweight REAL
);
```

Redundant Goal Variable Issue:

Whilst trying to make a test account I realised that maintaining a separate goal variable was unnecessary, as it introduced logical conflicts when user inputs were inconsistent (e.g. if a user selected “lose weight” while setting a goal weight greater than the current weight). Removing this variable simplified the logic and improved data integrity across the application.

```
ALTER TABLE Users
DROP COLUMN goal;

```

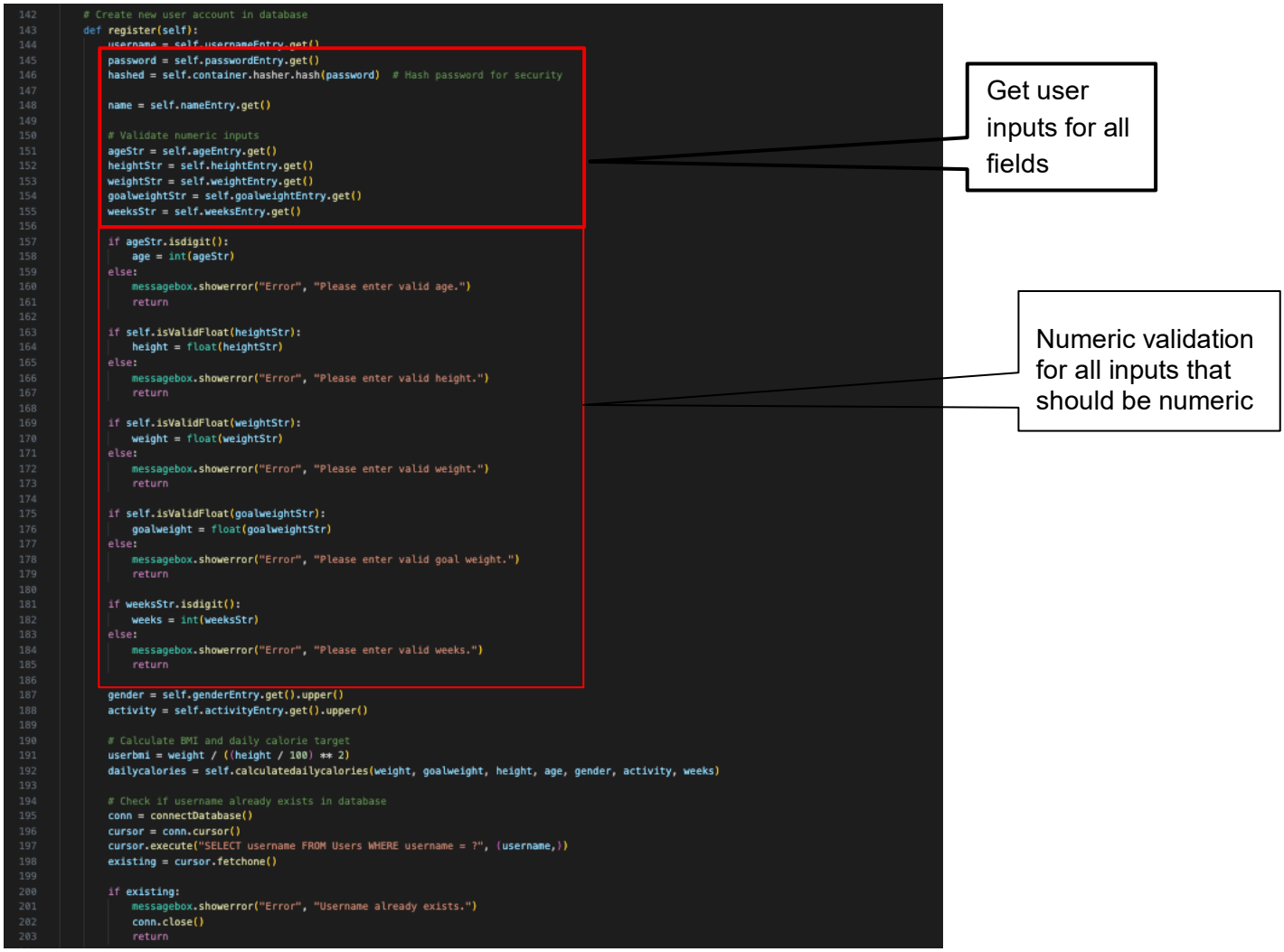
I removed the goal variable to make the sign up process easier for the user.

Sign up class without the goal variable:

```
# User registration screen
class SignUp(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        # Create input fields for user details
        self.usernameLabel = tk.Label(self, text="Username")
        self.usernameLabel.pack()
        self.usernameEntry = tk.Entry(self)
        self.usernameEntry.pack()
        self.passwordLabel = tk.Label(self, text="Password")
        self.passwordLabel.pack()
        self.passwordEntry = tk.Entry(self)
        self.passwordEntry.pack()
        self.passwordEntry.config(show="*")  # Hide password text
        self.nameLabel = tk.Label(self, text="Name")
        self.nameLabel.pack()
        self.nameEntry = tk.Entry(self)
        self.nameEntry.pack()
        self.ageLabel = tk.Label(self, text="Age")
        self.ageLabel.pack()
        self.ageEntry = tk.Entry(self)
        self.ageEntry.pack()
        self.genderLabel = tk.Label(self, text="Gender")
        self.genderLabel.pack()
        self.genderEntry = tk.Entry(self)
        self.genderEntry.pack()
        self.heightLabel = tk.Label(self, text="Height (cm)")
        self.heightLabel.pack()
        self.heightEntry = tk.Entry(self)
        self.heightEntry.pack()
        self.weightLabel = tk.Label(self, text="Weight (kg)")
        self.weightLabel.pack()
        self.weightEntry = tk.Entry(self)
        self.weightEntry.pack()
        self.activityLabel = tk.Label(self, text="Activity Level (Sedentary, Light, Moderate, Active, Very Active)")
        self.activityLabel.pack()
        self.activityEntry = tk.Entry(self)
        self.activityEntry.pack()
        self.goalweightLabel = tk.Label(self, text="Goal weight (kg)")
        self.goalweightLabel.pack()
        self.goalweightEntry = tk.Entry(self)
        self.goalweightEntry.pack()
        self.weeksLabel = tk.Label(self, text="the number of weeks you want to take to reach your goal weight within")
        self.weeksLabel.pack()
        self.weeksEntry = tk.Entry(self)
        self.weeksEntry.pack()
        self.submitButton = tk.Button(self, text="Sign Up", command=lambda: self.register()).pack(pady=10)
```

register() subroutine:




```
# Create new user account in database
def register(self):
    username = self.usernameEntry.get()
    password = self.passwordEntry.get()
    hashed = self.container.hasher.hash(password)  # Hash password for security

    name = self.nameEntry.get()

    # Validate numeric inputs
    ageStr = self.ageEntry.get()
    heightStr = self.heightEntry.get()
    weightStr = self.weightEntry.get()
    goalweightStr = self.goalweightEntry.get()
    weeksStr = self.weeksEntry.get()

    if ageStr.isdigit():
        age = int(ageStr)
    else:
        messagebox.showerror("Error", "Please enter valid age.")
        return

    if self.isValidFloat(heightStr):
        height = float(heightStr)
    else:
        messagebox.showerror("Error", "Please enter valid height.")
        return

    if self.isValidFloat(weightStr):
        weight = float(weightStr)
    else:
        messagebox.showerror("Error", "Please enter valid weight.")
        return

    if self.isValidFloat(goalweightStr):
        goalweight = float(goalweightStr)
    else:
        messagebox.showerror("Error", "Please enter valid goal weight.")
        return

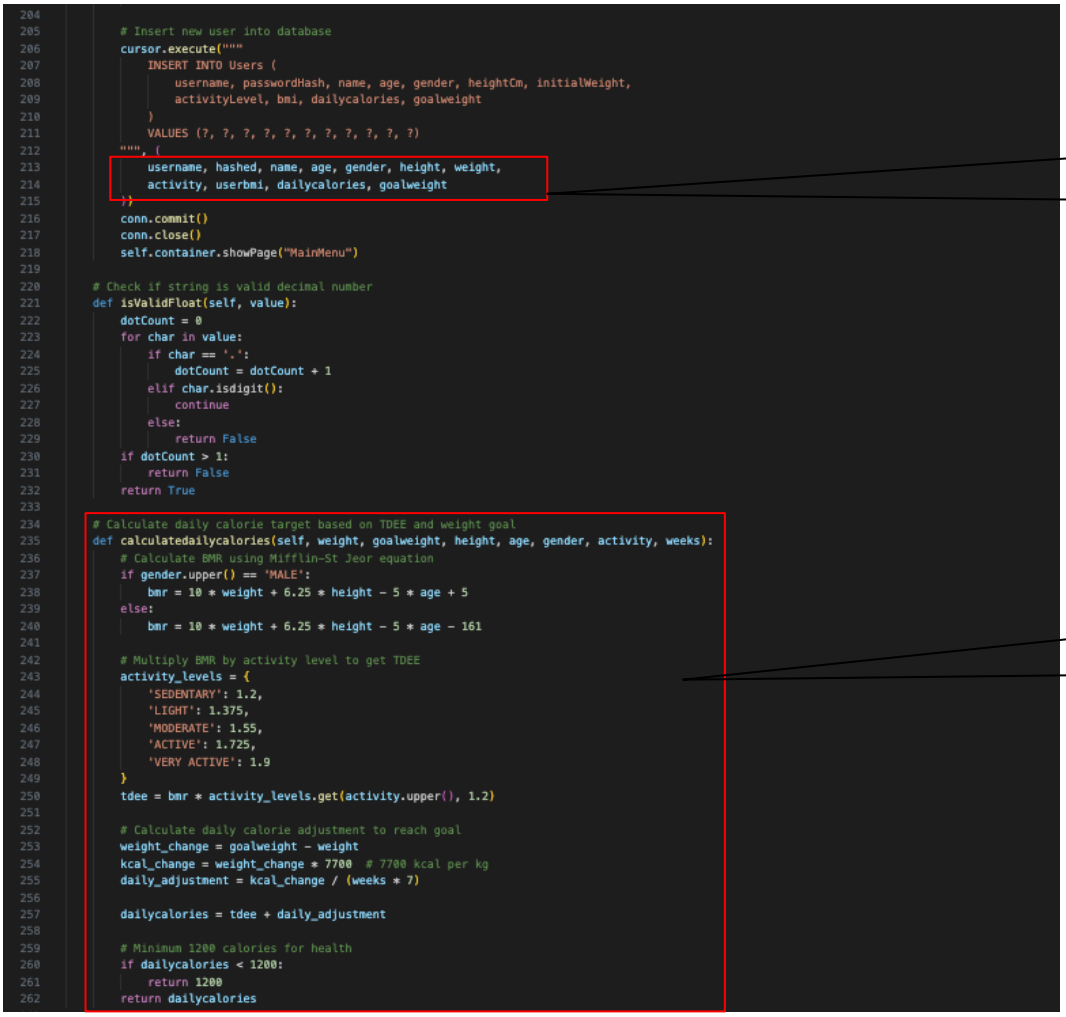
    if weeksStr.isdigit():
        weeks = int(weeksStr)
    else:
        messagebox.showerror("Error", "Please enter valid weeks.")
        return

    gender = self.genderEntry.get().upper()
    activity = self.activityEntry.get().upper()

    # Calculate BMI and daily calorie target
    userbmi = weight / ((height / 100) ** 2)
    dailycalories = self.calculatedailycalories(weight, goalweight, height, age, gender, activity, weeks)

    # Check if username already exists in database
    conn = connectDatabase()
    cursor = conn.cursor()
    cursor.execute("SELECT username FROM Users WHERE username = ?", (username,))
    existing = cursor.fetchone()

    if existing:
        messagebox.showerror("Error", "Username already exists.")
        conn.close()
        return
```



```
        # Insert new user into database
        cursor.execute("""
            INSERT INTO Users (
                username, passwordHash, name, age, gender, heightCm, initialWeight,
                activityLevel, bmi, dailycalories, goalweight
            )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            username, hashed, name, age, gender, height, weight,
            activity, userbmi, dailycalories, goalweight
        ))
        conn.commit()
        conn.close()
        self.container.showPage("MainMenu")

    # Check if string is valid decimal number
    def isValidFloat(self, value):
        dotCount = 0
        for char in value:
            if char == '.':
                dotCount = dotCount + 1
            elif char.isdigit():
                continue
            else:
                return False
        if dotCount > 1:
            return False
        return True

    # Calculate daily calorie target based on TDEE and weight goal
    def calculatedailycalories(self, weight, goalweight, height, age, gender, activity, weeks):
        # Calculate BMR using Mifflin-St Jeor equation
        if gender.upper() == 'MALE':
            bmr = 10 * weight + 6.25 * height - 5 * age + 5
        else:
            bmr = 10 * weight + 6.25 * height - 5 * age - 161

        # Multiply BMR by activity level to get TDEE
        activity_levels = {
            'SEDENTARY': 1.2,
            'LIGHT': 1.375,
            'MODERATE': 1.55,
            'ACTIVE': 1.725,
            'VERY ACTIVE': 1.9
        }
        tdee = bmr * activity_levels.get(activity.upper(), 1.2)

        # Calculate daily calorie adjustment to reach goal
        weight_change = goalweight - weight
        kcal_change = weight_change * 7700  # 7700 kcal per kg
        daily_adjustment = kcal_change / (weeks * 7)

        dailycalories = tdee + daily_adjustment

        # Minimum 1200 calories for health
        if dailycalories < 1200:
            return 1200
        return dailycalories
```

Inserting all values into users table (including all calculated values)

Carrying out all necessary calculations (BMR ,TDEE,BMI)

I implemented the functionality to capture the users information during registration and store it securely in the database.

I also ensured the system automatically calculates daily caloric needs and BMI at the point of registration, to make sure that users get personalised tracking from the outset.

Then I developed the login system for existing users. It verifies their identity through a string comparison of the password entered and the password hash stored in the database. This was done to ensure only authorised access for users.

```python
# Login screen
class Login(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.usernameLabel = tk.Label(self, text="Username")
        self.usernameLabel.pack()
        self.usernameEntry = tk.Entry(self)
        self.usernameEntry.pack()
        self.passwordLabel = tk.Label(self, text="Password")
        self.passwordLabel.pack()
        self.passwordEntry = tk.Entry(self)
        self.passwordEntry.pack()
        self.passwordEntry.config(show="*")
        self.submitButton = tk.Button(self, text="Login", command=lambda: self.login()).pack(pady=10)

    # Verify credentials and log user in
    def login(self):
        username = self.usernameEntry.get()
        password = self.passwordEntry.get()

        # Fetch user from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT id, passwordHash FROM Users WHERE username = ?", (username,))
        result = cursor.fetchone()
        conn.close()

        if result:
            # Verify password hash matches
            if self.container.hasher.verify(password, result[1]):
                self.container.currentUserId = result[0]
                self.container.showPage("MainMenu")
                return

        messagebox.showerror("Error", "Invalid credentials")
```

Creating All input elements: Using the same pattern as the sign up class: **Label → Entry Widget**

Using hasher function to hash the password entered and carry out a bitwise comparison to the password stored in the database.

Now that the Authentication menu is in place i created the main menu to serve as the central hub which will allow users to access different sections of the application. This design provides a clear pathway to core features such as food logging and workout logging.

```python
# Main navigation menu
class MainMenu(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.title = tk.Label(self, text="Main Menu", font=("Helvetica", 20, "bold"), bg="#e6f2ff")
        self.title.pack(pady=30)

        self.foodButton = tk.Button(self, text="Log Food", width=25, height=2, command=lambda: self.container.showPage("LogFood"))
        self.foodButton.pack(pady=30)
        self.workoutButton = tk.Button(self, text="Log Workout", width=25, height=2, command=lambda: self.container.showPage("LogWorkout"))
        self.workoutButton.pack(pady=30)
        self.statsButton = tk.Button(self, text="View Stats", width=25, height=2, command=lambda: self.showStats())
        self.statsButton.pack(pady=30)
        self.settingsButton = tk.Button(self, text="Settings", width=25, height=2, command=lambda: self.container.showPage("Settings"))
        self.settingsButton.pack(pady=30)
        self.exitButton = tk.Button(self, text="Exit", width=25, height=2, command=lambda: self.container.destroy())
        self.exitButton.pack(pady=30)

    # Load stats before showing stats page
    def showStats(self):
        self.container.pages["ViewStats"].getStats()
        self.container.showPage("ViewStats")
```

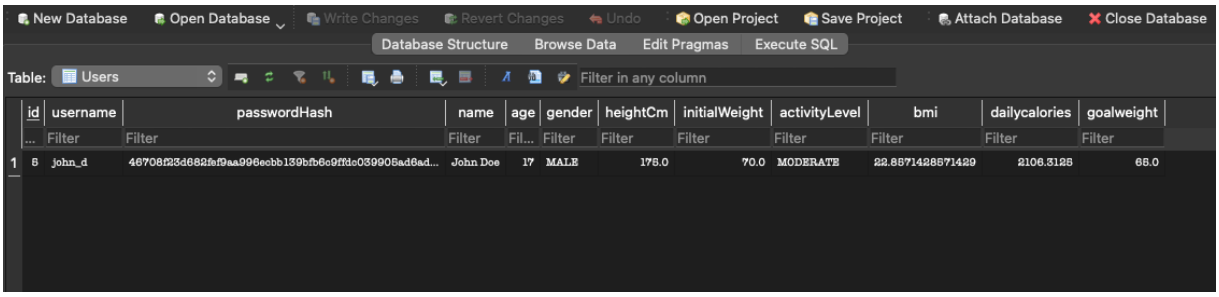
Now that the Authentication Menu and Main Menu have been programmed i will begin testing to ensure the Authentication menu works as proposed:

Iteration 1-Testing

Test 1.1: Create New User

Test	Input	Expected Output	Actual Outcome
Create new user	username: john_d password: Pass123! name: John Doe age: 17 height: 175 weight: 70 gender: MALE activity: MODERATE goal: 65 weeks: 10	Account created, user stored in Users table	Account was created and was stored in the Users table

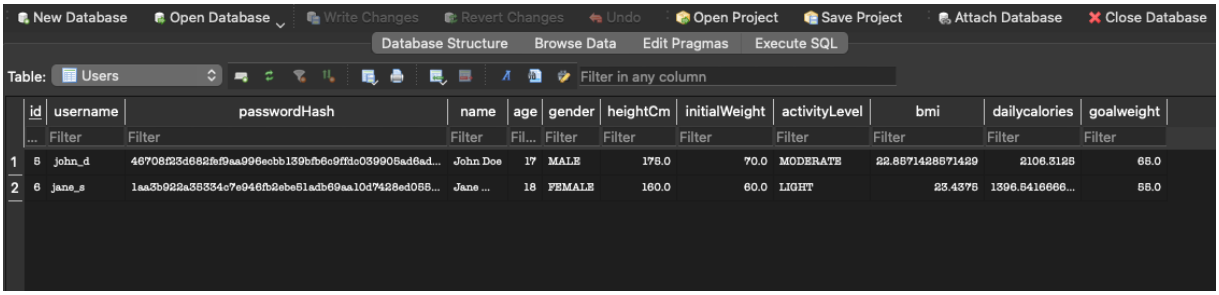
Proof:



Test 1.2: Create Second User

Test	Input	Expected Output	Actual Outcome
Create second user	username: jane_s password: Jane456! name: Jane Smith age: 18 height: 160 weight: 60 gender: FEMALE activity: LIGHT goal: 55 weeks: 12	Account created, user stored in Users table	Account was created and user was stored in the users table

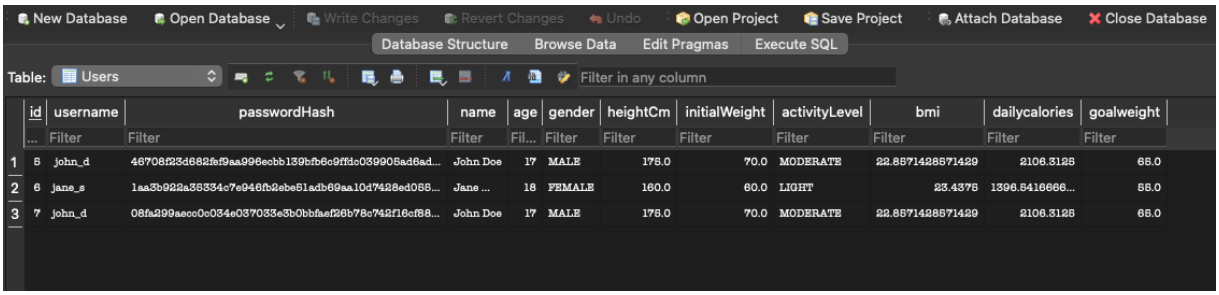
Proof:



Test 1.3: Duplicate Username

Test	Input	Expected Output	Actual Outcome

Duplicate username	username: john_d password: Pass123! (attempting to register again)	Error: "Username already exists."	Error did not appear , the user was added to the table with the same info.



	id	username	passwordHash	name	age	gender	heightCm	initialWeight	activityLevel	bmi	dailycalories	goalweight
		Filter	Filter	Filter	Fil...	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	5	john_d	46708f23d682fef9aa996ecbb139bfb6c9ffdc039905ad6ad...	John Doe	17	MALE	176.0	70.0	MODERATE	22.8571428571429	2106.3125	66.0
2	6	jane_s	1aa3b922a35334c7e946fb2ebe61adb69aa10d7428ed055...	Jane ...	18	FEMALE	160.0	60.0	LIGHT	23.4375	1396.6416666...	55.0
3	7	john_d	08fa299aeec0c034e0370333e3b0bbfaef26b78c742f16cf88...	John Doe	17	MALE	176.0	70.0	MODERATE	22.8571428571429	2106.3125	66.0

I reviewed my code and identified the issue. My original code looked like this :

```
# Check if username already exists in database
conn = connectDatabase()
cursor = conn.cursor()
cursor.execute("SELECT username FROM Users WHERE username = ?", (username,))
existing = cursor.fetchone()

if existing:
    messagebox.showerror("Error", "Username already exists.")
    conn.close()
    return
```

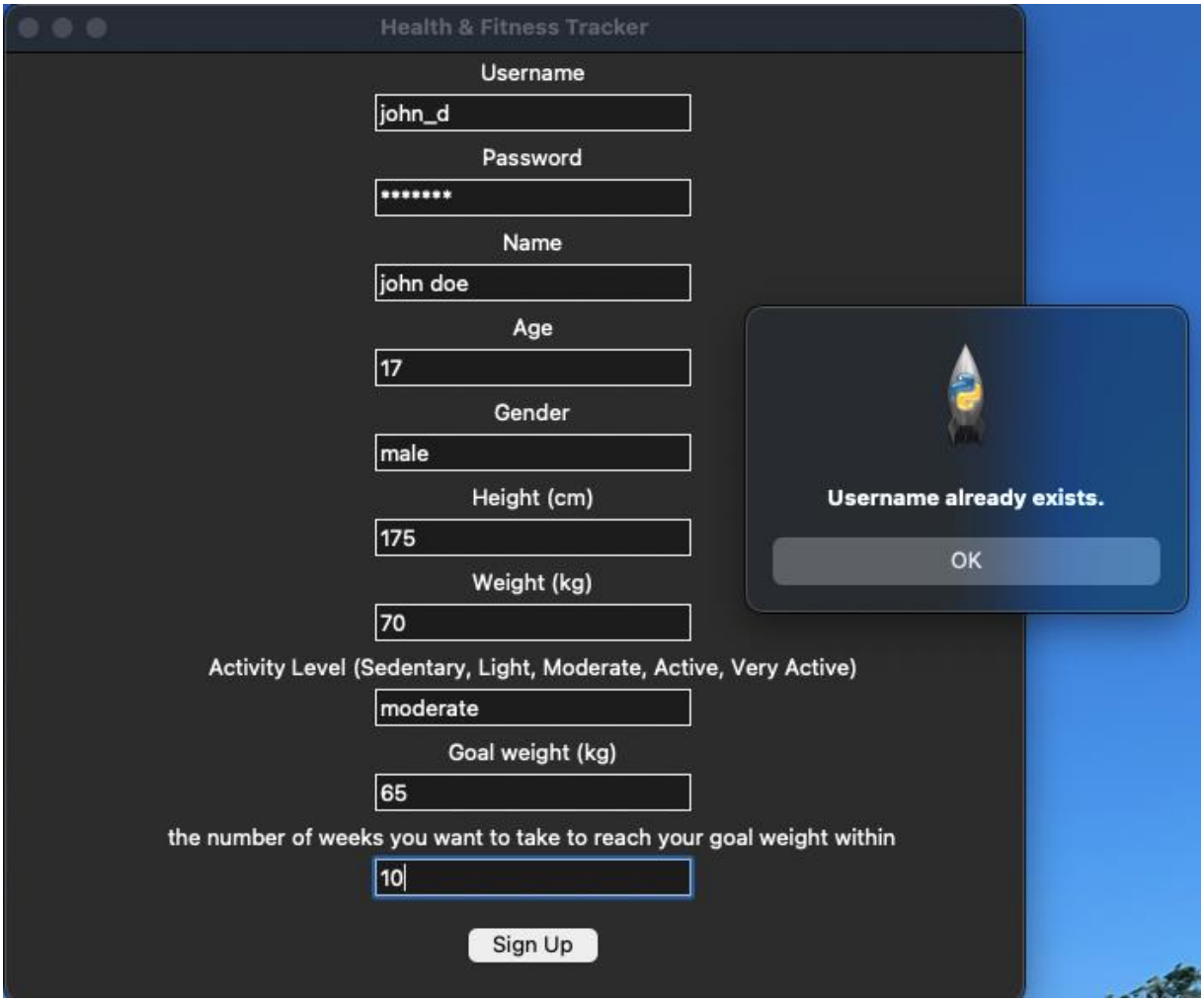
When the error popup appears first, it pauses the code and keeps the database connection open in the background. This open connection can block the database from being used properly when you try to register again or perform other operations. Closing the connection before the popup ensures the database is immediately freed up and is ready for the next action.

I changed my code to this:

```
# Check if username already exists in database
conn = connectDatabase()
cursor = conn.cursor()
cursor.execute("SELECT username FROM Users WHERE username = ?", (username,))
existing = cursor.fetchone()

if existing:
    conn.close()
    messagebox.showerror("Error", "Username already exists.")
    return
```

I then tried to create the account again and was met with the expected outcome:



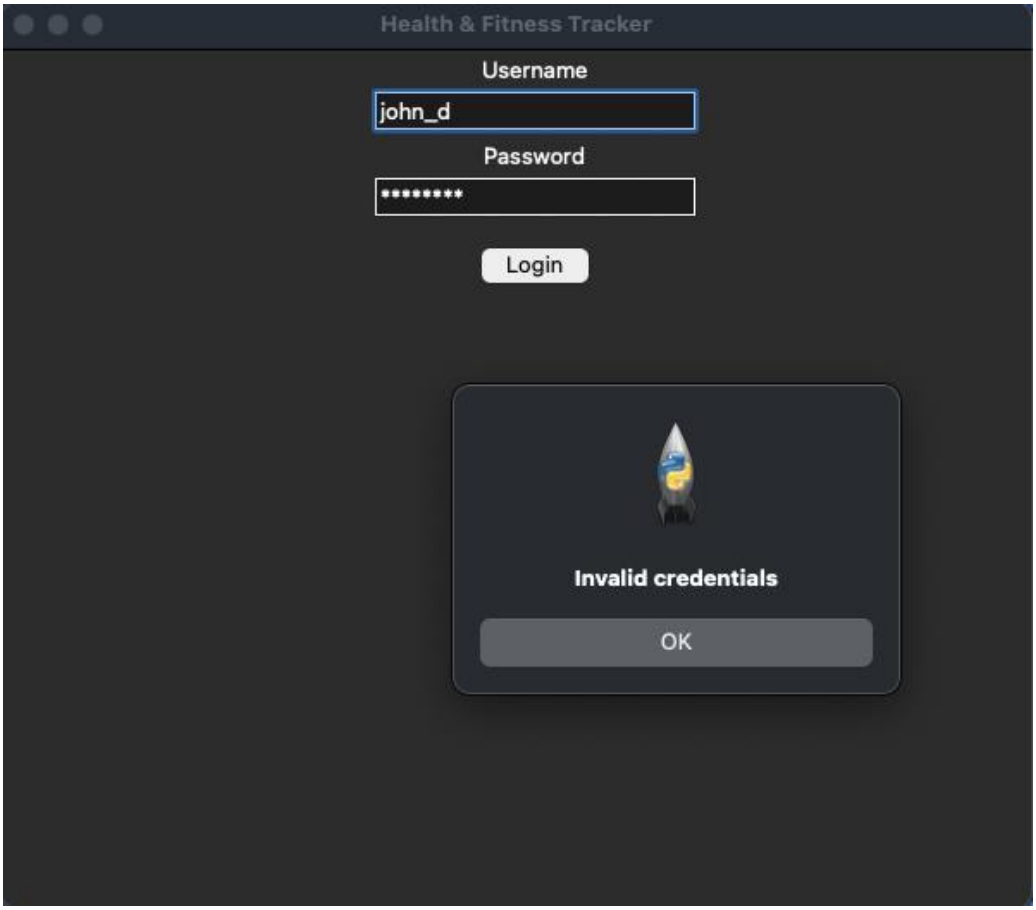
Updated test 1.3: Duplicate Username

Test	Input	Expected Output	Actual Outcome
Duplicate username	username: john_d password: Pass123! (attempting to register again)	Error: "Username already exists."	Error did appear and the account was not created

Test 1.4: Correct Credentials

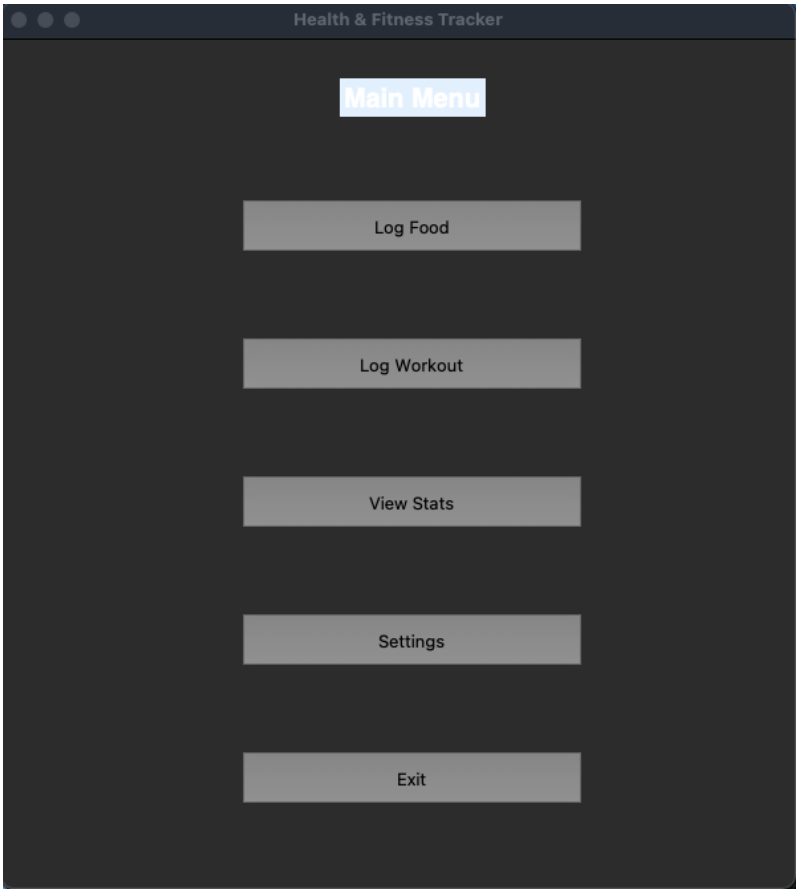
Test	Input	Expected Output	Actual Outcome

Correct credentials	username: john_d password: Pass123!	Login success, main menu displayed	Login was not successful



Initially I examined my code and didn't see any errors. Next I moved down to the database layer. I deleted all of the entries in the user table for users with id 5 and 7 and created the test account for John Doe again. I then attempted to login and this was successful.

Proof:



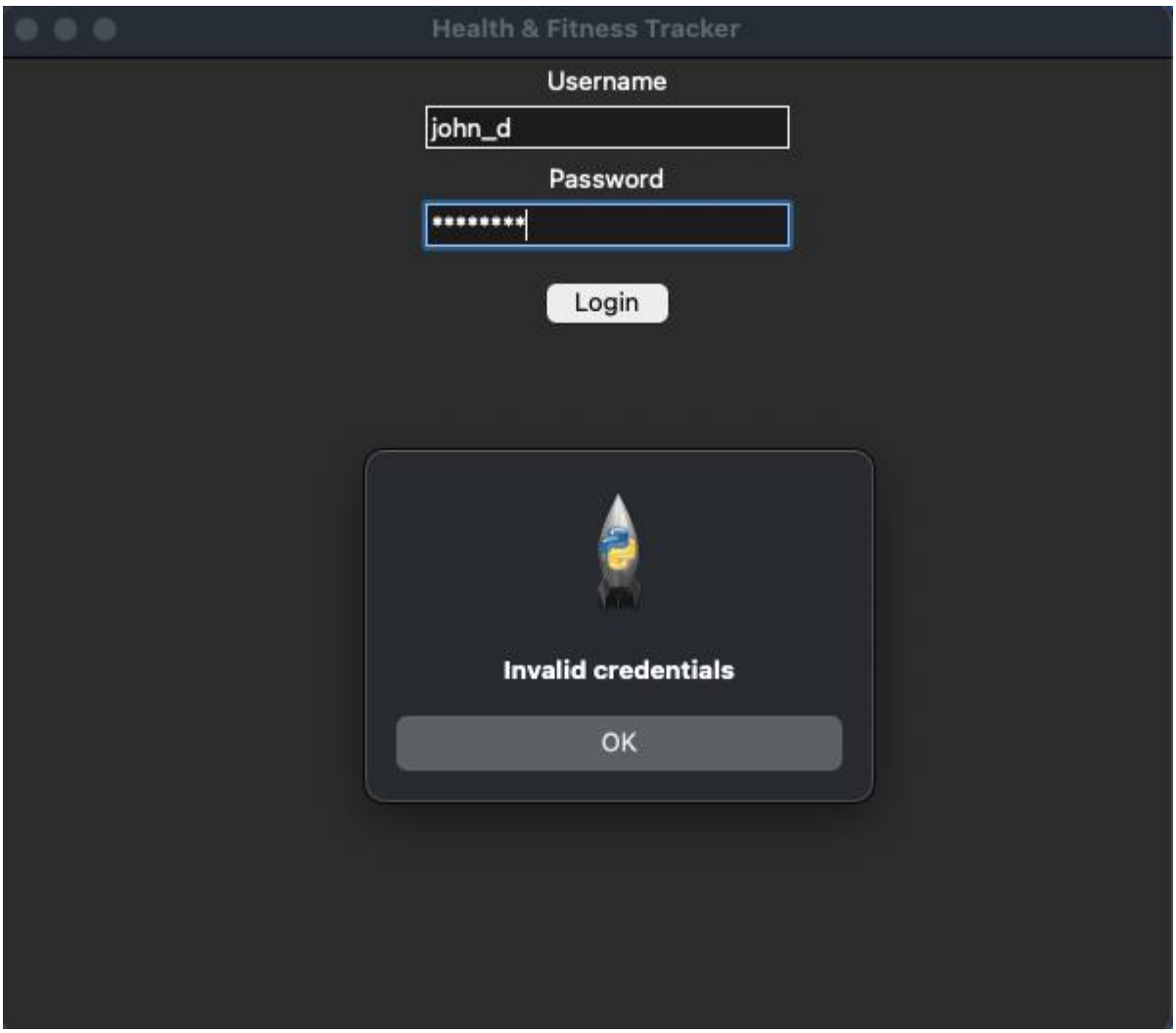
Updated Test 1.4: Correct Credentials

Test	Input	Expected Output	Actual Outcome
Correct credentials	username: john_d password: Pass123!	Login success, main menu displayed	Login was successful, main menu displayed

Test 1.5: Wrong Password

Test	Input	Expected Output	Actual Outcome
Wrong password	username: john_d password: wrong	Error: "Invalid credentials."	Error was received

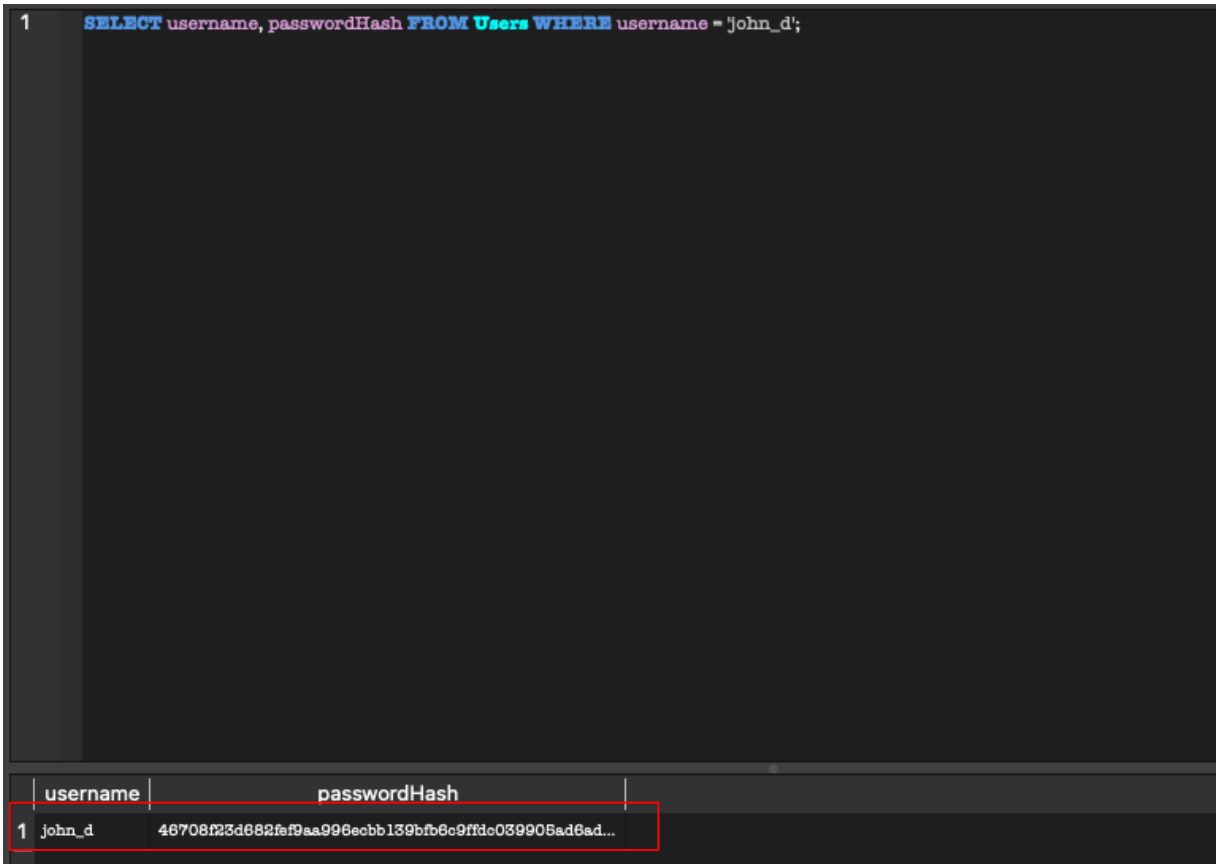
Proof:



Test 1.6: Check Stored Password

Test	Input	Expected Output	Actual Outcome
Check stored password	Query database for john_d passwordHash field	Password stored as SHA-256 hash (64-character hexadecimal string, not plain text)	Password was stored as its hash

Proof:



Testing for the Authentication menu is now complete and all of the elements have been passed.I will now be moving onto iteration number 2.

Iteration 2 - Food Logging

This iteration implements the Log Food module that was identified in the decomposition diagram. This section is responsible for querying the database for foods and using binary search to find the foods requested by the user, logging meals to specific categories and managing meal entries using a stack.

Development

Before I start implementing food logging it is essential that I populate the food table in the database.

I decided to populate the food table with 10 USDA verified foods.

The reason I only added ten is because a larger food database is not required at this stage of development, the goal is to show the food logging functionality ,macro tracking and binary search all work properly , Using a smaller dataset helped me ensure that the nutritional data for the foods were all accurate. In future versions a larger database would be used to allows users to track a variety of different foods.

	foodId	name	calories	protein	carbs	fat	portionsize
	Filter	Filter	Filter	Filter	Filter	Fil...	Filter
1	11	EGG	78.0	6.3	0.6	5.3	per egg
2	12	CHICKEN BREAST	165.0	31.0	0.0	3.6	per 100 grams
3	13	BANANA	105.0	1.3	27.0	0.3	per banana
4	14	SALMON	208.0	20.0	0.0	13.0	per 100 grams
5	15	BROCCOLI	55.0	3.7	11.2	0.6	per 100 grams
6	16	ALMONDS	579.0	21.2	21.6	49.9	per 100 grams
7	17	APPLE	95.0	0.5	25.0	0.3	per apple
8	18	TOFU	76.0	8.0	1.9	4.8	per 100 grams
9	19	RICE (WHITE)	130.0	2.7	28.2	0.3	per 100 grams
10	20	YOGURT (PLAIN)	59.0	10.0	3.6	0.4	per 100 grams

I decided to code the stack data structure that will be used to implement the undo feature during the food logging page.

```
326  # Stack data structure for undo functionality
327  class Stack:
328      def __init__(self):
329          self.items = []
330
331      # Add item to top of stack
332      def push(self, item):
333          self.items.append(item)
334
335      # Remove and return top item
336      def pop(self):
337          if len(self.items) == 0:
338              return None
339          return self.items.pop()
340
341      # View top item without removing
342      def peek(self):
343          if len(self.items) == 0:
344              return None
345          return self.items[-1]
346
347      # Check if stack is empty
348      def isEmpty(self):
349          if len(self.items) == 0:
350              return True
351          return False
352
353      # Get number of items in stack
354      def size(self):
355          return len(self.items)
356
```

I implemented the dedicated log food class to manage the process of recording meals and linking entries to the normalised database schema. This design ensures accurate tracking of food items, macros, and meal types.

```python
# Food logging screen
class LogFood(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container
        self.mealStack = Stack()  # Track meal entries for undo

        # Daily summary display
        self.dailymacrosHeader = tk.Label(self, text="Calories & Macronutrients")
        self.dailymacrosHeader.pack(pady=(20, 5))
        self.dailymacroslistbox = tk.Listbox(self, width=40, height=6)
        self.dailymacroslistbox.pack(padx=80, pady=(0, 30))

        # Food search input
        self.searchfoodLabel = tk.Label(self, text="Search Food:")
        self.searchfoodLabel.pack(pady=(0, 5))
        self.searchfoodEntry = tk.Entry(self)
        self.searchfoodEntry.pack()
        self.submitfoodButton = tk.Button(self, text="search", command=lambda: self.searchfood())
        self.submitfoodButton.pack(pady=(10, 30))

        # Search results display
        self.foodsearchHeader = tk.Label(self, text="Food Search Results")
        self.foodsearchHeader.pack(pady=(0, 5))
        self.foodsearchlistbox = tk.Listbox(self, width=40, height=5)
        self.foodsearchlistbox.pack(padx=80, pady=(0, 20))

        # Buttons to log food to different meals
        self.logfoodBreakfastButton = tk.Button(self, text="Log Food for Breakfast", command=lambda: self.logselectedfood("Breakfast"))
        self.logfoodBreakfastButton.pack(padx=40, pady=(0, 10))
        self.logfoodLunchButton = tk.Button(self, text="Log Food for Lunch", command=lambda: self.logselectedfood("Lunch"))
        self.logfoodLunchButton.pack(padx=40, pady=(0, 10))
        self.logfoodDinnerButton = tk.Button(self, text="Log Food for Dinner", command=lambda: self.logselectedfood("Dinner"))
        self.logfoodDinnerButton.pack(padx=40, pady=(0, 10))

        # Undo last logged meal
        self.undoButton = tk.Button(self, text="Undo Last Entry", command=lambda: self.undolastmealentry())
        self.undoButton.pack(padx=40, pady=(10, 20))

        # Display logged meals
        self.breakfastHeader = tk.Label(self, text="Breakfast")
        self.breakfastHeader.pack(pady=(0, 5))
        self.breakfastlistbox = tk.Listbox(self, width=40, height=5)
        self.breakfastlistbox.pack(padx=40, pady=(0, 25))

        self.lunchHeader = tk.Label(self, text="Lunch")
        self.lunchHeader.pack(pady=(0, 5))
        self.lunchlistbox = tk.Listbox(self, width=40, height=5)
        self.lunchlistbox.pack(padx=40, pady=(0, 25))

        self.dinnerHeader = tk.Label(self, text="Dinner")
        self.dinnerHeader.pack(pady=(0, 5))
        self.dinnerlistbox = tk.Listbox(self, width=40, height=5)
        self.dinnerlistbox.pack(padx=40, pady=(0, 30))

        self.backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
        self.backButton.pack(padx=40, pady=(0, 20))
```

Listboxes for all 3 meals to ensure tracking is functional and usable

This design improves the organisation of the system, it allows users to quickly review their logged meals and the associated nutritional data.

I implemented the Binary search directly inside the LogFood class because it's only used for searching foods when logging meals. No other part of the app needs to use the binary search, so there's no point making it a global function.

```
    # Refresh meals when page is shown
    def tkraise(self):
        super().tkraise()
        self.loadTodaysMeals()
        self.updateDailyMacros()

    # Binary search to find food in sorted list
    def binarySearch(self, foodList, searchTerm):
        left = 0
        right = len(foodList) - 1
        results = []

        while left <= right:
            mid = (left + right) // 2
            midFood = foodList[mid][0]

            if searchTerm == midFood:
                results.append(foodList[mid])
                return results
            elif searchTerm < midFood:
                right = mid - 1
            else:
                left = mid + 1

        return results

    # Search for food in database using binary search
    def searchfood(self):
        searchedfood = self.searchfoodEntry.get().upper()

        # Fetch all foods sorted by name from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT name, calories, protein, carbs, fat FROM Food ORDER BY name")
        allFoods = cursor.fetchall()
        conn.close()

        # Binary search through sorted food list
        results = self.binarySearch(allFoods, searchedfood)

        # Display search results
        self.foodsearchlistbox.delete(0, tk.END)

        if len(results) == 0:
            self.foodsearchlistbox.insert(tk.END, "No foods found. Try a different search.")
            return

        for food in results:
            displayText = "NAME: " + str(food[0]) + "  CALORIES: " + str(food[1]) + "  P: " + str(food[2]) + "g  C: " + str(food[3]) + "g  F: " + str(food[4]) + "g"
            self.foodsearchlistbox.insert(tk.END, displayText)

```

The food is searched for using this simple structure I defined in the design phase:

1. User searches for food
2. Input is turned into uppercase
3. All food is queried from the database
4. User input is searched for using the binary search function
5. If food is found all necessary information is displayed in the listbox
6. If food is not found an error is shown

The tkraise method is used as it automatically refreshes the Log Food page whenever it's displayed, loading today's meals and calculating total calories consumed and remaining. The searchfood method allows users to search for food items in the database by name, displaying matching results with their nutritional information so users can select and log them to their meals.

I used datetime and timedelta as datetime was essential for getting today's date when logging meals/workouts to make sure all foods consumed on a certain day were reset for the next day and stored in the database for later when statistics need to be viewed in view stats.

The logSelectedFood method takes a meal type (Breakfast, Lunch, or Dinner) and logs a chosen food from the search results to that meal. It first checks whether a food has been selected in the listbox.

When a food is selected, it retrieves the food name, queries the database for its full nutritional information, and gets today's date using datetime.now().strftime("%Y-%m-%d") so the meal is logged correctly for the given day. The quantity is set to an automatic 1.0 (one serving. The data is then inserted into the MealLogs table and the display is refreshed to show any newly logged foods.

```
    # Log selected food to specified meal in database
    def logselectedfood(self, meal):
        selection = self.foodsearchlistbox.curselection()
        if len(selection) == 0:
            return

        index = selection[0]
        selectedfood = self.foodsearchlistbox.get(index)

        if "NAME: " in selectedfood:
            foodName = selectedfood.split("  ")[0].replace("NAME: ", "").strip()

            # Get food details from database
            conn = connectDatabase()
            cursor = conn.cursor()
            cursor.execute("SELECT foodId, calories, protein, carbs, fat FROM Food WHERE name = ?", (foodName,))
            foodData = cursor.fetchone()

            if foodData:
                foodId = foodData[0]
                calories = foodData[1]
                protein = foodData[2]
                carbs = foodData[3]
                fat = foodData[4]

                todayDate = datetime.now().strftime("%Y-%m-%d")
                quantity = 1.0
                totalCal = calories * quantity
                totalProtein = protein * quantity
                totalCarbs = carbs * quantity
                totalFat = fat * quantity
                mealType = meal.lower()

                # Insert meal log into database
                cursor.execute("""
                    INSERT INTO MealLogs (userId, date, mealType, foodId, quantity, totalCal, totalProtein, totalCarbs, totalFat)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, (
                    self.container.currentUserId,
                    todayDate,
                    mealType,
                    foodId,
                    quantity,
                    totalCal,
                    totalProtein,
                    totalCarbs,
                    totalFat
                ))
                conn.commit()

                # Add to undo stack
                self.mealStack.push(cursor.lastrowid)

                conn.close()

                self.loadTodaysMeals()
                self.updateDailyMacros()
                return

            conn.close()
```

Now I created the section where I use the stack for undo functionality (stack subroutine was created earlier in the code) :

```
531         # Undo last meal entry using stack
532         def undolastmealentry(self):
533             if self.mealStack.isEmpty() == False:
534                 lastMealId = self.mealStack.pop()
535 
536                 # Delete meal from database
537                 conn = connectDatabase()
538                 cursor = conn.cursor()
539                 cursor.execute("DELETE FROM MealLogs WHERE logId = ?", (lastMealId,))
540                 conn.commit()
541                 conn.close()
542 
543                 self.loadTodaysMeals()
544                 self.updateDailyMacros()
545 
```

I then created the loadTodaysMeals method that gets all meals the user has logged today and displays them in the correct meal listboxes.

It first gets today's date using datetime.now().strftime("%Y-%m-%d"), then clears the breakfast, lunch, and dinner listboxes using delete(0, tk.END).

It connects to the database and runs a SQL query to retrieve meal details and food names for the current user and today's date. The results are returned using fetchall(), and the method loops through each entry, getting the meal type, food name, calories, protein, carbs, and fat. It formats these values into a display string with rounded nutrition data and inserts each item into the appropriate listbox based on whether it is logged for breakfast, lunch, or dinner.

I decided to use "m" and "f" as aliases for the MealLogs and Food tables, I used these to make the SQL query shorter and clearer for future development.

Loadtodaysmeals():

```
546     # Load today's logged meals from database
547     def loadTodaysMeals(self):
548         todayDate = datetime.now().strftime("%Y-%m-%d")
549 
550         # Clear existing displays
551         self.breakfastlistbox.delete(0, tk.END)
552         self.lunchlistbox.delete(0, tk.END)
553         self.dinnerlistbox.delete(0, tk.END)
554 
555         # Fetch today's meals from database
556         conn = connectDatabase()
557         cursor = conn.cursor()
558 
559         cursor.execute("""
560             SELECT m.mealType, f.name, m.totalCal, m.totalProtein, m.totalCarbs, m.totalFat
561             FROM MealLogs m
562             JOIN Food f ON m.foodId = f.foodId
563             WHERE m.userId = ? AND m.date = ?
564             ORDER BY m.logId
565         """, (self.container.currentUserId, todayDate))
566 
567         meals = cursor.fetchall()
568         conn.close()
569 
570         # Display meals in appropriate listbox
571         for meal in meals:
572             mealType = meal[0]
573             foodName = meal[1]
574             totalCal = meal[2]
575             totalProtein = meal[3]
576             totalCarbs = meal[4]
577             totalFat = meal[5]
578 
579             displayText = str(foodName) + "  " + str(round(totalCal, 0)) + " cal  P: " + str(round(totalProtein, 1)) + "g  C: " + str(round(totalCarbs, 1)) + "g  F: " + str(round(totalFat, 1)) + "g"
580 
581             if mealType == "breakfast":
582                 self.breakfastlistbox.insert(tk.END, displayText)
583             if mealType == "lunch":
584                 self.lunchlistbox.insert(tk.END, displayText)
585             if mealType == "dinner":
586                 self.dinnerlistbox.insert(tk.END, displayText)
```

I then created the updatedailymacros method to calculate and display the user's daily nutritional totals.

It works by getting today's date and retrieves the user's daily calorie goal from the database. Then it sums the calories, protein, carbs, and fat from all meals logged today. If no meals are logged, totals are set to 0. It calculates the remaining calories by subtracting total calories from the daily goal and updates the listbox to show the date, total calories, calories remaining and total protein, carbs, and fats .

Updatedailymacros():

```
461
462     def updateDailyMacros(self):
463         todayDate = datetime.now().strftime("%Y-%m-%d")
464
465         conn = connectDatabase()
466         cursor = conn.cursor()
467
468         cursor.execute("SELECT dailycalories FROM Users WHERE id = ?", (self.container.currentUserId,))
469         userResult = cursor.fetchone()
470         if userResult:
471             if userResult[0]:
472                 dailyGoal = userResult[0]
473             else:
474                 dailyGoal = 2000
475         else:
476             dailyGoal = 2000
477
478         cursor.execute("""
479             SELECT
480                 SUM(totalCal) as totalCalories,
481                 SUM(totalProtein) as totalProtein,
482                 SUM(totalCarbs) as totalCarbs,
483                 SUM(totalFat) as totalFat
484             FROM MealLogs
485             WHERE userId = ? AND date = ?
486         """, (self.container.currentUserId, todayDate))
487
488         totals = cursor.fetchone()
489         conn.close()
490
491         if totals[0] is None:
492             totalCalories = 0
493             totalProtein = 0
494             totalCarbs = 0
495             totalFat = 0
496         else:
497             totalCalories = int(totals[0])
498             totalProtein = int(totals[1])
499             totalCarbs = int(totals[2])
500             totalFat = int(totals[3])
501
502         caloriesRemaining = int(dailyGoal - totalCalories)
503
504         self.dailymacroslistbox.delete(0, tk.END)
505         self.dailymacroslistbox.insert(tk.END, "Date: " + todayDate)
506         self.dailymacroslistbox.insert(tk.END, "Total Calories: " + str(totalCalories) + " cal")
507         self.dailymacroslistbox.insert(tk.END, "Calories Remaining: " + str(caloriesRemaining) + " cal")
508         self.dailymacroslistbox.insert(tk.END, "Protein: " + str(totalProtein) + "g")
509         self.dailymacroslistbox.insert(tk.END, "Carbs: " + str(totalCarbs) + "g")
510         self.dailymacroslistbox.insert(tk.END, "Fat: " + str(totalFat) + "g")
```

```python
    # Update daily calorie and macro totals display
    def updateDailyMacros(self):
        todayDate = datetime.now().strftime("%Y-%m-%d")

        # Get user's daily calorie target from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("SELECT dailycalories FROM Users WHERE id = ?", (self.container.currentUserId,))
        userResult = cursor.fetchone()
        if userResult:
            if userResult[0]:
                dailycalories = userResult[0]
            else:
                dailycalories = 2000
        else:
            dailycalories = 2000

        # Sum today's totals from database
        cursor.execute("""
            SELECT
                SUM(totalCal) as totalCalories,
                SUM(totalProtein) as totalProtein,
                SUM(totalCarbs) as totalCarbs,
                SUM(totalFat) as totalFat
            FROM MealLogs
            WHERE userId = ? AND date = ?
        """, (self.container.currentUserId, todayDate))

        totals = cursor.fetchone()
        conn.close()

        if totals[0] is None:
            totalCalories = 0
            totalProtein = 0
            totalCarbs = 0
            totalFat = 0
        else:
            totalCalories = int(totals[0])
            totalProtein = int(totals[1])
            totalCarbs = int(totals[2])
            totalFat = int(totals[3])

        caloriesRemaining = int(dailycalories - totalCalories)

        # Display summary
        self.dailymacroslistbox.delete(0, tk.END)
        self.dailymacroslistbox.insert(tk.END, "Date: " + todayDate)
        self.dailymacroslistbox.insert(tk.END, "Total Calories: " + str(totalCalories) + " cal")
        self.dailymacroslistbox.insert(tk.END, "Calories Remaining: " + str(caloriesRemaining) + " cal")
        self.dailymacroslistbox.insert(tk.END, "Protein: " + str(totalProtein) + "g")
        self.dailymacroslistbox.insert(tk.END, "Carbs: " + str(totalCarbs) + "g")
        self.dailymacroslistbox.insert(tk.END, "Fat: " + str(totalFat) + "g")
```

Updates all necessary information and loads it into the dailymacros listbox

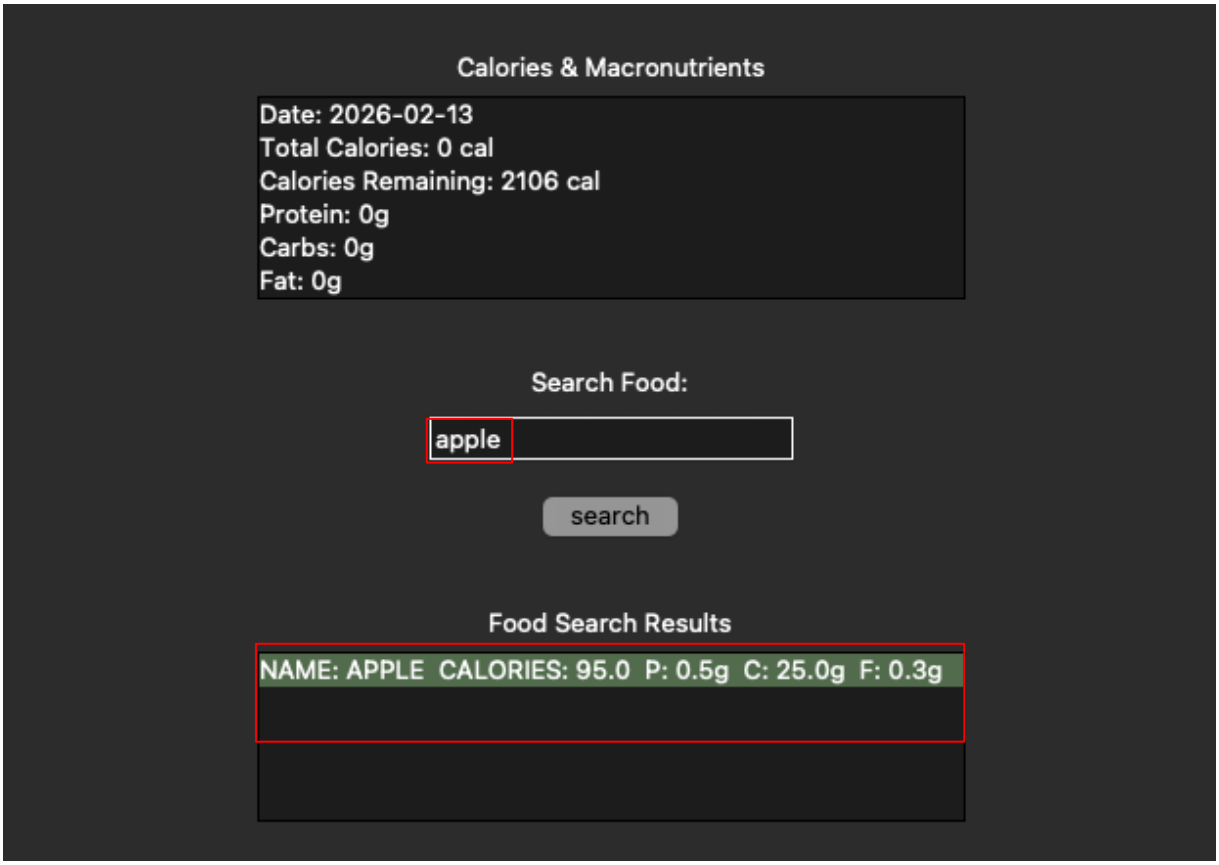
Now that i have completed the food logging section i will begin the testing.

Iteration 2-Testing

Test 2.1: Exact Food Search

Test	Input	Expected Output	Actual Outcome
Exact search	Search for: APPLE (Type in search box, click Search button)	Food shown in search results with name, calories, protein, carbs, fat	Food shown with name,calories,protein, carbs,fat

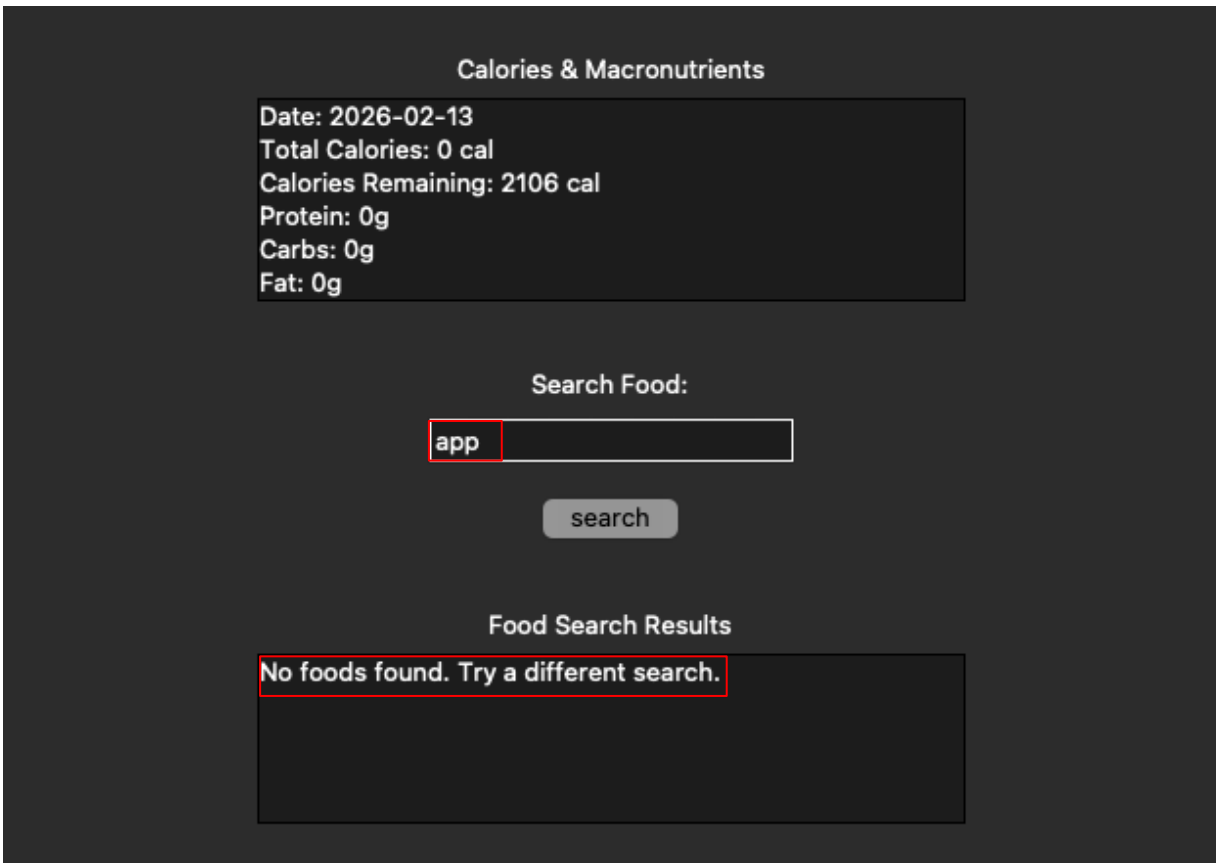
Proof:



Test 2.2: Partial Food Search

Test	Input	Expected Output	Actual Outcome
Partial search	Search for: APP (Type incomplete name in search box, click Search button)	"No foods found. Try a different search."	Error shown

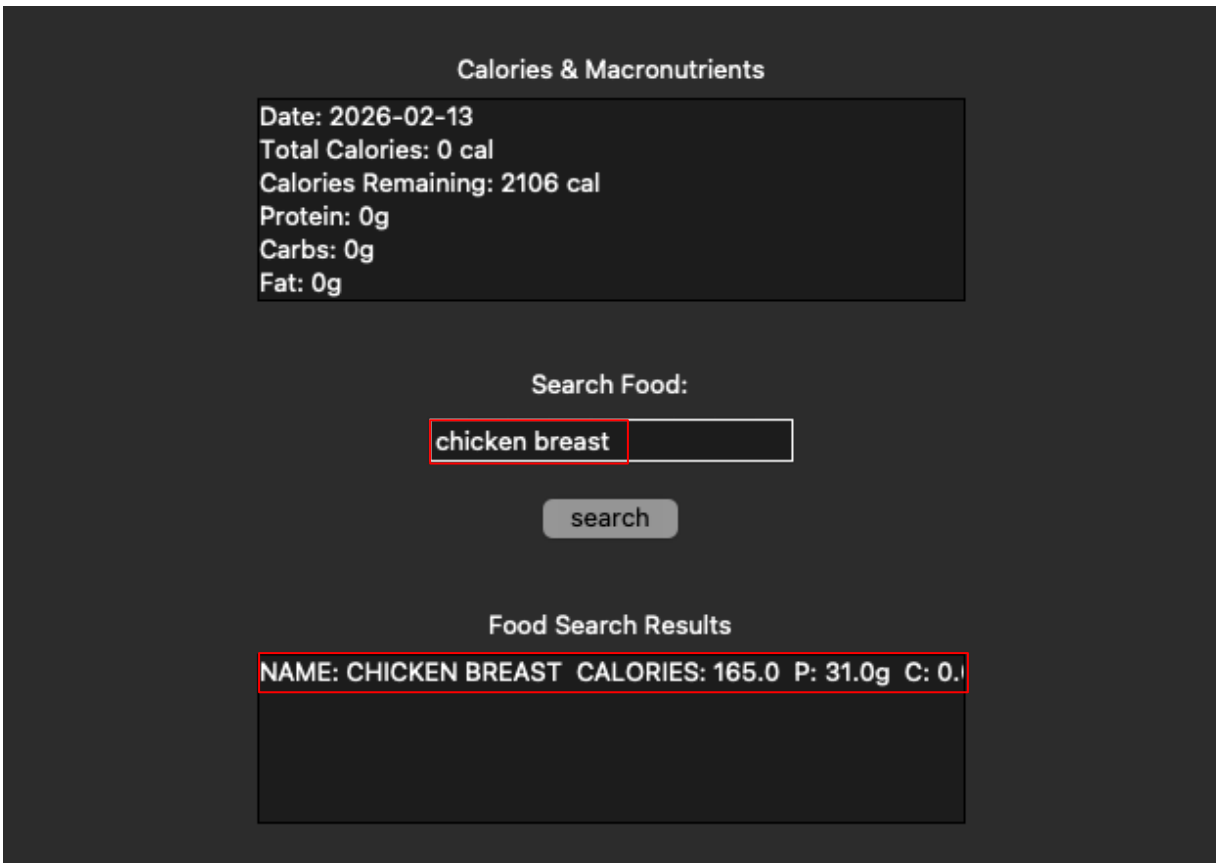
Proof:



Test 2.3: Binary Search Verification

Test	Input	Expected Output	Actual Outcome
Sorted database search	Search for any exact food name that exists in database (e.g., CHICKEN BREAST)	Food found and displayed (confirms binary search worked on sorted database)	Food found and displayed

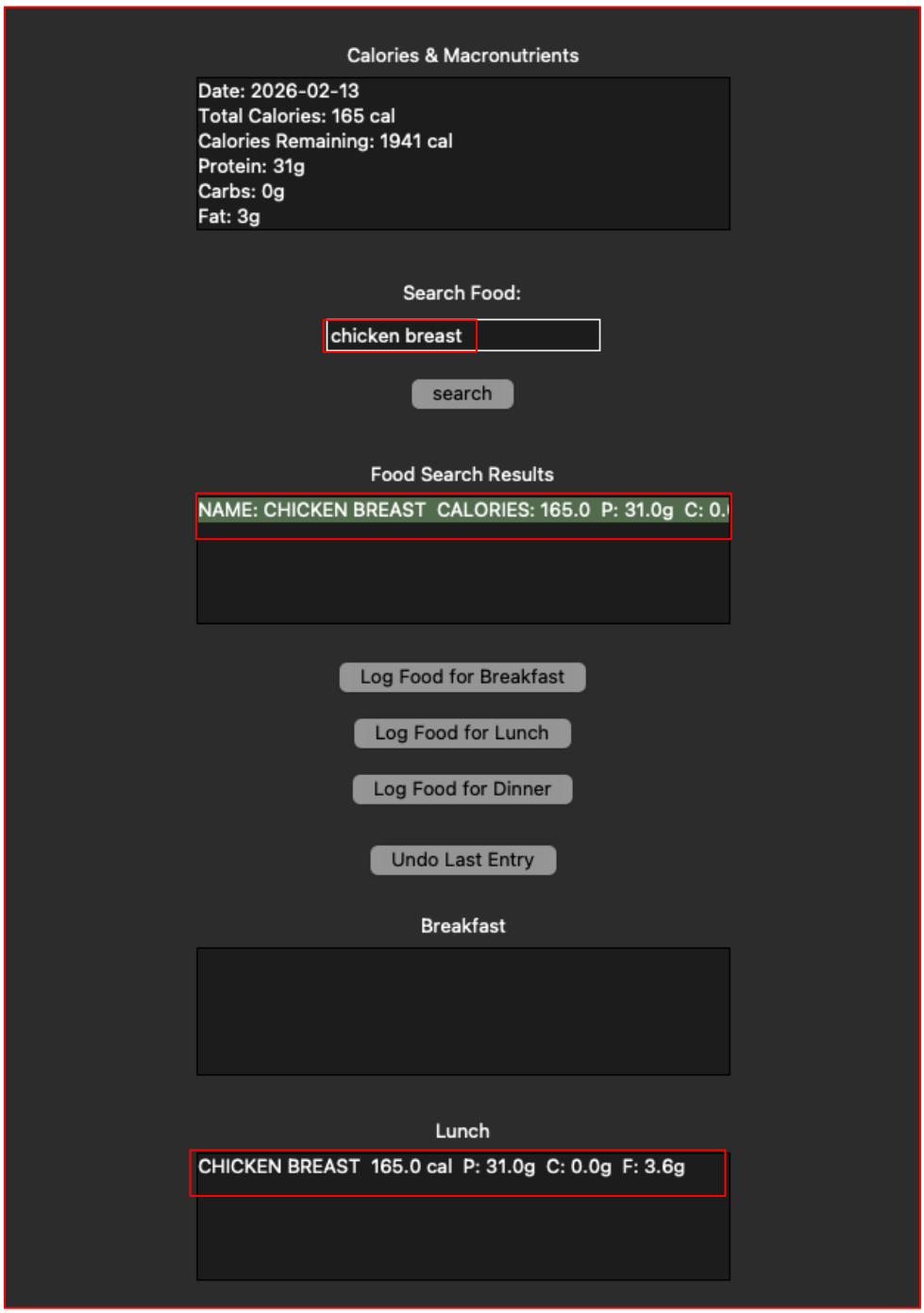
Proof:



Test 2.4 & 2.5: Nutritional Display

Test	Input	Expected Output	Actual Outcome
Nutritional display	Log any food to any meal (e.g., CHICKEN BREAST to Lunch)	Food entry in listbox shows: NAME, CALORIES, Protein (P), Carbs (C), Fat (F) in grams	Succesful

Proof:


Calories & Macronutrients
Date: 2026-02-13
Total Calories: 165 cal
Calories Remaining: 1941 cal
Protein: 31g
Carbs: 0g
Fat: 3g
Search Food:
chicken breast
search
Food Search Results
NAME: CHICKEN BREAST CALORIES: 165.0 P: 31.0g C: 0.
Log Food for Breakfast
Log Food for Lunch
Log Food for Dinner
Undo Last Entry
Breakfast
Lunch
CHICKEN BREAST 165.0 cal P: 31.0g C: 0.0g F: 3.6g

Test 2.6: Undo Last Meal Entry

Test	Input	Expected Output	Actual Outcome
Undo meal	1. Log a food to any meal 2. Click "Undo Last Entry" button	Most recently logged meal removed from listbox and database	Most recent food removed

Proof:



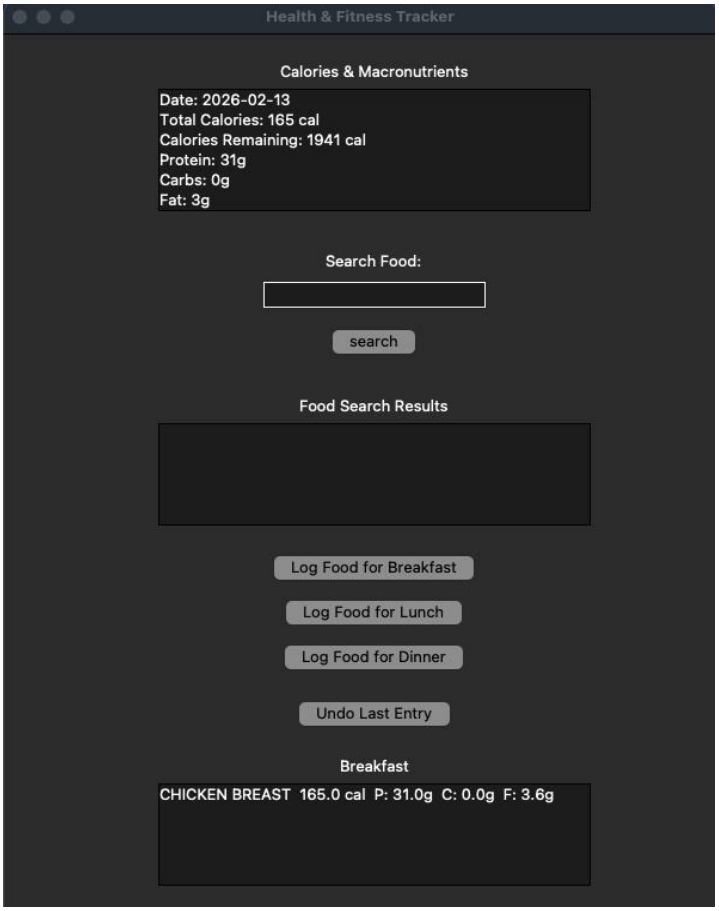
Test 2.7: Undo After App Restart

Test	Input	Expected Output	Actual Outcome
Undo after restart	1. Close application completely 2. Reopen application 3. Navigate to Log Food page 4. Click "Undo Last Entry" button	No change (stack is empty after restart, nothing to undo)	No change

I added a new entry and closed the program.

I ran the code again, logged in and went to the log food section. When Undo last entry was pressed there was no change as the stack was empty.

Proof:



Test 2.8: Auto Load Today's Meals

Test	Input	Expected Output	Actual Outcome
Auto load meals	1. Log 2-3 meals to Breakfast, Lunch, Dinner 2. Navigate away to Main Menu 3. Return to Log Food page	All meals logged today automatically displayed in their respective meal listboxes (Breakfast, Lunch, Dinner)	Passed

Proof:



Now that all of the testing for Iteration 2 is complete I will move onto Iteration 3.

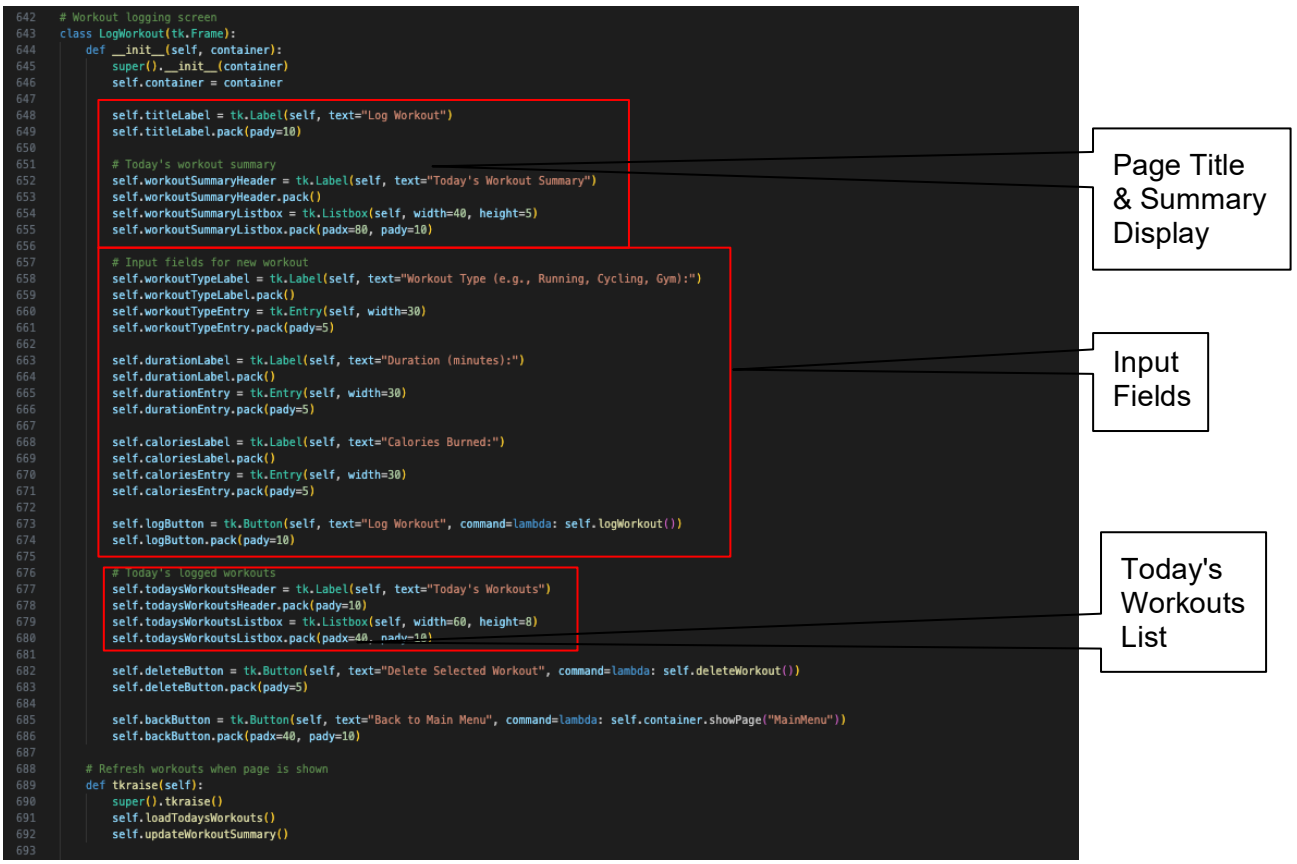
Iteration 3 - Workout Logging

This iteration implements the Log Workout module, it is responsible for allowing users to log workouts by type, duration, and calories burned,displaying the days workout summary and allowing users to delete workouts.

Development

I have now begun programming the LogWorkout class. I will be following a very similar structure to the logfood class as it seemed to work as I proposed with no errors at the testing phase.

The LogWorkout class:




```python
# Workout logging screen
class LogWorkout(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.titleLabel = tk.Label(self, text="Log Workout")
        self.titleLabel.pack(pady=10)

        # Today's workout summary
        self.workoutSummaryHeader = tk.Label(self, text="Today's Workout Summary")
        self.workoutSummaryHeader.pack()
        self.workoutSummaryListbox = tk.Listbox(self, width=40, height=5)
        self.workoutSummaryListbox.pack(padx=80, pady=10)

        # Input fields for new workout
        self.workoutTypeLabel = tk.Label(self, text="Workout Type (e.g., Running, Cycling, Gym):")
        self.workoutTypeLabel.pack()
        self.workoutTypeEntry = tk.Entry(self, width=30)
        self.workoutTypeEntry.pack(pady=5)

        self.durationLabel = tk.Label(self, text="Duration (minutes):")
        self.durationLabel.pack()
        self.durationEntry = tk.Entry(self, width=30)
        self.durationEntry.pack(pady=5)

        self.caloriesLabel = tk.Label(self, text="Calories Burned:")
        self.caloriesLabel.pack()
        self.caloriesEntry = tk.Entry(self, width=30)
        self.caloriesEntry.pack(pady=5)

        self.logButton = tk.Button(self, text="Log Workout", command=lambda: self.logWorkout())
        self.logButton.pack(pady=10)

        # Today's logged workouts
        self.todaysWorkoutsHeader = tk.Label(self, text="Today's Workouts")
        self.todaysWorkoutsHeader.pack(pady=10)
        self.todaysWorkoutsListbox = tk.Listbox(self, width=60, height=8)
        self.todaysWorkoutsListbox.pack(padx=40, pady=10)

        self.deleteButton = tk.Button(self, text="Delete Selected Workout", command=lambda: self.deleteWorkout())
        self.deleteButton.pack(pady=5)

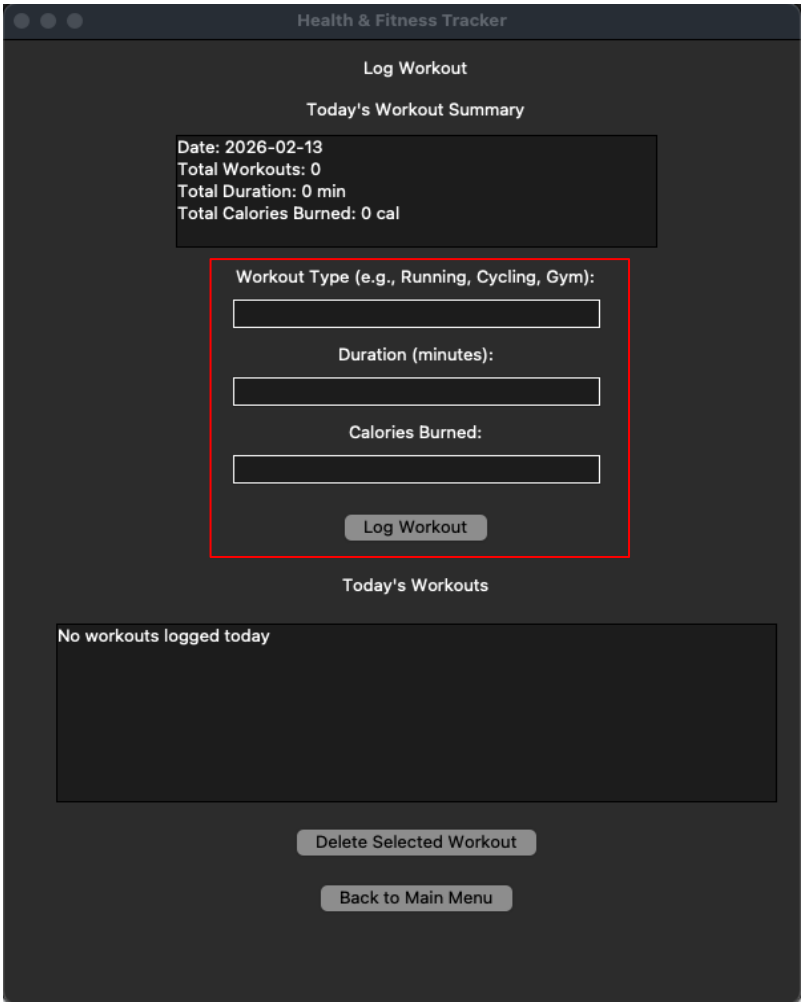
        self.backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
        self.backButton.pack(padx=40, pady=10)

    # Refresh workouts when page is shown
    def tkraise(self):
        super().tkraise()
        self.loadTodaysWorkouts()
        self.updateWorkoutSummary()
```

The LogWorkout class creates a Tkinter frame for logging workouts.

When initialised, it sets up labels and a listbox to show a summary of today's workouts, entry fields for the user to input workout type, duration, and calories burned, and buttons to log a workout, delete a selected workout, and return to the main menu. It also includes a listbox showing all of today's logged workouts individually.

Layout of the screen :



I then decided to program the logWorkout method, which lets a user log a new workout.

It works by first retrieving the inputs for workout type, duration, and calories burned. It then makes sure that all of the fields are filled in and that duration and calories are both positive integers. It then gets today's date and inserts the workout into the WorkoutLogs table in the database, and then it clears the entry fields. Finally, it reloads today's workouts using the subroutines LoadTodaysWorkouts() and it updates the workout summary using UpdateWorkoutSummary() to show the new entry in the listboxes. These 2 subrouitnes are shown below in more detail.

logWorkout method:

```
    # Save workout to database
    def logWorkout(self):
        workoutType = self.workoutTypeEntry.get().strip()
        durationStr = self.durationEntry.get().strip()
        caloriesStr = self.caloriesEntry.get().strip()

        # Validate inputs
        if len(workoutType) == 0:
            return
        if len(durationStr) == 0:
            return
        if len(caloriesStr) == 0:
            return

        if durationStr.isdigit():
            duration = int(durationStr)
        else:
            return

        if caloriesStr.isdigit():
            caloriesBurned = int(caloriesStr)
        else:
            return

        if duration <= 0:
            return
        if caloriesBurned <= 0:
            return

        todayDate = datetime.now().strftime("%Y-%m-%d")

        # Insert workout into database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO WorkoutLogs (userId, date, workoutType, durationMin, caloriesBurned)
            VALUES (?, ?, ?, ?, ?)
        """, (
            self.container.currentUserId,
            todayDate,
            workoutType,
            duration,
            caloriesBurned
        ))
        conn.commit()
        conn.close()

        # Clear input fields
        self.workoutTypeEntry.delete(0, tk.END)
        self.durationEntry.delete(0, tk.END)
        self.caloriesEntry.delete(0, tk.END)

        self.loadTodaysWorkouts()
        self.updateWorkoutSummary()
```

Loadtodaysworkout subroutine:

LoadTodaysWorkouts retrieves all individual workouts from the database, showing each workout's type, duration, and calories burned in a listbox, or a message if no workouts are logged.

```
749     # Load today's workouts from database
750     def loadTodaysWorkouts(self):
751         todayDate = datetime.now().strftime("%Y-%m-%d")
752 
753         self.todaysWorkoutsListbox.delete(0, tk.END)
754 
755         # Fetch workouts from database
756         conn = connectDatabase()
757         cursor = conn.cursor()
758 
759         cursor.execute("""
760             SELECT logId, workoutType, durationMin, caloriesBurned
761             FROM WorkoutLogs
762             WHERE userId = ? AND date = ?
763             ORDER BY logId
764         """, (self.container.currentUserId, todayDate))
765 
766         workouts = cursor.fetchall()
767         conn.close()
768 
769         if len(workouts) == 0:
770             self.todaysWorkoutsListbox.insert(tk.END, "No workouts logged today")
771             return
772 
773         # Display workouts
774         for workout in workouts:
775             logId = workout[0]
776             workoutType = workout[1]
777             duration = workout[2]
778             calories = workout[3]
779             displayText = str(workoutType) + "  " + str(duration) + " min   " + str(calories) + " cal burned "
780             self.todaysWorkoutsListbox.insert(tk.END, displayText)
781 
```

Display workouts in an easily readable format for inexperienced users

Updateworkoutsummary subroutine:

UpdateWorkoutSummary calculates totals from today's workouts, including the total number of workouts, total duration, and total calories burned, and displays this summary in a separate listbox.

```python
    # Calculate and display workout summary
    def updateWorkoutSummary(self):
        todayDate = datetime.now().strftime("%Y-%m-%d")

        # Sum workout totals from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT
                SUM(durationMin) as totalDuration,
                SUM(caloriesBurned) as totalCalories,
                COUNT(*) as workoutCount
            FROM WorkoutLogs
            WHERE userId = ? AND date = ?
        """, (self.container.currentUserId, todayDate))

        totals = cursor.fetchone()
        conn.close()

        if totals[0] is None:
            totalDuration = 0
            totalCalories = 0
            workoutCount = 0
        else:
            totalDuration = int(totals[0])
            totalCalories = int(totals[1])
            workoutCount = int(totals[2])

        # Display summary
        self.workoutSummaryListbox.delete(0, tk.END)
        self.workoutSummaryListbox.insert(tk.END, "Date: " + str(todayDate))
        self.workoutSummaryListbox.insert(tk.END, "Total Workouts: " + str(workoutCount))
        self.workoutSummaryListbox.insert(tk.END, "Total Duration: " + str(totalDuration) + " min")
        self.workoutSummaryListbox.insert(tk.END, "Total Calories Burned: " + str(totalCalories) + " cal")
```

LoadTodaysWorkouts and updateWorkoutSummary work together to show the user's workouts for today.

Delete Workout Issue :

I then moved onto the deleteworkout function, when trying to create a deleteworkout function i realised in order for users to be able to delete a workout entry, each listbox item needs to be paired with a unique identifier such as the workout log ID from the database.

I initially tried parsing the entry by name or duration but it didn't work as these fields are not unique to each workout as multiple workouts can have the same name or duration, so the program wouldn't know which one to delete. By including the log ID in each listbox entry, the program can accurately identify and remove the specific workout the user selects, even if other entries have the same name or duration.

This is the new loadTodaysWorkouts function with "logid" being put in the listbox.

```python
    # Load today's workouts from database
    def loadTodaysWorkouts(self):
        todayDate = datetime.now().strftime("%Y-%m-%d")

        self.todaysWorkoutsListbox.delete(0, tk.END)

        # Fetch workouts from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT logId, workoutType, durationMin, caloriesBurned
            FROM WorkoutLogs
            WHERE userId = ? AND date = ?
            ORDER BY logId
        """, (self.container.currentUserId, todayDate))

        workouts = cursor.fetchall()
        conn.close()

        if len(workouts) == 0:
            self.todaysWorkoutsListbox.insert(tk.END, "No workouts logged today")
            return

        # Display workouts
        for workout in workouts:
            logId = workout[0]
            workoutType = workout[1]
            duration = workout[2]
            calories = workout[3]
            displayText = str(workoutType) + "  " + str(duration) + " min   " + str(calories) + " cal burned [ID: " + str(logId) + "]"
            self.todaysWorkoutsListbox.insert(tk.END, displayText)

```

To allow the delete workout method to parse the listbox

This now allows the program to correctly parse the listbox entry to be able to locate the workout and therefore remove it from the database. split() is used twice to remove all information other than the id so we are able to query the logid in the database and delete the correct workout.

```python
    # Delete selected workout from database
    def deleteWorkout(self):
        selection = self.todaysWorkoutsListbox.curselection()

        if len(selection) == 0:
            return

        index = selection[0]
        selectedItem = self.todaysWorkoutsListbox.get(index)

        # Extract workout ID from display text
        if "[ID: " in selectedItem:
            logIdStr = selectedItem.split("[ID: ")[1].split("]")[0]
            logId = int(logIdStr)

            # Delete from database
            conn = connectDatabase()
            cursor = conn.cursor()
            cursor.execute("DELETE FROM WorkoutLogs WHERE logId = ? AND userId = ?", (logId, self.container.currentUserId))
            conn.commit()
            conn.close()

            self.loadTodaysWorkouts()
            self.updateWorkoutSummary()
```

Parsing using [ID: and] and getting contents in the middle

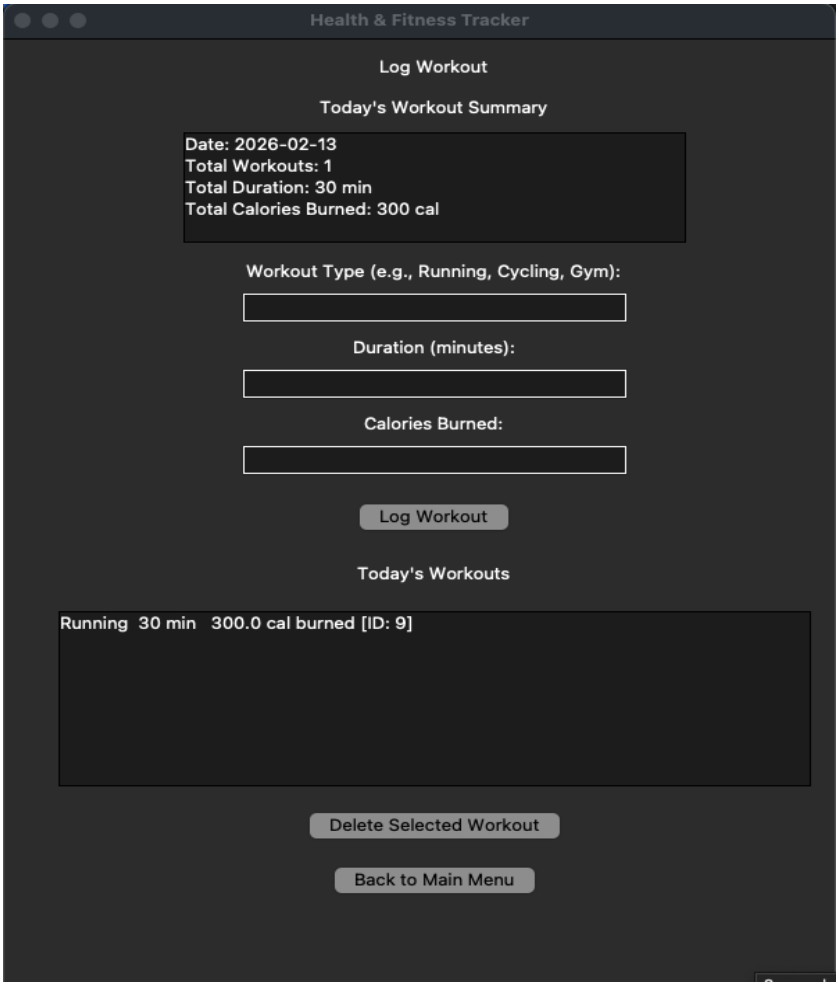
Now that the code for the Log Workout section is complete I will begin testing to identify any errors I may have made along the way .

Iteration 3 - Testing

Test 3.1: Log Workout

Test	Input	Expected Output	Actual Outcome
Add workout	1. Workout Type: Running 2. Duration: 30 3. Calories Burned: 300 4. Click "Log Workout" button	Workout logged and appears in "Today's Workouts" listbox	Workout logged

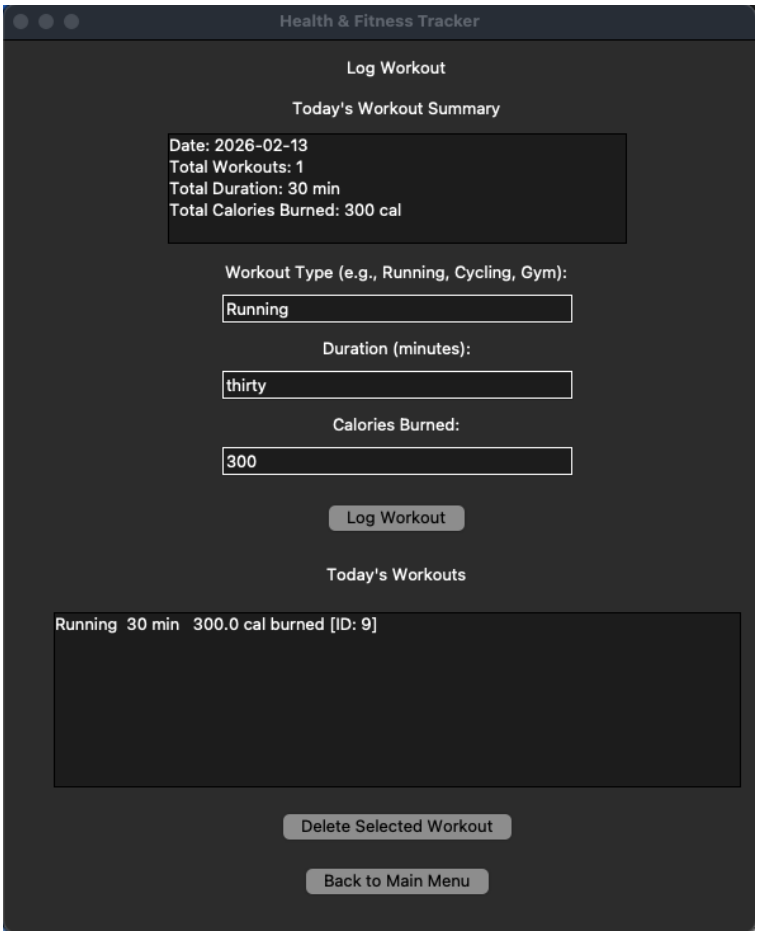
Proof:



Test 3.2: Non-Numeric Duration Validation

Test	Input	Expected Output	Actual Outcome
Non-numeric duration	1. Workout Type: Running 2. Duration: thirty (text instead of number) 3. Calories Burned: 300 4. Click "Log Workout" button	Error message: "Please enter valid numeric values" or workout rejected	Workout was not logged but also no error message

Does not allow you to log workout but also does not display an Error Message



I realised i had forgotten to code this aspect so i went back to change it .

Old code:

```
700         # Validate inputs
701         if len(workoutType) == 0:
702             return
703         if len(durationStr) == 0:
704             return
705         if len(caloriesStr) == 0:
706             return
707 
708         if durationStr.isdigit():
709             duration = int(durationStr)
710         else:
711             return
712 
713         if caloriesStr.isdigit():
714             caloriesBurned = int(caloriesStr)
715         else:
716             return
717 
718         if duration <= 0:
719             return
720         if caloriesBurned <= 0:
721             return
```

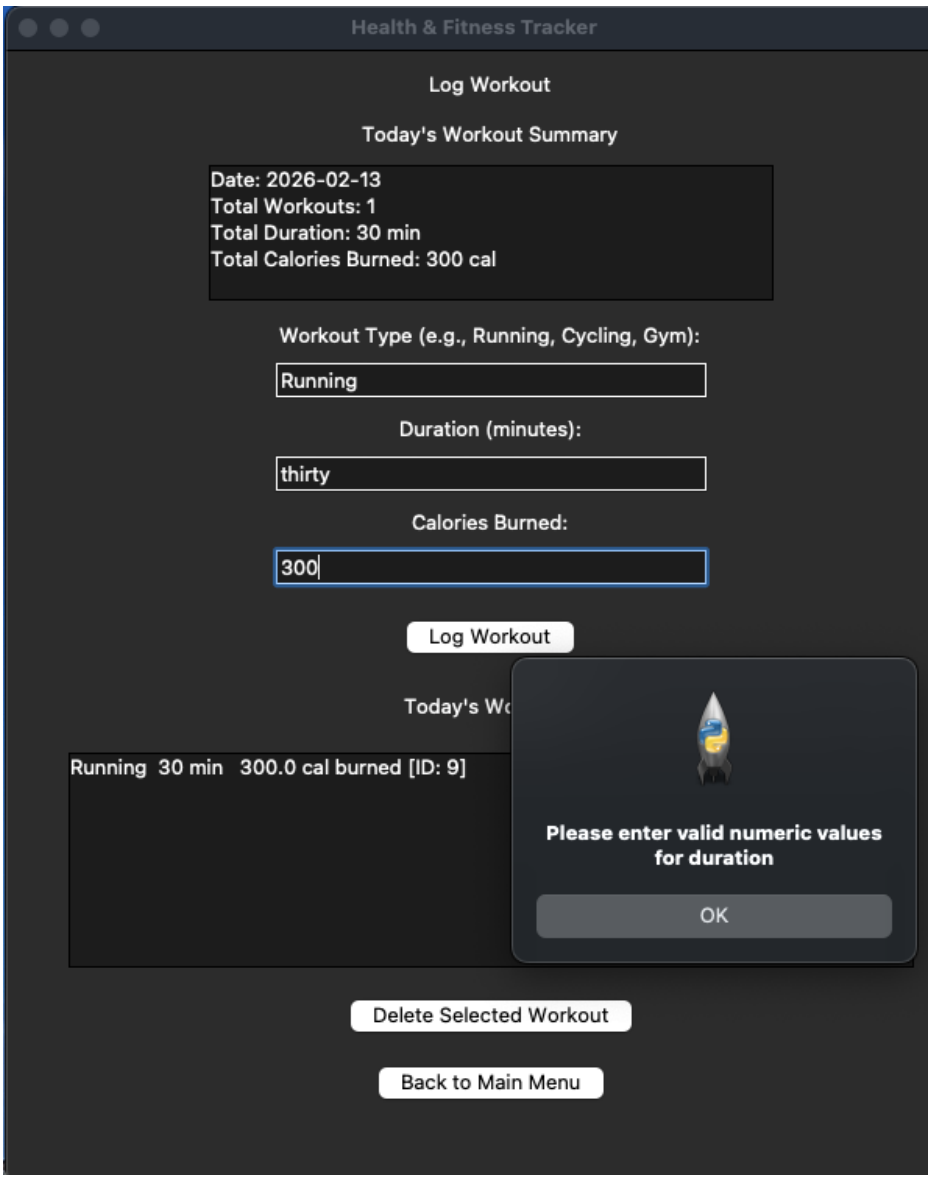
I also noticed that I hadn't coded messages for any other invalid inputs so I changed this .

New code with validation for all fields:

```
700         # Validate inputs
701         if len(workoutType) == 0:
702             return
703         if len(durationStr) == 0:
704             return
705         if len(caloriesStr) == 0:
706             return
707 
708         if durationStr.isdigit():
709             duration = int(durationStr)
710         else:
711             messagebox.showerror("Error", "Please enter valid numeric values for duration")
712             return
713 
714         # Validate calories is a digit
715         if caloriesStr.isdigit():
716            caloriesBurned = int(caloriesStr)
717         else:
718             messagebox.showerror("Error", "Please enter valid numeric values for calories")
719             return
720 
721         # Validate positive values
722         if duration <= 0:
723             messagebox.showerror("Error", "Duration must be a positive number")
724             return
725 
726         if caloriesBurned <= 0:
727             messagebox.showerror("Error", "Calories must be a positive number")
728             return
729         todayDate = datetime.now().strftime("%Y-%m-%d")
730 
```

Now the error message shows up :

Proof:



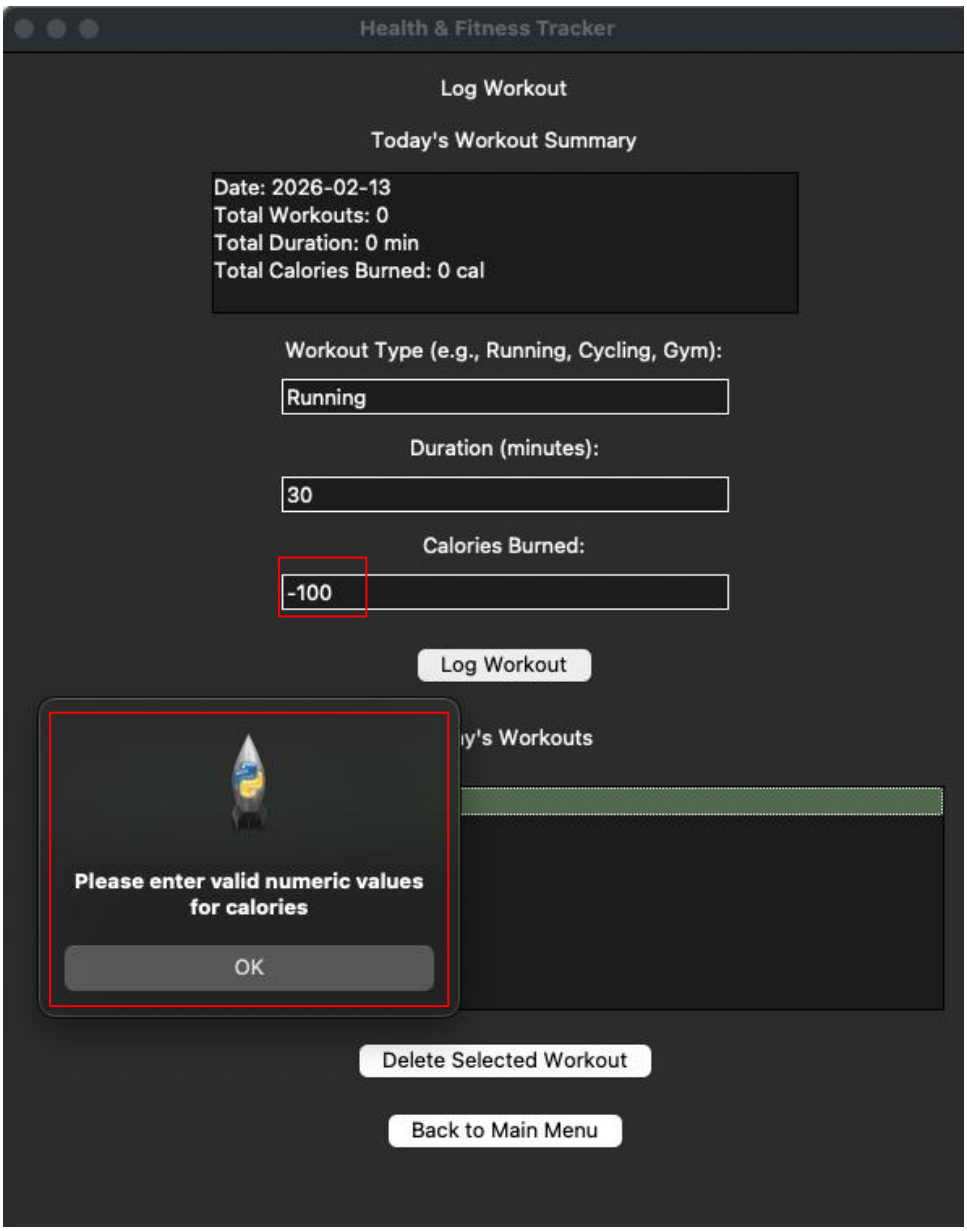
Test	Input	Expected Output	Actual Outcome
Non-numeric duration	1. Workout Type: Running 2. Duration: thirty (text instead of number) 3. Calories Burned: 300	Error message: "Please enter valid numeric values" or workout rejected	Workout was not logged and also error message shown

	4. Click "Log Workout" button		

Test 3.3: Negative Calories Validation

Test	Input	Expected Output	Actual Outcome
Negative calories	1. Workout Type: Running 2. Duration: 30 3. Calories Burned: -100 4. Click "Log Workout" button	Error message: "Please enter valid numeric values" or workout rejected	Error message and workout rejected

Proof:

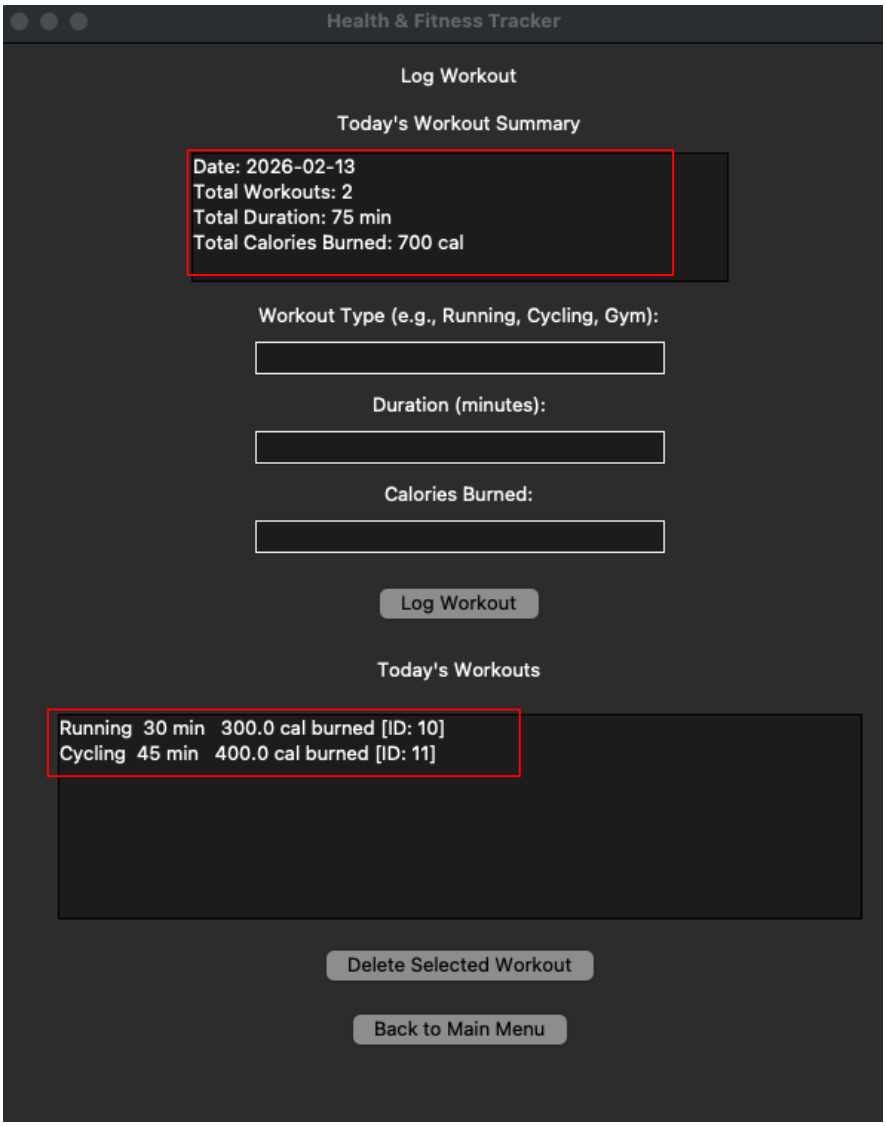


Test 3.4: Auto Totals Summary

Test	Input	Expected Output	Actual Outcome

Multiple workouts summary	1. Log workout 1: Running, 30 min, 300 cal 2. Log workout 2: Cycling, 45 min, 400 cal 3. Check "Today's Workout Summary" listbox	Summary shows: Total workouts: 2, Total duration: 75 minutes, Total calories burned: 700	Summary shows: Total workouts: 2, Total duration: 75 minutes, Total calories burned: 700

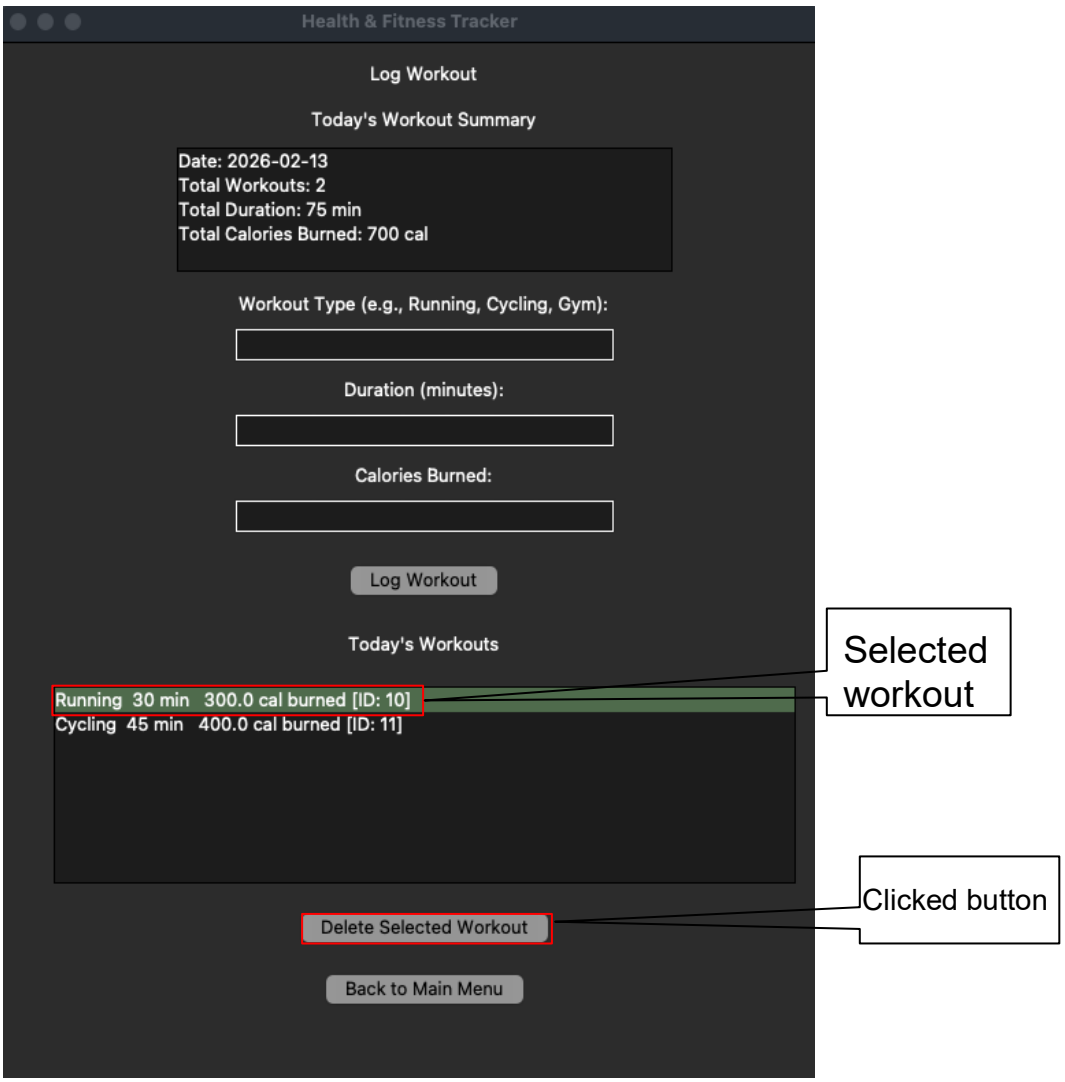
Proof:



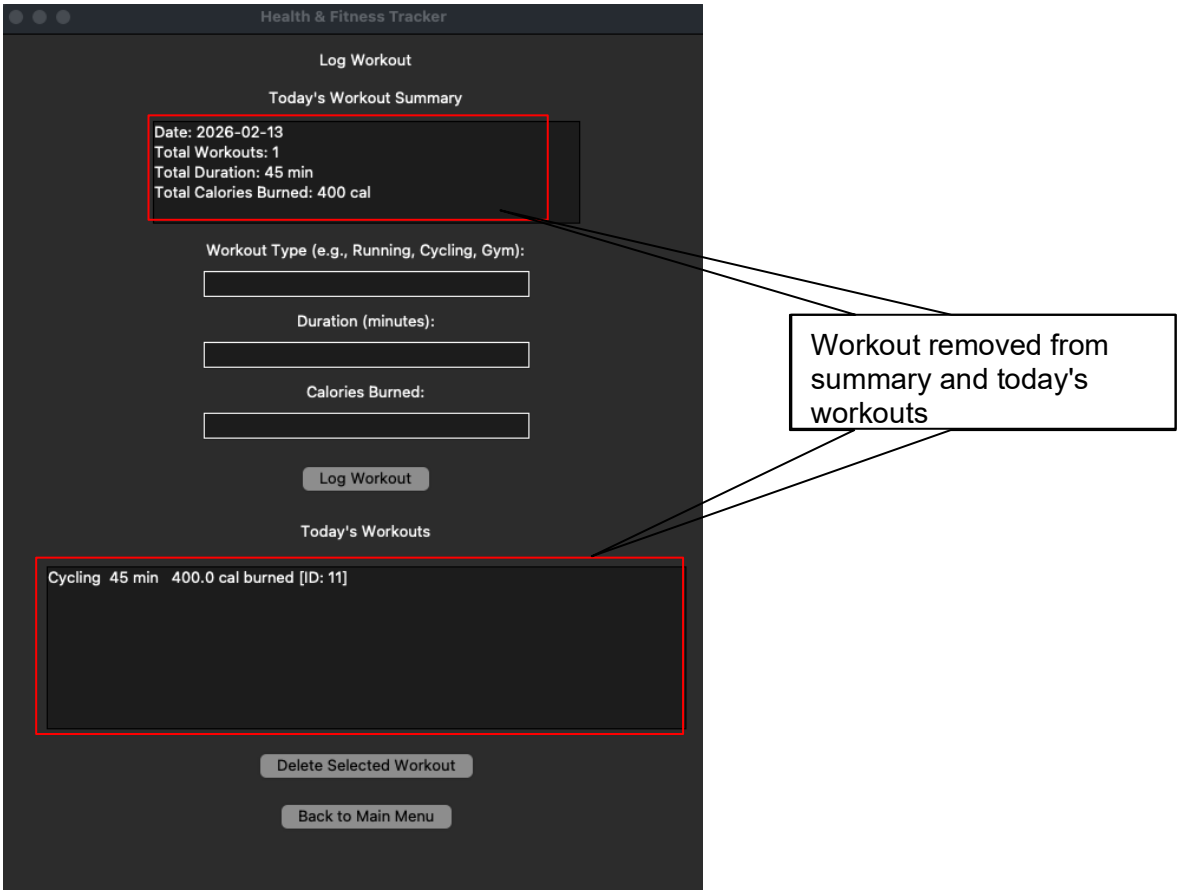
Test 3.5: Delete Workout

Test	Input	Expected Output	Actual Outcome
Delete by list	1. Log a workout 2. Select workout from "Today's Workouts" listbox 3. Click "Delete Selected Workout" button	Selected workout removed from listbox and database	Selected workout removed from listbox and database

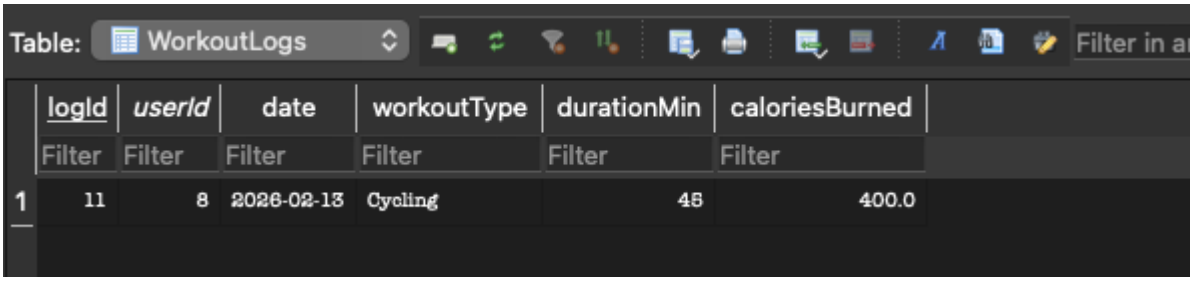
Proof:



Result:



Also deleted from database :



Testing for the Log workout section is now complete with all sections passed. Now i can move onto the View stats section.

Iteration 4 - View Stats

This iteration implements the View Stats module . It is responsible for displaying health statistics and generating matplotlib graphs.

Development

The ViewStats class creates a Tkinter frame for displaying the user's stats and graphs.

```
# Statistics and graphs screen
class ViewStats(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

    # Load and display user stats
    def getStats(self):
        # Fetch user stats from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT name, initialWeight, heightCm, bmi FROM users WHERE id = ?", [str(self.container.currentUserId)])

        stats = cursor.fetchall()
        conn.close()

        # Display basic stats
        todolist = tk.Listbox(self, width=70, height=20, font=("Arial", 12))
        todolist.grid(row=0, column=0, padx=20, pady=20)

        for stat in stats:
            todolist.insert(tk.END, "Name: " + str(stat[0]) + " | Weight: " + str(stat[1]) + "kg | Height: " + str(stat[2]) + "m | BMI: " + str(stat[3]))

        # Buttons to show different graphs
        graphButton1 = tk.Button(self, text="Calorie Intake (Last 7 Days)", width=30, command=lambda: self.showCalorieGraph())
        graphButton1.grid(row=1, column=0, pady=5)

        graphButton2 = tk.Button(self, text="Workout Minutes (Last 7 Days)", width=30, command=lambda: self.showWorkoutGraph())
        graphButton2.grid(row=2, column=0, pady=5)

        graphButton3 = tk.Button(self, text="Macronutrient Breakdown (Today)", width=30, command=lambda: self.showMacroGraph())
        graphButton3.grid(row=3, column=0, pady=5)

        backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
        backButton.grid(row=4, column=0, pady=10)

```

Getting users stats from the database (name ,initial weight ,height and BMI)

Format of page (listbox,buttons)

The getStats method connects to the database and retrieves the user's name, weight, height, and BMI. It then creates a listbox to display this information in a readable format for the user.

Below the listbox, it adds three buttons to show different graphs: calorie intake over the last 7 days, workout minutes over the last 7 days, and today's macronutrient breakdown. Finally, I added the back button to return to the main menu.

This frame allows the user to see their current stats and access all the visual summaries I decided on in the design.

The window is formatted as follows:



I will now program the function showCalorieGraph . I will use definite iteration to retrieve the data from the last 7 days to be able to graph visually the change in calorie consumption over time. I will be using matplotlib to create these graphs and datetime to retrieve the information for each day.

```python
# Display bar chart of last 7 days calorie intake
def showCalorieGraph(self):
    conn = connectDatabase()
    cursor = conn.cursor()

    dates = []
    calories = []

    # Get data for each of last 7 days
    for i in range(7):
        date = (datetime.now() - timedelta(days=6 - i)).strftime("%Y-%m-%d")
        dates.append(date[-5:])  # Format as MM-DD

        # Sum calories from database for each day
        cursor.execute("""
            SELECT SUM(totalCal) FROM MealLogs
            WHERE userId = ? AND date = ?
        """, (self.container.currentUserId, date))

        result = cursor.fetchone()
        if result[0]:
            calories.append(result[0])
        else:
            calories.append(0)

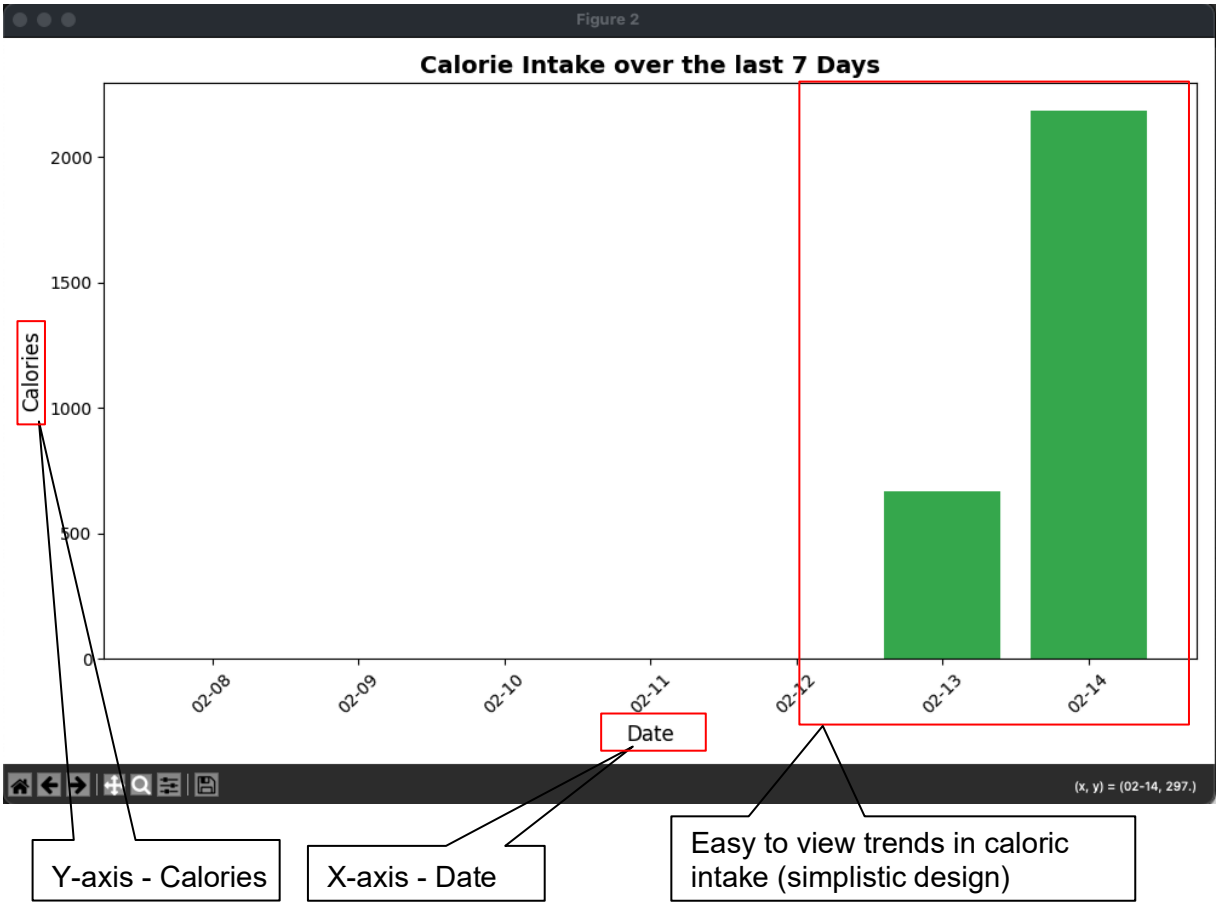
    conn.close()

    # Create bar chart
    plt.figure(figsize=(10, 6))
    plt.bar(dates, calories, color='#4CAF50')
    plt.xlabel('Date', fontsize=12)
    plt.ylabel('Calories', fontsize=12)
    plt.title('Calorie Intake over the last 7 Days', fontsize=14, fontweight='bold')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

Definite iteration to get information for the past 7 days

Creating a bar chart using matplot lib functions such as .bar,.x label,.ylabel

This is how the graph looks:



The Function showWorkoutGraph will use the exact same structure but instead fetch data from the WorkoutLogs table.

```python
# Display bar chart of last 7 days workout minutes
def showWorkoutGraph(self):
    conn = connectDatabase()
    cursor = conn.cursor()

    dates = []
    minutes = []

    # Get data for each of last 7 days
    for i in range(7):
        date = (datetime.now() - timedelta(days=6 - i)).strftime("%Y-%m-%d")
        dates.append(date[-5:])

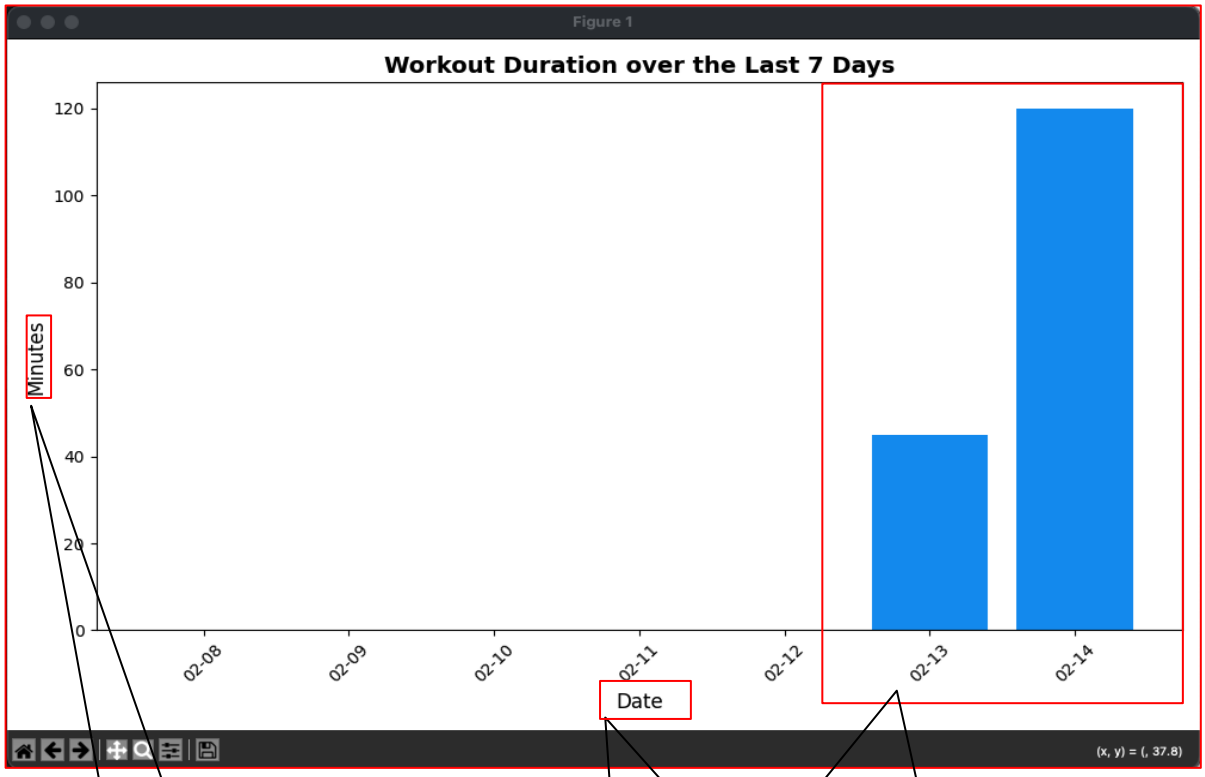
        # Sum workout duration from database for each day
        cursor.execute("""
            SELECT SUM(durationMin) FROM WorkoutLogs
            WHERE userId = ? AND date = ?
        """, (self.container.currentUserId, date))

        result = cursor.fetchone()
        if result[0]:
            minutes.append(result[0])
        else:
            minutes.append(0)

    conn.close()

    # Create bar chart
    plt.figure(figsize=(10, 6))
    plt.bar(dates, minutes, color='#2196F3')
    plt.xlabel('Date', fontsize=12)
    plt.ylabel('Minutes', fontsize=12)
    plt.title('Workout Duration over the Last 7 Days', fontsize=14, fontweight='bold')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

Same overall structure as showCalorieGraph()



Y-axis - Minutes

X-axis - Date

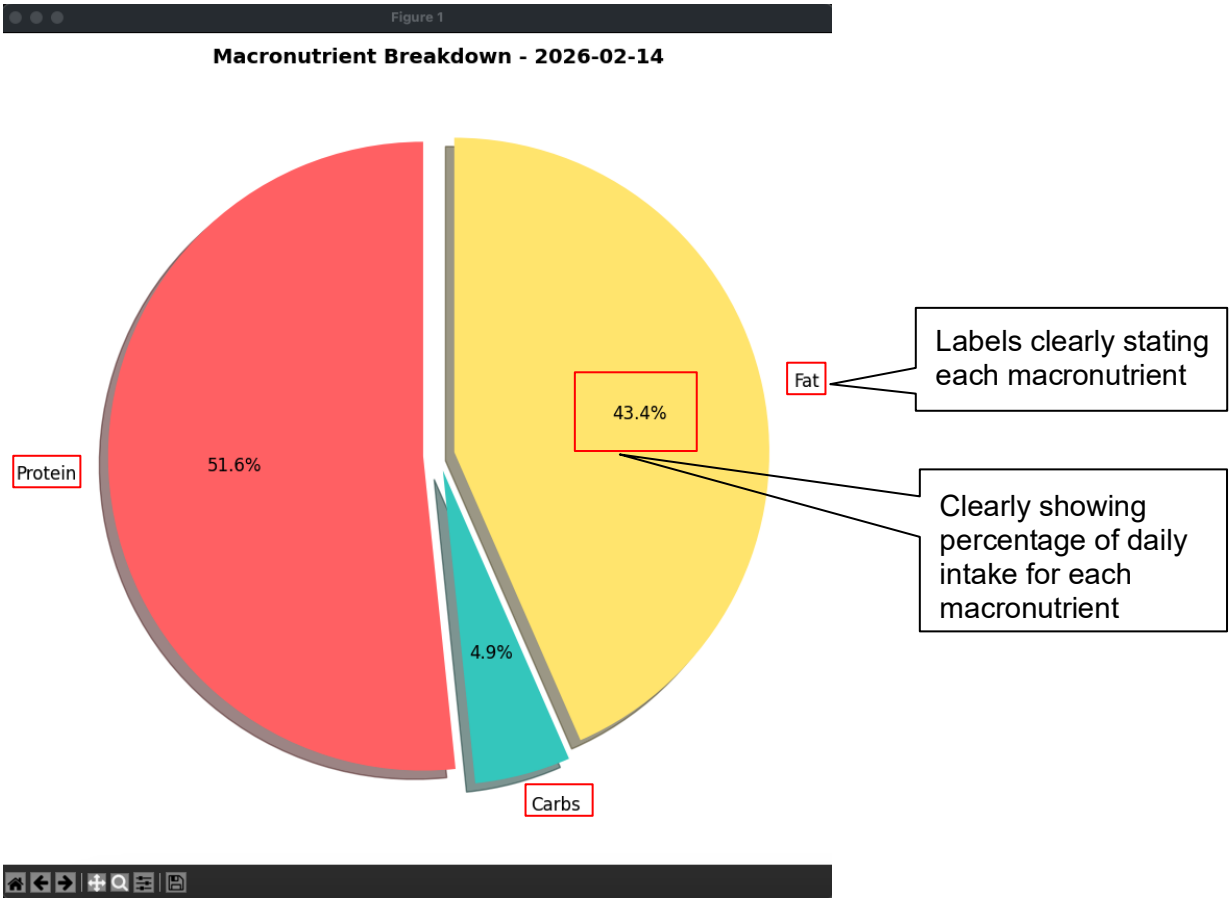
Easy to view trends in caloric intake (simplistic design)

The showMacroGraph will follow a slightly different structure as it is a pie chart. It retrieves the macronutrient totals (protein, carbs, fat) for the day. While this only uses a single day it still gives an insightful breakdown of the users macronutrient distribution to help them see changes that need to be made to their diet.

```
959     # Display pie chart of today's macronutrient breakdown
960     def showMacroGraph(self):
961         conn = connectDatabase()
962         cursor = conn.cursor()
963 
964         todayDate = datetime.now().strftime("%Y-%m-%d")
965 
966         # Get today's macro totals from database
967         cursor.execute("""
968             SELECT SUM(totalProtein), SUM(totalCarbs), SUM(totalFat)
969             FROM MealLogs
970             WHERE userId = ? AND date = ?
971         """, (self.container.currentUserId, todayDate))
972 
973         result = cursor.fetchone()
974         conn.close()
975 
976         # Don't show graph if no data
977         if result[0] is None:
978             return
979         if result[0] == 0 and result[1] == 0 and result[2] == 0:
980             return
981 
982         if result[0]:
983             protein = result[0]
984         else:
985             protein = 0
986 
987         if result[1]:
988             carbs = result[1]
989         else:
990             carbs = 0
991 
992         if result[2]:
993             fat = result[2]
994         else:
995             fat = 0
996 
997         # Create pie chart
998         labels = ['Protein', 'Carbs', 'Fat']
999         sizes = [protein, carbs, fat]
1000        colors = ['#FF6B6B', '#4ECDC4', '#FFE66D']
1001        explode = (0.05, 0.05, 0.05)
1002
1003        plt.figure(figsize=(8, 8))
1004        plt.pie(sizes, explode=explode, labels=labels, colors=colors, autopct='%1.1f%%',
1005                shadow=True, startangle=90, textprops={'fontsize': 12})
1006        plt.title('Macronutrient Breakdown - ' + str(todayDate), fontsize=14, fontweight='bold')
1007        plt.axis('equal')
1008        plt.tight_layout()
1009        plt.show()
```

Coloured for usability and so that data is easy to understand

This is what the pie chart looks like :

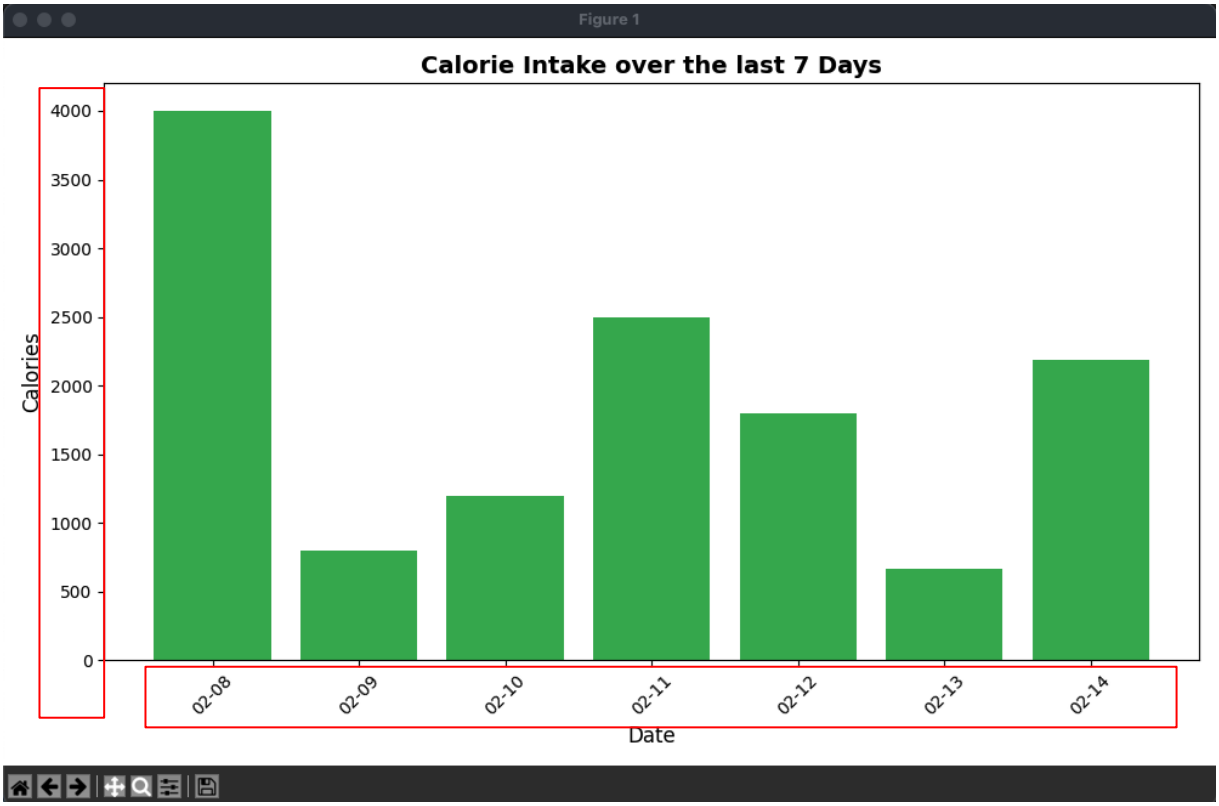


Now that I have completed programming the view stats section I will continue to the testing phase.

Iteration 4 - Testing

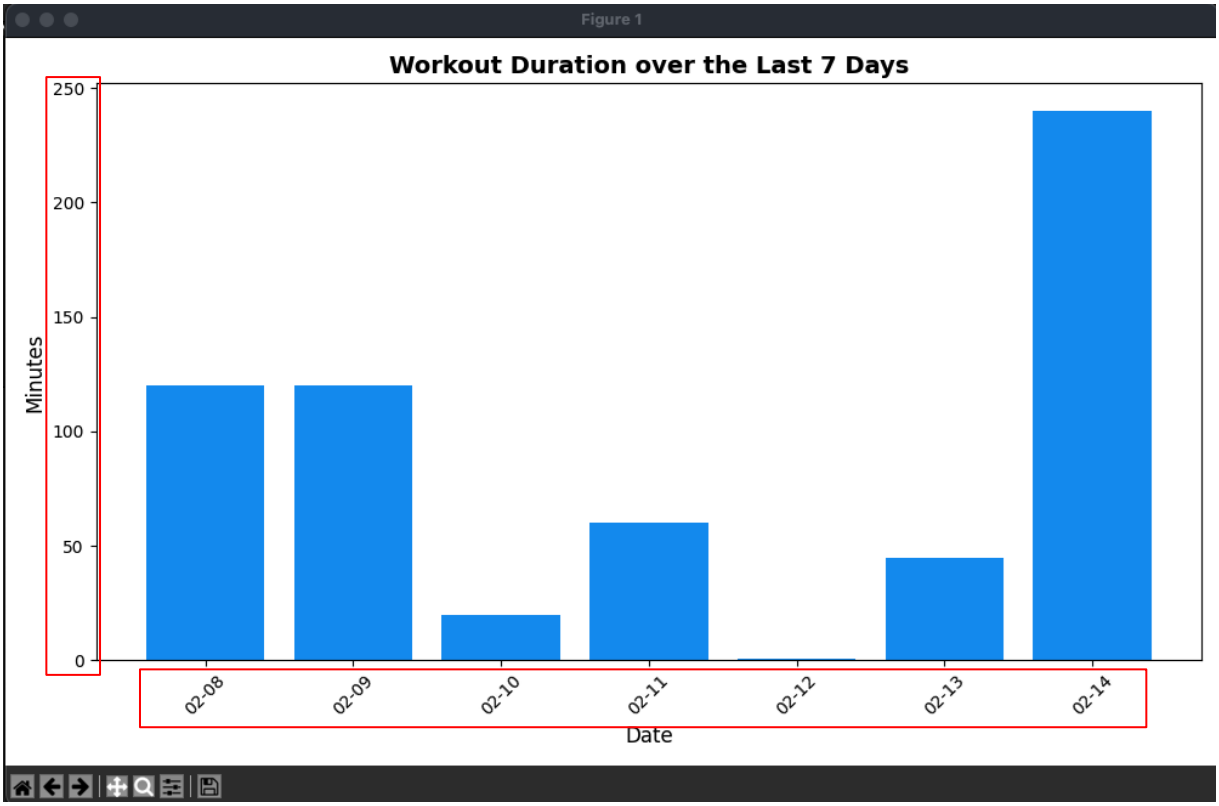
Test 4.1: 7-Day Calorie Graph

Test	Input	Expected Output	Actual Outcome
7-day calorie data	1. Log meals across 7 different days (or use pre-existing data) 2. Navigate to View Stats page 3. Click "Calorie Intake Graph" button	Bar chart displays with 7 bars showing daily calorie totals for the last 7 days with date labels	Bar chart with 7 bars showing daily calorie totals for the last 7 days with date labels



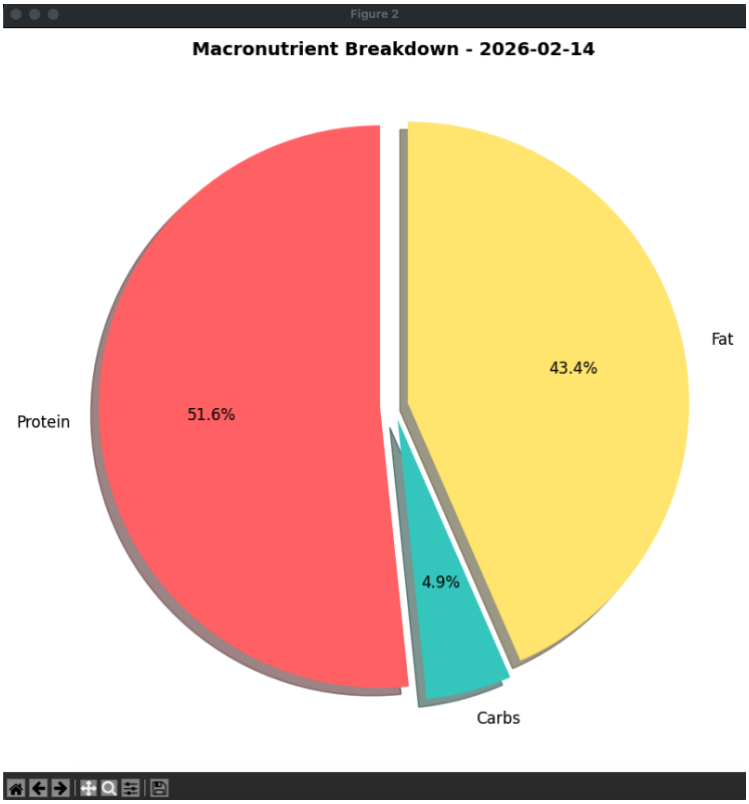
Test 4.2: 7-Day Workout Graph

Test	Input	Expected Output	Actual Outcome
7-day workout data	1. Log workouts across 7 different days (or use pre-existing data) 2. Navigate to View Stats page 3. Click "Workout Duration Graph" button	Bar chart displays with 7 bars showing daily workout duration (minutes) for the last 7 days with date labels	Bar chart displays 7 bars showing daily workout duration for the last 7 days with date labels



Test 4.3: Today's Macro Pie Chart

Test	Input	Expected Output	Actual Outcome
Today's macros	1. Log 2-3 meals today with varying macros 2. Navigate to View Stats page 3. Click "Macro Breakdown Graph" button	Pie chart displays three sections (Protein, Carbs, Fat) showing proportional breakdown of today's macronutrients with percentages	Pie chart shows 3 sections showing proportional breakdown with percentages



Test 4.4: User Statistics Display

Test	Input	Expected Output	Actual Outcome
View stats	Navigate to View Stats page	Listbox displays: Name, Weight (kg), Height (cm), BMI	Listbox displays name, weight, height, BMI

Proof:



All stages of testing for the Stats section have been passed successfully and now I will begin iteration 5 .

Iteration 5 – Settings

This iteration implements the Settings module. It is responsible for allowing users to update their current and goal weights and allowing the user to change their password.

Development

The Settings class creates a Tkinter frame for updating user information. It allows the user to update their current weight and goal weight through 2 entry fields and a save button. It also provides a section to change the password, with an entry for the new password and a button to submit the change.

Finally, there is the button to return to the main menu. This frame organises the settings in a clear way, separating weight updates from password changes clearly for usability reasons.

```
841  class Settings(tk.Frame):
842      def __init__(self, container):
843          super().__init__(container)
844          self.container = container
845
846          tk.Label(self, text="Settings", font=("Helvetica", 18, "bold")).pack(pady=20)
847
848          tk.Label(self, text="Update Current Weight (kg):").pack(pady=5)
849          self.weightEntry = tk.Entry(self, width=30)
850          self.weightEntry.pack(pady=5)
851
852          tk.Label(self, text="Update Goal Weight (kg):").pack(pady=5)
853          self.goalWeightEntry = tk.Entry(self, width=30)
854          self.goalWeightEntry.pack(pady=5)
855
856          tk.Button(self, text="Save Changes", width=20, command=self.updateSettings).pack(pady=20)
857
858          tk.Label(self, text="Change Password", font=("Helvetica", 12, "bold")).pack(pady=15)
859
860          tk.Label(self, text="New Password:").pack(pady=5)
861          self.newPasswordEntry = tk.Entry(self, width=30, show="*")
862          self.newPasswordEntry.pack(pady=5)
863
864          tk.Button(self, text="Update Password", width=20, command=self.changePassword).pack(pady=15)
865
866          tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu")).pack(pady=10)
867
```

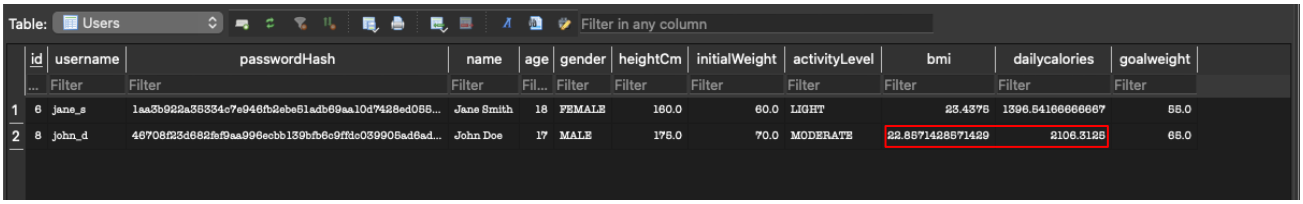
The updateSettings method gets the user's input for current and goal weight, checks that the fields are not empty, and validates that the input is a proper float. If the inputs are valid, it updates the values in the database and commits the changes .

```
868      def updateSettings(self):
869          weightStr = self.weightEntry.get().strip()
870          goalWeightStr = self.goalWeightEntry.get().strip()
871
872          if len(weightStr) == 0 and len(goalWeightStr) == 0:
873              return
874
875          conn = connectDatabase()
876          cursor = conn.cursor()
877
878          if len(weightStr) > 0 and weightStr.replace('.', '', 1).isdigit() and weightStr.count('.') <= 1:
879              weight = float(weightStr)
880              cursor.execute("UPDATE Users SET initialWeight = ? WHERE id = ?", (weight, self.container.currentUserId))
881
882          if len(goalWeightStr) > 0 and goalWeightStr.replace('.', '', 1).isdigit() and goalWeightStr.count('.') <= 1:
883              goalWeight = float(goalWeightStr)
884              cursor.execute("UPDATE Users SET goalweight = ? WHERE id = ?", (goalWeight, self.container.currentUserId))
885
886          conn.commit()
887          conn.close()
888
889          self.weightEntry.delete(0, tk.END)
890          self.goalWeightEntry.delete(0, tk.END)
891
892
893          self.weightEntry.delete(0, tk.END)
894          self.goalWeightEntry.delete(0, tk.END)
895
```

Daily Calories not Recalculating on Weight Update Issue:

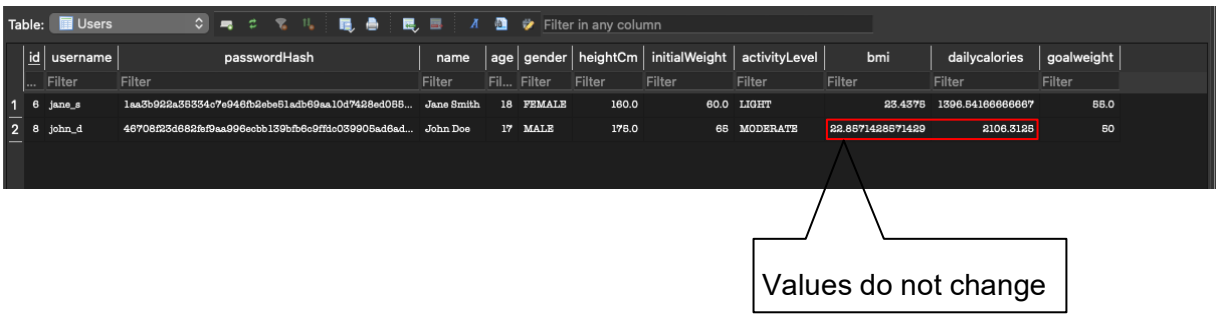
I identified an issue as, when initial weight and goal weight are changed the daily calories for the user do not change.

Database before changing weights:



	id	username	passwordHash	name	age	gender	heightCm	initialWeight	activityLevel	bmi	dailycalories	goalweight
1	6	jane_s	1aa3b922a36334c7e946fb2ebe51adb69aa10d7428ed088...	Jane Smith	18	FEMALE	160.0	60.0	LIGHT	23.4375	1396.6416666667	55.0
2	8	john_d	46708f23d682fef9aa996ecbb139bfb6c9ffdc039905ad6ad...	John Doe	17	MALE	175.0	70.0	MODERATE	22.857142857142	2106.3125	65.0

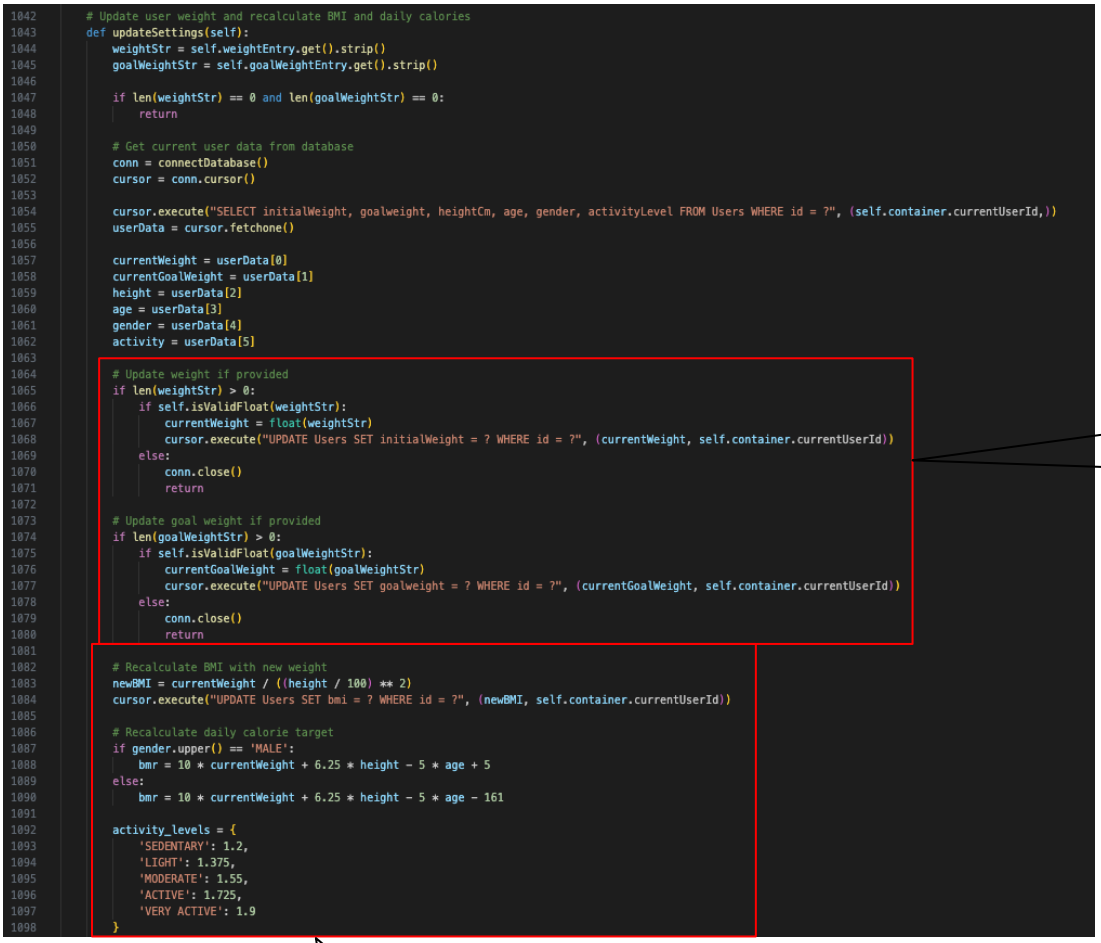
After changing weights in settings:



	id	username	passwordHash	name	age	gender	heightCm	initialWeight	activityLevel	bmi	dailycalories	goalweight
1	6	jane_s	1aa3b922a36334c7e946fb2ebe51adb69aa10d7428ed088...	Jane Smith	18	FEMALE	160.0	60.0	LIGHT	23.4375	1396.6416666667	55.0
2	8	john_d	46708f23d682fef9aa996ecbb139bfb6c9ffdc039905ad6ad...	John Doe	17	MALE	175.0	65	MODERATE	22.857142857142	2106.3125	60

These values should change and be updated as the weight of the user is updated, this would mislead users and stop them from reaching their desired outcome.

I modified the updateSettings method to not only store new weight values in the database, but also to automatically recalculate and update the user's BMI and daily calorie target whenever their weight changes.I used the same code for the calculations as i created earlier in the sign up menu .



```python
    # Update user weight and recalculate BMI and daily calories
    def updateSettings(self):
        weightStr = self.weightEntry.get().strip()
        goalWeightStr = self.goalWeightEntry.get().strip()

        if len(weightStr) == 0 and len(goalWeightStr) == 0:
            return

        # Get current user data from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("SELECT initialWeight, goalweight, heightCm, age, gender, activityLevel FROM Users WHERE id = ?", (self.container.currentUserId,))
        userData = cursor.fetchone()

        currentWeight = userData[0]
        currentGoalWeight = userData[1]
        height = userData[2]
        age = userData[3]
        gender = userData[4]
        activity = userData[5]

        # Update weight if provided
        if len(weightStr) > 0:
            if self.isValidFloat(weightStr):
                currentWeight = float(weightStr)
                cursor.execute("UPDATE Users SET initialWeight = ? WHERE id = ?", (currentWeight, self.container.currentUserId))
            else:
                conn.close()
                return

        # Update goal weight if provided
        if len(goalWeightStr) > 0:
            if self.isValidFloat(goalWeightStr):
                currentGoalWeight = float(goalWeightStr)
                cursor.execute("UPDATE Users SET goalweight = ? WHERE id = ?", (currentGoalWeight, self.container.currentUserId))
            else:
                conn.close()
                return

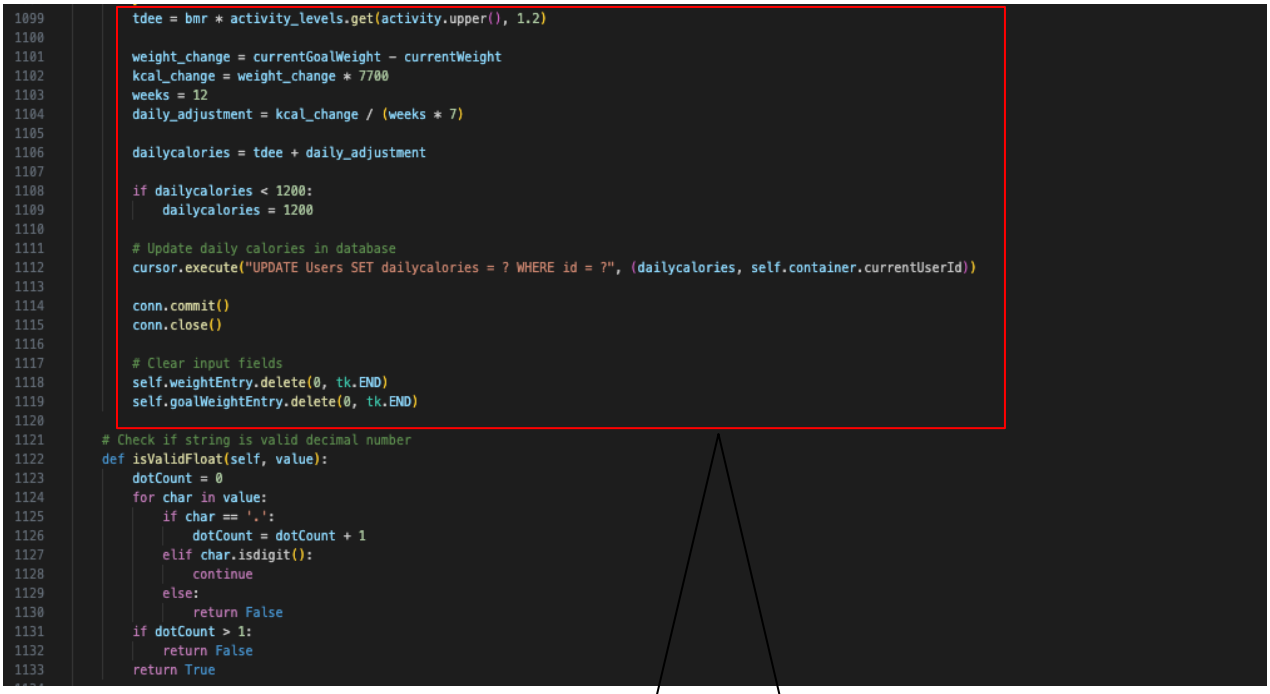
        # Recalculate BMI with new weight
        newBMI = currentWeight / ((height / 100) ** 2)
        cursor.execute("UPDATE Users SET bmi = ? WHERE id = ?", (newBMI, self.container.currentUserId))

        # Recalculate daily calorie target
        if gender.upper() == 'MALE':
            bmr = 10 * currentWeight + 6.25 * height - 5 * age + 5
        else:
            bmr = 10 * currentWeight + 6.25 * height - 5 * age - 161

        activity_levels = {
            'SEDENTARY': 1.2,
            'LIGHT': 1.375,
            'MODERATE': 1.55,
            'ACTIVE': 1.725,
            'VERY ACTIVE': 1.9
        }
```

Change weight in DB

Re calculating BMI BMR at the point when the user changes their weight



```
        tdee = bmr * activity_levels.get(activity.upper(), 1.2)

        weight_change = currentGoalWeight - currentWeight
        kcal_change = weight_change * 7700
        weeks = 12
        daily_adjustment = kcal_change / (weeks * 7)

        dailycalories = tdee + daily_adjustment

        if dailycalories < 1200:
            dailycalories = 1200

        # Update daily calories in database
        cursor.execute("UPDATE Users SET dailycalories = ? WHERE id = ?", (dailycalories, self.container.currentUserId))

        conn.commit()
        conn.close()

        # Clear input fields
        self.weightEntry.delete(0, tk.END)
        self.goalWeightEntry.delete(0, tk.END)

    # Check if string is valid decimal number
    def isValidFloat(self, value):
        dotCount = 0
        for char in value:
            if char == '.':
                dotCount = dotCount + 1
            elif char.isdigit():
                continue
            else:
                return False
        if dotCount > 1:
            return False
        return True
```

Calculate new TDEE and update in database

How this change ensures data integrity:
Previously, if a user updated their weight for example, from 80 kg to 70 kg the database would reflect the new weight but retain the old daily calorie target calculated for 80 kg. This caused the data to be largely inconsitent, as the calorie goal no longer matched the user's actual weight, which would lead to inaccurate tracking and make it harder to reach health and fitness goals.

With the new approach, whenever weight or goal weight is updated, the method now retrieves all the relevant data and then recalculates their BMI,BMR, and daily calorie target. This means that all related data is actually representative and the daily calorie target always reflects the user's current weight and goals. This prevents misleading information and helps users to achieve their desired outcome.

I then created the final subroutine changePassword to allow users to change their password. I used the same logic as was used in the account registration setting no limits on the users password .

```
1135    # Update password in database
1136    def changePassword(self):
1137        newpassword = self.newPasswordEntry.get()
1138        hashed = self.container.hasher.hash(newpassword)  # Hash new password
1139
1140        # Update password hash in database
1141        conn = connectDatabase()
1142        cursor = conn.cursor()
1143        cursor.execute("UPDATE Users SET passwordHash = ? WHERE id = ?", (hashed, self.container.currentUserId))
1144        conn.commit()
1145        conn.close()
1146
1147        self.newPasswordEntry.delete(0, tk.END)
1148
1149        self.newPasswordEntry.delete(0, tk.END)
```

Finally I instantiated the Passwordhasher class to manage password encryption and verification all across the application. Next I created an instance of the HealthTrackerApp class, passing the hasher object as a parameter so that all parts of the app use the same hasher function for consistent password handling during signup and login. Finally, app.mainloop() starts the GUI, keeping the application window open and responsive to user interactions until it is closed by the user.

```
1150
1151    # Run the application
1152    if __name__ == "__main__":
1153        hasher = Passwordhasher()
1154        app = HealthTrackerApp(hasher)
1155        app.mainloop()
```

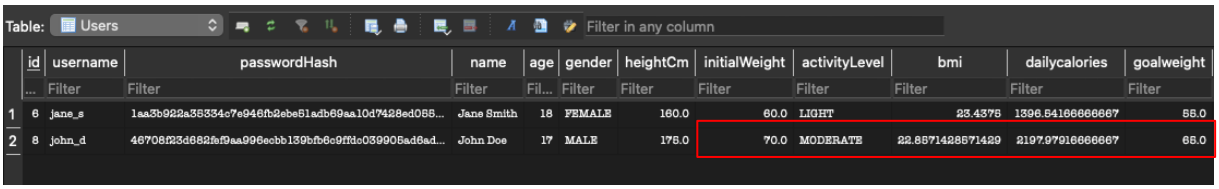
Now that the whole of the code is completed i can begin the final testing section of the User interface and the Settings page.

Iteration 5 - Testing

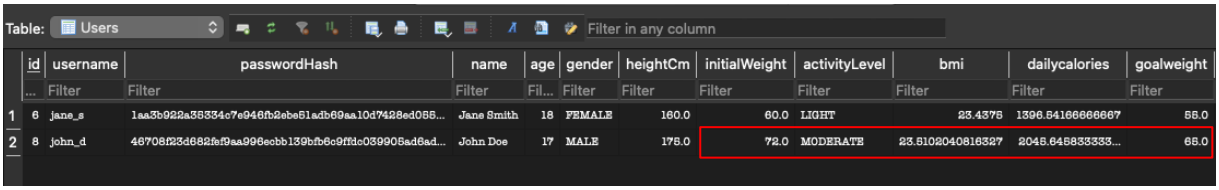
Test 5.1: Update Weight

Test	Input	Expected Output	Actual Outcome
Update weight	1. Navigate to Settings page 2. Enter new weight: 72 3. Click "Update Settings" button	Weight updated in database, BMI recalculated, daily calories recalculated	Weight updated in database,BMI and daily calories were recalculated

Before:



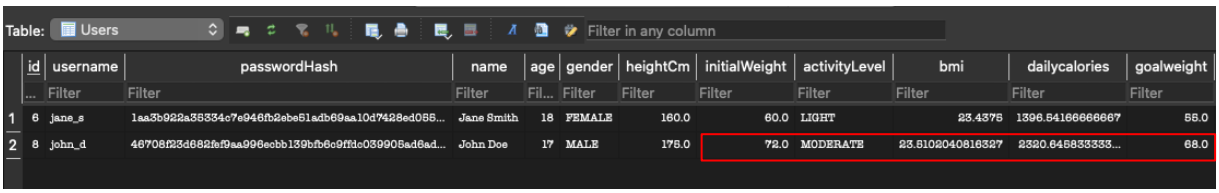
After:



Test 5.2: Update Goal Weight

Test	Input	Expected Output	Actual Outcome
Update goal	1. Navigate to Settings page 2. Enter goal weight: 68 3. Click "Update Settings" button	Goal weight updated in database, daily calories recalculated	Goal weight updated and daily calories and Bmi recalculated

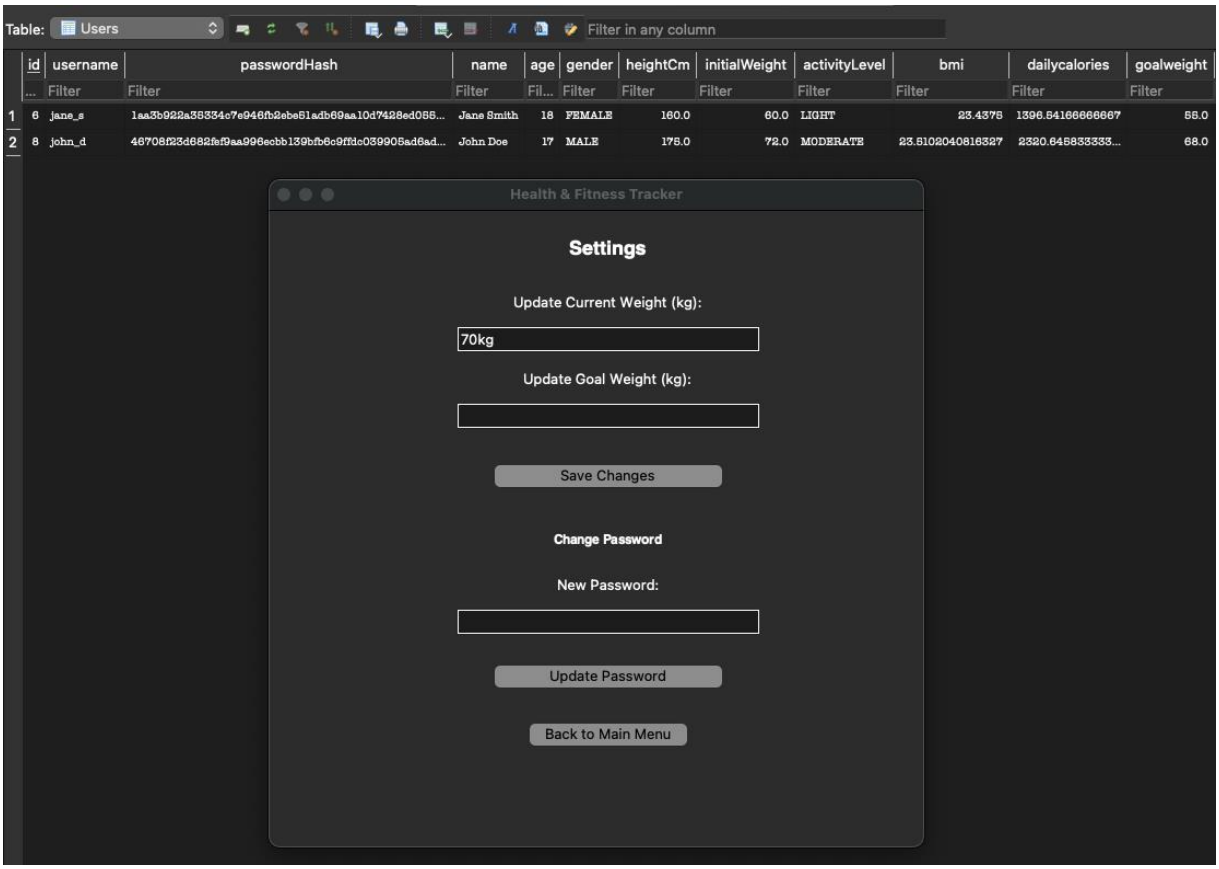
Proof:



Test 5.3: Invalid Weight Validation

Test	Input	Expected Output	Actual Outcome
Invalid weight	**1. Navigate to Settings page** **2. Enter weight: 70kg (text included)** **3. Click "Update Settings" button**	**Error message: "Please enter valid weight" or input rejected**	No error message

When the test data was used there was no update to the database which means the value had been rejected but there was no error message .



After reviewing my code I had realised that when isValidFloat() returns False, the code just closes the connection and returns without showing an error message.

```
# Update weight if provided
if len(weightStr) > 0:
    if self.isValidFloat(weightStr):
        currentWeight = float(weightStr)
        cursor.execute("UPDATE Users SET initialWeight = ? WHERE id = ?", (currentWeight, self.container.currentUserId))
    else:
        conn.close()
        return
```

There should be an error message

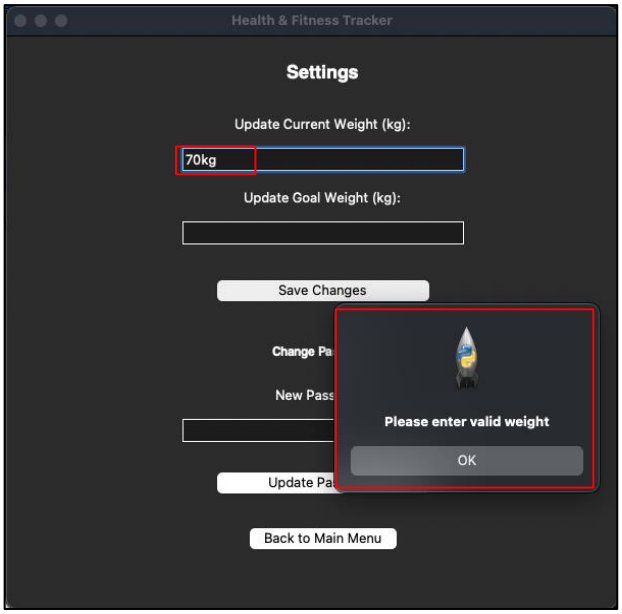
I went back and changed both the goal weight and current weight validation

```
# Update weight if provided
if len(weightStr) > 0:
    if self.isValidFloat(weightStr):
        currentWeight = float(weightStr)
        cursor.execute("UPDATE Users SET initialWeight = ? WHERE id = ?", (currentWeight, self.container.currentUserId))
    else:
        conn.close()
        messagebox.showerror("Error", "Please enter valid weight")
        return

# Update goal weight if provided
if len(goalWeightStr) > 0:
    if self.isValidFloat(goalWeightStr):
        currentGoalWeight = float(goalWeightStr)
        cursor.execute("UPDATE Users SET goalweight = ? WHERE id = ?", (currentGoalWeight, self.container.currentUserId))
    else:
        conn.close()
        messagebox.showerror("Error", "Please enter valid goal weight")
        return
```

I went back and tried the test again and i was met with the expected outcome .

Proof:



Test	Input	Expected Output	Actual Outcome
Invalid weight	**1. Navigate to Settings page** **2. Enter weight: 70kg (text included)** **3. Click "Update Settings" button**	**Error message: "Please enter valid weight" or input rejected**	Error message “ please enter valid weight”

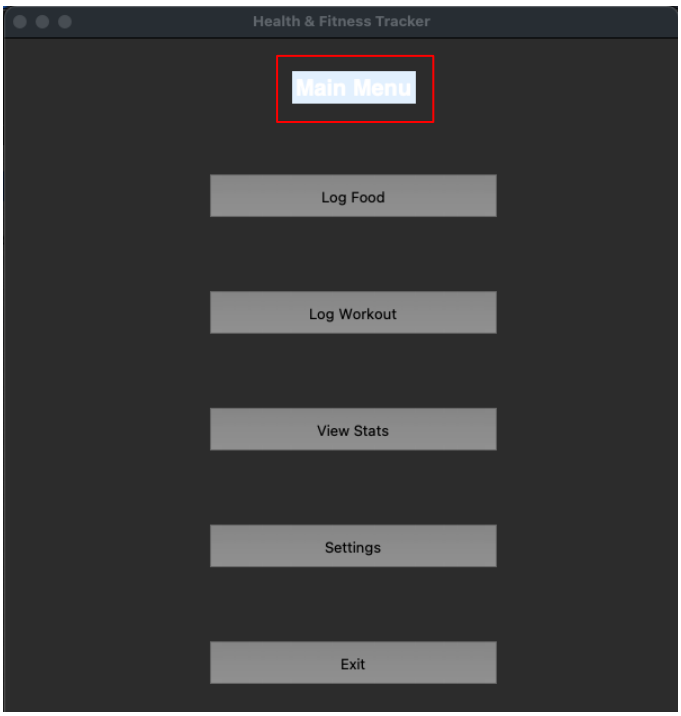
GUI Testing

Tests 5.4 and 5.5 have been skipped and will be addressed later

Test 5.6: Consistent Styling

Test	Input	Expected Output	Actual Outcome
UI styling check	**Open application and navigate through all pages**	**All pages have consistent font styles, colours , and layout**	**Consistent layout and styles but one page had inconsistent colours**

Main Menu screen before:



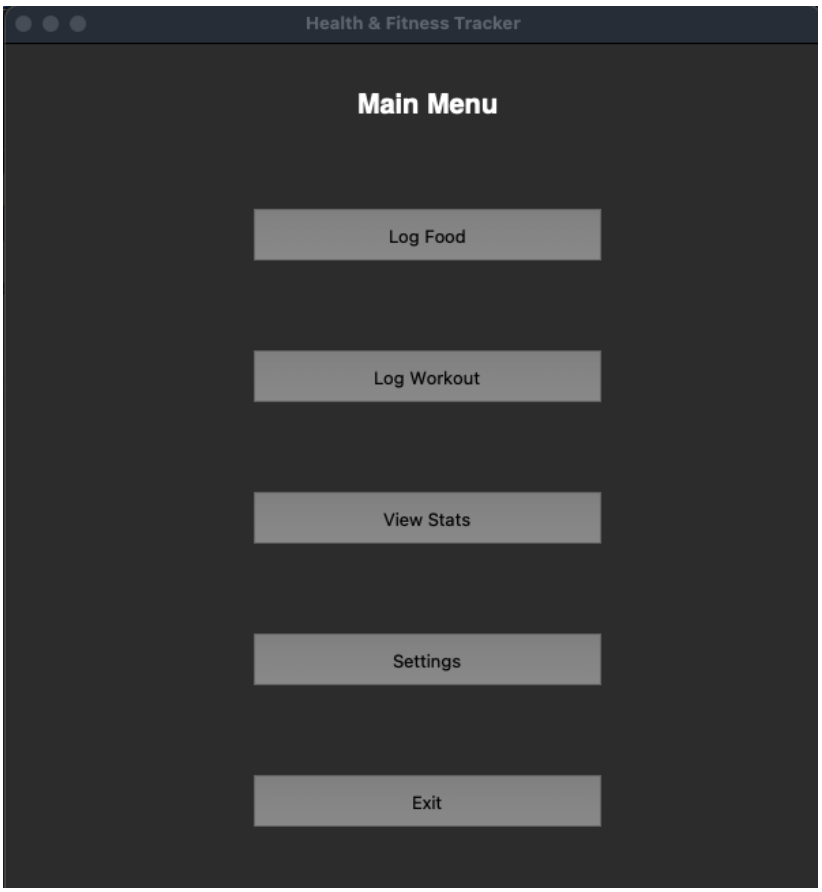
I decided to edit the code to fix this issue:

```
# Main navigation menu
class MainMenu(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.title = tk.Label(self, text="Main Menu", font=("Helvetica", 20, "bold"), bg="#e6f2ff")
        self.title.pack(pady=30)
```

Removed

Main Menu screen after:



Test	Input	Expected Output	Actual Outcome
UI styling check	Open application and navigate through all pages	All pages have consistent font styles, colours , and layout	Consistent layout , styles and colours

Test 5.7: Readable Lists

Test	Input	Expected Output	Actual Outcome
List formatting	1. Log meals to Breakfast, Lunch, Dinner 2. Log workouts 3. Check all listboxes	Text in listboxes is readable with proper formatting (e.g., "NAME: CHICKEN CALORIES: 165 P: 31g C: 0g F: 3.6g")	Readable text with proper formatting

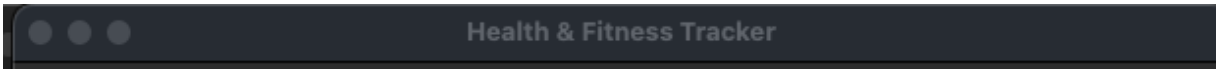
Proof :



Test 5.8: Window Title

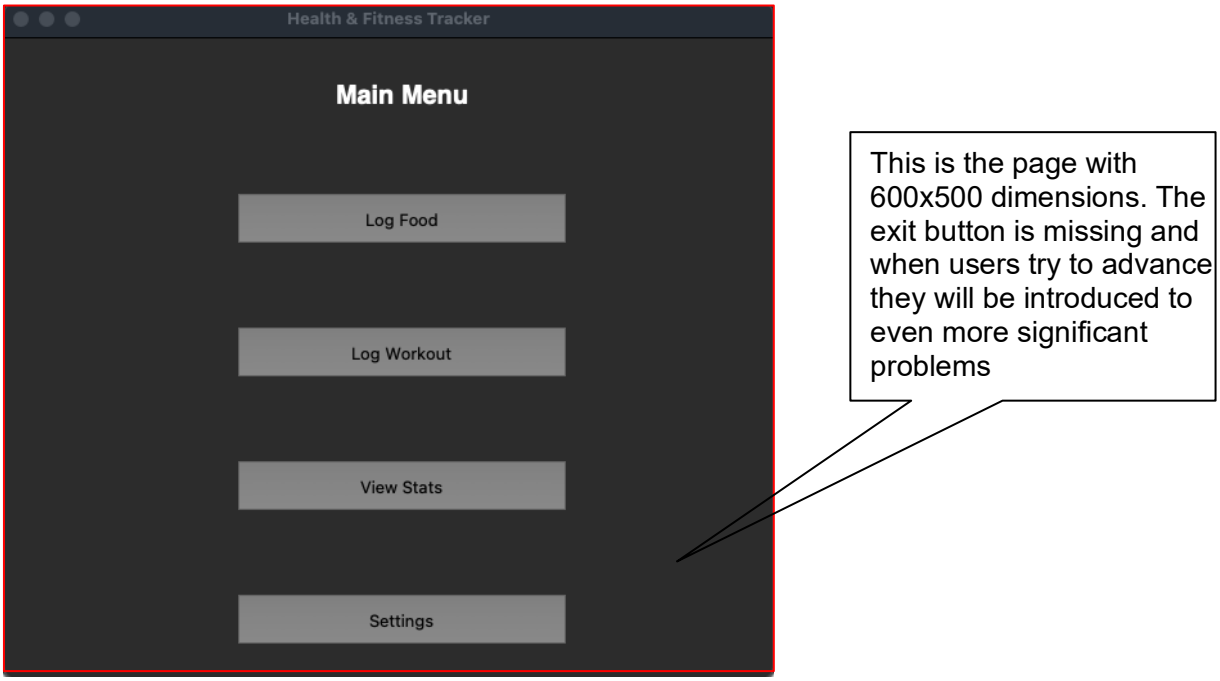
Test	Input	Expected Output	Actual Outcome
Window title check	Launch application	Window title displays "Health & Fitness Tracker"	Passed

Proof :

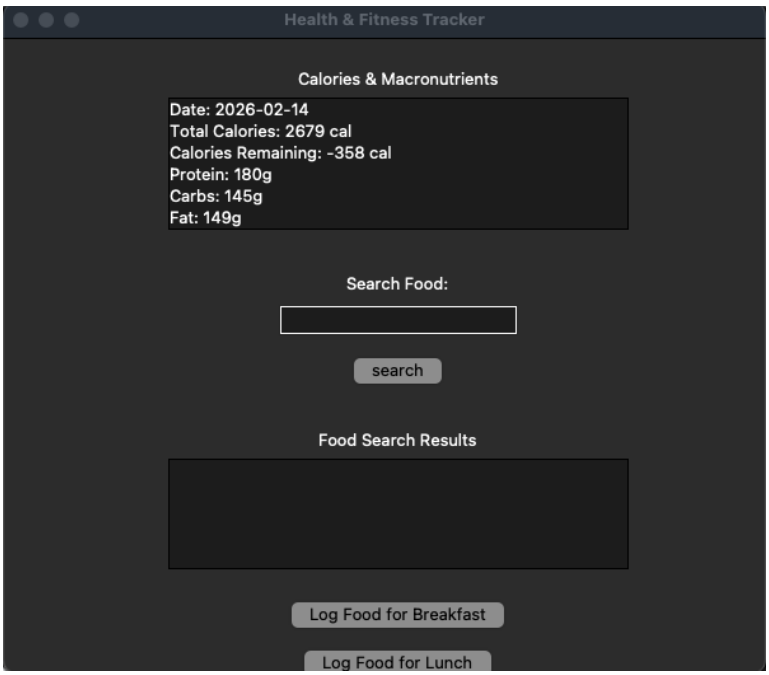


Test 5.9: Window Size

Test	Input	Expected Output	Actual Outcome
Window size check	Launch application	Window size is 600x500 pixels (appropriate dimensions, not too small or too large)	No consistent window size



For example this is the Log Food menu at 600x500 dimensions:



As we can see only a select few buttons are able to fit onto the screen . This is what it looks like in fullscreen :



Justification for Test 5.9 Failure

Why Test 5.9 Failed:

The window size is not consistent across all of the pages because different pages contain different amounts of content. Pages with more input fields, listboxes, and buttons (like LogFood and Settings) require more vertical space than simpler pages (like MainMenu or the AuthMenu).

Explanation:

Tkinter's .pack() geometry manager automatically adjusts the window size to fit all widgets on each page. When navigating from a simple page (e.g., MainMenu with 5 buttons) to a complex page (e.g., LogFood with multiple listboxes, search boxes, and buttons), the window expands vertically to accommodate for the additional content.

While the initial geometry is set to 600x500 in the HealthTrackerApp.__init__() method, this only defines the starting size. As pages with different content amounts are raised using tkraise(), the window adapts to fit all the visible widgets.

Why fixing this would require significant redesign:

Enforcing a fixed window size across all pages would require:

1. Using .grid() or .place() instead of .pack() - This would mean having to rewrite the entire UI layout for all 8 Frame classes, which would take a significant amount of time.
2. Adding scrollbars to every page - Each frame would need a canvas widget with embedded scrollbars, adding complexity to my program and also reducing usability for pages that don't really need scrolling.
3. Removing content from larger pages - This would compromise functionality, as pages like LogFood do require multiple listboxes (Breakfast, Lunch, Dinner, Daily Macros, Search Results) to meet the project objectives and would not function properly without them.

Justification for Current Design:

The dynamic window sizing approach was chosen to:

- Prioritise functionality over rigid dimensions - Each page displays all the necessary information without forcing users to scroll
- Improve user experience - Users can see all of the pages content at once rather than scrolling
- Maintain development efficiency - The .pack() geometry manager simplifies layout code and reduces development time

While the window size variation is not ideal from a UI consistency perspective, it does not impact the core functionality of the application. All of the core functionalities of my program work correctly regardless of window dimensions.

Future Improvement:

In a production version, this could be addressed by:

- Migrating to .grid() layout manager with fixed row/column weights
- Using a scrollable frame for all pages

However, for my project the current dynamic sizing approach is acceptable as changing this would take a significant amount of time.

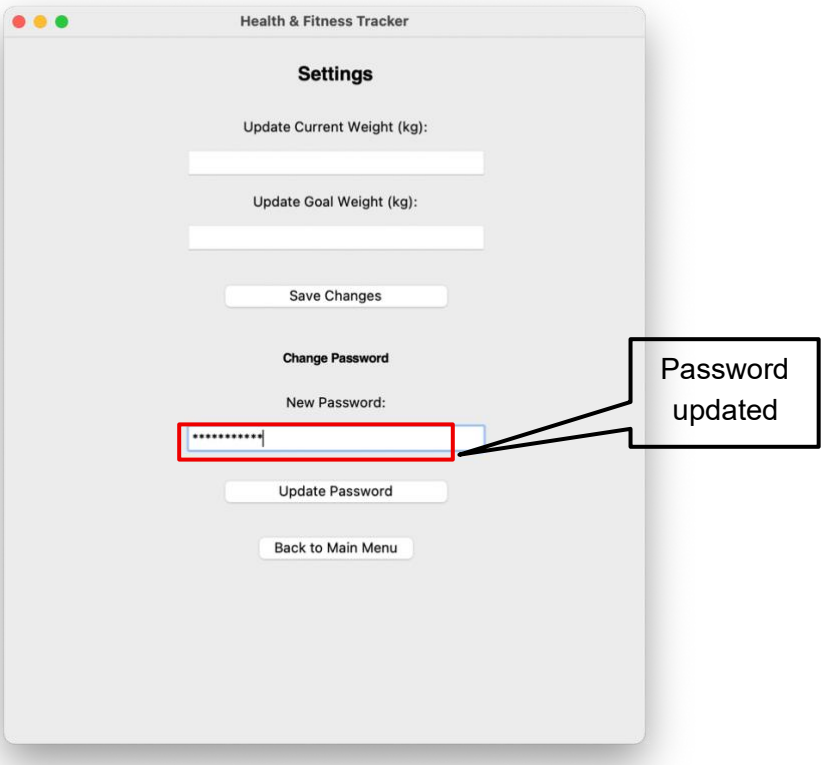
Remaining testing

The tests 1.7, 5.4 and 5.5 were left incomplete. The reason I left these incomplete is because the program needed to be complete first.

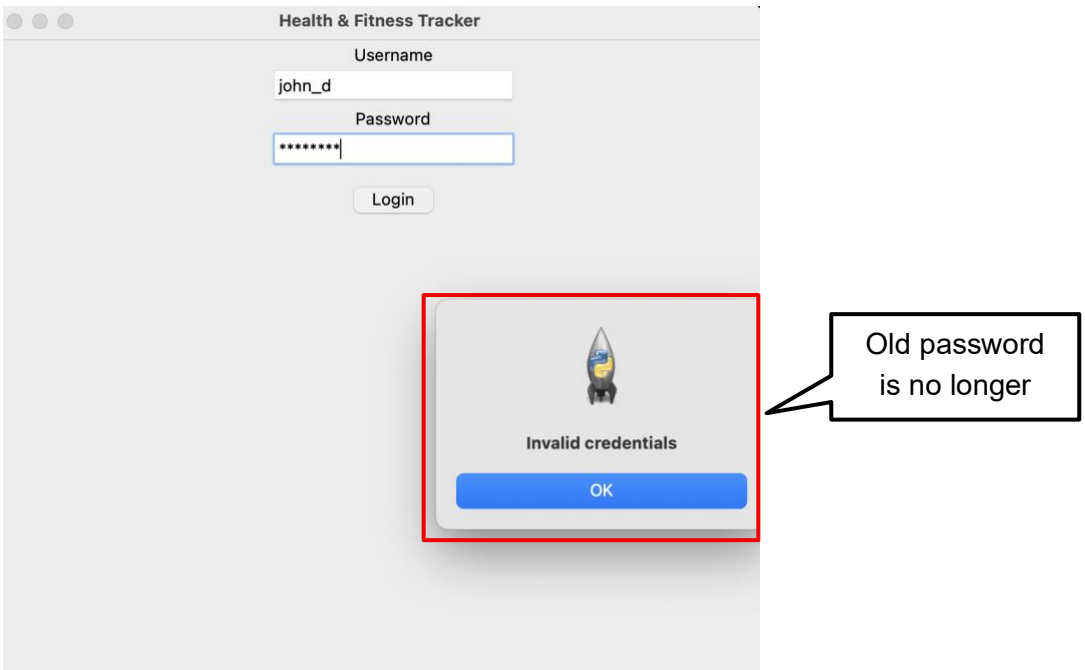
For test 1.7 the reason I couldn't complete it was because the settings module needed to be created first before I could change the password , now that the program is complete I will do this test to ensure that all tests are complete.

Test	Input	Expected Output	Actual Outcome
1.7 Change Password	New password: NewPass123!	Password hash updated, login works with new password	The password hash was updated in the database, login verified with new password.

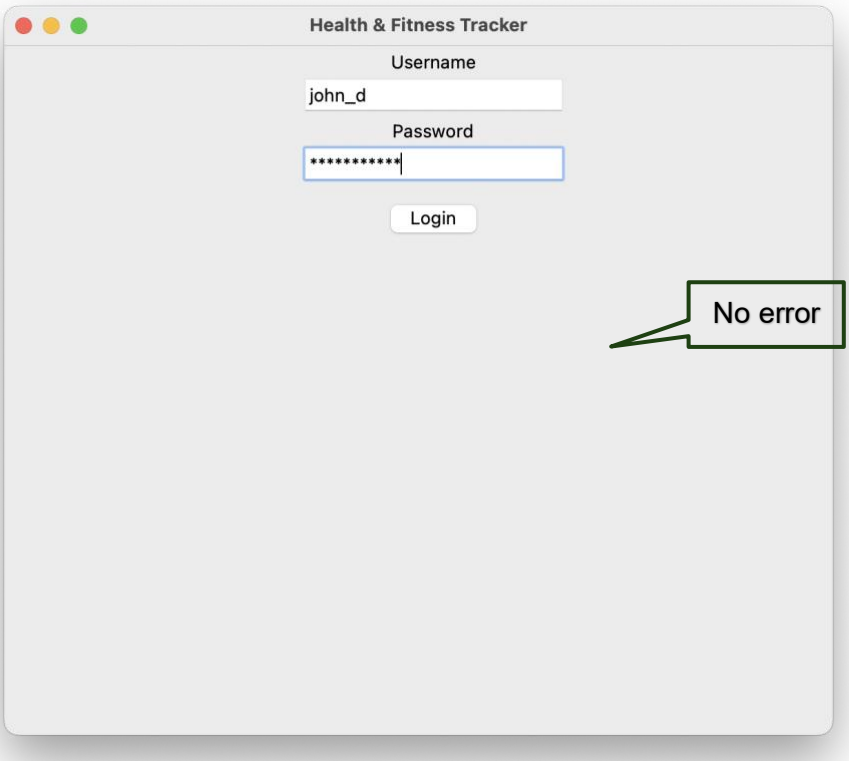
Updating the password :



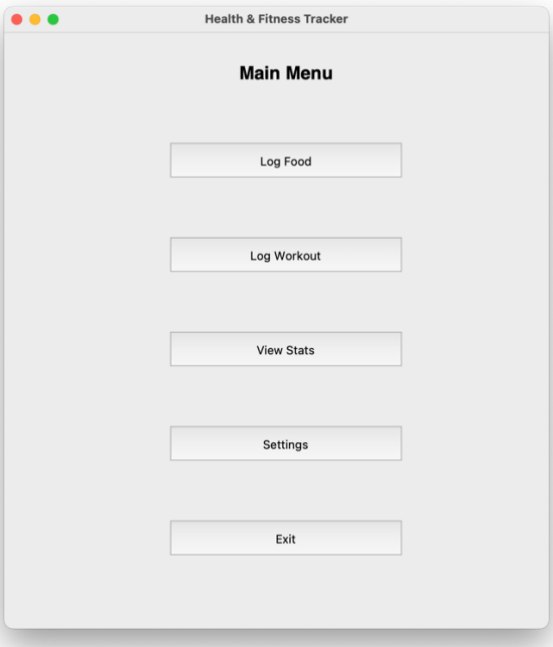
Attempting to log in using old password:



Attempting to log in with new password:



With the updated password login was successful:



Now that test 1.7 was successful we can move on to the final two tests.

I decided to create this video showing my entire project to verify my UI testing ,show all error messages and also complete the two missing tests. The time stamps for tests will be left below .

Test 5.4 - starts at time 0:42 and and all buttons are pressed by 3:38

Test 5.5 - starts at 1:20 and all “Back to Main Menu” buttons are pressed by 3:38

Test	Input	Expected Output	Actual Outcome
5.4 Navigation	From Main Menu, click each button: Log Food, Log Workout, View Stats, Settings	Each page opens successfully when the button is clicked	Demonstrated all throughout video from time 0:42 - 3:38 . Every page opens correctly from main menu. Scan the QR code for the video evidence.
5.5 Back Buttons	Navigate to each page, click "Back to Main Menu"	Each back button returns to Main Menu successfully	Demonstrated all throughout video from time 1:20 - 3:38 . All of the back buttons return to main menu successfully. Scan QR code for the video evidence.

Database testing:

The last section of the development testing is the database testing . In this section I will be comparing my database schema to the proposed one using test data I made during the design.

Below is the test table and I will verify each test with a screenshot of the specific tables schema in which I retrieved using the PRAGMA table_info() command in SQLite.

Users table test:

Objective	Test	Expected Output	Actual Outcome
6.1 Users table	Check table has required fields	Users table has: id, username, passwordHash, name, age, gender, heightCm, initialWeight, activityLevel, bmi, dailycalories, goalweight ,goal	The required fields are present. See the screenshot below.

	cid	name	type	notnull	dflt_value	pk
1	0	id	INTEGER	0	NULL	1
2	1	username	TEXT	1	NULL	0
3	2	passwordHash	TEXT	1	NULL	0
4	3	name	TEXT	0	NULL	0
5	4	age	INTEGER	0	NULL	0
6	5	gender	TEXT	0	NULL	0
7	6	heightCm	REAL	0	NULL	0
8	7	initialWeight	REAL	0	NULL	0
9	8	activityLevel	TEXT	0	NULL	0
10	9	bmi	REAL	0	NULL	0
11	10	dailycalories	REAL	0	NULL	0
12	11	goalweight	REAL	0	NULL	0

Not all fields in the initial design were present , this is because I removed the goal field during iteration 1 as it was unneccesary and introduced conficts with other data . This removal was justified during iteration 1 so I considered my outcome a pass.

Food table test:

Objective	Test	Expected Output	Actual Outcome
6.2 Food table	Check table has required fields	Food table has: foodId, name, calories, protein, carbs, fat , portionsize	All of the required fields were present. See the screenshot below.

	cid	name	type	notnull	dflt_value	pk
1	0	foodId	INTEGER	0	NULL	1
2	1	name	TEXT	1	NULL	0
3	2	calories	REAL	1	NULL	0
4	3	protein	REAL	1	NULL	0
5	4	carbs	REAL	1	NULL	0
6	5	fat	REAL	1	NULL	0
7	6	portionsize	TEXT	0	NULL	0

MealLogs table test:

Objective	Test	Expected Output	Actual Outcome
6.3 MealLogs table	Check table has required fields	MealLogs has: logId, userId, date, mealType, foodId, quantity, totalCal, totalProtein, totalCarbs, totalFat	All of the required fields are present. See the screenshot below.

	cid	name	type	notnull	dflt_value	pk
1	0	logId	INTEGER	0	NULL	1
2	1	userId	INTEGER	1	NULL	0
3	2	date	TEXT	1	NULL	0
4	3	mealType	TEXT	0	NULL	0
5	4	foodId	INTEGER	1	NULL	0
6	5	quantity	REAL	0	NULL	0
7	6	totalCal	REAL	0	NULL	0
8	7	totalProtein	REAL	0	NULL	0
9	8	totalCarbs	REAL	0	NULL	0
10	9	totalFat	REAL	0	NULL	0

WorkoutLogs table test :

Objective	Test	Expected Output	Actual Outcome
6.4 WorkoutLogs table	Check table has required fields	WorkoutLogs has: logId, userId, date, workoutType, durationMin, caloriesBurned	All the required fields are present. See the screenshot below.

	cid	name	type	notnull	dflt_value	pk
1	0	logId	INTEGER	0	*NULL*	1
2	1	userId	INTEGER	1	*NULL*	0
3	2	date	TEXT	1	*NULL*	0
4	3	workoutType	TEXT	0	*NULL*	0
5	4	durationMin	INTEGER	0	*NULL*	0
6	5	caloriesBurned	REAL	0	*NULL*	0

Conclusion:

All four of the tables contained the required fields as they were specified during the design phase. The only change i made from the original design was removing the goal column from the Users table during iteration 1. All foreign key relationships are working correctly, meaning meal and workout logs are always correctly linked to the right user and overall the database functions as it was proposed seeing as the program is able to complete all the essential tasks that were designed.

Post Development Testing:

Now that I have completed all my coding I will complete the post development test to test my final code against the success criteria of my project.

This will check what success criteria have been achieved as well as ensure that the application works together as a whole.

Objective	Expected Output	Actual Outcome

1.1 **The User can create an account**	Account created, user stored in Users table	Account created and stored in Users table
1.2 The User can Login	Login success, main menu shown	Login successful, main menu displayed
1.3 Multiple accounts can be created	Successful, multiple accounts have been created	Multiple account created
1.4 The Users Password is hashed	Password stored as SHA-256 hash (64-character hex, not plain text)	Password stored as 64-character hexadecimal hash
1.5 The User can change password	Password hash updated, login works with new password	Password hash updated, login verified with new password
2.1 The User can search food	Food shown with name, calories, protein, carbs, fat	APPLE displayed with all nutritional information
2.2 Binary search is used for lookup of foods	Binary search is used to search for foods	Binary search is used to search for foods

2.3 The User can log food for breakfast	Entry appears under Breakfast listbox	Food appeared in Breakfast listbox
2.3 The User can log food for lunch	Entry appears under Lunch listbox	Food appeared in Lunch listbox
2.3 The User can log food for dinner	Entry appears under Dinner listbox	Food appeared in Dinner listbox
2.4 Nutritional display	Listbox shows: NAME, CALORIES, P: Xg, C: Xg, F: Xg	All nutritional values displayed in correct format
2.5 The User can undo meal entries	Most recently logged meal removed from listbox and database	Most recent entry removed successfully
2.6 Auto load meals	All today's meals automatically displayed	Meals loaded automatically without manual refresh
3.1 The User can view daily calories	Total calories for today displayed	Total calories displayed and updated after each log
3.2 Macros display	Protein, carbs and fat totals shown in grams	Protein, carbs and fat totals displayed correctly
3.3 The User can view	Calories remaining = daily	Remaining calories calculated

calories remaining	goal minus consumed	and displayed correctly
3.4 Daily goal calculation	Daily calorie goal calculated from BMR, TDEE and weekly adjustment	Daily calories calculated correctly using Mifflin-St Jeor
3.5 Recalculate on update of weight	Daily calorie target automatically recalculated	Daily calories recalculated and updated in database
3.6 BMI update	BMI recalculated and updated	BMI updated correctly in database
4.1 The User can log workouts	Workout logged and appears in Today's Workouts listbox	Workout logged and displayed correctly
4.2 The User can view their workout summary	Total workouts: 2, Duration: 75 min, Calories burned: 700	Summary showed correct totals
4.3 The User can view their workout details	Listbox shows type, duration, calories and ID	Workout displayed with all details and log ID
4.4 The User can delete a workout	Workout removed from listbox and database	Workout removed from listbox and confirmed deleted from database

4.5 Validation	Error "Please enter valid numeric values for duration"	Error shown and workout not logged
4.5 Validation	Error "Calories must be a positive number"	Error shown and workout not logged
5.1 The User can view Calorie graph	Bar chart with 7 bars and MM-DD date labels	Bar chart displayed with correct 7-day data
5.2 The User can view Workout graph	Bar chart with 7 bars showing duration in minutes	Bar chart displayed with correct workout data
5.3 The User can view Macro pie chart	Pie chart with Protein, Carbs, Fat sections and percentages	Pie chart displayed with correct proportions
5.4 User statistics are shown	Listbox shows Name, weight, height and BMI	Name, weight, height and BMI displayed correctly
5.5 Graphs use timedelta to calculate dates for past week	Graphs display values for last week (using timedelta)	Graphs display values for last week (using timedelta)
6.1 Users table	Users table has all required fields including bmi and dailycalories	All required fields present in Users table

6.2 Food table	Food table has foodId, name, calories, protein, carbs, fat	All required fields present in Food table
6.3 MealLogs table	MealLogs has logId, userId, date, mealType, foodId, quantity, totals	All required fields present in MealLogs table
6.4 WorkoutLogs table	WorkoutLogs has logId, userId, date, workoutType, durationMin, caloriesBurned	All required fields present in WorkoutLogs table
6.5 Information can be stored, retrieved, and deleted from the database	All operations complete successfully	All store, retrieve and delete operations functioned correctly
6.6 Referential integrity	Data linked correctly via foreign keys	Foreign key relationships maintained correctly
6.7 Second Normal Form	All tables use single primary keys, no partial dependencies	Database confirmed in Second Normal Form (2NF)
7.1 Update weight	Weight updated to 72 in database	Weight updated successfully

7.2 Update goal weight	Goal weight updated to 68 in database	Goal weight updated successfully
7.3 Calorie recalculation	Daily calorie target automatically recalculated	Daily calories recalculated after weight updates
7.4 BMI recalculation	BMI recalculated and stored	BMI updated correctly in database
7.5 Change password	Password hash updated in database	Password updated and login verified
7.6 Input validation	Error "Please enter valid weight", input rejected	Error shown and database not updated
8.1 The User can navigate the main menu	Each page opens correctly	All pages opened correctly from main menu
8.2 Back buttons	Each back button returns to Main Menu	All back buttons returned to Main Menu correctly
8.3 Consistent styling	Consistent fonts, colours and layout throughout	Consistent white background and black text across all pages
8.4 Readable lists	Formatted text e.g. "NAME: X CALORIES: Y P: Zg"	Readable formatted text in all listboxes

8.5 Window title and size	Title "Health & Fitness Tracker", consistent 600×500 dimensions	Title correct. Window size varies between pages as .pack() adapts to content .does not affect functionality

Analysis of post development testing

From this I can now see more clearly when I do my evaluation which of the success criteria i have met and which i have not met . It has shown me that all parts are working as expected apart from one that I was not able to meet fully . I will evaluate this in the main evaluation section .

Usability Testing:

I will carry out a usability test on my final application which I planned in the design

stage of my project. I will give my application to two potential users with no prior knowledge of the system for 7 days, to give an unbiased perspective on the app's usability and functionality .This will allow me to see any issues I may have missed during the development of the program as well as any missing features that I could add in the further development.

Tester 1- Student :

Ryan, Age 19, Student

1. Were you able to use the app to log your food, workouts, and personal details successfully? “Yes, I was able to log food and workouts without any issues. Creating an account and entering my details was straightforward.

2. Did the app help you keep track of your calories, macros, and fitness progress more easily? “Yes, seeing my calories and macros broken down after each meal made it much easier to understand what I was eating compared to just guessing.”

3.Was the application easy to use? “Overall yes. It felt simple and didn't overwhelm me with too many options at once.”

4. Was it clear what each part of the application was for? “Mostly yes. The main menu made it obvious where to go for food logging and workout logging.”

5. Which feature did you find most helpful when using the app? “The calorie target shown on the dashboard . It was useful to see how many calories I had left for the day.”

6. Were there any problems or issues you encountered while using the app? “Not really. The only thing I noticed was that I wasn't sure which activity level to pick during sign up. A small description next to each option would have helped.”

7. Were there any features you think would improve the app further? “It would be good to have a progress graph over time so you can visually see your weight changing week by week rather than just reading numbers.”

8. Would you use this app again, or could you see yourself using it in the long term? “Yes, I think so. It's simple enough that it wouldn't feel like a chore to log every day, which is what puts me off other apps.”

9. Do you have any other comments or suggestions? “No major issues. I thought it was a solid app overall, especially for something free with no paywalls.”

Tester 2 - Type 2 diabetic:

Sarah, Age 34, Type 2 Diabetic

1. Were you able to use the app to log your food, workouts, and personal details successfully? “Yes, I was able to log my meals and personal details without difficulty. The sign up process was simple and I was up and running quickly.”

2. Did the app help you keep track of your calories, macros, and fitness progress more easily? “Yes, particularly the carbohydrate tracking. As a diabetic, I need to know exactly how many carbs are in each meal before eating, and the app displays this clearly after every food entry. This is something I previously had to look up manually every time.”

3. Was the application easy to use? “Yes, it was straightforward. I was concerned it might be too technical but the layout was clean and easy to follow.”

4. Was it clear what each part of the application was for? “Yes, the menu options were clearly labelled and I had no trouble navigating between food logging and viewing my stats.”

5. Which feature did you find most helpful when using the app? “Definitely the meal by meal macro breakdown. For managing my blood sugar levels, knowing the carbohydrate content of each individual meal not just a daily total is essential for calculating my insulin correctly. The app made this much easier than my previous method of using food packaging labels and a notebook.”

6. Were there any problems or issues you encountered while using the app? “Nothing serious. I did notice that when I searched for a food item that wasn't in the database, there was no message telling me it couldn't be found ,the results just appeared empty. A clear "no results found" message would make that less confusing.”

7. Were there any features you think would improve the app further? “It would be helpful to have a carbohydrate warning or alert if a single meal exceeds a certain threshold, as diabetics need to be especially careful about large carb spikes in one sitting.”

8. Would you use this app again, or could you see yourself using it in the long term? “Yes, absolutely. Managing my diet is something I have to do every single day and this app makes it considerably less time consuming. I would use it regularly.”

9. Do you have any other comments or suggestions? “I was impressed that the nutritional data appeared reliable and consistent. For someone in my situation, inaccurate food data is a genuine health risk, so it was reassuring that the information felt trustworthy.”

Analysis of Usability Testing

Overall the feedback was very positive .Both testers were able to complete all core tasks successfully, indicating the application is intuitive and accessible. Ryan's feedback highlighted that the calorie dashboard and simple navigation made daily logging feel manageable, while Sarah's responses demonstrated the app's value beyond general fitness ,particularly for users with medical conditions where precise macro tracking is critical. (It is important to note that both of the testers were under 35, this means the accessibility concerns relevant to the elderly stakeholder group identified in the analysis were not directly captured in this usability test.)

Two minor issues were raised:

- The lack of descriptive labels for activity levels during registration
- The absence of a "no results found" message when a food search returns nothing.

Both of these are relatively small improvements that could be addressed in future development.

Overall, usability testing confirms the application meets its core objectives of being free, functional,having a clear and usable layout and being accessible to a range of users. (It is important to note that both of the testers were under 35, this means the accessibility concerns relevant to the elderly stakeholder group identified in the analysis were not directly captured in these interviews).

The two issues will be evaluated in the Usabilty features evaluation section below.

Evaluation:

Evaluation against the Success criteria defined during the analysis phase:

Success Criteria	Test Evidence	Success Status	Evaluation	Further Development
1.1 - User can create an account with username and password	Test 1: Create new user. Username "john_d", password "Pass123!", all profile data successfully stored in database	Fully Met	The registration form captures all required data and validates them. isValidFloat() validates the numeric inputs. Account is then stored in Users table with hashed password.	N/A

1.2 - User can login to their account	Test 4: Correct credentials. Login with "john_d" and "Pass123!" authenticated and navigated to Main Menu	Fully Met	Login queries the database, retrieves passwordHash, uses hasher.verify() to compare the passwrods succesfully. Sets currentUserId on success to keep the user logged on to the system the whole time.	N/A
1.3 - Multiple accounts can be created so that multiple users can access the system	Test 2: Create second user. Username "jane_s" successfully created	Fully Met	The database supports multiple users via auto-incrementing primary keys. Each user is assigned a unique id number that is specific to their account.	N/A
1.4 - Passwords should be run under a hashing algorithm (SHA-256) to securely store user passwords in the database	Test 6: Check stored password. Database showed 64-character hexadecimal hash, not plain text	Fully Met	Passwordhasher class implements the SHA-256 algorithm via hashlib. Password hashed and then stored as hexadecimal number. Plain text is never persisted so this was successfully done.	One limitation of the hashing algorithm I used is that it does not use salting.Without a salt passwords are susceptible to rainbow attacks. For further development a different hashing algorithm could be used such as bcrypt.
1.5 - User can change their password through the settings page	Settings class code review confirmed changePassword() method exists and	Fully Met	New password is hashed with SHA-256 and then the database is updated.	N/A

	functions correctly			
2.1 - Users can search for foods in the database	Test 1: Exact search. "APPLE" search displayed food with all nutritional info	Partially Met	searchfood() queries database with ORDER BY name, calls binarySearch(), displays formatted results.	Could enhance with partial matching (e.g., "APP" finds "APPLE"). The same issue was brought up during usability testing so this will be addressed fully in the usability evaluation.
2.2 - Binary search algorithm is implemented for efficient food lookup	Test 3: Binary search verification. Code review confirmed correct implementation	Fully Met	The binary search successfully reduces the lookup time for items from O(n) to O(log n) and now requires significantly less comparisons.	
2.3 - Users can log foods to specific meals (breakfast, lunch, dinner)	Test 4: Log breakfast. OATS logged to Breakfast appeared in correct listbox	Fully met	Three listboxes are present and users can filter by meal type.	N/A

2.4 - Food entries display nutritional information including calories, protein, carbs, and fat	Test 5: Nutritional display. All macros displayed in readable format	Fully Met	Format: "NAME: X CAL: Y P: Zg C: Ag F: Bg". All four values (calories, protein, carbs, fat) shown.	N/A
2.5 - Users can undo their last meal entry using stack data structure	Test 6: Undo. Last meal removed. Test 7: Undo after restart. Stack empty as expected	Fully Met	Stack class implements LIFO with push/pop. logselectedfood() pushes logId, undolastmealentry() pops and deletes. Limitation: in memory, it clears on restart of the program.	N/A
2.6 - Today's meals are automatically loaded when the log food page is opened	Test 8: Auto load. Meals automatically appeared when returning to page	Fully Met	tkraise() overridden to call loadTodaysMeals() and updateDailyMacros(). No manual refresh is needed.	N/A
3.1 - Daily calorie intake is calculated and displayed	Test 8: Auto load Showed "Total Calories: 517 cal"	Fully Met	SUM(totalCal) aggregates daily intake. Updates after every meal logged/undone.	N/A
3.2 - Daily macronutrients (protein, carbs, fat) are calculated	Test 8: Auto load. Showed protein, carbs, fat totals	Fully Met	SUM(totalProtein), SUM(totalCarbs), SUM(totalFat) displayed with gram units.	N/A

and displayed				
3.3 - Calories remaining based on user's daily goal are calculated and shown	Test 8: Auto load. Showed "Calories Remaining: 1621 cal"	Fully Met	Queries daily goal from Users table, subtracts consumed calories, displays remaining.	Could add colour coding (green/red) for visual feedback
3.4 - User's daily calorie goal is calculated using Mifflin St Jeor equation and activity level	Test 1: Create new user. Manual calculation verified correct formula implementation	Fully Met	Male: BMR = 10×W + 6.25×H - 5×A + 5. Female: BMR = 10×W + 6.25×H - 5×A - 161. Multiplies by activity level, adds daily adjustment, enforces 1200 minimum.	N/A
3.5 - Daily calorie goal automatically recalculates when user updates weight or goal weight in settings	Test 1: Update weight. Weight change from 70kg to 72kg recalculated calories	Fully Met	updateSettings() retrieves user data, applies new weight/goalweight, recalculates BMR/TDEE/adjustment and then updates database.	N/A
3.6 - BMI is calculated and updated when weight changes	Test 1: Update weight. BMI updated from 22.86 to 23.51	Fully Met	Formula: weight / ((height/100)**2). Calculated on registration and when weight changes.	N/A

4.1 - Users can log workouts with type, duration, and calories burned	Test 1: Add workout. Running, 30min, 300cal logged successfully	Fully Met	Three entry fields capture type, duration, calories. Inserts into WorkoutLogs table with userId and date.	N/A
4.2 - Today's workout summary displays total workouts, duration, and calories burned	Test 4: Auto totals. Summary showed total workouts, duration, calories	Fully Met	SELECT SUM(durationMin), SUM(caloriesBurned), COUNT(*) aggregates daily totals. Displayed in workout summary listbox.	N/A
4.3 - Individual workouts are listed with all details	Test 1: Add workout. Workout appeared in "Today's Workouts" list with all details	Fully Met	loadTodaysWorkouts() queries and displays: "TYPE DURATION min CALORIES cal burned [ID: X]".	N/A
4.4 - Users can delete selected workouts from today's logs	Test 5: Delete workout. Selected workout removed from list and database	Fully Met	deleteWorkout() extracts logId from selected item, executes DELETE FROM WorkoutLogs WHERE logId = ?.	N/A
4.5 - Workout data is validated to ensure numeric values and positive numbers	Test 2: Non-numeric duration. "thirty" rejected. Test 3: Negative calories "-100" rejected	Fully Met	Validates isdigit() for duration and calories. Checks positive values. Shows specific error messages.	N/A

5.1 - Bar graph showing calorie intake over the last 7 days using matplotlib	Test 1: 7-day calorie graph. Bar chart showed 7 bars with dates and values	Fully Met	Matplotlib bar chart queries last 7 days with timedelta. Date labels formatted MM-DD. X-axis=dates, Y-axis=calories.	N/A
5.2 - Bar graph showing workout duration over the last 7 days using matplotlib	Test 2: 7-day workout graph. Bar chart showed workout durations	Fully Met	Similar to the calorie graph. Duration in minutes displayed. Uses timedelta for date calculations.	N/A
5.3 - Pie chart showing macronutrient breakdown for today using matplotlib	Test 3: Macro pie chart. Pie chart showed protein/carbs/fat proportions with percentages	Fully Met	Queries today's SUM(totalProtein/Carbs/Fat). Three sections which are all colour coded making it visually appealing and easy for users to view and understand.	N/A
5.4 - User statistics including name, weight, height, and BMI are displayed	Test 4: View stats. Stats page showed name, weight, height, BMI	Fully Met	Queries Users table for current user, displays profile information in listbox.	N/A
5.5 - Graphs use timedelta to calculate dates for the past week	Test 1 & 2: Graph tests confirmed 7-day date range calculated correctly	Fully Met	calculates dates for the last 7 days. Used in both calorie and workout graphs.	N/A

6.1 - A database table for user accounts is present including username, passwordHash, name, age, gender, height, weight, activity level, BMI, daily calories, and goal weight	Database inspection confirmed table structure with all required columns	Fully Met	Users table contains: id (PK), username, passwordHash, name, age, gender, heightCm, initialWeight, activityLevel, bmi, dailycalories, goalweight. “Goal” was removed during development .	N/A
6.2 - A database table for foods is present including foodId, name, calories, protein, carbs, and fat	Database inspection confirmed Food table structure	Fully Met	Food table contains: foodId (PK), name, calories, protein, carbs, fat. Stores nutritional database.	N/A
6.3 - A database for meal logs is present including logId, userId, date, mealType, foodId, quantity, and total	Database inspection confirmed MealLogs table structure	Fully Met	MealLogs table contains: logId (PK), userId (FK), date, mealType, foodId (FK), quantity, totalCal, totalProtein, totalCarbs, totalFat.	N/A

nutritional values				
6.4 - A database table for workout logs is present including logId, userId, date, workoutType, duration, and calories burned	Database inspection confirmed WorkoutLogs table structure	Fully Met	WorkoutLogs table contains: logId (PK), userId (FK), date, workoutType, durationMin, caloriesBurned.	N/A
6.5 - Information can be stored, retrieved, and deleted from these databases	All tests demonstrated store, retrieve, delete operations successfully	Fully Met	Registration stores users. Food/workout logging stores data. Queries retrieve data for display. Delete removes workouts. All operations functional.	N/A
6.6 - Database maintains referential integrity with foreign key relationships	Foreign key relationships confirmed. userId in MealLogs/WorkoutLogs	Fully Met	Cannot insert meal/workout without valid userId. Maintains data consistency.	N/A

	references Users(id)			
6.7 - Database design follows Second Normal Form (2NF) with no partial dependencies	Database review confirmed Second Normal Form compliance	Fully Met	All tables have single attribute primary keys (no composite keys), eliminating partial dependencies. Some calculated values (BMI, daily calories) stored for performance (deliberate denormalisation trade-off).	Because I used 2NF I experienced anomalies in iteration 5 when users try and update their weight. Using 3NF would cause there to be better data integrity. Changing the whole database structure may take a significant amount of time but could potentially pose beneficial in further development as it would improve data integrity and remove unnecessary logic from the code.
7.1 - Users can update their current weight	Test 1: Update weight. Weight changed from 70kg to 72kg successfully	Fully Met	Settings page validates input with isValidFloat(), updates initialWeight in database.	N/A
7.2 - Users can update their goal weight	Test 2: Update goal. Goal weight updated to 68kg successfully	Fully Met	Settings page updates goalweight column. Triggers calorie recalculation.	N/A

7.3 - Daily calorie target automatically recalculates when weights are updated	Test 1: Update weight Calories recalculated automatically after weight change	Fully Met	updateSettings() recalculates BMR, TDEE, daily adjustment, updates daily calories column.	N/A
7.4 - BMI automatically recalculates when weight is updated	Test 1: Update weight. BMI updated automatically	Fully Met	New BMI calculated and stored: UPDATE Users SET bmi = ?.	N/A
7.5 - Users can change their password	Settings code review confirmed password change functionality exists	Fully Met	changePassword() hashes new password with SHA-256, updates passwordHash in database.	N/A
7.6 - Input validation ensures only valid numeric values are accepted for weight	Test 3: Invalid weight. "70kg" rejected with error message	Fully Met	isValidFloat() validates numeric input only. Rejects text characters. Displays: "Please enter valid weight".	N/A
8.1 - Main menu provides clear navigation to all main features	Test 4: Navigation. All buttons opened correct pages	Fully Met	Dictionary self.pages enables O(1) page lookup. Buttons call showPage(pageName) with string keys.	N/A

8.2 - All pages have back buttons to return to main menu	Test 5: Back buttons. All pages return to main menu	Fully Met	Every page has a "Back to Main Menu" button calling container.showPage("MainMenu"). Consistent navigation pattern.	N/A
8.3 - Application uses consistent styling with colours and fonts	Test 6: Styling. Pages gave no consistent colour scheme to maths users device preferences(light mode/dark mode)	Fully Met	Consistent colour scheme throughout for usability reasons.	N/A
8.4 - Listboxes display information in readable formatted text	Test 7: List formatting. Listboxes display formatted, readable text	Fully Met	Formatted strings with labels: "NAME: X CALORIES: Y P: Zg". Clear, readable presentation of data.	N/A
8.5 - Application window has appropriate title and dimensions	Test 8: Window title. Title correct. Test 9: Window size - FAIL . Size varies between pages	Partially Met	Title "Health & Fitness Tracker" displayed correctly. Initial geometry 600x500 set. However, .pack() geometry manager dynamically resizes when navigating between pages with different content amounts.	Lock window with self.resizable(False, False) and add scrollbars to content heavy pages. Alternatively the geometry manager could be changed to .grid()with fixed dimensions.

Summary of success criteria evaluation:

The Health & Fitness Tracker application achieved an overall success rate of 44 out of 45 when evaluated against the success criteria.

Out of 45 criteria

43 were fully met

2 were partially met

0 were not met

One of the partially met criterion was 8.5 - Application window has appropriate title and dimensions. While the window title "Health & Fitness Tracker" displayed correctly, the window dimensions varied between pages due to the .pack() geometry manager dynamically adjusting to content size.

This is a issue that does not impact the core functionality of my program, and could be addressed in further development by using a grid() geometry and adding scrollbars to content heavy pages such as the log food page.

All of the core computational requirements were successfully implemented, including:

Algorithms:

- Binary search (Log food)
- SHA-256 password hashing (authentication menu and settings)

Data Structures:

- Stack (Log food)
- Dictionary (authentication menu and settings and main application class)
- Lists (View Stats)

Database Design:

- Second Normal Form (2NF)
- referential integrity with foreign keys
- support for multiple users

Calculations:

- Mifflin-St Jeor equation for BMR (authentication menu and settings)
- TDEE calculations and recalculations with activity multipliers (authentication menu and settings)
- automatic calculation/recalculation of BMI and daily calories (authentication menu and settings)

This high success rate validates my problem decomposition, modular class design , and effective implementation of computational thinking principles throughout the iterative development process. Overall the solution successfully addressed all of the core needs of all four stakeholder groups that I identified in the analysis.

The other partially met criteria was also brought up during my usability testing so it will be addressed in the usability features evaluation section.

Further development from success criteria :

Criteria	Why it wasn't fully resolved	Further Development
2.1 – Users can search for foods in the database	Binary search only works on exact full name matches. Typing "APP" returns nothing because the algorithm requires the search term to exactly equal a food name at the comparison. This is an inherent limitation of binary search .	Add a secondary SQL query using the LIKE operator . Binary search runs first for speed, LIKE query runs as a fallback. LIKE is O(n) but only triggers on failed exact searches so the efficiency remains acceptable as the Like operator is only used if no exact match is found .
3.3 – Calories remaining based on user's daily goal are calculated and shown	Calories remaining are calculated correctly but displayed as plain black text regardless of whether the user is under or over their target. There is no immediate visual signal when a user exceeds their goal. This is a usability gap rather than a functional failure.	Add conditional colour coding to the calories remaining display using a tk.Label that calls label.config(fg="green") when remaining calories is positive and label.config(fg="red") when remaining calories is negative, updated every time updateDailyMacros() is called. Valuable for all users as they get an immediate visual warning when intake is too high.

8.5 – Application window has appropriate title and dimensions	.pack() automatically expands the window to fit whichever frame is raised. Moving from simple pages like Main Menu to complex pages like Log Food causes the window to expand vertically. self.geometry("600x500") only sets the starting size and is overridden by .pack() as content changes.	Change layout from .pack() to .grid() . For content heavy pages a scroll bar could be used so users can scroll within a fixed window instead of the window expanding.

Usability Features Evaluation

No.	Criteria	Was it met?	How could it be improved?	User feedback / evidence (from Usability testing Interviews)
1	Main menu navigation with buttons for all key tasks	Yes	N/A	Ryan: "The main menu made it obvious where to go for food logging and workout logging." Sarah: "The menu options were clearly labelled and I had no trouble navigating between food logging and viewing my stats."
2	Minimal steps per action	Yes	N/A	Ryan: "It felt simple and didn't overwhelm me with too many options at once." Sarah: "I was able to log my meals and personal details without difficulty."
3	Numeric validation (for fields like age, weight, height)	Yes	N/A	No user evidence but this was evaluated during the iterative development process.

4	Instant feedback after actions	Yes	N/A	Ryan: "The calorie target shown on the dashboard. It was useful to see how many calories I had left for the day." Sarah: "The app displays this clearly after every food entry. This is something I previously had to look up manually every time."
5	Consistent layout	Yes	N/A	Ryan: "It felt simple and didn't overwhelm me with too many options at once." Sarah: "I was concerned it might be too technical but the layout was clean and easy to follow."
6	Error prompts	Partially met	A "no results found" message should be shown when a food search returns no matches, rather than displaying an empty list.	Ryan: "Not really. The only thing I noticed was that I wasn't sure which activity level to pick during sign up." Sarah: "When I searched for a food item that wasn't in the database, there was no message telling me it couldn't be found ,the results just appeared empty. A clear 'no results found' message would make that less confusing."

Summary of usability features

The Health & Fitness Tracker application overall passed 5 out of 6 criteria .

Out of 6 criteria:

5 were fully met

1 was partially met

0 were not met

The one criteria that was partially met was number 6 (No results found message). Also during the interview for my usability testing with Ryan, he brought up another issue which did

not fall into any of the 6 criteria. The issues and how they can be improved during further development have been explained below.

Further development from usability tests and interviews:

Issue	Comment	Further Development
Activity level descriptions during sign up	Ryan was not sure which activity level to select during sign up. He said "a small description next to each option would have helped." The input field functioned correctly and all of the validation was in place, but the lack of context made it difficult for users to make an informed decision .This is a usability issue rather than a functional one , the feature works but is not as clear as it could be.	I could add a label next to each activity level option explaining what each one means. For example, clarifying that "Sedentary" means 0–2 hours of exercise per week while "Active" means 5–7 hours of exercise per week. This would require minimal changes to the Tkinter sign up frame and would significantly improve the experience for new users.
No results found message when searching for food	Sarah identified that when she searched for a food item not in the database, no feedback was given the results list simply appeared empty. She stated "there was no message telling me it couldn't be found, the results just appeared empty," which was confusing as it was unclear whether the search had failed or whether the item genuinely did not exist.	Add a conditional check after the database query, if the returned list is empty, display a message such as "No results found" in the listbox. This would require only a small addition to the existing search logic and would make the application considerably more usable .

GUI	Although this was not mentioned in any of the interviews I feel that the use of tkinter makes the GUI less visually appealing as opposed to other Gui routines .	Other GUI routines such as Kivy, PyQt5 could be used to provide a more visually appealing user interface to help motivate users to be consistent with their food / workout tracking.

Further Development

These are features that with more time I would like to add to my program to achieve all of the success criteria above. These features would make the program easier to use, clearer and much more user friendly.

Delete account - Adding the ability for users to delete their account would improve data security by ensuring personal data is not kept longer than it needs to be. It would also reduce the database query time as fewer records would need to be searched through, overall this would improve the applications performance.

More appealing user interface - Although the current interface was intentionally designed to be professional and minimalistic, adding subtle use of colour could make the application more inviting to less experienced users such as the elderly stakeholder group identified in the analysis. A more visually engaging design would improve long term engagement without significantly affecting the clean layout.

Medication tracker - Adding a medication tracking feature would make the application a central health manager and would be particularly valuable for users with medical conditions where they would need to track medication .Users would be able to track food, workouts, and medication all within a single application rather than switching between multiple different programs.

Streak counter – Adding a streak counter to track how many consecutive days a user has logged their food and workouts for would be a motivational tool, encouraging consistent daily use.

A larger food database - The current food database size is sufficient to demonstrate how the application functions, but expanding it would allow users to track a much wider range of meals without leaving food entries unlogged. Untracked food directly affects the accuracy of

calorie counting, so a larger database would significantly improve the reliability of the tracking data for all of the users and stakeholder groups.

Limitations

I will now be analysing any limitations of my current solution and how I could address these if possible .

Desktop Only - The application is built using Tkinter which only supports desktop, meaning users cannot access it on a smartphone or tablet. This is a significant limitation for practical day to day tracking, as most users eat meals and complete workouts away from their computer. Having to remember everything to log later at a desktop introduces inaccuracy and reduces the likelihood of consistent use. A mobile version would allow users to log food and workouts in real time, for example immediately after a meal at a restaurant or directly after finishing a gym session, which would improve both accuracy and engagement across all users .

How to address this issue :

This could be addressed by developing a dedicated mobile application using a framework such as React Native or Flutter, both of which allow a single codebase to produce applications for both iOS and Android. The application would be published to the Apple App Store and Google Play Store, allowing users to download and install it directly onto their smartphones in the same way they would download any other fitness app such as MyFitnessPal or Lifesum. This would allow users to log food and workouts in real time from anywhere, for example immediately after a meal or directly after finishing a gym session, which would significantly improve the consistency of daily tracking across all of the stakeholder groups. The core logic including the calorie calculations, database queries, and validation would remain the same for the most part, with only the interface layer needing to be rebuilt for a mobile environment.

Hardcoded Database File Path The database connection in connectDatabase() uses an absolute file path hardcoded to a specific location on my computer . This means the application will immediately have a database connection error on any other computer because that file path will not exist. Any stakeholder who attempts to run the program on their own machine would be unable to use it at all without first editing the code.

How to address this issue :

Switch from a local SQLite file to a cloud hosted database service, which is the approach that is used by commercial fitness applications such as MyFitnessPal and Cronometer. A service like ,Amazon Web Services (AWS), Google Cloud, or Microsoft Azure could be used to host the database on remote servers, with the application connecting to it over the internet. This would mean the application would work identically on any machine without requiring any changes to be made to the code, as the database address would be a fixed server URL rather than a path tied to my specific computer.

An additional benefit is that user data would be automatically backed up and protected by the cloud provider, removing the risk of permanent data loss if a user's machine is damaged or if the local database file was to be accidentally deleted. This is the same infrastructure model used by most modern web and mobile applications .

Calories Burned Must Be Manually Entered When logging a workout, users are required to manually input the number of calories burned. Most users have no way of knowing this figure without a fitness wearable, which not all stakeholders will own. This means that users are likely to either guess, leave the field blank causing a validation error, or enter an inaccurate value ,all of which make the workout data inaccurate . A better approach would be to automatically estimate calories burned using an established formula based on the user's weight, age, workout type, and duration, all of which are already stored in or entered into the application. While not as precise as a wearable device, this would provide a reasonable estimate that is significantly more accurate than guessing and would remove a major barrier to usability for users without fitness trackers.

How to address this issue :

This could be addressed by removing the calories burned input field entirely and instead automatically calculating an estimate using MET (Metabolic Equivalent of Task) values. Each workout type would be assigned a MET value ,for example, running has a MET of approximately 9.8 and cycling has a MET value of approximately 7.5 . Calories burned would be calculated using the formula:

Calories = MET × user weight in kg × duration in hours.

The user's weight is already stored in the database and the duration is already entered as part of workout logging, so only the MET values would need to be added.

A dictionary mapping common workout types to their MET values could be implemented with a default value for unrecognised workout types. While this would still be an estimate rather than a precise measurement, it would be considerably more accurate than manual guessing and would remove the need for users to own a fitness wearable.

Maintenance

Throughout development I have included comments across all algorithms and code. This will make it easier for other developers who may work on the program in the future to understand what each section does, reducing the time needed to locate and edit any sections of code. It will also reduce confusion about what the code does as the purpose of each function is clearly documented therefore making it easier to maintain.

Furthermore, the code is divided into separate classes each with a clearly defined responsibility . SignUp, Login, LogFood, LogWorkout, ViewStats, Settings, and Stack. The use of object orientation means that a developer will be able to know exactly which class to look at for any given feature. This means that individual modules can be updated, replaced, or extended without affecting the rest of the system. The modularity also promotes code

reusability , for example, the Stack class and the calculatedailycalories() function are self contained components that could be reused in a future mobile or web based version of the application without having to be modified. For example, if a future developer needed to modify how workouts are validated, they would go directly to the LogWorkout class without needing to read any unrelated code. This separation makes the code significantly easier to navigate and to update for maintenance of the system.

The main maintenance concern is the database because as more users create accounts and log meals and workouts over time, the database will continue to grow. Records from inactive or deleted accounts will accumulate over time, making query times longer and reducing the overall performance. A system to identify and remove accounts that have not been accessed for an extended period of time would likely need to be implemented to keep the database at a manageable size and to maintain consistent response times for users.

A further maintenance consideration is the food database. The nutritional values for foods can change over time as manufacturers alter their products or as more accurate data becomes available. The current verified food database would need to be reviewed and updated every so often to ensure the nutritional information remains accurate, which is particularly important to stakeholders such as diabetic patients who rely heavily on precise carbohydrate values.

Finally, as the application grows and stores an increasing amount of sensitive personal health data including weights, calorie intake, and medical conditions, the security of that data becomes a more significant responsibility.

Password hashing using SHA-256 is already implemented, but at larger scale it would likely require additional security measures such as encrypted database connections, rate limiting on the number of login attempts to try and prevent brute force attacks, and potentially professional security review to ensure all user data is sufficiently protected.

Code:

```python
import tkinter as tk
from tkinter import messagebox
import sqlite3
import hashlib
from datetime import datetime, timedelta
import matplotlib.pyplot as plt




# Password hashing using SHA-256
class Passwordhasher():
    # Hash password for secure storage
    def hash(self, password):
        return hashlib.sha256(password.encode("utf8")).hexdigest()

    # Verify password matches stored hash
    def verify(self, password, storedHash):
        if self.hash(password) == storedHash:
            return True
        return False

# Connect to SQLite database
def connectDatabase():
    return sqlite3.connect("/Users/aadamanwar/Desktop/coursework/fitness tracker database (1).db")

# Create Users table in database if it doesn't exist
def createUserTable():
    conn = connectDatabase()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS Users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            passwordHash TEXT NOT NULL,
            name TEXT,
            age INTEGER,
            gender TEXT,
            heightCm REAL,
            initialWeight REAL,
            activityLevel TEXT,
            goal TEXT
        )
    """)
    conn.commit()
    conn.close()

# Main application window
class HealthTrackerApp(tk.Tk):
    def __init__(self, hasher):
        super().__init__()
        self.title("Health & Fitness Tracker")
        self.geometry("600x500")
        self.configure(bg="#e6f2ff")

        self.hasher = hasher
        self.currentUserId = -1  # Store logged-in user's ID

        # Create all pages/screens
        self.pages = {}
        self.pages["AuthMenu"] = AuthMenu(self)
        self.pages["SignUp"] = SignUp(self)
        self.pages["Login"] = Login(self)
        self.pages["MainMenu"] = MainMenu(self)
        self.pages["LogFood"] = LogFood(self)
        self.pages["LogWorkout"] = LogWorkout(self)
        self.pages["ViewStats"] = ViewStats(self)
        self.pages["Settings"] = Settings(self)

        # Grid all pages in same position
        self.pages["AuthMenu"].grid(column=0, row=0, sticky="nesw")
        self.pages["SignUp"].grid(column=0, row=0, sticky="nesw")
        self.pages["Login"].grid(column=0, row=0, sticky="nesw")
        self.pages["MainMenu"].grid(column=0, row=0, sticky="nesw")
        self.pages["LogFood"].grid(column=0, row=0, sticky="nesw")
        self.pages["LogWorkout"].grid(column=0, row=0, sticky="nesw")
        self.pages["ViewStats"].grid(column=0, row=0, sticky="nesw")
        self.pages["Settings"].grid(column=0, row=0, sticky="nesw")
        self.showPage("AuthMenu")
```

```
80
81      # Bring specified page to front
82      def showPage(self, pageName):
83          pageToShow = self.pages[pageName]
84          pageToShow.tkraise()
85
86  # Initial login/signup screen
87  class AuthMenu(tk.Frame):
88      def __init__(self, container):
89          super().__init__(container)
90          tk.Label(self, text="Welcome to Fitness Tracker", font=("Helvetica", 20, "bold")).pack(pady=30)
91          tk.Button(self, text="Login", width=25, height=2, command=lambda: container.showPage("Login")).pack(pady=10)
92          tk.Button(self, text="Sign Up", width=25, height=2, command=lambda: container.showPage("SignUp")).pack(pady=10)
93
94  # User registration screen
95  class SignUp(tk.Frame):
96      def __init__(self, container):
97          super().__init__(container)
98          self.container = container
99
100         # Create input fields for user details
101         self.usernameLabel = tk.Label(self, text="Username")
102         self.usernameLabel.pack()
103         self.usernameEntry = tk.Entry(self)
104         self.usernameEntry.pack()
105         self.passwordLabel = tk.Label(self, text="Password")
106         self.passwordLabel.pack()
107         self.passwordEntry = tk.Entry(self)
108         self.passwordEntry.pack()
109         self.passwordEntry.config(show="*")  # Hide password text
110         self.nameLabel = tk.Label(self, text="Name")
111         self.nameLabel.pack()
112         self.nameEntry = tk.Entry(self)
113         self.nameEntry.pack()
114         self.ageLabel = tk.Label(self, text="Age")
115         self.ageLabel.pack()
116         self.ageEntry = tk.Entry(self)
117         self.ageEntry.pack()
118         self.genderLabel = tk.Label(self, text="Gender")
119         self.genderLabel.pack()
120         self.genderEntry = tk.Entry(self)
121         self.genderEntry.pack()
122         self.heightLabel = tk.Label(self, text="Height (cm)")
123         self.heightLabel.pack()
124         self.heightEntry = tk.Entry(self)
125         self.heightEntry.pack()
126         self.weightLabel = tk.Label(self, text="Weight (kg)")
127         self.weightLabel.pack()
128         self.weightEntry = tk.Entry(self)
129         self.weightEntry.pack()
130         self.activityLabel = tk.Label(self, text="Activity Level (Sedentary, Light, Moderate, Active, Very Active)")
131         self.activityLabel.pack()
132         self.activityEntry = tk.Entry(self)
133         self.activityEntry.pack()
134         self.goalweightLabel = tk.Label(self, text="Goal weight (kg)")
135         self.goalweightLabel.pack()
136         self.goalweightEntry = tk.Entry(self)
137         self.goalweightEntry.pack()
138         self.weeksLabel = tk.Label(self, text="the number of weeks you want to take to reach your goal weight within")
139         self.weeksLabel.pack()
140         self.weeksEntry = tk.Entry(self)
141         self.weeksEntry.pack()
142         self.submitButton = tk.Button(self, text="Sign Up", command=lambda: self.register()).pack(pady=10)
143
144     # Create new user account in database
145     def register(self):
146         username = self.usernameEntry.get()
147         password = self.passwordEntry.get()
148         hashed = self.container.hasher.hash(password)  # Hash password for security
149
150         name = self.nameEntry.get()
151
152         # Validate numeric inputs
153         ageStr = self.ageEntry.get()
154         heightStr = self.heightEntry.get()
155         weightStr = self.weightEntry.get()
156         goalweightStr = self.goalweightEntry.get()
157         weeksStr = self.weeksEntry.get()
```

```
158
159         if ageStr.isdigit():
160             age = int(ageStr)
161         else:
162             messagebox.showerror("Error", "Please enter valid age.")
163             return
164
165         if self.isValidFloat(heightStr):
166             height = float(heightStr)
167         else:
168             messagebox.showerror("Error", "Please enter valid height.")
169             return
170
171         if self.isValidFloat(weightStr):
172             weight = float(weightStr)
173         else:
174             messagebox.showerror("Error", "Please enter valid weight.")
175             return
176
177         if self.isValidFloat(goalweightStr):
178             goalweight = float(goalweightStr)
179         else:
180             messagebox.showerror("Error", "Please enter valid goal weight.")
181             return
182
183         if weeksStr.isdigit():
184             weeks = int(weeksStr)
185         else:
186             messagebox.showerror("Error", "Please enter valid weeks.")
187             return
188
189         gender = self.genderEntry.get().upper()
190         activity = self.activityEntry.get().upper()
191
192         # Calculate BMI and daily calorie target
193         userbmi = weight / ((height / 100) ** 2)
194         dailycalories = self.calculatedailycalories(weight, goalweight, height, age, gender, activity, weeks)
195
196         # Check if username already exists in database
197         conn = connectDatabase()
198         cursor = conn.cursor()
199         cursor.execute("SELECT username FROM Users WHERE username = ?", (username,))
200         existing = cursor.fetchone()
201
202         if existing:
203             conn.close()
204             messagebox.showerror("Error", "Username already exists.")
205             return
206
207         # Insert new user into database
208         cursor.execute("""
209             INSERT INTO Users (
210                 username, passwordHash, name, age, gender, heightCm, initialWeight,
211                 activityLevel, bmi, dailycalories, goalweight
212             )
213             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
214         """, (
215             username, hashed, name, age, gender, height, weight,
216             activity, userbmi, dailycalories, goalweight
217         ))
218         conn.commit()
219         conn.close()
220         self.container.showPage("MainMenu")
221
222     # Check if string is valid decimal number
223     def isValidFloat(self, value):
224         dotCount = 0
225         for char in value:
226             if char == '.':
227                 dotCount = dotCount + 1
228             elif char.isdigit():
229                 continue
230             else:
231                 return False
232         if dotCount > 1:
233             return False
234         return True
```

```python
    # Calculate daily calorie target based on TDEE and weight goal
    def calculatedailycalories(self, weight, goalweight, height, age, gender, activity, weeks):
        # Calculate BMR using Mifflin-St Jeor equation
        if gender.upper() == 'MALE':
            bmr = 10 * weight + 6.25 * height - 5 * age + 5
        else:
            bmr = 10 * weight + 6.25 * height - 5 * age - 161

        # Multiply BMR by activity level to get TDEE
        activity_levels = {
            'SEDENTARY': 1.2,
            'LIGHT': 1.375,
            'MODERATE': 1.55,
            'ACTIVE': 1.725,
            'VERY ACTIVE': 1.9
        }
        tdee = bmr * activity_levels.get(activity.upper(), 1.2)

        # Calculate daily calorie adjustment to reach goal
        weight_change = goalweight - weight
        kcal_change = weight_change * 7700  # 7700 kcal per kg
        daily_adjustment = kcal_change / (weeks * 7)

        dailycalories = tdee + daily_adjustment

        # Minimum 1200 calories for health
        if dailycalories < 1200:
            return 1200
        return dailycalories

# Login screen
class Login(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.usernameLabel = tk.Label(self, text="Username")
        self.usernameLabel.pack()
        self.usernameEntry = tk.Entry(self)
        self.usernameEntry.pack()
        self.passwordLabel = tk.Label(self, text="Password")
        self.passwordLabel.pack()
        self.passwordEntry = tk.Entry(self)
        self.passwordEntry.pack()
        self.passwordEntry.config(show="*")
        self.submitButton = tk.Button(self, text="Login", command=lambda: self.login()).pack(pady=10)

    # Verify credentials and log user in
    def login(self):
        username = self.usernameEntry.get()
        password = self.passwordEntry.get()

        # Fetch user from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT id, passwordHash FROM Users WHERE username = ?", (username,))
        result = cursor.fetchone()
        conn.close()

        if result:
            # Verify password hash matches
            if self.container.hasher.verify(password, result[1]):
                self.container.currentUserId = result[0]
                self.container.showPage("MainMenu")
                return

        messagebox.showerror("Error", "Invalid credentials")

# Main navigation menu
class MainMenu(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        self.title = tk.Label(self, text="Main Menu", font=("Helvetica", 20, "bold"))
        self.title.pack(pady=30)
```

(variable) activity_levels: dict[str, float]

```
312
313         self.foodButton = tk.Button(self, text="Log Food", width=25, height=2, command=lambda: self.container.showPage("LogFood"))
314         self.foodButton.pack(pady=30)
315         self.workoutButton = tk.Button(self, text="Log Workout", width=25, height=2, command=lambda: self.container.showPage("LogWorkout"))
316         self.workoutButton.pack(pady=30)
317         self.statsButton = tk.Button(self, text="View Stats", width=25, height=2, command=lambda: self.showStats())
318         self.statsButton.pack(pady=30)
319         self.settingsButton = tk.Button(self, text="Settings", width=25, height=2, command=lambda: self.container.showPage("Settings"))
320         self.settingsButton.pack(pady=30)
321         self.exitButton = tk.Button(self, text="Exit", width=25, height=2, command=lambda: self.container.destroy())
322         self.exitButton.pack(pady=30)
323
324     # Load stats before showing stats page
325     def showStats(self):
326         self.container.pages["ViewStats"].getStats()
327         self.container.showPage("ViewStats")
328
329 # Stack data structure for undo functionality
330 class Stack:
331     def __init__(self):
332         self.items = []
333
334     # Add item to top of stack
335     def push(self, item):
336         self.items.append(item)
337
338     # Remove and return top item
339     def pop(self):
340         if len(self.items) == 0:
341             return None
342         return self.items.pop()
343
344     # View top item without removing
345     def peek(self):
346         if len(self.items) == 0:
347             return None
348         return self.items[-1]
349
350     # Check if stack is empty
351     def isEmpty(self):
352         if len(self.items) == 0:
353             return True
354         return False
355
356     # Get number of items in stack
357     def size(self):
358         return len(self.items)
359
360
361 # Food logging screen
362 class LogFood(tk.Frame):
363     def __init__(self, container):
364         super().__init__(container)
365         self.container = container
366         self.mealStack = Stack()  # Track meal entries for undo
367
368         # Daily summary display
369         self.dailymacrosHeader = tk.Label(self, text="Calories & Macronutrients")
370         self.dailymacrosHeader.pack(pady=(20, 5))
371         self.dailymacroslistbox = tk.Listbox(self, width=40, height=6)
372         self.dailymacroslistbox.pack(padx=80, pady=(0, 30))
373
374         # Food search input
375         self.searchfoodLabel = tk.Label(self, text="Search Food:")
376         self.searchfoodLabel.pack(pady=(0, 5))
377         self.searchfoodEntry = tk.Entry(self)
378         self.searchfoodEntry.pack()
379         self.submitfoodButton = tk.Button(self, text="search", command=lambda: self.searchfood())
380         self.submitfoodButton.pack(pady=(10, 30))
381
382         # Search results display
383         self.foodsearchHeader = tk.Label(self, text="Food Search Results")
384         self.foodsearchHeader.pack(pady=(0, 5))
385         self.foodsearchlistbox = tk.Listbox(self, width=40, height=5)
386         self.foodsearchlistbox.pack(padx=80, pady=(0, 20))
387
388         # Buttons to log food to different meals
```

```
        self.logfoodBreakfastButton = tk.Button(self, text="Log Food for Breakfast", command=lambda: self.logselectedfood("Breakfast"))
        self.logfoodBreakfastButton.pack(padx=40, pady=(0, 10))
        self.logfoodLunchButton = tk.Button(self, text="Log Food for Lunch", command=lambda: self.logselectedfood("Lunch"))
        self.logfoodLunchButton.pack(padx=40, pady=(0, 10))
        self.logfoodDinnerButton = tk.Button(self, text="Log Food for Dinner", command=lambda: self.logselectedfood("Dinner"))
        self.logfoodDinnerButton.pack(padx=40, pady=(0, 10))

        # Undo last logged meal
        self.undoButton = tk.Button(self, text="Undo Last Entry", command=lambda: self.undolastmealentry())
        self.undoButton.pack(padx=40, pady=(10, 20))

        # Display logged meals
        self.breakfastHeader = tk.Label(self, text="Breakfast")
        self.breakfastHeader.pack(pady=(0, 5))
        self.breakfastlistbox = tk.Listbox(self, width=40, height=5)
        self.breakfastlistbox.pack(padx=40, pady=(0, 25))

        self.lunchHeader = tk.Label(self, text="Lunch")
        self.lunchHeader.pack(pady=(0, 5))
        self.lunchlistbox = tk.Listbox(self, width=40, height=5)
        self.lunchlistbox.pack(padx=40, pady=(0, 25))

        self.dinnerHeader = tk.Label(self, text="Dinner")
        self.dinnerHeader.pack(pady=(0, 5))
        self.dinnerlistbox = tk.Listbox(self, width=40, height=5)
        self.dinnerlistbox.pack(padx=40, pady=(0, 30))

        self.backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
        self.backButton.pack(padx=40, pady=(0, 20))

    # Refresh meals when page is shown
    def tkraise(self):
        super().tkraise()
        self.loadTodaysMeals()
        self.updateDailyMacros()

    # Binary search to find food in sorted list
    def binarySearch(self, foodList, searchTerm):
        left = 0
        right = len(foodList) - 1
        results = []

        while left <= right:
            mid = (left + right) // 2
            midFood = foodList[mid][0]

            if searchTerm == midFood:
                results.append(foodList[mid])
                return results
            elif searchTerm < midFood:
                right = mid - 1
            else:
                left = mid + 1

        return results

    # Search for food in database using binary search
    def searchfood(self):
        searchedfood = self.searchfoodEntry.get().upper()

        # Fetch all foods sorted by name from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT name, calories, protein, carbs, fat FROM Food ORDER BY name")
        allFoods = cursor.fetchall()
        conn.close()

        # Binary search through sorted food list
        results = self.binarySearch(allFoods, searchedfood)

        # Display search results
        self.foodsearchlistbox.delete(0, tk.END)

        if len(results) == 0:
            self.foodsearchlistbox.insert(tk.END, "No foods found. Try a different search.")
            return
```

```
465
466             for food in results:
467                 displayText = "NAME: " + str(food[0]) + "  CALORIES: " + str(food[1]) + "  P: " + str(food[2]) + "g  C: " + str(food[3]) + "g  F: " + str(food[4]) + "g"
468                 self.foodsearchlistbox.insert(tk.END, displayText)
469
470         # Log selected food to specified meal in database
471         def logselectedfood(self, meal):
472             selection = self.foodsearchlistbox.curselection()
473             if len(selection) == 0:
474                 return
475
476             index = selection[0]
477             selectedfood = self.foodsearchlistbox.get(index)
478
479             if "NAME: " in selectedfood:
480                 foodName = selectedfood.split("  ")[0].replace("NAME: ", "").strip()
481
482                 # Get food details from database
483                 conn = connectDatabase()
484                 cursor = conn.cursor()
485                 cursor.execute("SELECT foodId, calories, protein, carbs, fat FROM Food WHERE name = ?", (foodName,))
486                 foodData = cursor.fetchone()
487
488                 if foodData:
489                     foodId = foodData[0]
490                     calories = foodData[1]
491                     protein = foodData[2]
492                     carbs = foodData[3]
493                     fat = foodData[4]
494
495                     todayDate = datetime.now().strftime("%Y-%m-%d")
496                     quantity = 1.0
497                     totalCal = calories * quantity
498                     totalProtein = protein * quantity
499                     totalCarbs = carbs * quantity
500                     totalFat = fat * quantity
501                     mealType = meal.lower()
502
503                     # Insert meal log into database
504                     cursor.execute("""
505                         INSERT INTO MealLogs (userId, date, mealType, foodId, quantity, totalCal, totalProtein, totalCarbs, totalFat)
506                         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
507                     """, (
508                         self.container.currentUserId,
509                         todayDate,
510                         mealType,
511                         foodId,
512                         quantity,
513                         totalCal,
514                         totalProtein,
515                         totalCarbs,
516                         totalFat
517                     ))
518                     conn.commit()
519
520                     # Add to undo stack
521                     self.mealStack.push(cursor.lastrowid)
522
523                     conn.close()
524
525                     self.loadTodaysMeals()
526                     self.updateDailyMacros()
527                     return
528
529                 conn.close()
530
531         # Undo last meal entry using stack
532         def undolastmealentry(self):
533             if self.mealStack.isEmpty() == False:
534                 lastMealId = self.mealStack.pop()
535
536                 # Delete meal from database
537                 conn = connectDatabase()
538                 cursor = conn.cursor()
539                 cursor.execute("DELETE FROM MealLogs WHERE logId = ?", (lastMealId,))
540                 conn.commit()
541                 conn.close()
```

```
542
543             self.loadTodaysMeals()
544             self.updateDailyMacros()
545
546     # Load today's logged meals from database
547     def loadTodaysMeals(self):
548         todayDate = datetime.now().strftime("%Y-%m-%d")
549
550         # Clear existing displays
551         self.breakfastlistbox.delete(0, tk.END)
552         self.lunchlistbox.delete(0, tk.END)
553         self.dinnerlistbox.delete(0, tk.END)
554
555         # Fetch today's meals from database
556         conn = connectDatabase()
557         cursor = conn.cursor()
558
559         cursor.execute("""
560             SELECT m.mealType, f.name, m.totalCal, m.totalProtein, m.totalCarbs, m.totalFat
561             FROM MealLogs m
562             JOIN Food f ON m.foodId = f.foodId
563             WHERE m.userId = ? AND m.date = ?
564             ORDER BY m.logId
565         """, (self.container.currentUserId, todayDate))
566
567         meals = cursor.fetchall()
568         conn.close()
569
570         # Display meals in appropriate listbox
571         for meal in meals:
572             mealType = meal[0]
573             foodName = meal[1]
574             totalCal = meal[2]
575             totalProtein = meal[3]
576             totalCarbs = meal[4]
577             totalFat = meal[5]
578
579             displayText = str(foodName) + "  " + str(round(totalCal, 0)) + " cal  P: " + str(round(totalProtein, 1)) + "g  C: " + str(round(totalCarbs, 1)) + "g  F: " + str(ro
580
581             if mealType == "breakfast":
582                 self.breakfastlistbox.insert(tk.END, displayText)
583             if mealType == "lunch":
584                 self.lunchlistbox.insert(tk.END, displayText)
585             if mealType == "dinner":
586                 self.dinnerlistbox.insert(tk.END, displayText)
587
588     # Update daily calorie and macro totals display
589     def updateDailyMacros(self):
590         todayDate = datetime.now().strftime("%Y-%m-%d")
591
592         # Get user's daily calorie target from database
593         conn = connectDatabase()
594         cursor = conn.cursor()
595
596         cursor.execute("SELECT dailycalories FROM Users WHERE id = ?", (self.container.currentUserId,))
597         userResult = cursor.fetchone()
598         if userResult:
599             if userResult[0]:
600                 dailycalories = userResult[0]
601             else:
602                 dailycalories = 2000
603         else:
604             dailycalories = 2000
605
606         # Sum today's totals from database
607         cursor.execute("""
608             SELECT
609                 SUM(totalCal) as totalCalories,
610                 SUM(totalProtein) as totalProtein,
611                 SUM(totalCarbs) as totalCarbs,
612                 SUM(totalFat) as totalFat
613             FROM MealLogs
614             WHERE userId = ? AND date = ?
615         """, (self.container.currentUserId, todayDate))
616
617         totals = cursor.fetchone()
618         conn.close()
```

```
619
620         if totals[0] is None:
621             totalCalories = 0
622             totalProtein = 0
623             totalCarbs = 0
624             totalFat = 0
625         else:
626             totalCalories = int(totals[0])
627             totalProtein = int(totals[1])
628             totalCarbs = int(totals[2])
629             totalFat = int(totals[3])
630 
631         caloriesRemaining = int(dailycalories - totalCalories)
632 
633         # Display summary
634         self.dailymacroslistbox.delete(0, tk.END)
635         self.dailymacroslistbox.insert(tk.END, "Date: " + todayDate)
636         self.dailymacroslistbox.insert(tk.END, "Total Calories: " + str(totalCalories) + " cal")
637         self.dailymacroslistbox.insert(tk.END, "Calories Remaining: " + str(caloriesRemaining) + " cal")
638         self.dailymacroslistbox.insert(tk.END, "Protein: " + str(totalProtein) + "g")
639         self.dailymacroslistbox.insert(tk.END, "Carbs: " + str(totalCarbs) + "g")
640         self.dailymacroslistbox.insert(tk.END, "Fat: " + str(totalFat) + "g")
641 
642 # Workout logging screen
643 class LogWorkout(tk.Frame):
644     def __init__(self, container):
645         super().__init__(container)
646         self.container = container
647 
648         self.titleLabel = tk.Label(self, text="Log Workout")
649         self.titleLabel.pack(pady=10)
650 
651         # Today's workout summary
652         self.workoutSummaryHeader = tk.Label(self, text="Today's Workout Summary")
653         self.workoutSummaryHeader.pack()
654         self.workoutSummaryListbox = tk.Listbox(self, width=40, height=5)
655         self.workoutSummaryListbox.pack(padx=80, pady=10)
656 
657         # Input fields for new workout
658         self.workoutTypeLabel = tk.Label(self, text="Workout Type (e.g., Running, Cycling, Gym):")
659         self.workoutTypeLabel.pack()
660         self.workoutTypeEntry = tk.Entry(self, width=30)
661         self.workoutTypeEntry.pack(pady=5)
662 
663         self.durationLabel = tk.Label(self, text="Duration (minutes):")
664         self.durationLabel.pack()
665         self.durationEntry = tk.Entry(self, width=30)
666         self.durationEntry.pack(pady=5)
667 
668         self.caloriesLabel = tk.Label(self, text="Calories Burned:")
669         self.caloriesLabel.pack()
670         self.caloriesEntry = tk.Entry(self, width=30)
671         self.caloriesEntry.pack(pady=5)
672 
673         self.logButton = tk.Button(self, text="Log Workout", command=lambda: self.logWorkout())
674         self.logButton.pack(pady=10)
675 
676         # Today's logged workouts
677         self.todaysWorkoutsHeader = tk.Label(self, text="Today's Workouts")
678         self.todaysWorkoutsHeader.pack(pady=10)
679         self.todaysWorkoutsListbox = tk.Listbox(self, width=60, height=8)
680         self.todaysWorkoutsListbox.pack(padx=40, pady=10)
681 
682         self.deleteButton = tk.Button(self, text="Delete Selected Workout", command=lambda: self.deleteWorkout())
683         self.deleteButton.pack(pady=5)
684 
685         self.backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
686         self.backButton.pack(padx=40, pady=10)
687 
688     # Refresh workouts when page is shown
689     def tkraise(self):
690         super().tkraise()
691         self.loadTodaysWorkouts()
692         self.updateWorkoutSummary()
693 
694     # Save workout to database
695     def logWorkout(self):
```

```
        workoutType = self.workoutTypeEntry.get().strip()
        durationStr = self.durationEntry.get().strip()
        caloriesStr = self.caloriesEntry.get().strip()

        # Validate inputs
        if len(workoutType) == 0:
            return
        if len(durationStr) == 0:
            return
        if len(caloriesStr) == 0:
            return

        if durationStr.isdigit():
            duration = int(durationStr)
        else:
            messagebox.showerror("Error", "Please enter valid numeric values for duration")
            return

        # Validate calories is a digit
        if caloriesStr.isdigit():
           caloriesBurned = int(caloriesStr)
        else:
            messagebox.showerror("Error", "Please enter valid numeric values for calories")
            return

        # Validate positive values
        if duration <= 0:
            messagebox.showerror("Error", "Duration must be a positive number")
            return

        if caloriesBurned <= 0:
            messagebox.showerror("Error", "Calories must be a positive number")
            return
        todayDate = datetime.now().strftime("%Y-%m-%d")

        # Insert workout into database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO WorkoutLogs (userId, date, workoutType, durationMin, caloriesBurned)
            VALUES (?, ?, ?, ?, ?)
        """, (
            self.container.currentUserId,
            todayDate,
            workoutType,
            duration,
            caloriesBurned
        ))
        conn.commit()
        conn.close()

        # Clear input fields
        self.workoutTypeEntry.delete(0, tk.END)
        self.durationEntry.delete(0, tk.END)
        self.caloriesEntry.delete(0, tk.END)

        self.loadTodaysWorkouts()
        self.updateWorkoutSummary()

    # Load today's workouts from database
    def loadTodaysWorkouts(self):
        todayDate = datetime.now().strftime("%Y-%m-%d")

        self.todaysWorkoutsListbox.delete(0, tk.END)

        # Fetch workouts from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT logId, workoutType, durationMin, caloriesBurned
            FROM WorkoutLogs
            WHERE userId = ? AND date = ?
            ORDER BY logId
        """, (self.container.currentUserId, todayDate))

        workouts = cursor.fetchall()
```

```
        conn.close()

        if len(workouts) == 0:
            self.todaysWorkoutsListbox.insert(tk.END, "No workouts logged today")
            return

        # Display workouts
        for workout in workouts:
            logId = workout[0]
            workoutType = workout[1]
            duration = workout[2]
            calories = workout[3]
            displayText = str(workoutType) + "  " + str(duration) + " min   " + str(calories) + " cal burned [ID: " + str(logId) + "]"
            self.todaysWorkoutsListbox.insert(tk.END, displayText)

    # Calculate and display workout summary
    def updateWorkoutSummary(self):
        todayDate = datetime.now().strftime("%Y-%m-%d")

        # Sum workout totals from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT
                SUM(durationMin) as totalDuration,
                SUM(caloriesBurned) as totalCalories,
                COUNT(*) as workoutCount
            FROM WorkoutLogs
            WHERE userId = ? AND date = ?
        """, (self.container.currentUserId, todayDate))

        totals = cursor.fetchone()
        conn.close()

        if totals[0] is None:
            totalDuration = 0
            totalCalories = 0
            workoutCount = 0
        else:
            totalDuration = int(totals[0])
            totalCalories = int(totals[1])
            workoutCount = int(totals[2])

        # Display summary
        self.workoutSummaryListbox.delete(0, tk.END)
        self.workoutSummaryListbox.insert(tk.END, "Date: " + str(todayDate))
        self.workoutSummaryListbox.insert(tk.END, "Total Workouts: " + str(workoutCount))
        self.workoutSummaryListbox.insert(tk.END, "Total Duration: " + str(totalDuration) + " min")
        self.workoutSummaryListbox.insert(tk.END, "Total Calories Burned: " + str(totalCalories) + " cal")

    # Delete selected workout from database
    def deleteWorkout(self):
        selection = self.todaysWorkoutsListbox.curselection()

        if len(selection) == 0:
            return

        index = selection[0]
        selectedItem = self.todaysWorkoutsListbox.get(index)

        # Extract workout ID from display text
        if "[ID: " in selectedItem:
            logIdStr = selectedItem.split("[ID: ")[1].split("]")[0]
            logId = int(logIdStr)

            # Delete from database
            conn = connectDatabase()
            cursor = conn.cursor()
            cursor.execute("DELETE FROM WorkoutLogs WHERE logId = ? AND userId = ?", (logId, self.container.currentUserId))
            conn.commit()
            conn.close()

            self.loadTodaysWorkouts()
            self.updateWorkoutSummary()

# Statistics and graphs screen
```

```python
class ViewStats(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

    # Load and display user stats
    def getStats(self):
        # Fetch user stats from database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("SELECT name, initialWeight, heightCm, bmi FROM users WHERE id = ?", [str(self.container.currentUserId)])

        stats = cursor.fetchall()
        conn.close()

        (variable) todolist: Listbox
        todolist = tk.Listbox(self, width=70, height=20, font=("Arial", 12))
        todolist.grid(row=0, column=0, padx=20, pady=20)

        for stat in stats:
            todolist.insert(tk.END, "Name: " + str(stat[0]) + " | Weight: " + str(stat[1]) + "kg | Height: " + str(stat[2]) + "m | BMI: " + str(stat[3]))

        # Buttons to show different graphs
        graphButton1 = tk.Button(self, text="Calorie Intake (Last 7 Days)", width=30, command=lambda: self.showCalorieGraph())
        graphButton1.grid(row=1, column=0, pady=5)

        graphButton2 = tk.Button(self, text="Workout Minutes (Last 7 Days)", width=30, command=lambda: self.showWorkoutGraph())
        graphButton2.grid(row=2, column=0, pady=5)

        graphButton3 = tk.Button(self, text="Macronutrient Breakdown (Today)", width=30, command=lambda: self.showMacroGraph())
        graphButton3.grid(row=3, column=0, pady=5)

        backButton = tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu"))
        backButton.grid(row=4, column=0, pady=10)

    # Display bar chart of last 7 days calorie intake
    def showCalorieGraph(self):
        conn = connectDatabase()
        cursor = conn.cursor()

        dates = []
        calories = []

        # Get data for each of last 7 days
        for i in range(7):
            date = (datetime.now() - timedelta(days=6 - i)).strftime("%Y-%m-%d")
            dates.append(date[-5:])  # Format as MM-DD

            # Sum calories from database for each day
            cursor.execute("""
                SELECT SUM(totalCal) FROM MealLogs
                WHERE userId = ? AND date = ?
            """, (self.container.currentUserId, date))

            result = cursor.fetchone()
            if result[0]:
                calories.append(result[0])
            else:
                calories.append(0)

        conn.close()

        # Create bar chart
        plt.figure(figsize=(10, 6))
        plt.bar(dates, calories, color='#4CAF50')
        plt.xlabel('Date', fontsize=12)
        plt.ylabel('Calories', fontsize=12)
        plt.title('Calorie Intake over the last 7 Days', fontsize=14, fontweight='bold')
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()

    # Display bar chart of last 7 days workout minutes
    def showWorkoutGraph(self):
        conn = connectDatabase()
        cursor = conn.cursor()
```

```
926
927         dates = []
928         minutes = []
929
930         # Get data for each of last 7 days
931         for i in range(7):
932             date = (datetime.now() - timedelta(days=6 - i)).strftime("%Y-%m-%d")
933             dates.append(date[-5:])
934
935             # Sum workout duration from database for each day
936             cursor.execute("""
937                 SELECT SUM(durationMin) FROM WorkoutLogs
938                 WHERE userId = ? AND date = ?
939             """, (self.container.currentUserId, date))
940
941             result = cursor.fetchone()
942             if result[0]:
943                 minutes.append(result[0])
944             else:
945                 minutes.append(0)
946
947         conn.close()
948
949         # Create bar chart
950         plt.figure(figsize=(10, 6))
951         plt.bar(dates, minutes, color='#2196F3')
952         plt.xlabel('Date', fontsize=12)
953         plt.ylabel('Minutes', fontsize=12)
954         plt.title('Workout Duration over the Last 7 Days', fontsize=14, fontweight='bold')
955         plt.xticks(rotation=45)
956         plt.tight_layout()
957         plt.show()
958
959     # Display pie chart of today's macronutrient breakdown
960     def showMacroGraph(self):
961         conn = connectDatabase()
962         cursor = conn.cursor()
963
964         todayDate = datetime.now().strftime("%Y-%m-%d")
965
966         # Get today's macro totals from database
967         cursor.execute("""
968             SELECT SUM(totalProtein), SUM(totalCarbs), SUM(totalFat)
969             FROM MealLogs
970             WHERE userId = ? AND date = ?
971         """, (self.container.currentUserId, todayDate))
972
973         result = cursor.fetchone()
974         conn.close()
975
976         # Don't show graph if no data
977         if result[0] is None:
978             return
979         if result[0] == 0 and result[1] == 0 and result[2] == 0:
980             return
981
982         if result[0]:
983             protein = result[0]
984         else:
985             protein = 0
986
987         if result[1]:
988             carbs = result[1]
989         else:
990             carbs = 0
991
992         if result[2]:
993             fat = result[2]
994         else:
995             fat = 0
996
997         # Create pie chart
998         labels = ['Protein', 'Carbs', 'Fat']
999         sizes = [protein, carbs, fat]
1000        colors = ['#FF6B6B', '#4ECDC4', '#FFE66D']
1001        explode = (0.05, 0.05, 0.05)
```

```
        plt.figure(figsize=(8, 8))
        plt.pie(sizes, explode=explode, labels=labels, colors=colors, autopct='%1.1f%%',
                shadow=True, startangle=90, textprops={'fontsize': 12})
        plt.title('Macronutrient Breakdown - ' + str(todayDate), fontsize=14, fontweight='bold')
        plt.axis('equal')
        plt.tight_layout()
        plt.show()


# Settings screen for updating user data
class Settings(tk.Frame):
    def __init__(self, container):
        super().__init__(container)
        self.container = container

        tk.Label(self, text="Settings", font=("Helvetica", 18, "bold")).pack(pady=20)

        # Input fields for updating weight goals
        tk.Label(self, text="Update Current Weight (kg):").pack(pady=5)
        self.weightEntry = tk.Entry(self, width=30)
        self.weightEntry.pack(pady=5)

        tk.Label(self, text="Update Goal Weight (kg):").pack(pady=5)
        self.goalWeightEntry = tk.Entry(self, width=30)
        self.goalWeightEntry.pack(pady=5)

        tk.Button(self, text="Save Changes", width=20, command=self.updateSettings).pack(pady=20)

        # Password change section
        tk.Label(self, text="Change Password", font=("Helvetica", 12, "bold")).pack(pady=15)

        tk.Label(self, text="New Password:").pack(pady=5)
        self.newPasswordEntry = tk.Entry(self, width=30, show="*")
        self.newPasswordEntry.pack(pady=5)

        tk.Button(self, text="Update Password", width=20, command=self.changePassword).pack(pady=15)

        tk.Button(self, text="Back to Main Menu", command=lambda: self.container.showPage("MainMenu")).pack(pady=10)

    # Update user weight and recalculate BMI and daily calories
    def updateSettings(self):
        weightStr = self.weightEntry.get().strip()
        goalWeightStr = self.goalWeightEntry.get().strip()

        if len(weightStr) == 0 and len(goalWeightStr) == 0:
            return

        # Get current user data from database
        conn = connectDatabase()
        cursor = conn.cursor()

        cursor.execute("SELECT initialWeight, goalweight, heightCm, age, gender, activityLevel FROM Users WHERE id = ?", (self.container.currentUserId,))
        userData = cursor.fetchone()

        currentWeight = userData[0]
        currentGoalWeight = userData[1]
        height = userData[2]
        age = userData[3]
        gender = userData[4]
        activity = userData[5]

        # Update weight if provided
        if len(weightStr) > 0:
            if self.isValidFloat(weightStr):
                currentWeight = float(weightStr)
                cursor.execute("UPDATE Users SET initialWeight = ? WHERE id = ?", (currentWeight, self.container.currentUserId))
            else:
                conn.close()
                messagebox.showerror("Error", "Please enter valid weight")
                return
```

```
        # Update goal weight if provided
        if len(goalWeightStr) > 0:
            if self.isValidFloat(goalWeightStr):
                currentGoalWeight = float(goalWeightStr)
                cursor.execute("UPDATE Users SET goalweight = ? WHERE id = ?", (currentGoalWeight, self.container.currentUserId))
            else:
                conn.close()
                messagebox.showerror("Error", "Please enter valid goal weight")
                return

        # Recalculate BMI with new weight
        newBMI = currentWeight / ((height / 100) ** 2)
        cursor.execute("UPDATE Users SET bmi = ? WHERE id = ?", (newBMI, self.container.currentUserId))

        # Recalculate daily calorie target
        if gender.upper() == 'MALE':
            bmr = 10 * currentWeight + 6.25 * height - 5 * age + 5
        else:
            bmr = 10 * currentWeight + 6.25 * height - 5 * age - 161

        activity_levels = {
            'SEDENTARY': 1.2,
            'LIGHT': 1.375,
            'MODERATE': 1.55,
            'ACTIVE': 1.725,
            'VERY ACTIVE': 1.9
        }
        tdee = bmr * activity_levels.get(activity.upper(), 1.2)

        weight_change = currentGoalWeight - currentWeight
        kcal_change = weight_change * 7700
        weeks = 12
        daily_adjustment = kcal_change / (weeks * 7)

        dailycalories = tdee + daily_adjustment

        if dailycalories < 1200:
            dailycalories = 1200

        # Update daily calories in database
        cursor.execute("UPDATE Users SET dailycalories = ? WHERE id = ?", (dailycalories, self.container.currentUserId))

        conn.commit()
        conn.close()

        # Clear input fields
        self.weightEntry.delete(0, tk.END)
        self.goalWeightEntry.delete(0, tk.END)

    # Check if string is valid decimal number
    def isValidFloat(self, value):
        dotCount = 0
        for char in value:
            if char == '.':
                dotCount = dotCount + 1
            elif char.isdigit():
                continue
            else:
                return False
        if dotCount > 1:
            return False
        return True

    # Update password in database
    def changePassword(self):
        newpassword = self.newPasswordEntry.get()
        hashed = self.container.hasher.hash(newpassword)  # Hash new password

        # Update password hash in database
        conn = connectDatabase()
        cursor = conn.cursor()
        cursor.execute("UPDATE Users SET passwordHash = ? WHERE id = ?", (hashed, self.container.currentUserId))
        conn.commit()
        conn.close()
```

```

        self.newPasswordEntry.delete(0, tk.END)

        self.newPasswordEntry.delete(0, tk.END)

# Run the application
if __name__ == "__main__":
    hasher = Passwordhasher()
    app = HealthTrackerApp(hasher)
    app.mainloop()
```